O'REILLY®

2nd Edition



High Performance Spark

Best Practices for Scaling and Optimizing Apache Spark

Holden Karau,
Adi Polak &
Rachel Warren

"The best Spark book has been updated with coverage of the cutting edge of Apache Spark. With guidance on GenAI, Spark Connect, Adaptive Query Execution, and almost everything added to the project since the first edition, it's amazing it all fits in a single volume."

Russell Spitzer, PMC member of the Apache Iceberg and Apache Polaris Projects, principal engineer at Snowflake

High Performance Spark

Apache Spark is amazing when everything clicks. But if you haven't seen the performance improvements you expected or still don't feel confident enough to use Spark in production, this practical book is for you. Authors Holden Karau, Adi Polak, and Rachel Warren take you through the secrets of the Spark code base and demonstrate performance optimizations that will help your data pipelines run faster, scale to larger datasets, and avoid costly antipatterns.

Ideal for data engineers, software engineers, data scientists, and system administrators, the second edition of *High Performance Spark* presents new use cases, code examples, and best practices for Spark 3.x, 4.x, and beyond. This book gives you a fresh perspective on this continually evolving framework and shows you how to navigate bumps on your Spark and PySpark journey.

- **Accelerate your ML workflows with integrations including PyTorch**
- **Take advantage of GPU acceleration with RAPIDS and resource profiles**
- **Handle key skew and take advantage of Spark's new dynamic partitioning**
- **Deploy Spark on Kubernetes and similar environments**
- **Use Spark to productionize exploratory data science projects**
- **Gain faster insights by reducing pipeline running times**

Holden Karau is a transgender Canadian software engineer and an Apache Spark committer specializing in PySpark and machine learning. She speaks internationally on Spark, contributes actively to open source, and holds a degree in computer science from the University of Waterloo.

Adi Polak is a software engineer specializing in data and machine learning who has contributed to Apache Spark since 2013, teaching Spark to thousands of students worldwide. She is an official Databricks ambassador, the author of *Scaling Machine Learning with Spark*, and a respected international keynote presenter.

Rachel Warren is a data scientist and software engineer at Alpine Data Labs, where she uses Spark to address real-world data processing challenges. She has experience working as an analyst in both industry and academia. Rachel graduated with a degree in computer science from Wesleyan University.

DATA / DATABASES

US $65.99 CAN $82.99

ISBN: 978-1-098-14585-9



O'REILLY®

Praise for the Second Edition of *High Performance Spark*

The best Spark book has been updated with coverage of the cutting-edge of Apache Spark. With guidance on GenAI, Spark Connect, adaptive query execution, and almost everything added to the project since the first edition, it's amazing it all fits in a single volume. I thought I'd never remove my first edition copy from my desk, but it looks like that time has come!

—Russell Spitzer, PMC member of the Apache Iceberg and Apache Polaris Projects, principal engineer at Snowflake

High Performance Spark bridges the gap between writing Spark code and understanding how it actually executes, giving engineers the intuition needed to debug and optimize real-world Spark workloads at scale. Backed by years of hands-on experience and contributions to the Spark community, the authors bring credibility to every concept in this book.

—Dipankar Mazumdar, director of Developer Relations at Cloudera, author of Engineering Lakehouses

Scaling Spark is easy until it isn't. Also, Spark has changed a lot since they published the first edition of this great book. Holden, Adi, and Rachel cut through the noise to deliver a practical, no-nonsense masterclass on maximizing Spark's potential. What I like is that they write in a personable way, drawing from deep first hand experience. If you're serious about engineering with Spark, this book is required reading.

—Joe Reis, coauthor of Fundamentals of Data Engineering

You can't build an AI-first business without the data layer, and Spark remains one of the most consequential data technologies. This book does a great job laying out the fundamentals of Spark and how to achieve scalable data transformations. It also covers how to make Spark work on GPUs and its integration with DeepSpeed, TensorFlow, and PyTorch, making Spark ready for the AI era.

— *Chip Huyen, author of* Designing Machine Learning Systems and AI Engineering

I make my living telling people their data isn't as big as they think. But when it actually is, I want them holding this book. Holden, Adi, and Rachel have turned years of hard-won Spark scars into the kind of practical, internals-deep guidance that can turn a 3 AM job failure into a one-line config change.

—Ryan Boyd, cofounder, MotherDuck

2ND EDITION

High Performance Spark

Best Practices for Scaling and Optimizing Apache Spark

Holden Karau, Adi Polak, and Rachel Warren

O'REILLY®

High Performance Spark
by Holden Karau, Adi Polak, and Rachel Warren

Copyright © 2026 Pigs Can Fly Labs LLC and Adi Polak. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Aaron Black
Development Editor: Shira Evans
Production Editor: Christopher Faucher
Copyeditor: nSight, Inc.
Proofreader: Kim Cofer

Indexer: WordCo Indexing Services, Inc.
Cover Designer: Susan Thompson
Cover Illustrator: Karen Montgomery
Interior Designer: David Futato
Interior Illustrator: Kate Dullea

May 2017: First Edition
June 2026: Second Edition

Revision History for the Second Edition
2026-05-29: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781098145859* for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *High Performance Spark*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-14585-9

[LSI]

Table of Contents

Preface xiii

1. Introduction to High Performance Spark 1
What Is Spark and Why Performance Matters 1
What You Can Expect to Get from This Book 2
Spark Versions 3
Why the Focus on Scala and Python? 3
To Be a Spark Expert You Have to Be Able to Read a Little Scala Anyway 4
The Spark Scala and Python APIs Are Easier to Use Than the Java APIs 4
Why Not Scala? 4
Learning Scala 5
Conclusion 6

2. How Spark Works 7
How Spark Fits into the Big Data Ecosystem 8
Cluster Managers 9
Catalog System 10
Multitool Pipelines 10
Spark Components 11
Spark Model of Parallel Computing: RDDs 13
Lazy Evaluation 13
In-Memory Persistence and Memory Management 17
Immutability and the RDD Interface 18
Types of RDDs 19
Functions on RDDs: Transformations Versus Actions 20
Wide Versus Narrow Dependencies 21
Spark Job Scheduling 23
Resource Allocation 23

The Spark Application 23
Default Spark Scheduler and Concurrent Actions 25
The Anatomy of a Spark Job 26
The DAG 26
Jobs 27
Stages 27
Tasks 28
Spark Connect 30
Conclusion 32

3. Upgrading Spark. 33
Finding What You Need to Change 33
Compile-Time Changes 34
Exceptions at Runtime 34
Differing Results 35
Updating Your Code 35
Scala 35
Python 38
SQL 39
Verifying Correctness and Performance 39
Places Where Performance Can Get Worse 42
Conclusion 43

4. What's New in Spark 4.2 Since 2.4. 45
The Bad News: GraphX 45
The Good News 46
No Code Changes Required 46
Partition Pruning and Predicate Pushdown 46
Joins 47
Debugging 48
Additional Upgrades Required (Not Code) 48
Code and Logic Changes Needed 49
General 49
Python Specific 51
Scala Specific 51
Prioritizing Upgrades 51
Totally New Features 52
Spark Declarative Pipelines 52
SQL UDFs/UDTFs 53
Looking Ahead to the Future 53
Conclusion 53

5. DataFrames, Datasets, and Spark SQL. 55
Getting Started with the SparkSession (or HiveContext or SQLContext) 57
Spark SQL Dependencies 58
Basics of Schemas 60
Column Metadata 62
DataFrame API 63
Transformations 63
Multi-DataFrame Transformations 76
Plain Old SQL Queries and Interacting with Metastore Tables (Hive, Iceberg, etc.) 77
Data Representation in DataFrames and Datasets 78
Tungsten 78
Datasets 79
Interoperability with RDDs, DataFrames, and Local Collections 79
Compile-Time Strong Typing 81
Easier Functional (RDD "like") Transformations 81
Relational Transformations 81
Multi-Dataset Relational Transformations 82
Grouped Operations on Datasets 82
Extending with User-Defined Functions (UDFs), Aggregate Functions (UDAFs), and Expressions 83
Custom Expressions 87
Custom Encoders 89
User-Defined Table Functions 89
Data Loading and Saving Functions 90
DataFrameWriter and DataFrameReader 90
Formats 90
Save Modes 99
Merge 100
Partitions (Discovery and Writing) 101
Query Optimizer 102
Logical and Physical Plans 102
Static Optimizer Rules You Should Understand (and Working Around Them) 103
Understanding Spark Adaptive Query Execution 104
Code Generation 105
Large Query Plans and Iterative Algorithms 105
SparkSession Extensions and Custom Optimizers 106
Debugging Spark SQL Queries 108
JDBC/ODBC Server 108
Conclusion 109

6. Joins (SQL and Core)..... 111
Core Spark Joins 111
Choosing a Join Type 113
Choosing an Execution Plan 115
Spark SQL Joins 118
DataFrame Joins 118
Concrete SQL Join Execution Operators 123
Dataset Joins 125
Conclusion 126

7. Effective Transformations..... 127
Narrow Versus Wide Transformations 128
Implications for Performance 130
Implications for Fault Tolerance 131
The Special Case of coalesce 132
What Type of RDD Does Your Transformation Return? 132
Minimizing Object Creation 134
Reusing Existing Objects 134
Using Smaller Data Structures 138
Iterator-to-Iterator Transformations with mapPartitions 141
What Is an Iterator-to-Iterator Transformation? 142
Space and Time Advantages 143
An Example 143
Set Operations 147
Reducing Setup Overhead 148
Shared Variables 149
Broadcast Variables 149
Accumulators 150
Reusing RDDs 155
Cases for Reuse 155
Deciding if Recompute Is Inexpensive Enough 157
Types of Reuse: Cache, Persist, Checkpoint, Shuffle Files 159
Alluxio (nee Tachyon) 163
LRU Caching 163
Shuffle Files 164
Noisy Cluster Considerations 166
Interaction with Accumulators 167
Conclusion 167

8. Working with Key/Value Data..... 169
The Goldilocks Example 171
Goldilocks Version 0: Iterative Solution 172

How to Use PairRDDFunctions and OrderedRDDFunctions 174
Actions on Key/Value Pairs 175
What's So Dangerous About the groupByKey Function? 176
Goldilocks Version 1: groupByKey Solution 176
Choosing an Aggregation Operation 180
Dictionary of Aggregation Operations with Performance Considerations 180
Preventing Out-Of-Memory Errors with Aggregation Operations 182
Multiple RDD Operations 184
Co-Grouping 184
Partitioners and Key/Value Data 185
Using the Spark Partitioner Object 186
Hash Partitioning 186
Range Partitioning 186
Custom Partitioning 187
Preserving Partitioning Information Across Transformations 188
Leveraging Co-Located and Co-Partitioned RDDs 188
Dictionary of Mapping and Partitioning Functions PairRDDFunctions 190
Dictionary of OrderedRDDOperations 192
Sorting by Two Keys with sortByKey 193
Secondary Sort and repartitionAndSortWithinPartitions 193
Leveraging repartitionAndSortWithinPartitions for a Group by Key and Sort Values Function 195
How Not to Sort by Two Orderings 198
Goldilocks Version 2: Secondary Sort 198
A Different Approach to Goldilocks 202
Goldilocks Version 3: Sort on Cell Values 207
Straggler Detection and Unbalanced Data 208
Back to Goldilocks (Again) 210
Goldilocks Version 4: Reduce to Distinct on Each Partition 210
Conclusion 216

9. Going Beyond Scala. 219
Beyond Scala Within the JVM 221
Custom Code Beyond Scala and Beyond the JVM 225
How SparklyR Works 234
Spark on the Common Language Runtime (CLR)—C# and Friends 236
Calling Other Languages from Spark 236
Using Pipe and Friends 236
Java Native Interface 238
Java Native Access (JNA) 241
Project Panama 242
Underneath Everything Is Fortran 242

Getting to the GPU 243
Going Beyond the JVM with Spark Accelerators 243
Databricks Photon 245
Apache Arrow DataFusion Comet 246
Project Gluten 247
Spark RAPIDS 250
Application-Specific Integrated Circuits (ASICs) 252
Managing Memory Outside of the JVM 253
The Future (from ~2026) 253
Conclusion 253

10. Spark: A Thoughtful Step into Generative AI. 255
So, What Is This Generative AI Thing? 256
How Generative AI Works: Training the Creator 257
A World of Applications: Generative AI at Work 258
How Is GenAI Related to Deep Learning? 259
Can You Spark It? 260
Effortless TensorFlow Scaling with Spark's TensorFlowDistributor 261
Scaling PyTorch with Spark's TorchDistributor 262
Supercharging Distributed Training with DeepSpeed 263
Wait, Your Processor Matters! 266
Inference in the Real World: Beyond Batch Prediction 271
Batch Prediction: Scaling Predictions with Spark 271
Model as a Service: Real-Time Predictions at Scale 272
Model in a Service: Embedding Inference into Workflows 272
A World of Inference Possibilities 273
Evaluation of Models 273
Case Studies 274
Fight Health Insurance 275
Uber's Enhanced Agentic-RAG (EAg-RAG):
When Chatbots Deliver Near-Human Precision 277
The Model Dilemma: Hosting or Leveraging a Third-Party REST API 287
Layer 1: Use Case and Domain Alignment 287
Layer 2: Performance Requirements 287
Layer 3: Scalability Needs 288
Layer 4: Cost Considerations 288
Layer 5: Data Sensitivity and Security 288
Layer 6: Integration and Ecosystem Fit 289
Layer 7: Time-to-Market 289
The Decision 289
Alex's Reflection Moment 289
The Story of an Evolving Industry 290

Conclusion: The Future of Generative AI with Spark 290
Spark's Role in the Generative AI Revolution 290
Generative AI and the Creative Industries 291
The Path Forward 291
Your Next Step 291

11. Testing, Validation, and Side-By-Side Runs. 293
Unit Testing 295
Factoring Your Code for Testability 295
Mocking RDDs 296
Core Spark Jobs (Testing with RDDs) 297
Testing Streaming 298
Testing DataFrames 301
Testing Codegen 304
Getting Test Data 304
Generating Large Datasets 305
Sampling 306
Property Checking with ScalaCheck 307
Computing RDD Difference 308
Integration Testing 310
Choosing Your Integration Testing Environment 310
Verifying Performance 311
Spark Counters for Verifying Performance 311
Projects for Verifying Performance 313
Validation (or Audits) 313
Data Validation 313
Accumulators and Built-In Counters 322
Side-by-Side Runs 323
Table-Level Assertions 323
Conclusion 323

12. Spark Components and Packages. 325
GraphX 327
Using Community Packages and Libraries 327
Popular Packages 330
Creating a Spark Package 331
Building Your Own Spark or Using Packages 333
Conclusion 334

A. Just Enough Iceberg and Friends. 335

B. Spark Connect. 339

C. When Not to Use Spark. 343

D. Advanced Task Scheduling: Gangs and Resource Profiles. 349

E. Spark Streaming. 351

F. The Spark Web UI: Debugging and Optimizing Your Jobs. 365

Index. 377

Preface

We wrote this book for data engineers, data scientists, and ML practitioners who are looking to get the most out of Spark. If you've been working with Spark and invested in Spark but your experience so far has been mired by memory errors and mysterious, intermittent failures, this book is for you. If you have been using Spark for some exploratory work or experimenting with it on the side but have not felt confident enough to put it into production, this book may help. If you are enthusiastic about Spark but have not seen the performance improvements from it that you expected, we hope this book can help. This book is intended for those who have some working knowledge of Spark and may be difficult to understand for those with little or no experience with Spark or distributed computing. For recommendations of more introductory literature, see "Supporting Books and Materials" on page xiv.

We expect this text will be most useful to those who care about optimizing repeated queries in production, rather than to those who are primarily doing exploratory work. While writing highly performant queries is perhaps more important to the data engineer, writing those queries with Spark, in contrast to other frameworks, requires a good knowledge of the data, which is usually more intuitive to the data scientist. Thus, it may be more useful to a data engineer who may be less experienced with thinking critically about the statistical nature, distribution, and layout of data when considering performance. We hope that this book will help data engineers think more critically about their data as they put pipelines into production. Similarly for data scientists we hope to provide more understanding of how Spark works so they can use their knowledge of the data for high performance queries. We want to help our readers ask questions such as "How is my data distributed?" "Is it skewed?" "What is the range of values in a column?" and "How do we expect a given value to group?" and then apply the answers to those questions to the logic of their Spark queries.

However, even for data scientists using Spark mostly for exploratory purposes, this book should cultivate some important intuition about writing performant Spark queries, so that as the scale of the exploratory analysis inevitably grows, you may have a better shot of getting something to run the first time. We hope to guide data

scientists, even those who are already comfortable thinking about data in a distributed way, to think critically about how their programs are evaluated, empowering them to explore their data more fully and more quickly and to communicate effectively with anyone helping them put their algorithms into production.

Regardless of your job title, it is likely that the amount of data with which you are working is growing quickly. Your original solutions may need to be scaled, and your old techniques for solving new problems may need to be updated. We hope this book will help you leverage Apache Spark to tackle new problems more easily and old problems more efficiently.

Second Edition Notes

You are reading the second edition of *High Performance Spark*, and for that, we thank you! If you find errors or mistakes or have ideas for ways to improve this book, please reach out to us at *high-performance-spark@googlegroups.com*. If you wish to be included in a "thanks" section in future editions of the book, please include your preferred display name.

Supporting Books and Materials

For data scientists and developers new to Spark, *Learning Spark*, 2nd edition, by Jules S. Damji, Brooke Wenig, Tathagata Das, and Denny Lee is an excellent introduction.[1] If you're interested in a deeper dive into ML pipelines, check out *Scaling Machine Learning with Spark* by Adi Polak and *Advanced Analytics with Spark* by Sandy Ryza, Uri Laserson, Sean Owen, and Josh Wills. Both are great books for interested data scientists and ML practitioners. For individuals more interested in streaming, *Stream Processing with Apache Spark* by Gerard Maas and François Garillot may also be of use.

Beyond books, there is also a collection of intro-level Spark training material available. For individuals who prefer video. Commercially, Databricks as well as Cloudera and other Hadoop/Spark vendors offer Spark training. Previous recordings of Data + AI Summit, Spark Summit, and Spark camps, as well as many other great resources, have been posted on the Apache Spark documentation page.

1 Though we may be biased, Holden was one of the coauthors for the first edition.

If you don't have experience with Scala, we do our best to convince you to pick up Scala in Chapter 1, and if you are interested in learning, *Programming Scala*, 3rd Edition, by Dean Wampler is a good introduction.[2]

For people wanting to dive into the internals of Spark, Jacek Laskowski's "The Internals of Spark Core" remains one of the best online resources.

If you want to introduce your children, or executives, to the world of Apache Spark, Holden would like to recommend her own soon-to-be-published "Distributed Computing 4 Kids (and Executives)".

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

2 Although it's important to note that some of the practices suggested in this book are not common practice in Spark code.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download from the High Performance Spark GitHub repository and some of the testing code is available at the spark-testing-base GitHub repository and the Spark Validator repo. Structured Streaming machine learning examples are available at *https://github.com/holdenk/spark-structured-streaming-ml*.

If you have a technical question or a problem using the code examples, please send email to *support@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*High Performance Spark*, 2nd edition by Holden Karau, Adi Polack, and Rachel Warren (O'Reilly). Copyright 2026 Pigs Can Fly Labs LLC and Adi Polak, 978-1-098-14585-9."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Online Learning



For more than 40 years, O'Reilly Media has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

For feedback, email the authors at *high-performance-spark@googlegroups.com*. For random ramblings, occasionally about Spark, follow us on "social media":

Holden Karau: *https://linkedin.com/in/holdenkarau*, *https://twitch.tv/holdenkarau*, *https://bsky.app/profile/holdenkarau.com*, *https://tech.lgbt/@holden*

Adi Polak: *https://linkedin.com/in/polak-adi*

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
141 Stony Circle, Suite 195
Santa Rosa, CA 95401
800-889-8969 (in the United States or Canada)
707-827-7019 (international or local)
707-829-0104 (fax)
support@oreilly.com
https://oreilly.com/about/contact.html

We have a web page for this book, where we list errata and any additional information. You can access this page at *https://oreil.ly/high-performance-spark-2e*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly*.

Watch us on YouTube: *https://youtube.com/oreillymedia*.

General Acknowledgments

Second Edition

In the 2nd edition we've had some amazing technical reviewers, including Tristen Wentling, Scott Haines, Anya Bida, and Zubair Muhammad. In addition we also had a copy review from Kat Tory on some of our new content.

Two appendices were contributed externally. Appendix E was written in a large part by Tomasz Magdanski with Scott Haines contributing an update for the new "real-time" mode. Appendix F was contributed by Daniel Aronovich. All mistakes remain our fault of course. This project would not have been possible without the amazing O'Reilly staff we've worked with throughout the process (we're sorry it took so long) including Shira Evans, Sara Hunter, Christopher Faucher, Kristen Brown, and more.

None of this would have been possible without the community that makes Apache Spark, from the core developers and PMC to the users; thank you. The 2nd edition would not have been possible without the 1st edition, and all who contributed to it.

First Edition

The authors would like to acknowledge everyone who has helped with comments and suggestions on early drafts of our work. Special thanks to Anya Bida, Jakob Odersky, and Katharine Kearnan for reviewing early drafts and diagrams. We'd like to thank Mahmoud Hanafy for reviewing and improving the sample code as well as early drafts. We'd also like to thank Michael Armbrust for reviewing and providing feedback on early drafts of the SQL chapter. Justin Pihony has been one of the most active early readers, suggesting fixes in every respect (language, formatting, etc.).

Thanks to all of the readers of our O'Reilly early release who have provided feedback on various errata, including Kanak Kshetri and Rubén Berenguel.

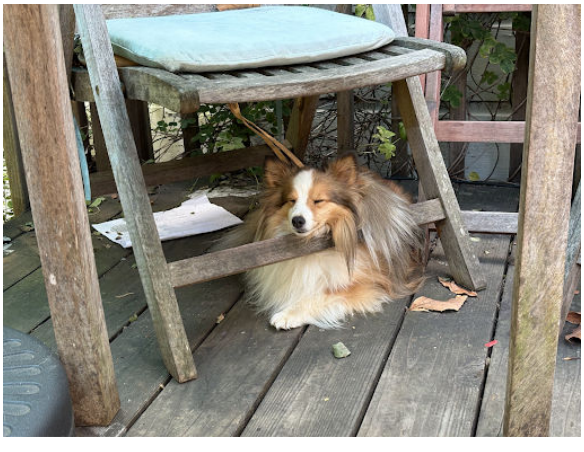
We'd also like to thank our dedicated (official) technical reviewers from the first edition Neelesh Srinivas Salian and Denny Lee, who read through every page, providing detailed feedback, and helped us decide what content belonged where.

Finally, thank you to our respective employers for being understanding as we've worked on this book. Especially Lawrence Spracklen, who insisted we mention him here :p.

Personal Acknowledgments

Holden Karau

I would also like to thank my wife and partners for putting up with my long in-the-bathtub writing sessions. A special thank you to Timbit for guarding the house and generally giving me a reason to get out of bed (albeit often a bit too early for my taste). An extra thank you to Paco Nathan and Ann Spencer whose advice have helped me navigate this book and so many more things.



Adi Polak

This book is dedicated to my husband and Arya, my daughter. Thank you for being my grounding force, my source of joy, and the reason behind so much of what I do.

To my husband, thank you for enduring the long writing sessions, the endless technical tangents, and the many moments when I was physically present but mentally buried somewhere inside a Spark execution plan. Your patience, support, and ability to pull me back to earth made this book possible.

To Arya, thank you for shining so brightly and for bringing light, laughter, and perspective into every day. Even during the most intense deadlines, you reminded me what truly matters.

I also want to thank the people who challenged and sharpened my thinking over the years through deep technical conversations, difficult questions, and honest feedback. A special thank you to Holden for the opportunity to coauthor, and to the mentors, peers, and collaborators whose guidance shaped not only this book, but many parts of my career.

And finally, to my cat, Chupa, for consistently reminding me that no matter how critical Spark performance may seem, an empty food bowl is always the highest-priority production incident.





CHAPTER 1

Introduction to High Performance Spark

This chapter provides an overview of what we hope you will be able to learn from this book and does its best to convince you to learn to read some Scala. Feel free to skip ahead to Chapter 2 if you already know what you're looking for.

What Is Spark and Why Performance Matters

ASF (currently) stands for *Apache Software Foundation*, although there are calls to rename the foundation.[1] Spark is a high performance, general-purpose data-parallel distributed computing system that has become the most active ASF open source project, with more than 2,000 active contributors. Spark enables us to process large quantities of data, beyond what can fit on a single machine, with a high-level, relatively easy-to-use API. Spark's design and interface are unique, and it is one of the fastest systems of its kind. Uniquely, Spark allows us to write the logic of data transformations and machine learning algorithms in a parallelizable way while being relatively system agnostic.[2]

However, despite its many advantages and the excitement around Spark, the simplest implementation of many common data science routines in Spark can be much slower and less robust than the best version. Since the computations we are concerned with may involve data at a very large scale, the time and resource gains from tuning code for performance are enormous. Performance does not just mean running faster; often, at this scale, it means getting something to run at all. It is possible to construct

1 Kevin Purdy, "Indigenous Tech Group Asks Apache Foundation to Change Its Name," Ars Technica, January 12, 2023.

2 Spark runs on the majority of common operating systems and architectures as well as many types of cluster managers (YARN, Kubernetes, Mesos, etc.). It even runs on less common systems, like z/OS.

a Spark query that succeeds on megabytes while failing on gigabytes, but when refactored and adjusted with an eye toward the structure of the data and the requirements of the cluster, it succeeds on the same system with terabytes of data. In the authors' experience writing production Spark code, we have seen the same tasks, run on the same clusters, run 100× faster using some of the optimizations discussed in this book. In terms of data processing, time is money, and we hope this book pays for itself through a reduction in data infrastructure costs and developer hours.

Not all of these techniques apply to every use case. In fact, some techniques, when applied to the wrong datasets, can make your job slower. Especially because Spark is highly configurable and is exposed at a higher level than other computational frameworks of comparable power, we can reap tremendous benefits just by becoming more attuned to the shape and structure of our data. Some techniques can work well on certain data sizes or even certain key distributions, but not all. As a simple example, using `groupByKey` in Spark can very easily cause the dreaded out-of-memory (OOM) exceptions, but for data with few duplicates, this operation can be just as quick as the alternatives that we will present. Learning to understand your particular use case and system and how Spark will interact with it is a must to solve the most complex data science problems with Spark.

What You Can Expect to Get from This Book

Our hope is that this book will help you take your Spark queries and make them faster, able to handle larger data sizes, while using fewer resources. This book covers a broad range of tools and scenarios. You will likely pick up some techniques that might not apply to the problems you are working with but might apply to a problem in the future and may help shape your understanding of Spark more generally. Most of the chapters in this book are written with enough context to allow the book to be used as a reference; however, the structure of this book is intentional, and reading the sections in order should give you not only a few scattered tips but also a comprehensive understanding of Spark and how to make it sing.

It's equally important to point out what you will likely not get from this book. This book is not intended to introduce Spark, Scala, or Python; several other books and video series are available to get you started. The authors may be a little biased in this regard,[3] but we think *Learning Spark*, 2nd edition, by Jules Damji, Brooke Wenig, Tathagata Das, and Denny Lee (O'Reilly, 2020), is an excellent option for Spark beginners, and there may be a third edition by the time you are reading this. While this book focuses on performance, it is not an operations book, so topics such as setting up a cluster and multitenancy are not covered. We assume you already have

3 Holden cowrote the first edition of *Learning Spark*, and while she receives no royalties on it, she thinks the second edition is great.

a way to use Spark in your system, so we won't provide much assistance in making higher-level architecture decisions.

Spark Versions

Spark attempts to follow semantic versioning with the standard [MAJOR].[MINOR].[MAINTENANCE] with API stability for public *non-experimental nondeveloper* APIs within minor and maintenance releases. Many of these experimental components are some of the more exciting ones from a performance standpoint, including things like custom aggregations, and expressions to accelerate Spark's Datasets evaluation.

Spark aims for binary API compatibility between releases, using MiMa;[4] so if you are using the stable API theoretically, you generally should not need to recompile to run a job against a new version of Spark unless the major version has changed. In practice, we recommend recompiling all Spark jobs against the latest MINOR version, as mistakes in binary compatibility have been known to happen.



This book was created using the Spark 4.x APIs, but much of the code will work in earlier versions of Spark as well. In places where this is not the case, we have attempted to call that out.

Why the Focus on Scala and Python?

In this book, we will focus on Spark's Scala and Python APIs with callouts to other languages where relevant. Part of this decision is simply in the interest of time and space; we trust readers wanting to use Spark in another language can probably read one of (if not both) Scala and Python.



In the first edition, we said that we believed most users seeking high performance Spark would have to use some Scala. However, improvements in Spark's Python integrations mean that this is no longer necessary.

While we don't expect you to know how to write Scala and Python code, we expect that you'll be able to *read* Scala or Python and mentally translate to your favorite language.

4 MiMa is the Migration Manager for Scala and tries to catch binary incompatibilities between releases.

To Be a Spark Expert You Have to Be Able to Read a Little Scala Anyway

Although Python and Java are more commonly used languages, learning *to read* Scala is a worthwhile investment for anyone interested in delving deep into Spark development. Spark's documentation can be uneven. However, the readability of the codebase is world-class. Perhaps more than with other frameworks, the advantages of cultivating a sophisticated understanding of the Spark codebase are integral to the advanced Spark user. Because Spark is written in Scala, it will be difficult to interact with the Spark source code without the ability, at least, to read Scala code. The methods in the *resilient distributed datasets* (RDDs) class closely mimic those in the Scala collections API. RDD functions, such as `map`, `filter`, `flatMap`, `reduce`, and `fold`, have nearly identical specifications to their Scala equivalents.[5] Even folks who primarily use the Datasets API will appreciate being able to understand the underlying RDDs.

Fundamentally Spark is a functional framework, relying heavily on concepts like immutability and lambda definition, so using the Spark API may be more intuitive with some knowledge of functional programming. Programmers familiar with functional programming in Python or Scala (map, filter, etc.) will have the easiest time.

The Spark Scala and Python APIs Are Easier to Use Than the Java APIs

Once you have learned Scala or Python, you will quickly find that writing Spark in Scala or Python is less painful than writing Spark in Java. The Spark shell can be a powerful tool for debugging and development and is only available in languages with existing read–eval–print loops (REPLs), as in Scala, Python, and R.

Why Not Scala?

There are several good reasons to develop with Spark in other languages. One of the more important reasons is developer/team preference. Existing code, both internal and in libraries, can also be a strong reason to use a different language. Python is one of the most supported languages today, with some of the best machine learning tools available.

While writing Java code can be clunky and sometimes lag slightly in terms of API, there is very little performance cost to writing in another Java virtual machine (JVM) language (at most some object conversions). Spark's Java APIs also tend to be "longer-lived" than its Scala APIs, by the simple nature of Scala not (historically)

5 Although, as we explore in this book, the performance implications and evaluation semantics are quite different.

providing binary compatibility between versions.[6] This should be better in Scala 3+, but as Spark is still on Scala 2.13, the eventual migration to 3 is something to keep in mind.

Spark SQL is not only simpler to understand and write, it also has performance optimizations that one would have to do by hand in RDDs. In some ways, Spark SQL is like the "autopilot" option of Spark; you should still keep an eye on it, but it does a pretty good job. Since the code is transpiled behind the scenes, the ability to *read* Scala and Java code can be useful. This transpiling does much to minimize the performance difference when using a non-JVM language. Chapter 9 looks at options to work effectively in Spark with languages outside of the JVM, including Spark's supported languages of Python and R. This section also offers guidance on how to use Fortran, C, and GPU-specific code to reap additional performance improvements. Even if we are developing most of our Spark applications in Scala, we shouldn't feel tied to doing everything in Scala, because specialized libraries in other languages can be well worth the overhead of going outside the JVM.

Learning Scala

If after all of this we've convinced you to use Scala, there are several excellent options for learning Scala. Spark 4 is built against Scala 2.13, while the latest version of Scala is 3; the community's migration to Scala 3 is slow. Depending on how much we've convinced you to learn Scala and what your resources are, there are several different options ranging from books to massive open online courses (MOOCs) to professional training.

For books, *Programming Scala*, 3rd edition, by Dean Wampler and Alex Payne (O'Reilly, 2014) can be great, although much of the actor system references are not relevant while working in Spark. The Scala language website also maintains a list of Scala books.

In addition to books focused on Spark, there are online courses for learning Scala. Functional Programming Principles in Scala, taught by Martin Odersky, its creator, is on Coursera. A number of different companies also offer video-based Scala courses, none of which the authors have personally experienced.

For those who prefer a more interactive approach, professional training is offered by several companies. Be sure to verify the company's approach to software licensing early in your exploration.

6 Despite the implications in issuing releases, they rarely achieve these goals in meaningful ways; see "Scala is a Maintenance Nightmare" (plus the empirical evidence of cross-building Scala versions on Maven Central).

Conclusion

Although you will likely be able to get the most out of Spark performance if you have some understanding of Scala, working in Spark does not require knowledge of Scala. Special techniques to consider when working with other languages will be covered in Chapter 9.

This book is aimed at individuals who already have a grasp of the basics of Spark, and we thank you for choosing *High Performance Spark* to deepen your knowledge of Spark. The next chapter will introduce some of Spark's general design and evaluation paradigms that are important to understanding how to efficiently utilize Spark.

CHAPTER 2
How Spark Works

This chapter introduces the overall design of Spark as well as its place in the big data ecosystem. Spark used to be compared to ASF MapReduce, since Spark can also be used for distributed data processing with Hadoop.[1] Now it is also often thought of as the "scaled-up" version of pandas directly, by those not familiar with MapReduce.

As we will discuss in this chapter, Spark's design principles are quite different from those of MapReduce. Unlike Hadoop MapReduce, Spark does not need to be run in tandem with ASF Hadoop—although it often is. Increasingly Spark is run without Hadoop on cloud providers or using Kubernetes with disaggregated storage.

Spark has inherited parts of its API, design, and supported formats from other existing computational frameworks, particularly DryadLINQ.[2] Spark's internals, especially how it handles failures, differ from many traditional distributed systems.

Spark's ability to leverage lazy evaluation within memory computations makes it particularly unique. Spark's creators believe it to be the first high-level programming language for fast, distributed data processing.[3]

1 MapReduce is a programmatic paradigm that defines programs in terms of *map* procedures that filter and sort data onto the nodes of a distributed system and *reduce* procedures that aggregate the data on the mapper nodes. Implementations of MapReduce have been written in many languages, but the term usually refers to a popular implementation called *Hadoop MapReduce*, packaged with the distributed filesystem, ASF Hadoop Distributed File System.

2 DryadLINQ is a Microsoft research project that puts the .NET Language-Integrated Query (LINQ) on top of the Dryad distributed execution engine. Like Spark, the DryadLINQ API defines an object representing a distributed dataset and then exposes functions to transform data as methods defined on that dataset object. DryadLINQ is lazily evaluated, and its scheduler is similar to Spark's. However, DryadLINQ doesn't use in-memory storage. For more information, see the DryadLINQ documentation.

3 See the original Spark Paper and other Spark papers.

Since the introduction of Spark, some similar systems have been created, the most notable of which is Dask, which, while aimed at smaller (but still large) workloads, also uses lazy evaluation with recompute on failure.

The successor to the research lab that created the technology that became ASF Spark has created Ray, which is another distributed system but more focused on distributed state and machine learning. Ray and Spark often work together, with Spark handling the data and Ray handling tasks like model training.

If you are interested in Ray or Dask, Holden has cowritten books on each, which she recommends (with some bias, of course).

This chapter will mostly focus on the core building blocks of Spark, while leaving coverage of the higher-level components, like Datasets and SQL, to their respective chapters. We believe that while Datasets offer many great performance benefits over RDDs, it is still important to understand RDDs, as much of the Dataset work is effectively executed using RDDs.

To get the most out of Spark, it is important to understand some of the principles used to design Spark and, at a cursory level, how Spark programs are executed. In this chapter, we will provide a broad overview of Spark's parallel computing model and a thorough explanation of the Spark scheduler and execution engine. We will refer to the concepts in this chapter throughout the book. Further, we hope this explanation will provide you with a more precise understanding of some of the terms you've heard tossed around by other Spark users and encountered in the Spark documentation.

How Spark Fits into the Big Data Ecosystem

ASF Spark is an open source framework that provides methods to process data in parallel that are generalizable; the same high-level Spark functions can be used to perform disparate data processing tasks on data of different sizes and structures. On its own, Spark is not a data storage solution; it performs computations on Spark JVMs (Java virtual machines) that last only for the duration of a Spark application. Spark can be run locally on a single machine with a single JVM (called local mode). More often, Spark is used in tandem with distributed storage systems (e.g., Hadoop Distributed File System [HDFS], Cassandra, or Amazon S3), catalog system/metadata stores (Iceberg, Hive, or Unity) and the cluster manager to orchestrate the distribution of Spark applications across the cluster.

Cluster Managers

Spark currently supports three active cluster managers: Standalone Cluster Manager, Kubernetes, and Hadoop YARN (see Figure 2-1). The Standalone Cluster Manager is included in Spark, but using the Standalone manager requires installing Spark on each node of the cluster. In addition to these traditional cluster managers, Databricks, Snowflake, and other commercial providers offer Spark Connect (and more may follow), allowing you to deploy on their proprietary cluster manager (see Table 2-1).

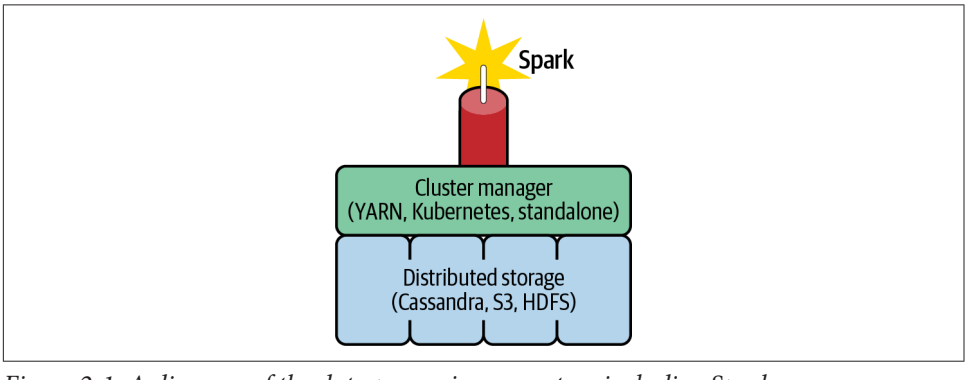


Figure 2-1. A diagram of the data processing ecosystem including Spark

Spark on Kubernetes is by far the newest open source entrant and is now deployed at scale with a number of users and vendors. If you are interested in learning more about the big data ecosystem on Kubernetes, check out *Managing Cloud Native Data on Kubernetes* by Jeff Carpenter and Patrick McFadin (O'Reilly, 2023).

Table 2-1. Spark cluster managers

Cluster manager	Biggest concern	Workload isolation	Dynamic scale
YARN	Difficult to manage system-level dependencies (including Python).	Medium	Great for Spark applications, so-so for underlying machines. Uses the terminate-and-stay-resident approach.
Kubernetes	Newest. Possible higher overhead from increased isolation. Lack of terminate-and-stay resident block server.	High	Requires block migrations, large recomputes, or an external shuffle service.
Standalone	No multitenancy (in apps or users).	None (except what you set up)	None (except what you do by hand).



Previously, Spark had both "fine-grained" and "coarse-grained" scheduler modes. In fine-grained mode, it would schedule executors for each task, but the overhead of this proved to be impractical, so in practice, all schedulers implement the coarse-grained APIs.

Catalog System

A large part of big data systems is figuring out which data to load. Keeping track of the location of the different data files, along with metadata to help Spark skip unneeded files or columns, has become increasingly important with the growth of Datasets. Using a catalog like Polaris, Unity, or Hive can help your Spark jobs run faster and also makes it easier to keep track of where your data is stored for data requests, including General Data Protection Regulation (GDPR), if applicable. We cover working with catalogs and their performance impact in "Metastore tables" on page 94. For those who do not yet have a catalog system in place, we explore some of the trade-offs to help you choose the right catalog for your organization in Appendix A.

Multitool Pipelines

Spark is also now often used as part of larger pipelines. These pipelines may be coordinated by systems like Argo, Airflow, or Kubeflow—and can involve multiple data tools as well as ML tools, such as TensorFlow and Ray.

Putting all of this together can give you a system diagram that looks something like Figure 2-2.

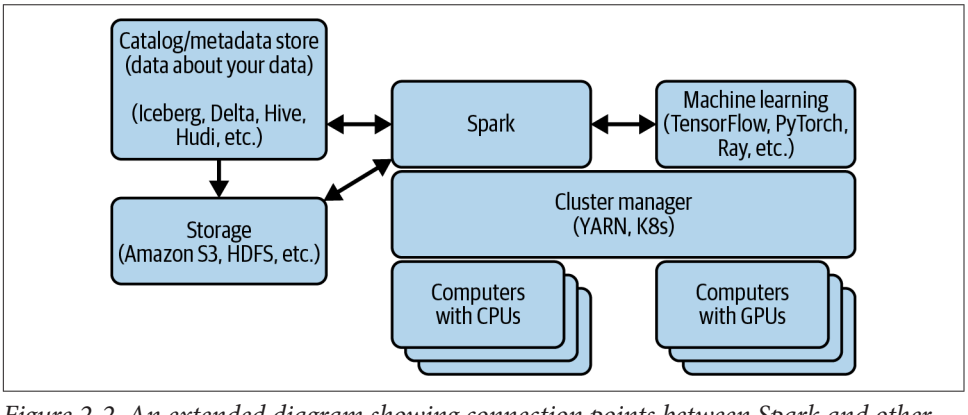


Figure 2-2. An extended diagram showing connection points between Spark and other tools in the data processing ecosystem

Spark Components

Spark provides a high-level query language to process data. Spark Core, the main data processing framework in the Spark ecosystem, has APIs in Scala, Java, and Python. Spark's core is a data abstraction called *resilient distributed datasets* (RDDs). RDDs are a representation of lazily evaluated, statically typed, distributed collections. RDDs have several predefined "coarse-grained" transformations (functions that are applied to the entire dataset), such as `map`, `join`, and `reduce` to manipulate the distributed datasets, as well as I/O functionality to read and write data between the distributed storage system and the Spark JVMs.



While Spark also supports R, at present the RDD interface is not available in that language. Some other languages, including the .NET ecosystem, have their own integration with Spark, which lives "outside" of Spark's main codebase (and R has also moved to SparklyR, which is external from RStudio). We will cover tips for using Java, Python, R, and other languages in detail in Chapter 9.

In addition to Spark Core, the Spark ecosystem includes many other first-party components, including Spark SQL (which powers Spark's DataFrame/Datasets), Spark ML, and Spark Streaming, which provide more specific data processing functionality. Additionally, many libraries are developed outside of Spark, allowing their development to happen at its own cadence. For example, the new standard for doing graph processing on Spark is developed in the GraphFrames library, separate from the primary Spark components. Similarly, there are Spark-specific data quality tools that are developed and released separately from Spark. Some of these components have the same generic performance considerations as the Core. However, some of them have unique considerations. Spark SQL, for example, has a different query optimizer than Spark Core.

Spark's Datasets/DataFrames are powered by the deceptively named "Spark SQL" component that can be used in tandem with Spark Core and has APIs in Scala, Java, Python, and R and basic SQL queries. In Python it's called DataFrames, not to be confused with Spark's SQL panda interfaces. In Scala and Java, it's accessed through Datasets (which also have a type alias for a non-type-safe version called DataFrame). Spark Datasets are also available in additional languages, like Go and Swift, through Spark Connect.

Spark SQL is a very important component for Spark performance, and much of what can be accomplished with Spark Core can be done by leveraging Spark SQL. We will cover Spark SQL in detail in Chapter 5 and compare the performance of joins in Spark SQL and Spark Core in "Core Spark Joins" on page 111.

Spark ML provides a higher-level API than MLlib with the goal of allowing users to create practical machine learning pipelines.[4] Spark's built-in ML tools have become less popular, with the focus shifting toward powering other ML tools (like TensorFlow). Spark ML has performance considerations from Spark Core and Spark SQL—we cover some of these in Chapter 10. Since the majority of popular ML tools today are not in the Java virtual machine (JVM), you'll also want to take a look at Chapter 9.

Spark Streaming uses the scheduling of the Spark Core for streaming analytics on mini-batches of data. Spark Streaming has many unique considerations, such as the window sizes used for batches. Like much of Spark, the streaming API has migrated from its low-level RDD-based origins to now run on Spark SQL, with this interface being called *Structured Streaming*. We offer some tips for using Spark Streaming in Appendix E.

GraphX is a graph processing framework built on top of Spark with an API for graph computations, which is no longer active.

This book will focus on optimizing programs written with Spark Core and Spark SQL. These two components form the building blocks of almost everything else in Spark, so even if you are mostly interested in streaming or ML, we still believe gaining an understanding of them will be helpful.

Maybe by the time you're done, you will be ready to start contributing to libraries built on top of Spark to help them better integrate.

In addition to these first-party components, the community has written a number of libraries that provide additional functionality, such as for testing or loading raw Cassandra tables and offer tools to connect it to different data sources. Previously, many libraries were listed at *http://spark-packages.org*. Nowadays, Awesome Spark, or just Maven Central, is the source of record for many packages, and most can be dynamically included at runtime with `spark-submit` or the `spark-shell` and added as build dependencies to your `maven` or `sbt` project. In practice, using standard Maven-style dependencies tends to be the better choice than using the spark-packages plugin as it gives you more flexibility. The Spark Packages website can still help for the discoverability of these libraries, and awesome-spark is an additional source for finding packages.

4 See the MLlib documentation.

Spark Model of Parallel Computing: RDDs

Spark allows users to write a program for the *driver* (or master node) on a cluster computing system that can perform operations on data in parallel. Spark represents large datasets as RDDs—immutable, distributed collections of objects—which are stored in the *executors* (or slave nodes). The objects that comprise RDDs are called *partitions* and may be (but do not need to be) computed on different nodes of a distributed system. The Spark cluster manager handles starting and distributing the Spark executors across a distributed system according to the configuration parameters set by the Spark application. The Spark execution engine itself distributes the loading of the data across the executors for a computation. (See Figure 2-5 later in this chapter.)

Rather than evaluating each transformation as soon as specified by the driver program, Spark evaluates RDDs lazily, computing RDD transformations only when the final RDD data needs to be computed (often by writing out to storage or collecting an aggregate to the driver). Spark can keep an RDD loaded in-memory on the executor nodes throughout the life of a Spark application for faster access in repeated computations. Spark RDDs are immutable, so transforming an RDD returns a new RDD rather than modifying the existing one.

As we will explore in this chapter, this paradigm of lazy evaluation, in-memory storage, and immutability allows Spark to be easy-to-use, fault-tolerant, scalable, and efficient.

Lazy Evaluation

Many other systems for in-memory storage are based on "fine-grained" updates to mutable objects, i.e., inserting or updating a value (or group of values) in a database like Cassandra. In contrast, evaluation of RDDs is delayed until necessary and almost completely.

Spark does not begin computing the partitions until an action is called. An action is a Spark operation that returns something other than an RDD, triggering evaluation of partitions and possibly returning some output to a non-Spark system (outside of the Spark executors); for example, bringing data back to the driver (with operations like `toLocalIterator` or `collect`) or writing data to an external storage storage system (such as `copyToHadoop`) or forcing computation with `count`.[5] Actions trigger the scheduler, which builds a *directed acyclic graph* (DAG), based on the dependencies between RDD transformations. In other words, Spark evaluates an action by working backward to define the series of steps it has to take to produce each object in the

5 Technically it does bring *some* data back, but just the count.

final distributed dataset (each partition). Then, using this series of steps, called the execution plan, the scheduler computes the missing partitions for each stage until it computes the result.



Not all transformations are 100% lazy. `sortByKey` needs to evaluate the RDD to determine the range of data, so it involves both a transformation and an action. This becomes more common in Spark SQL, where partial evaluation is sometimes needed for schema inference.

Performance and usability advantages of lazy evaluation

Lazy evaluation allows Spark to combine operations that don't require communication with the driver (called transformations with one-to-one dependencies) to avoid doing multiple passes through the data. For example, suppose a Spark program calls a `map` and a `filter` function on the same RDD. Spark can send the instructions for both the `map` and the `filter` to each executor. Then Spark can perform both the `map` and `filter` on each partition, which requires accessing the records only once, rather than sending two sets of instructions and accessing each partition twice. This theoretically reduces the computational complexity by up to half.

Spark's lazy evaluation paradigm is not only more efficient; it is also easier to implement the same logic in Spark than in a different framework—like MapReduce. MapReduce requires the developer to do the work to consolidate her mapping operations or reduce and focus on comparatively low-level implementation.

Spark's clever lazy evaluation strategy lets us be lazy and express the same logic in far fewer lines of code: we can chain together operations with narrow dependencies and let the Spark evaluation engine do the work of consolidating them.

Spark DataFrames/Datasets/SQL go beyond this, further delegating the responsibility to Spark with a more declarative programming interface, although, as always, the developer may need to override Spark's decisions with her expert judgment.

Consider the classic word count example that, given a dataset of documents, parses the text into words and then computes the count for each word. The ASF docs provide a word count example, which even in its simplest form comprises roughly 50 lines of code (excluding import statements) in Java. A comparable Spark implementation is roughly 15 lines of code in Java and 5 in Scala, available on the ASF website.

The example excludes the steps of reading the data mapping documents into words and counting the words. We have reproduced it in Example 2-1.

Example 2-1. Simple Scala word count

```
def simpleWordCount(rdd: RDD[String]): RDD[(String, Int)] = {
  val words = rdd.flatMap(_.split(" "))
  val wordPairs = words.map((_, 1))
  val wordCounts = wordPairs.reduceByKey(_ + _)
  wordCounts
}
```

A further benefit of the Spark implementation of word count is that it is easier to modify and improve. Suppose that we now want to modify this function to filter out some "stop words" and punctuation from each document before computing the word count. In MapReduce, this would require adding the filter logic to the mapper to avoid doing a second pass through the data. You can find an implementation of this routine for MapReduce on GitHub. In contrast, we can modify the preceding Spark routine by simply putting a `filter` step before the `map` step that creates the key/value pairs. Example 2-2 shows how Spark's lazy evaluation will consolidate the `map` and `filter` steps for us.

Example 2-2. Word count example with stop words filtered

```
def withStopWordsFiltered(
    rdd: RDD[String],
    illegalTokens: Array[Char],
    stopWords: Set[String]
): RDD[(String, Int)] = {
  val separators = illegalTokens ++ Array[Char](' ')
  val tokens: RDD[String] =
    rdd.flatMap(_.split(separators).map(_.trim.toLowerCase))
  val words =
    tokens.filter(token => !stopWords.contains(token) && (token.length > 0))
  val wordPairs = words.map((_, 1))
  val wordCounts = wordPairs.reduceByKey(_ + _)
  wordCounts
}
```

Lazy evaluation and fault tolerance

Spark is fault-tolerant, meaning Spark will generally not fail, lose data, or return inaccurate results in the event of a host machine or network failure. Spark's unique method of fault tolerance is achieved because each partition of the data contains the dependency information needed to recalculate the partition.

Most distributed computing paradigms that allow users to work with mutable objects provide fault tolerance by logging updates and duplicating data across machines. For example, Postgres works by both replicating the data and logging the updates in *write-ahead logs* between the leader and its followers. Open Search works by partitioning data, replicating files, and sharing a "translog" per partition. Depending

on how acceptable data loss is, an update might be accepted as soon as one machine has a copy, or once the transaction has been logged on multiple machines, or once the data has been fully replicated. This can range from unreliable on one side to slow and reliable on the other.

In contrast, Spark does not need to maintain a log of updates to each RDD or log the actual intermediary steps, since the RDD itself contains all the dependency information needed to replicate each of its partitions. Thus, if a partition is lost, the RDD has enough information about its lineage to recompute it, and that computation can be parallelized to make recovery faster.

This approach to fault tolerance requires that the graph representing the computations be not lost. This is stored in the driver program, so Spark is (generally) not fault-tolerant to the loss of the driver node. As such, you should not schedule the Spark driver on a likely unreliable node (for example, a spot instance). There can be some fault tolerance on the driver in streaming, but this involves tracking additional information to allow for re-creation of the driver, which is not done in "regular" Spark.

Lazy evaluation and debugging

Lazy evaluation has important consequences for debugging since it means that a Spark program will fail only at the point of action. For example, suppose that you were using the word count example and afterwards were collecting the results to the driver. If the value you passed in for the stop words was null (maybe because it was the result of a Java program), the code would of course fail with a null pointer exception in the `contains` check. However, this failure would not appear until the program evaluated the collect step. Even the stack trace will show the failure as first occurring at the collect step, suggesting that the failure came from the collect statement. For this reason, it is probably most efficient to develop in an environment that gives you access to complete debugging information.



Because of lazy evaluation, stack traces from failed Spark jobs (especially when embedded in larger systems) will often appear to fail consistently at the point of the action, even if the problem in the logic occurs in a transformation much earlier in the program.

A common technique for debugging is introducing checkpoints in Spark where Spark is forced to evaluate the data up to that point; we will discuss these more in "Checkpointing" on page 161.

In-Memory Persistence and Memory Management

Spark's performance advantage over MapReduce is greatest in use cases involving repeated computations. Much of this performance increase is due to Spark's use of in-memory persistence. Rather than writing to disk between each pass through the data, Spark has the option of keeping the data on the executors loaded into memory. That way, the data on each partition is available in-memory each time it needs to be accessed.

Spark offers three options for memory management: in-memory as deserialized data, in-memory as serialized data, and on disk. Each has different space and time advantages:

In memory as deserialized Java objects
: The most intuitive way to store objects in RDDs is as the original deserialized Java objects that are defined by the driver program. This form of in-memory storage is the fastest, since it reduces serialization time; however, it may not be the most memory efficient, since it requires the data to be stored as objects.

As serialized data
: Using the standard Java serialization library, Spark objects are converted into streams of bytes as they are moved around the network. This approach may be slower, since serialized data is more CPU-intensive to read than deserialized data; however, it is often more memory efficient, since it allows the user to choose a more efficient representation. While Java serialization is more efficient than full objects, Kryo serialization can be even more space efficient.

On disk
: RDDs, whose partitions are too large to be stored in RAM on each of the executors, can be written to disk if iterator-to-iterator transformations are used. This strategy is obviously slower for repeated computations but can be more fault-tolerant for long sequences of transformations and may be the only feasible option for enormous computations.

The `persist()` function in the RDD class lets the user control how the RDD is stored. By default, `persist()` stores an RDD as deserialized objects in memory, but the user can pass one of numerous storage options to the `persist()` function to control how the RDD is stored. We will cover the different options for RDD reuse in "Types of Reuse: Cache, Persist, Checkpoint, Shuffle Files" on page 159. When persisting RDDs in memory, the default implementation of RDDs evicts the least recently used partition (called LRU caching) if the space it takes is required to compute or to cache a new partition. However, you can change this behavior and control Spark's memory prioritization, with the `persistencePriority()` function in the RDD class. See "LRU Caching" on page 163.

Immutability and the RDD Interface

Spark defines an RDD interface with the properties that each type of RDD must implement. These properties include the RDD's dependencies and information about data locality that are needed for the execution engine to compute that RDD. Since RDDs are statically typed and immutable, calling a transformation on one RDD will not modify the original RDD but rather return a new RDD object with a new definition of the RDD's properties.

RDDs can be created in three ways: (1) by transforming an existing RDD; (2) from a `SparkContext`, which is the API's gateway to Spark for your application; and (3) converting a `DataFrame` or `Dataset` (created from the `SparkSession`). The `SparkContext` represents the connection between a Spark cluster and one running Spark application. The `SparkContext` can be used to create an RDD from a local collection (using the `makeRDD` or `parallelize` methods) or by reading from stable storage (text files, binary files, a Hadoop Context, or a Hadoop file). `DataFrames` and `Datasets` can be read using the Spark SQL equivalent to a `SparkContext`, the `SparkSession`.

Internally, Spark uses five main properties to represent an RDD. The three required properties are the list of partition objects that make up the RDD, a function for computing an iterator of each partition, and a list of dependencies on its parent RDDs. Optionally, RDDs also include a partitioner (for RDDs of rows of key/value pairs represented as Scala tuples) and a list of preferred locations for each partition to be computed. As an end user, you will rarely need these five properties and are more likely to use predefined RDD transformations. However, it is helpful to understand the properties and know how to access them for debugging and for a better conceptual understanding. These five properties correspond to the following five methods available to the end user (you):

`partitions()`
: Returns an array of the partition objects that make up the parts of the distributed dataset. In the case of an RDD with a partitioner, the value of the index of each partition will correspond to the value of the `getPartition` function for each key in the data associated with that partition.

`iterator(p, parentIters)`
: Computes the elements of partition `p` given iterators for each of its parent partitions. This function is called in order to compute each of the partitions in this RDD. This is not intended to be called directly by the user. Rather, this is used by Spark when computing actions. Still, referencing the implementation of this function can be useful in determining how each partition of an RDD transformation is evaluated.

`dependencies()`
: Returns a sequence of dependency objects. The dependencies let the scheduler know how this RDD depends on other RDDs. There are two kinds of dependencies: *narrow dependencies* (`NarrowDependency` objects), which represent partitions that depend on one or a small subset of partitions in the parent, and *wide dependencies* (`ShuffleDependency` objects), which are used when a partition can only be computed by rearranging all the data in the parent. We will discuss the types of dependencies in "Wide Versus Narrow Dependencies" on page 21.

`partitioner()`
: Returns a Scala option type of a `partitioner` object if the RDD has a function between `element` and `partition` associated with it, such as a `hashPartitioner`. This function returns `None` for all RDDs that are not of type tuple (do not represent key/value data). An RDD that represents an HDFS file (implemented in *NewHadoopRDD.scala*) has a partition for each block of the file. We will discuss partitioning in detail in "Partitioners and Key/Value Data" on page 185.

`preferredLocations(p)`
: Returns information about the data locality of a partition, `p`. Specifically, this function returns a sequence of strings representing some information about each of the nodes where the split `p` is stored. In an RDD representing an HDFS file, each string in the result of `preferredLocations` is the Hadoop name of the node where that partition is stored.

Types of RDDs

The implementation of the Spark Scala API contains an abstract class, `RDD`, which contains not only the five core functions of RDDs but also those transformations and actions that are available to all RDDs, such as `map` and `collect`. Functions defined only on RDDs of a particular type are defined in several RDD function classes, including `PairRDDFunctions`, `OrderedRDDFunctions`, and `GroupedRDDFunctions`. The additional methods in these classes are made available by implicit conversion from the abstract `RDD` class, based on type information or when a transformation is applied to an RDD.

The Spark API also contains implementations of the `RDD` class that define more specific behavior by overriding the core properties of the RDD. These include the `NewHadoopRDD` class discussed previously—which represents an RDD created from an HDFS—and `ShuffledRDD`, which represents an RDD that was already partitioned. Each of these RDD implementations contains functionality that is specific to RDDs of that type. Creating an RDD, either through a transformation or from a `SparkContext`, will return one of these implementations of the RDD class. Some RDD

operations have a different signature in Java than in Scala. These are defined in the `JavaRDD.java` class.



Find out what type an RDD is by using the `toDebugString` function, which is defined on all RDDs. This will tell you what kind of RDD you have and provide a list of its parent RDDs.

We will discuss the different types of RDDs and RDD transformations in detail in Chapters 7 and 8.

Functions on RDDs: Transformations Versus Actions

There are two types of functions defined on RDDs: *actions* and *transformations*. Actions are functions that return something that is not an RDD, including a side effect, and transformations are functions that return another RDD.

Each Spark program must contain an action, since actions either bring information back to the driver (or Spark Connect server) or write the data to stable storage. Actions are what force evaluation of a Spark program. Checkpoint calls also force evaluation but usually are not the only action in a Spark job since their result is automatically cleaned up. Actions that bring some data back to the driver include `collect`, `count`, `collectAsMap`, `toLocalIterator`, `reduce`, and `take`. For most big jobs the main action is likely going to be saving the data out somewhere besides the driver.



Some of these actions do not scale well, since they can cause memory errors in the driver. In general, it is best to use actions like `take`, `count`, and `reduce`, which bring back a fixed amount of data to the driver, rather than `collect` or `sample`.

Actions that write to storage include `saveAsTextFile`, `saveAsSequenceFile`, and `saveAsObjectFile`. Most actions that save to Hadoop are made available only on RDDs of key/value pairs; they are defined both in the `PairRDDFunctions` class (which provides methods for RDDs of tuple type by implicit conversion) and the `NewHadoopRDD` class, which is an implementation for RDDs that were created by reading from Hadoop. Some saving functions, like `saveAsTextFile` and `saveAsObjectFile`, are available on all RDDs, and they work by adding an implicit null key to each record (which is then ignored by the saving level). Functions that return nothing (`void` in Java, or `Unit` in Scala), such as `foreach`, are also actions: they force execution of a Spark job. `foreach` can be used to force evaluation of an RDD, but is also often used to write out to nonsupported formats (like web endpoints).

Most of the power of the Spark API is in its transformations. Spark transformations are coarse-grained transformations used to sort, reduce, group, sample, filter, and map distributed data. We will discuss transformations in detail in both Chapter 8, which deals exclusively with transformations on RDDs of key/value data, and Chapter 7.

Wide Versus Narrow Dependencies

For the purpose of understanding how RDDs are evaluated, the most important thing to know about transformations is that they fall into two categories: transformations with narrow dependencies and transformations with wide dependencies. The narrow versus wide distinction has significant implications for the way Spark evaluates a transformation and, consequently, for its performance. We will define narrow and wide transformations for the purpose of understanding Spark's execution paradigm in "Spark Job Scheduling" on page 23 of this chapter, but we will save the longer explanation of the performance considerations associated with them for Chapter 7.

Conceptually, narrow transformations are those in which each partition in the child RDD has simple, finite dependencies on partitions in the parent RDD. Dependencies are only narrow if they can be determined at design time, irrespective of the values of the records in the parent partitions, and if each parent has at most one child partition. Specifically, partitions in narrow transformations can either depend on one parent (such as in the `map` operator), or a unique subset of the parent partitions that is known at design time (`coalesce`). Thus, narrow transformations, like `map`, `flatMap`, or `filter`, can be executed on an arbitrary subset of the data without any information about the other partitions. In contrast, transformations with wide dependencies cannot be executed on arbitrary rows and instead require the data to be partitioned in a particular way (e.g., according to the value of their key). In `sort`, for example, records have to be partitioned so that keys in the same range are on the same partition. Transformations with wide dependencies include `sort`, `reduceByKey`, `groupByKey`, `join`, and anything that calls the `rePartition` function.

In certain instances, for example, when Spark already knows the data is partitioned in a certain way, operations with wide dependencies do not cause a shuffle. If an operation will require a shuffle to be executed, Spark adds a `ShuffledDependency` object to the dependency list associated with the RDD. In general, shuffles are expensive. They become more expensive with more data and when a greater proportion of that data has to be moved to a new partition during the shuffle. As we will discuss at length in Chapter 8, we can get a lot of performance gains out of Spark programs by doing fewer and less expensive shuffles.

The next two diagrams illustrate the difference in the dependency graph for transformations with narrow dependencies versus transformations with wide dependencies. Figure 2-3 shows narrow dependencies in which each child partition (each of the

squares on the bottom rows) depends on a known subset of parent partitions. Narrow dependencies are shown with dashed blue arrows. The left represents a dependency graph of narrow transformations (such as `map`, `filter`, `mapPartitions`, and `flatMap`). On the upper right are dependencies between partitions for `coalesce`, a narrow transformation. In this instance we try to illustrate that a transformation can still qualify as narrow if the child partitions may depend on multiple parent partitions, so long as the set of parent partitions can be determined regardless of the values of the data in the partitions.

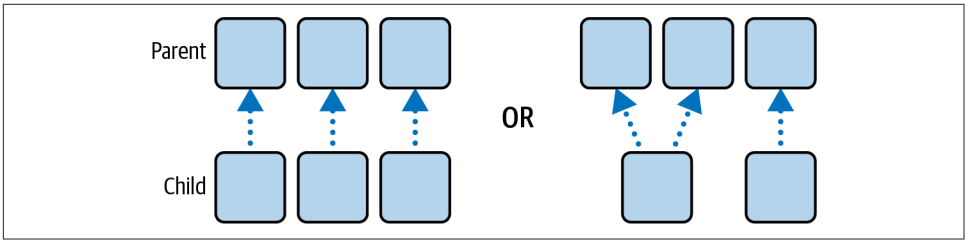


Figure 2-3. A simple diagram of dependencies between partitions for narrow transformations

Figure 2-4 shows wide dependencies between partitions. In this case the child partitions (shown at the bottom of Figure 2-4) depend on an arbitrary set of parent partitions. The wide dependencies (displayed as solid red arrows) cannot be known fully before the data is evaluated. In contrast to the `coalesce` operation, data is partitioned according to its value. The dependency graph for any operations that cause a shuffle (such as `groupByKey`, `reduceByKey`, `sort`, and `sortByKey`) follows this pattern.

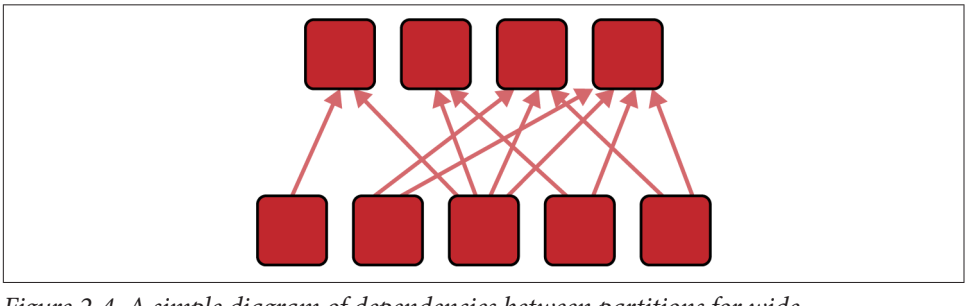


Figure 2-4. A simple diagram of dependencies between partitions for wide transformations

The join functions are a bit more complicated since they can have wide or narrow dependencies depending on how the two parent RDDs are partitioned. We illustrate the dependencies in different scenarios for the join operation in "Core Spark Joins" on page 111.

Spark Job Scheduling

A Spark application consists of a driver process, which is where the high-level Spark logic is written, and a series of executor processes that can be scattered across the nodes of a cluster. The Spark program itself runs in the driver node and sends some instructions to the executors. One Spark cluster can run several Spark applications concurrently. The applications are scheduled by the cluster manager and correspond to one `SparkContext`. Spark applications can, in turn, run multiple concurrent jobs. Jobs correspond to each action called on an RDD in a given application. In this section, we will describe the Spark application and how it launches Spark jobs: the processes that compute RDD transformations.

In addition to Spark's own scheduler, especially if you are in a shared environment, you'll want to pay attention to the cluster's scheduler. Environments like YARN and Kubernetes have mechanisms for sharing resources, and if your job is trying to run when the cluster is overloaded, this can result in poor performance even with the most excellent application code.

Resource Allocation

Spark offers two ways of allocating resources across applications: *static allocation* and *dynamic allocation*. With static allocation, each application is allotted a finite maximum of resources on the cluster and reserves them for the duration of the application (as long as the `SparkContext` is still running). In our experience, dynamic allocation is the way to go for almost all jobs.

Spark added resource profiles to support allocating different types of resources for different stages of your job. This is most commonly used for GPUs and other specialized expensive hardware. We'll explore resource profiles in Appendix D.

The Spark Application

A Spark application corresponds to a set of Spark jobs defined by one `SparkContext` in the driver program. A Spark application begins when a `SparkContext` is started. When the `SparkContext` is started, a driver and a series of executors are started on the worker nodes of the cluster. Each executor is its own Java virtual machine (JVM), and an executor cannot span multiple nodes, although one node may contain one or several executors.

The `SparkContext` determines how many tasks are allotted to each executor. When a Spark job is launched, each executor has slots for running the tasks needed to compute an RDD. The `SparkContext` is responsible for getting these executors from the cluster manager, and it can request different sizes of executor (GPU, etc.) as needed. In this way, we can think of one `SparkContext` as one set of configuration

parameters for running Spark jobs. These parameters are exposed in the `SparkConf` object, which is used to create a `SparkContext`. Applications often, but not always, correspond to users. That is, each Spark program running on your cluster likely uses one `SparkContext`.



RDDs cannot be shared between applications. Thus, transformations, such as `join`, that use more than one RDD must have the same `SparkContext`.

Figure 2-5 illustrates what happens when we start a `SparkContext`. First, the driver program pings the cluster manager. The cluster manager launches a number of Spark executors (JVMs shown as boxes in the diagram) on the worker nodes of the cluster (shown as circles). One node can have multiple Spark executors, but an executor cannot span multiple nodes. An RDD will be evaluated across the executors in partitions (shown as smaller rectangles). Each executor can have multiple partitions, but a partition cannot be spread across multiple executors.

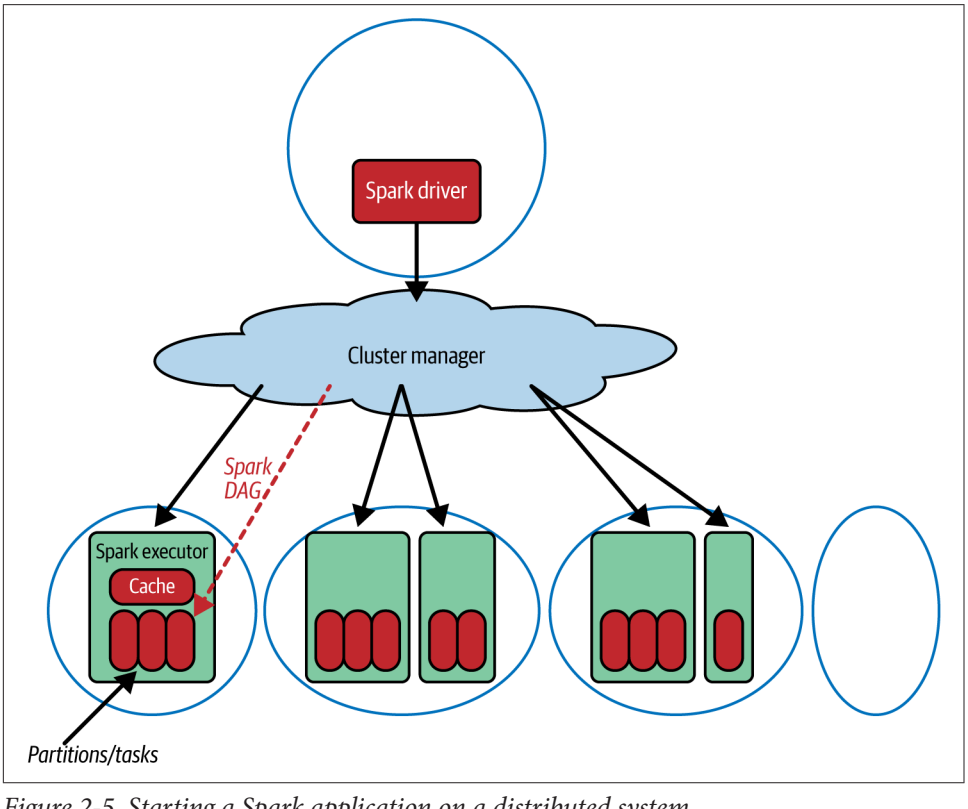


Figure 2-5. Starting a Spark application on a distributed system

Default Spark Scheduler and Concurrent Actions

By default, Spark schedules jobs on a first-in, first-out basis. However, Spark does offer a fair scheduler, which assigns tasks to concurrent jobs in round-robin fashion, i.e., parceling out a few tasks for each job until the jobs are all complete. The fair scheduler ensures that jobs get a more even share of cluster resources. The Spark application then launches jobs in the order that their corresponding actions were called on the `SparkContext`.

This tends to not matter, as most Spark actions are blocking, and only very few are asynchronous. You need to be careful when using async actions or threading alongside Spark. If you are using async actions or threading, it's important to make sure any shared dependencies are cached/persisted/checkpointed.

One common case where async (or parallel) actions make sense is when writing two different views of the same data, for example, one with and without personally identifiable information (PII). Note: this requires the shared dependencies are pre-calculated manually (with persist + count or similar).

In Python the `pyspark-asyncactions` project makes async actions easy, as shown in Example 2-3:

Example 2-3. Dual action/write Python

```
import asyncactions  # noqa # pylint: disable=unused-import


class DualWriteExample:
    def do_write(self, df, p1, p2):
        """
        Apply two concrete actions to a DataFrame in parallel.
        A common use case is two views of the same data, normally
        one with sensitive data and one scrubbed/clean.
        """
        # First we "persist" it (you can also checkpoint or choose a different
        # level of persistence.
        df.persist()
        df.count()
        # Create the distinct "safe" view.
        df1 = df.select("times")
        # Start the async actions
        async1 = df1.write.mode("append").format("parquet").saveAsync(p1)
        async2 = df.write.mode("append").format("parquet").saveAsync(p2)
        # Block until the writes are both finished.
        async1.result()
        async2.result()
```

In Scala/Java you can either use the built-in async actions from Spark or start different threads.

The Anatomy of a Spark Job

In the Spark lazy evaluation paradigm, a Spark application doesn't "do anything" until the driver program calls an action. With each action, the Spark scheduler builds an execution graph and launches a *Spark job*. Each job consists of *stages*, which are steps in the transformation of the data needed to materialize the final RDD. Each stage consists of a collection of *tasks* that represent each parallel computation and are performed on the executors.

Figure 2-6 shows a tree of the different components of a Spark application and how these correspond to the API calls. An application corresponds to starting a `SparkContext/SparkSession`. Each *application* may contain many jobs that correspond to one RDD action. Each *job* may contain several stages that correspond to each wide transformation. Each *stage* is composed of one or many tasks that correspond to a parallelizable unit of computation done in each stage. There is one *task* for each partition in the resulting RDD of that stage.

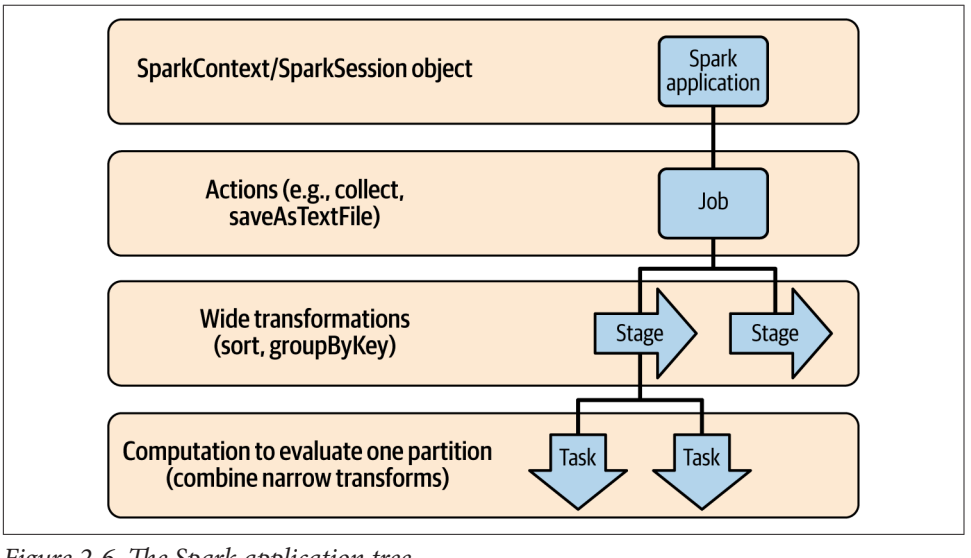


Figure 2-6. The Spark application tree

The DAG

Spark's high-level scheduling layer uses RDD dependencies to build a DAG of stages for each Spark job. In the Spark API, this is called the *DAG Scheduler*. As you have probably noticed, errors that have to do with connecting to your cluster, your configuration parameters, or launching a Spark job show up as DAG Scheduler errors. This is because the execution of a Spark job is handled by the DAG. The DAG builds a graph of stages for each job, determines the locations to run each task, and passes that

information on to the `TaskScheduler`, which is responsible for running tasks on the cluster. The `TaskScheduler` creates a graph with dependencies between partitions.[6]

Jobs

A Spark job is the highest element of core Spark's execution hierarchy. Each Spark job corresponds to one action, and each action is called by the driver program of a Spark application. As we discussed in "Functions on RDDs: Transformations Versus Actions" on page 20, one way to conceptualize an action is as something that brings data out of the RDD world of Spark into some other storage system (usually by bringing data to the driver or writing to some stable storage system).

The edges of the Spark execution graph are based on dependencies between the partitions in RDD transformations (as illustrated by Figures 2-3 and 2-4). Thus, an operation that returns something other than an RDD cannot have any children. In graph theory, we would say the action forms a "leaf" in the DAG. Thus, an arbitrarily large set of transformations may be associated with one execution graph. However, as soon as an action is called, Spark can no longer add to that graph. The application launches a job including those transformations that were needed to evaluate the final RDD that called the action.



You can provide Spark job names with `setJobGroup` on `SparkContext`, which can be especially useful when it comes time to debug large jobs.

Stages

Recall that Spark lazily evaluates transformations; transformations are not executed until an action is called. As mentioned previously, a job is defined by calling an action. The action may include one or several transformations, and wide transformations define the breakdown of jobs into *stages*. Stages can request different types of resources from Spark.

Each stage corresponds to a shuffle dependency created by a wide transformation in the Spark program. At a high level, one stage can be thought of as the set of computations (tasks) that can each be computed on one executor without communication with other executors or with the driver. In other words, a new stage begins whenever network communication between workers is required, for instance, in a shuffle.

6 See Spark Scheduler for a more thorough description of the `TaskScheduler`.

These dependencies that create stage boundaries are called `ShuffleDependencies`. As we discussed in "Wide Versus Narrow Dependencies" on page 21, shuffles are caused by those wide transformations, such as `sort` or `groupByKey`, which require the data to be redistributed across the partitions. Several transformations with narrow dependencies can be grouped into one stage.

As we saw in the word count example where we filtered stop words (Example 2-2), Spark can combine the `flatMap`, `map`, and `filter` steps into one stage since none of those transformations require a shuffle. Thus, each executor can apply the `flatMap`, `map`, and `filter` steps consecutively in one pass of the data.



Spark keeps track of how an RDD is partitioned so that it does not need to partition the same RDD by the same partitioner more than once. This has some interesting consequences for the DAG: the same operations on RDDs with known partitioners and RDDs without a known partitioner can result in different stage boundaries because there is no need to shuffle an RDD with a known partition (and thus the subsequent transformations happen in the same stage). We will discuss the evaluation consequence of known partitioners in Chapter 8.

Because the stage boundaries require communication with the driver, the stages associated with one job generally have to be executed in sequence rather than in parallel. It is possible to execute stages in parallel if they are used to compute different RDDs that are combined in a downstream transformation such as a `join`. However, the wide transformations needed to compute one RDD have to be computed in sequence. Thus, it is usually desirable to design your program to require fewer shuffles.

Tasks

A stage consists of tasks. The *task* is the smallest unit in the execution hierarchy, and each can represent one local computation. All of the tasks in one stage execute the same code on a different piece of the data. One task cannot be executed on more than one executor. However, each executor has a dynamically allocated number of slots for running tasks and may run many tasks concurrently throughout its lifetime. The number of tasks per stage corresponds to the number of partitions in the output RDD of that stage.

Figure 2-7 shows the evaluation of a Spark job that is the result of a driver program that calls the simple Spark program shown in Example 2-4.

Example 2-4. Different types of transformations showing stage boundaries

```
def simpleSparkProgram(rdd: RDD[Double]): Long = {
  // stage1
  rdd
    .filter(_ < 1000.0)
    .map(x => (x, x))
    // stage2
    .groupByKey()
    .map { case (value, groups) => (groups.sum, value) }
    // stage 3
    .sortByKey()
    .count()
}
```

The stages (blue boxes) are bounded by the shuffle operations `groupByKey` and `sortByKey`. Each stage consists of several tasks: one for each partition in the result of the RDD transformations (shown as red rectangles), which are executed in parallel.

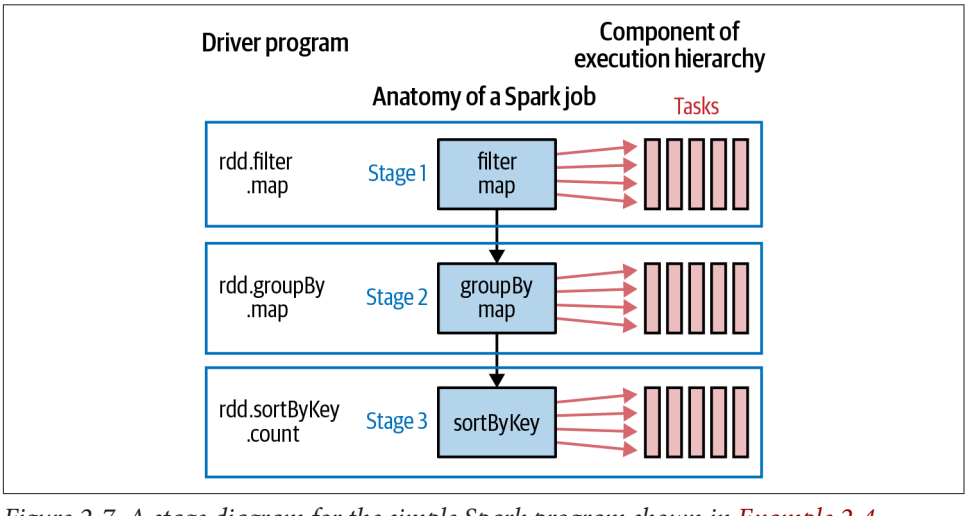


Figure 2-7. A stage diagram for the simple Spark program shown in Example 2-4

A cluster cannot necessarily run every task in parallel for each stage. Each executor has a number of cores. The number of cores per executor is configured at the application level but likely corresponds to the physical cores on a cluster. Spark can run no more tasks at once than the total number of executor cores allocated for the application. We can calculate the number of tasks from the settings from the Spark Conf as (total number of executor cores = # of cores per executor × number of executors). If there are more partitions (and thus more tasks) than the number of slots for running tasks, then the extra tasks will be allocated to the executors as the first round of tasks finishes and resources are available. In most cases, all the tasks for one stage must be completed before the next stage can start. The process

of distributing these tasks is done by the `TaskScheduler` and varies depending on whether the fair scheduler or FIFO scheduler is used (recall the discussion in "Default Spark Scheduler and Concurrent Actions" on page 25).

In some ways, the simplest way to think of the Spark execution model is that a Spark job is the set of RDD transformations needed to compute one final result. Each stage corresponds to a segment of work, which can be accomplished without involving the driver. In other words, one stage can be computed without moving data across the partitions. Within one stage, the tasks are the units of work done for each partition of the data.

Spark Connect

Spark Connect is an alternate way to interface with the Spark DataFrame API. Why do we need an alternate interface?

Spark was originally designed for the client application to be in the datacenter, tightly coupled with the cluster. In regular Spark applications, dependencies must be upgraded in lockstep with the server Spark and Scala version. Spark Connect somewhat relaxes these requirements for applications without any user-defined functions, which we believe are in the minority. Also, one application that overwhelms the Spark Driver causes failures on all the applications using that driver, which can happen with Livy or some Spark Connect use cases. Data professionals spend a considerable amount of their time addressing these complications.

Designed to decouple client applications from the Spark server, the Spark Connect API serves as a thin interface between the two. If Spark Connect is enabled, client calls to Spark's DataFrame API will not be sent directly to the Spark server. Instead, that client call will be translated into a logical parse plan. So if the client requests to read a file at a particular location, but the client doesn't know the file type, that's OK. The logical plan will be generated on the fly. So the client code can be written once and reused for a different file type. Or the client code can be written in a non-JVM language because Spark Connect is constructing the parse plan.

In Figure 2-8, Spark Connect serves as a thin layer between the client and the Spark server.

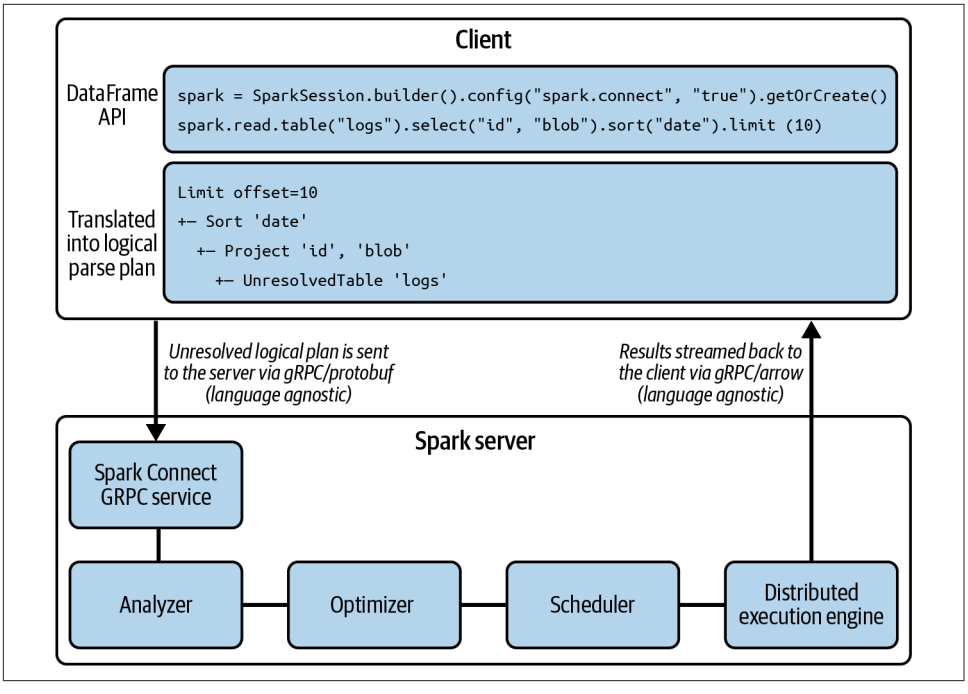


Figure 2-8. Spark Connect is designed to address some of the challenges in Spark Core. Reproduced from Spark Connect's High-Level Design Doc.

Spark Connect made four key design decisions:

1. The DataFrame API was chosen as the most used interface of Spark. Here the client can express the intent of what is needed (e.g., read a file without necessarily knowing the file type).
2. The DataFrame API call is translated into a logical parse plan. This is *not* the same as the catalyst optimizer logical plan; rather, it's one step up.
3. The unresolved logical plan is sent via protobuf to the Spark server, where it's translated into catalyst logical plans.
4. As tasks are completed on the distributed execution engine, the results are sent back to the client.

The claim is that Spark Connect is language agnostic, so you can call Spark Connect from Ruby, Go, Rust, Python, and others. But these authors are skeptical of the full range of support for these languages. Time will tell how complete the different implementations are.

We do currently know of some limitations: there's no support for RDDs, the `SparkContext` is not available on the client, and the client does not have direct access to the

JVM driver. Let's see how many IoT devices end up using Spark Connect. When the errors are pushed to the client, you can in theory scale up the number of clients using the Spark server.

Conclusion

Spark offers an innovative, efficient model of parallel computing that centers on lazily evaluated, immutable, distributed datasets, known as RDDs. Spark exposes RDDs as an interface, and RDD methods can be used without any knowledge of their implementation—but having an understanding of the details will help you write more performant code. Understanding RDDs is important since the higher-level libraries, like SQL, depend on them to perform the actual work. Because of Spark's ability to run jobs concurrently, to compute jobs across multiple nodes, and to materialize collections lazily, the performance implications of similar logical patterns may differ widely, and errors may surface from misleading places. Thus, it is important to understand how the execution model for your code is assembled in order to write and debug Spark code. Furthermore, it is often possible to accomplish the same tasks in many different ways using the Spark API, and a strong understanding of how your code is evaluated will help you optimize its performance. In this book, we will focus on ways to design Spark applications to minimize network traffic, memory errors, and the cost of failures.

CHAPTER 3

Upgrading Spark

When we started writing the second edition of this book, one of the first tasks we had to face was upgrading our examples from Spark 2.2 to Spark 3.3 (and then later 4; it took us a while to finish the book). In our day jobs, we also often face the task of helping people upgrade to new versions of Spark. Upgrading to new versions of Spark is important to be able to take advantage of its many performance improvements; some of these can be as simple as making your code run on the new engine, whereas in other cases, you may need to use newer APIs. In this chapter you will learn about how to identify areas of Spark that have changed and where you may need to update your codebase.

Upgrading to newer versions of Spark is not as simple as bumping the version and basking in the joy of a new engine. While Spark officially aims to follow semantic versioning (SemVer), where it maintains API compatibility within the same major version, it frequently falls down in important places as we'll explore in this chapter. Notably, "developer APIs"—which high performance users are most likely to take advantage of—are excluded from Spark's promises around API compatibility. Don't let these obstacles keep you from upgrading, though; let's explore how to find the changes that will impact you, not just in 4 but in any version of Spark.

Finding What You Need to Change

We like to classify breaking changes into three types: (1) those that break at compile time (Scala/Java/Kotlin only), (2) those that raise exceptions at runtime, and (3) those that produce differing output or behavior. These three categories are helpful as the techniques and impact differ between them. We will start with the easiest, compile-time changes, but it is important for you to pay attention to runtime output changes as they are the easiest to miss and can have an outsized negative impact.

Compile-Time Changes

From a documentation point of view, there are a few places you should read. Spark's general migration guide and the release notes are great starting points. While the migration guide is good at calling out breaking changes, the release notes are especially useful for highlighting new APIs you may want to consider using for increased performance. Depending on which components you use, you should also look at the component-specific guide, like Dataset/SQL or ML. Additionally, Spark uses the MiMa build checker, where the Spark developers have to create exceptions when they break Java/Scala APIs. While MiMa only directly checks the JVM APIs, this checker gives you an idea of where the Python APIs might be changing too.

While Spark does generally try to support language versions for relatively long periods of time, major version updates (e.g., Spark 3 versus Spark 2) may drop support of old versions of languages. While Spark 3 did drop Python 2 support, this was comparatively late in the game, with many popular libraries no longer supporting Python 2 by that point. Generally speaking, we believe Spark supports dependencies longer than other tools because Spark has such wide adoption.

Another important factor to consider when upgrading Spark is what other libraries you will need to upgrade along with it. For JVM users, Spark 3 and 4 both increased the *shading*, which means more of the libraries are renamed internally. Shading is intended to reduce the number of conflicts you encounter. However, especially if you're upgrading from Spark 2, you may find that you have taken a transitive dependency through Spark that you now need to make explicit. Thankfully most of these will show up at compile time. A good place to check what libraries have changed for Java users is Maven Central or look at the change log for the main build file. Now let's consider Python users. Spark has fewer direct Python dependencies; however, shading is not an option, and some of the requirements are implicit. For example, some of Spark's performance improvements require that the PyArrow library be installed, but it is not automatically installed. Check the history of the setup file to see what Python libraries have changed.

Exceptions at Runtime

End-to-end test coverage, even if incomplete, can catch most situations where Spark has introduced a new exception at runtime. While you'll sometimes need to evaluate a few values (e.g., empty data tests may not be enough) by having your pipeline run end-to-end, you should be aware of most of the relevant removed, restricted, private, or prohibited APIs (mostly from Java upgrades).

Good testing is an important part of all software development, but we realize the reality that not all your coworkers will either agree with that or work in a business environment that empowers them to spend the time needed to test their data pipelines. We cover testing in more detail in Chapter 11, but for the purpose of migration,

once you've got your code compiling and running, you can use your tests to find areas where your assumptions about Spark may need to be updated. If you find yourself in the situation where you need to migrate an untested pipeline, we encourage you to first add tests and then proceed with the migration.

Differing Results

Differing results is by far the most challenging part of Spark upgrades because this problem is silent; there's no error thrown. In addition to well-designed tests (the best prevention), we suggest running a side-by-side pipeline comparison. There are several different tools to simplify side-by-side pipeline comparisons, including LakeFS, Nessie, and Iceberg Write-Audit-Publish (WAP) snapshotting. We've created a proof-of-concept tool that uses each of these to compare the outputs.



Not every pipeline is simple to compare side-by-side; for example, if your pipeline outputs a machine learning model with some element of randomness involved in selecting the initial weights. Some random processes, like ML, do support configurable seed parameters, which can make them more deterministic. If the whole pipeline cannot be easily made comparable, try splitting your pipeline into the extract, transform, load (ETL) part, which you can more easily make assertions about, and your ML code. Not only will this allow you to run (partial) side-by-side comparisons, but it will likely make it easier to write better tests.

Updating Your Code

While the upgrading process can seem like quite a lot, there are tools that can help you with the migrations. It's important to note that these tools, while helpful, are not suitable for fully automated usage.



With the proliferation of AI coding tools, it's tempting to use them to update your Spark code, and while this can work well for tail-end upgrades, for those on the bleeding edge, the AI tools frequently do not have a sufficient training set to complete an upgrade (although it can be a good place to start).

Scala

Spark's Scala API is the least stable of the APIs, but its type system makes it the easiest to catch changes and errors.

Scalafix is a linting and refactoring tool for Scala. With Scalafix, we create rules to simplify upgrading to new versions of Spark. You can start by creating a *.scalafix.conf*

file with the rules (and their configuration) that you want to use. For example, in our automated upgrade project, we created the conf file shown in Example 3-1.

Example 3-1. Scalfix configuration example (./.scalafix.conf in the main example repo)

```
UnionRewrite.deprecatedMethod {
  "unionAll" = "union"
}

OrganizeImports {
  blankLines = Auto,
  groups = [
    "re:javax?\\."
    "scala."
    "org.apache.spark."
    "*"
  ],
  removeUnused = false
}

rules = [
//  DisableSyntax,
//  SparkAutoUpgrade,
//  MigrateHiveContext,
//  MigrateToSparkSessionBuilder,
//  MigrateDeprecatedDataFrameReaderFuns,
//  AccumulatorUpgrade,
//  onFailureFix,
//  ExecutorPluginWarn,
//  UnionRewrite,
//  GroupByKeyWarn,
//  GroupByKeyRewrite,
//  MetadataWarnQQ,
//  ScalaTestExtendsFix,
//  ScalaTestImportChange.
  TypelevelUnusedIO
  TypelevelMapSequence
  TypelevelAs
  TypelevelUnusedShowInterpolator
  TypelevelFs2SyncCompiler
  TypelevelHttp4sLiteralsSyntax
  TypelevelIORandomUUID
]
```

sbt users can add the scalafix plugin to their plugins configuration as shown in Example 3-2.

Example 3-2. Adding Scalafix plugin (./project/plugins.sbt)

```
addSbtPlugin("ch.epfl.scala" % "sbt-scalafix" % "0.14.4")
```

To bring in the Spark-specific rules, you'll need to add them as special Scalafix dependencies as shown in Example 3-3.

Example 3-3. Adding Scalafix rules (./build.sbt)

```
// Note: post upgrade you'll want to switch to modern rules
// or comment out because of Scala version.
ThisBuild / scalafixDependencies +=
 "com.holdenkarau" %% "spark-scalafix-rules-2.4.8" % "0.1.5"
ThisBuild / scalafixDependencies +=
 "com.github.liancheng" %% "organize-imports" % "0.6.0"
// Needs to be commented out post-upgrade because of Scala versions.
//ThisBuild / scalafixDependencies +=
//  "com.holdenkarau" %% "spark-scalafix-rules-2.4.8" % "0.1.5"
//ThisBuild / scalafixDependencies +=
//  "com.github.liancheng" %% "organize-imports" % "0.6.0"
```

These upgrades are semiautomatic and, in some situations, introduce nulls and/or produce warnings in places where the migration cannot succeed without human intervention. You can run all of the configured rules with `scalaFixAll`.

You can integrate Scalafix with most of the build tools in the Scala ecosystem, from gradle-scalafix to scalafix-maven-plugin. These all support the same *.scalafix.conf* file used in our previous example with `sbt`.

For example, our upgrade to our example source code to Spark 3.3 is shown in this pull request.



In theory, with Scala 3 forward, the problem of incompatible Scala versions should be gone thanks to Dotty, but Spark is still on Scala 2.13.

You can also use Scalafix to help smooth language version upgrades. There is a collection of community provided rules as well as the built-in rules. We've used the import reorder tool as part of our upgrade to Spark 3.



Some Scalafix rules do not play well with others. For example, many of the tools that manipulate imports do not play well together and need to be executed separately. You can execute individual rules with `scalaFixAll [rule]`.

Python

Since the pain of upgrading from Python 2 to Python 3, the Python community has become much more cautious when it comes to introducing breaking changes in its language. That being said, Spark will continue to drop support for old versions of Python 3, as new features and performance improvements are frequently only available in new versions. In some situations, where Spark does not yet want to drop support for the older version but does want to take advantage of new functionality, two different code paths may exist inside of the Spark code. These situations are less likely to be thoroughly tested,[1] so it's worthwhile to run a relatively recent version of Python when possible.

We know part of the appeal for many Python users is its dynamic typing. However, we believe that once you have moved beyond the experimentation phase, it is time to begin to annotate your pipelines with types. These type annotations can catch errors before running long pipelines.



Real Python has a great Python Type Checking (Guide) tutorial for those new to Python typing.

Some of the common PySpark types you will want to take advantage of are `RDD[U]` (a templated type) or `DataFrame` (a nontemplated type).

1 Tests for new functionality are more likely to be completed for merging initial code than for merging additional language support.

SQL

Spark SQL has the least amount of API changes but has the least amount of prerun checking. Also, unlike Python and Scala, which have been through large breaking migrations (e.g., 2 → 3) and have strong open source tools to help with migrations, SQL migration tools are both more limited and proprietary.[2] SQL also does not have the same testing culture or tools as Python or Scala. While many companies have been through painful SQL migrations, it's more often focused on switching SQL variants (like getting off Oracle). Tools like SQLFluff and Meta's UPM are comparatively early stage.[3]

There are a small number of SQL migration rules in the spark-upgrade repo. Both SQLFluff and the Spark SQL migration rules can be installed with pip, e.g.:

```
pip install sqlfluff && python -m pip install 'sqlfluff-plugin-sparksql-upgrade
@ git+https://github.com/holdenk/spark-upgrade#subdirectory=sql'
```

You can see which Spark-specific rules are installed with `sqlfluff rules |grep -i spark` and run the entire fix set with `sqlfluff --fix mysqlhere.sql`.

One good way to mitigate the risk during SQL migrations is to create small, sampled golden datasets for side-by-side runs.

On the plus side, SQL jobs tend to be well suited to side-by-side runs to verify correctness, which we will discuss more in the next section.

Verifying Correctness and Performance

In addition to the testing, which we will discuss in Chapter 11, a good practice to verify both the correctness and performance of the new pipeline is to run it side-by-side with the old pipeline.

The first step in most cases is to parameterize your pipelines so you configure their output tables. Then you can kick off two instances of your pipeline (new and old) with different output tables. Compare the final result with a combined diff or join as shown in Example 3-4.

2 Each SQL vendor has their own special "flavor" of SQL, and they generally do their best to maintain backward compatibility with that flavor, avoiding the need for such large migrations. Though migrations are a natural opportunity to consider changing vendors, these proprietary tools do encourage vendor stickiness.

3 UPM is not currently available outside of Facebook :((but we hope!)

Example 3-4. Compare the results of two pipelines (from https://github.com/holdenk/spark-upgrade)

```
# Future improvement: compare only the changed partitions
def compare_tables(control, target):
    if control.schema != target.schema:
        control.printSchema()
        target.printSchema()
        raise Exception("Control schema and target schema do not match")
    if args.compare_precision is not None:
        columns = control.columns
        schema = control.schema
        for c in control.columns:
            if isinstance(schema[c].dataType, FractionalType):
                control = control.withColumn(
                    c, round(control[c], parser.compare_precision))
                target = control.withColumn(
                    c, round(target[c], parser.compare_precision))
    control.persist()
    target.persist()
    control_count = control.count()
    target_count = target.count()
    # Do diffs on the data, but subtract doesn't support all data types so fall
back to strings.
    try:
        missing_rows = control.subtract(target)
        new_rows = target.subtract(control)
    except Exception as e:
        # Improvement: only convert the columns that need to be converted.
        eprint(f"Warning converting all to strings.... {e}")
        columns = control.columns
        for c in columns:
            control = control.withColumn(c, control[c].cast('string'))
            target = target.withColumn(c, target[c].cast('string'))
        missing_rows = control.subtract(target)
        new_rows = target.subtract(control)
    new_rows.cache()
    missing_rows.cache()
    new_rows_count = new_rows.count()
    if new_rows_count > 0:
        eprint(f"Found {new_rows_count} that were not in the control")
        new_rows.show()
    missing_rows_count = missing_rows.count()
    if missing_rows_count > 0:
        eprint(f"Found {missing_rows_count} missing from new new pipeline")
        missing_rows.show()
    changed_rows = new_rows_count + missing_rows_count
    row_diff_tol = args.row_diff_tolerance
    exact_tol = row_diff_tol * control_count
    if changed_rows > exact_tol:
        error(
            f"Data differs in table by more than {100 * row_diff_tol}%, failing.")
```

```
    else:
        eprint(f"Different {changed_rows} within {row_diff_tol}% ({exact_tol})")

    if control_count != target_count:
        eprint(f"Counts do not match! {control_count} {target_count}")
        try:
            # Handle duplicates, will fail on maps.
            counted_control = control.groupBy(
                *control.columns).count().persist()
            control_count = counted_control.count()
            counted_target = target.groupBy(*target.columns).count().persist()
            new_rows = counted_target.subtract(counted_control)
            missing_rows = counted_control.subtract(counted_target)
            new_rows_count = new_rows.count()
            if new_rows_count > 0:
                print(f"Found {new_rows_count} grouped that were not in the con
trol")
                new_rows.show()
            missing_rows_count = missing_rows.count()
            if missing_rows_count > 0:
                print(
                    f"Found {missing_rows_count} grouped missing from new new pipe
line")
                missing_rows.show()
            exact_tol = row_diff_tol * control_count
            changed_rows = new_rows_count + missing_rows_count
            if changed_rows > exact_tol:
                error(
                    f"Grouped data differs in table by more than {100 *
row_diff_tol}%, failing.")
            else:
                print(f"Grouped different {changed_rows} within {row_diff_tol}%
({exact_tol})")
        except Exception as e:
            error(
                f"Data counts differ! And we ran into: {e}")


def run_comparisions(tables):
    for (ctrl_name, target_name) in tables:
        # Handle snapshots
        if "@" in ctrl_name:
            c_diff_view = None
            t_diff_view = None
            try:
                (c_diff_view, t_diff_view) = get_cdc_views(spark, ctrl_name, tar
get_name)
            except Exception as e:
                # Otherwise fall through. Technically we could make CDC view from t0
                # But we are using the CDC view as just a hack for speed so not
worth it.
                eprint("Falling through to legacy compare")
```

```
                (ctrl_name, c_snapshot) = ctrl_name.split("@")
                (target_name, t_snapshot) = target_name.split("@")
                return compare_tables(
                    spark.read.option("snapshot-id", c_snapshot).table(ctrl_name),
                    spark.read.option("snapshot-id", t_snapshot).table(target_name))
            print(f"Using CDC view to compare {c_diff_view} and {t_diff_view}")
            return compare_tables(
                c_diff_view,
                t_diff_view)
        else:
            compare_tables(
                spark.read.table(ctrl_name),
                spark.read.table(target_name))


if __name__ == "__main__":
    args = parser.parse_args()
    if args.control_root is not None:
        for table in args.tables:
            control = spark.read.format(args.format).load(
                f"{args.control_root}/{table}")
            target = spark.read.format(args.format).load(
                f"{args.target_root}/{table}")
            compare_tables(control, target)
    else:
        tables = zip(args.control_tables, args.target_tables)
        run_comparisions(tables)
```

The previous example is a bit complicated, since it tries to handle floats and other inexact comparisons.



Tooling here is under active development, and hopefully by the time you read this book some parts of the validation will be able to be automated. While we are biased, we encourage you to check out Holden's spark-upgrade and spark-validator repos to see the latest.

Places Where Performance Can Get Worse

While Spark 3 (and 4) generally has better performance, there are some places where the tricks you did before don't work as well. As such, it's important to verify the performance of your job before and after upgrading.

One of the places where we experienced this is with the new data source/sink interface, which adds expected sort order and distribution from the storage/catalog level. This is generally good, as it allows Iceberg or Delta to have Spark partition your DataFrame to match the target you are writing to—sometimes the resulting distribution may result in less efficient computation than your previous carefully hand-crafted distribution. A concrete example of this would be if you had your own

custom partitioner on a DataFrame and then wrote it out to a table with a specific sort order. On upgrading to Spark 3 (from 2), Spark might introduce a global sort undoing your custom partitioning, which was useful for data parallelism earlier in your job. A general workaround for this is either disabling the write distribution modes or introducing a persist/cache right before the write to prevent the target write distribution from bubbling up to your computation.

Conclusion

Keeping your code updated with new versions of Apache Spark is an important part of taking advantage of performance improvements. While parts of these upgrades can be automated, upgrading still requires substantial manual work. The Spark upgrade tool and Scalafix can help with upgrading to new versions of Spark and Scala, respectively, but they don't provide any automation around other libraries you will likely need to upgrade as a result. Even for the Spark APIs, if you're using things that are less common (for example, accumulators with custom data types), these tools are unlikely to have a rule to help with that migration.[4] To keep your code "high performance," you will need to manually put in the effort to take advantage of new APIs after you have upgraded, and we'll look at some of these new APIs in the next chapter.

4 Community contributions welcomed!

CHAPTER 4

What's New in Spark 4.2 Since 2.4

Spark has changed significantly since 2.4, and one could fill an entire book of just the issues resolved. In this chapter, you will get an understanding of the most important performance-related changes in Spark that have occurred, as well as a bit of a look forward at some of the current proposals for large changes in the coming years.

Taking advantage of some of these changes can be as simple as upgrading to the latest version of Spark. Other changes require changing not just Spark but also other components in your ecosystem, like Iceberg or Delta Lake. Of course, some of the biggest improvements are in the form of new functionality, which you can use to either delete your own code or speed up existing tools.

It's not all good news, though; there are places where functionality is being removed or no longer maintained. While Spark has a fairly comprehensive test suite so that even less-maintained components are unlikely to break, the performance may degrade between versions, as not every component has benchmarking.

The Bad News: GraphX

We decided to address the elephant in the room: Spark's less-supported components. GraphX is much less well-maintained and could be considered functionally deprecated. In fact, there was a vote to deprecate it, which did pass (with some dissent), but it remains (technically) undeprecated as there is work being done to attempt to bring it back; we would be cautious before adopting it in any new project.

To a lesser degree, Spark ML has seen a decrease in active algorithm development. This does not mean you cannot do machine learning in Spark; far from it. Many improvements have been made to enable ML in Spark, but instead of using Spark's internal libraries alone, you now use Spark to power ML-specific libraries, like TensorFlow. In the Spark ML case, we are less worried about performance

degradation, but to keep up with the state-of-the-art models, we recommend pairing Spark with additional tools (as covered in Chapter 10).

It's important to remember that even in actively developed components, like Spark SQL, there will be the occasional performance regression. For example, in Spark 3.0, a bug was added where filter pushdowns could result in double evaluation of expensive expressions (only fixed in 4.2). So, while we do encourage everyone to keep up to date with new versions of Spark, always be alert to the potential for regressions.[1]

The Good News

On the flipside, the majority of Spark continues to be very actively developed, and performance-related changes remain an active area of contribution. One of the biggest things we've seen is the proliferation of Spark accelerators, covered in "Going Beyond the JVM with Spark Accelerators" on page 243, which can natively execute a lot of code with only configuration changes.

No Code Changes Required

Some recent changes to Spark address the most common challenges, such as improved debugging and better partitioning. Best of all, many of these don't require changing your code. Many of these are available regardless of the specific language you work in Spark with.

For those developing on Apple silicon (M1, M2, etc.) *you no longer need to run in emulation mode*, with direct support now present.

Partition Pruning and Predicate Pushdown

Spark SQL has seen huge improvements, with benchmarking showing over a *2x improvement* from Spark 2.4 days. This includes exciting improvements like the new *dynamic partition pruning*, allowing for more intelligent data source–level filtering even within joins. Spark's also gained *adaptive query execution* (AQE), which can improve partitioning and handle skewed joins in many situations. Predicate pushdown has been improved overall, including pushing down more complex operations such as `StringEndsWith`, using column indexes, and other optimizations when supported by the storage engine. These improvements also benefit most DataFrame/Dataset users.

When you are using a modern catalog, such as a recent Iceberg, column statistics can also be used for filtering, so you do not need to partition on every potential value you care about. For example, if a file does not contain any nulls (and the column

1 You can help catch these by testing your code against release candidates.

statistics in the metadata have that recorded), Spark may be able to skip the file if you are just querying for nulls. This is often less efficient than partition-based predicate pushdown since with partitioning the number of files matching tends to be smaller.[2]

Joins

Spark's joins have gotten large performance gains from improved existing algorithms to new join techniques. Spark has introduced bloom filter joins, ideal for "medium-sized" joins where the smaller table is too large to broadcast but still smaller than the other. Shuffle hash join has been introduced to reduce the amount of sorting required for all join types in Spark. Spark's query optimizer will automatically apply these join types, and there is now a full set of join hints you can also supply to help the optimizer, some of which are shown in Table 4-1. Additional join improvements are available with some upgrades to your infrastructure.

Table 4-1. Join hint types

Join hint	When to use
`BROADCAST` (also known as `BROADCASTJOIN` and `MAP JOIN`)	The table is small enough to fit in memory (and inside a broadcast variable). Ideally for less than 1 GB. Especially useful when you have a small set of common or repeated keys you are joining on (think like country or state). Ideal for high skew.
`MERGE`	Sort-based shuffle merge; should be used when both tables are large.
`SHUFFLE_HASH`	Large tables, equal key distribution (no skew).
`SHUFFLE_REPLICATE_NL`	Ideal for cartesian-product type joins[a] and the input tables must be very small.

[a] Ones where the join condition is always true.

Since it turns out this "cloud" thing might have something to it, SPARK-23977 added *new higher-performance S3A committers*, which can speed up large direct file writes,[3] which is now (as of Spark 4.1) the default.[4]

2 For example, assuming a random dataset with 1% nulls, if we put all of the nulls together in one file, we will always only have that one file to load, but if they are randomly distributed, we may need to load multiple files (but possibly not all) when using column stats.

3 This does not impact Iceberg tables, which use their own code path.

4 Added in SPARK-47618.

Debugging

Tuning and debugging Spark has been a challenge, but new versions of Spark aim to improve much of this. If you've ever had difficulty loading the Spark UI on your larger jobs, now is the time to give it another shot, as both the online UI and history servers have been improved to handle larger jobs. Streaming jobs also periodically "roll over" the UI to keep it from growing too large. Spark's infamously *difficult-to-read error messages have also been improved*, with many error messages going through a central system instead of depending on whatever the developer writing the exception is thinking about that day or tracebacks so lengthy that actual causes are truncated out of the message.

User-defined metrics are now also exported from executors for better debugging.

PySpark users frustrated with difficult-to-debug memory problems will benefit from a *new Python executor memory profiler*.

Additional Upgrades Required (Not Code)

Storage partitioning-aware joins can significantly reduce the shuffle cost of many operations. In the ideal case, if the two tables share the same partitioning key, Spark can treat them as co-partitioned and avoid the shuffle entirely; in other cases, one of the two tables will be shuffled to match. This requires upgrading the data catalog (Iceberg 1.2+) as well as Spark itself (3.3+). A simple example of the happy path, matching partitioned keys, is shown in Example 4-1.

Example 4-1. Spark storage partitioned-ware join

```
CREATE TABLE IF NOT EXISTS local.developers (
       username string,
       firstname string,
       lastname string)
USING iceberg
PARTITIONED BY (username);
CREATE TABLE IF NOT EXISTS local.projects (
       creator string,
       projectname string)
USING iceberg
PARTITIONED BY (creator);
INSERT INTO local.developers VALUES("krisnova", "Kris", "Nova");
INSERT INTO local.projects VALUES("krisnova", "aurae");
SELECT * FROM local.developers INNER JOIN local.projects ON local.projects.creator
= local.developers.username;
```

It can be difficult to know if a given join used your storage partitioning information, as the query plans look the same either way. However, you can verify that the join used storage partitioning information by checking the UI and seeing that

there is no shuffled data for that task. To ensure that a given job runs with storage partitioned-aware joining in the previous example, we also used some extra config (in production, you likely want to leave the broadcast join threshold on, since if the table is small enough, that's even better), as shown in Example 4-2.

Example 4-2. Spark storage partitioned-aware config (and other optimizations disabled for verification)

```
--conf spark.sql.sources.v2.bucketing.enabled=true
--conf spark.sql.iceberg.planning.preserve-data-grouping=true
--conf spark.sql.requireAllClusterKeysForCoPartition=false

--conf spark.sql.adaptive.enabled=false
--conf spark.sql.autoBroadcastJoinThreshold=-1
--conf spark.sql.shuffle.partitions=4
```

Spark supports recent programming languages and runtimes (from Scala 2.13 and Python 3.14 to JDK 25). These languages and runtimes bring substantial performance benefits, with Java 17 seeing large improvements over Java 8 and Python 3.11 being 10–60% faster than the earlier versions.[5]

Spark Kubernetes support has been added, with support for various schedulers (such as Volcano) also available. While not directly related, Spark now supports pure IPv6 deployment, which is sometimes found in modern Kube deployments.

Spark Streaming now supports a RocksDB backend for state storage, which can improve reliability and reduce memory pressure for intensive workloads.

Code and Logic Changes Needed

Taking the most advantage of the newest version of Spark will, of course, require updating your code to use new APIs and patterns.

General

Spark added accelerator-aware (GPU and friends) scheduling. Taking advantage of this requires creating "resource profiles" and a cluster manager that is accelerator-aware (thankfully, both YARN and Kubernetes can be made hardware accelerator-aware). We cover resource profiles in Appendix D.

Spark Datasets have new format support, from arbitrary binary files to *support for protocol buffers* and nested JSON and CSV structures. The `VARIANT` data type makes

5 The Spark developers have worked to take advantage of the new features in JDK9–17.

storing semistructured data much less expensive and is also supported by Iceberg and other table formats.

Spark SQL (and DataFrames/Datasets) now have a *much larger range of built-in functions*, ranging from date and time to encryption. If you need sorted values in your grouped data, you can now get sorted records while supporting spill-to-disk for large values. Per-column default values for tables are also added, simplifying handling for when your schema has evolved. If you have written custom user-defined functions (UDFs) or user-defined aggregate functions (UDAFs), now is the time to check and see if you can delete them 😄. In addition to general utility functions, Spark SQL has greatly improved ANSI SQL support.



ANSI SQL is now the default in Spark 4, so when upgrading, pay careful attention to your inline SQL.

Parameterized SQL now makes it easier to write "safe" SQL code for Spark without worrying about SQL injection problems.



Have you ever tried to insert records into a table, only to get a confusing error message about type casting because the insert is by column number rather than by name? `INSERT BY NAME` and `INSERT OVERWRITE BY NAME` have been added in Spark 3.4, and we *strongly* recommend using by-name insertions when possible.

You can now make multiple stateful operations on the same stream instead of creating multiple streams.

A new, less language-specific API was added to Spark called Spark Connect, allowing you to have long-lived Spark sessions that send DataFrame code to be executed remotely. Think Java Database Connectivity (JDBC) on steroids (or at least DataFrames + JDBC + Arrow Flight + magic). There is work being undertaken to extend this less language-specific API to also include non-Python or JVM UDFs, but no concrete timeline.

Spark Streaming now supports new execution modes and APIs (including `transformWithState`), enabling it to handle data with lower latency and higher overall throughput. While it's still not a drop-in replacement for all your streaming needs, it's now good enough for many situations at a lower cost than many more real-time alternatives.

Python Specific

Some of the most exciting changes come to Spark through what has been called *Project Zen*, bringing a distributed pandas-like DataFrame experience (for a time known as Koalas). Python UDFs can now take advantage of Mypy type information, reducing the amount of boilerplate and improving correctness.

The *pandas on Spark interface* is the biggest and most exciting change to come to PySpark since the 2.x days. You can also use some of the internals, like `mapInArrow`, to power Arrow-capable libraries beyond pandas (like many ML libraries).

Dependency management has long been a pain point for PySpark users, and you can now ship *most of your Python dependencies* regardless of the deployment mechanism. With the new `--archive` option, you can ship relocatable virtualenvs, Conda, or Python EXecutable (PEX) for the driver and executor. Native dependency (C/C+\+, etc.) libraries still pose challenges outside of Dockerized environments, but this is a great step forward.

It is now possible to define data sources in Python, making it easier to interact with file formats without JVM libraries. When JVM libraries are available, we generally recommend using them to make a data source instead, as the performance is normally better and there are more opportunities to build pushdowns with the Scala API.

You can now do arbitrary stateful stream processing in PySpark using pandas-powered UDFs.

Spark now has built-in distributed training for PyTorch models, which can take advantage of two other features, the gang scheduler and the accelerator-aware scheduling.

Scala Specific

There are very few Scala-specific changes, but the biggest benefit is the new support of Scala 2.13, with which Scala 3 has a source-compatible mode for the eventual upgrade. In addition, the Dataset APIs now have many of the higher-order functions, so you don't need to write as much inline SQL and maintain your type safety.

Prioritizing Upgrades

Once you've moved to a modern version of Spark, it's important to prioritize where you upgrade and tune your code. We think a good first postupgrade step is to upgrade the language and other libraries so you can both take advantage of all of the benefits and give yourself some time before your next forced upgrade. After that, reexamine your existing join hints and see if they would benefit from a new, more specific join type. From there, we recommend you take a look at the improved and

more reliable Spark UI to narrow down the more expensive components of your job and focus on those. Good luck with your move to Spark 4 and beyond!

Totally New Features

There are also totally new features/functionality in Spark that enable you to do new exciting things.

Spark Declarative Pipelines

Spark 4.1 adds declarative data pipelines, which allow you to more easily express pipelines with semiautomatic dependency analysis with minimal streaming support. For pure PySpark and Spark SQL jobs, Spark Pipelines can serve much of the purpose of Airflow, dbt, or similar, but with better engine/pipeline understanding. If you've ever said to yourself, "I just want to process 5TB of data after these two other Spark jobs finish," this could be the answer for you. You can get started quickly with `spark-pipelines init --name my_pipeline`, which creates a templated Spark pipeline. The core specification is a YAML file that points at your Python- and SQL-defined steps as shown in Example 4-3.

Example 4-3. Create a simple Spark pipeline

```
name: spark-pipeline-ex
# Where should the steps output be stored, must be absolute (file:// for testing,
but hdfs/sf for real)
storage: file://tmp/pipeline-storage
libraries:
  - glob:
      # Can also use wildcard includes.
      include: transformations/project_pipeline.py
# Optional Spark configs
configuration:
  # Note: the values need to be strings, Spark side they get parsed.
  spark.sql.shuffle.partitions: "2"
```

Pipeline steps are "just" functions that return DataFrames (streaming or batch) with decorators like `@dp.temporary_view`. You can read views (and tables and materialized views) using the desired previous steps function name as a table name. The final materialized view in our pipeline reads from `popular_projects` and `creator_stats` shown in Example 4-4.

Example 4-4. Define the steps for the Spark pipeline

```
@dp.materialized_view
def project_summary() -> DataFrame:
    """Join popular projects with their creator stats."""
    return (
        spark.table("popular_projects")
        .join(spark.table("creator_stats"), on="creator")
        .select("creator", "projectname", "stars", "num_projects", "avg_stars")
    )
```

Everything you can do with Spark declarative pipelines can also be accomplished with "regular" Spark with more control, but a lack of support for mixing JVM/Python jobs makes us hesitant to fully recommend it just yet.

SQL UDFs/UDTFs

You can now write UDFs and UDTFs in SQL, which is especially appealing for data analysts and amenable to optimization by Spark's Catalyst engine.

Looking Ahead to the Future

While we don't have a crystal ball yet, Spark project improvement proposals (SPIP) can give us a bit of an idea of some of the larger changes that people are thinking about for Spark. You can see the current SPIPs in the Spark Jira, but while reading it's important to remember that not all SPIPs will be adopted or completed.

Holden is, of course, excited (cautiously) about her current SPIP to add transpilation to PySpark UDFs, which would allow people to write simple lambda expressions instead of Spark SQL especially when working in pandas on Spark.

When a new version of Spark comes out, the release notes can give you an idea of what you might want to be aware of, although for dot releases they tend to be on the sparse side; you'll need to dig through the Jira instead.

Conclusion

Apache Spark has a great many new features in both 3 and 4, which you should take the time to become familiar with to take the most advantage of Spark. Not all of the new features have been covered here, but we hope this chapter has got you thinking about what changes you might want to be making to your existing applications. The following chapters will look at how to use many of these new features in more detail, with the next chapter diving deeper into Spark's Datasets and DataFrames.

CHAPTER 5

DataFrames, Datasets, and Spark SQL

In 2017, we predicted that the future of Spark would be defined by Spark SQL DataFrames and Datasets interfaces' simplicity and ease of adoption. Today they are the most used capabilities among Spark developers. With more efficient storage options, an advanced optimizer, native accelerators, and direct operations on serialized data, this has become the go-to distributed data processing capabilities for data engineers, data analysts, and data scientists.

These components are super important for getting the best of Spark performance (see Figure 5-1). Datasets are a Scala/Java-specific concept versus DataFrames, which are supported across languages, including PySpark. For the most part we'll use *Dataset* instead of spelling out *Dataset and DataFrame* except for the section headers, where we've separated out some Dataset-specific information. When talking about things that apply to Spark SQL regardless of interface (SQL, DataFrame, or Dataset) we'll just say *Spark SQL*.

Datasets, and their earlier untyped DataFrames, specify more of what the goal (declarative) is versus specifying how to compute the value (imperative) as in Spark RDDs. The declarative approach opens up even more optimizations on top of those opened up by Spark RDDs' functional APIs with lazy evaluation. When possible, we generally recommend using Datasets in most instances instead of RDDs.

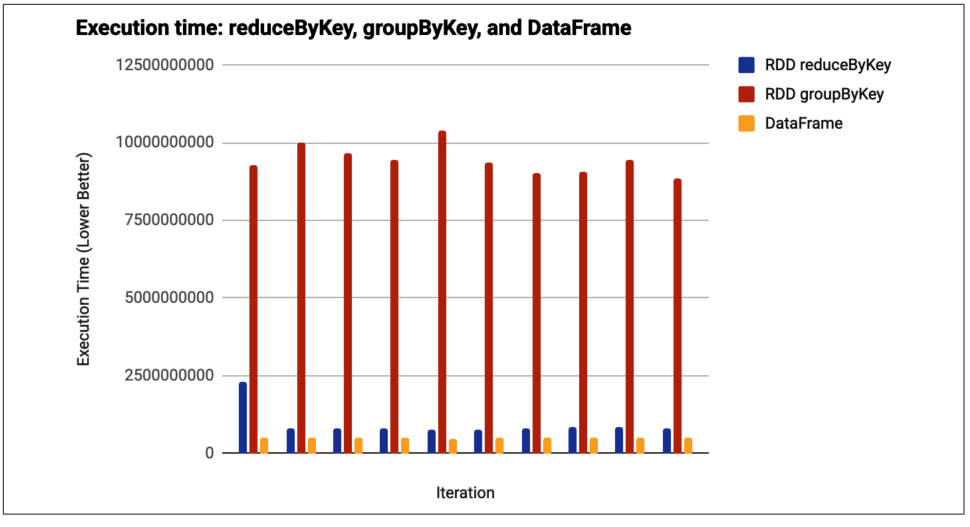


Figure 5-1. Relative performance for RDD versus DataFrames based on SimplePerfTest computing aggregate average fuzziness of pandas

For folks going beyond the traditional languages in Spark, Spark Connect only supports the Dataset APIs.



Spark's DataFrames have very different performance and somewhat different functionality compared to traditional DataFrames like pandas or R.[1] While these all deal with structured data, it is important not to depend on your existing intuition surrounding DataFrames.

Like RDDs, Datasets represent distributed collections. Datasets have additional schema information not found in RDDs, and their query graph has more information about the operations being performed instead of "black boxes" of functions. This additional schema information is used to provide a more efficient storage layer (Tungsten versus Java Serialization). The query details allow the optimizer (Catalyst) to perform additional optimizations (like reordering operations to reduce Dataset size sooner). Beyond schema information, the operations performed on Datasets and DataFrames are such that the optimizer can inspect the logical meaning rather than arbitrary functions. DataFrames are Datasets of a special `Row` object, which doesn't provide any compile-time type checking. The strongly typed Dataset API shines, especially for use with more RDD-like functional operations. Compared to

1 PySpark has a closer to pandas DataFrame interface, taken over from the Koalas project, but what's fast in pandas versus PySpark pandas is very different.

working with RDDs, DataFrames allow Spark's optimizer to better understand our code and our data, which allows for a new class of optimizations we explore in "Query Optimizer" on page 102.



While Spark SQL provides many excellent enhancements on top of Spark's core RDDs, they can be more difficult to reason about as the optimizer or code generator can substantially change your query into one it believes is more efficient (even when this is not the case).[2]

Getting Started with the SparkSession (or HiveContext or SQLContext)

Much as the `SparkContext` is the de facto entry point for all Spark applications, the `SparkSession` serves as the entry point for Spark SQL. Like with all of the Spark components, you need to import a few extra components as shown in Example 5-1.



If you are using the Spark shell, you will automatically get a `SparkSession` called `spark` to accompany the `SparkContext` called `sc`.

Example 5-1. Spark SQL imports

```
import org.apache.spark.sql.{Dataset, DataFrame, SparkSession, Row}
import org.apache.spark.sql.catalyst.expressions.aggregate._
import org.apache.spark.sql.expressions._
import org.apache.spark.sql.functions._
```



Scala's type alias of `DataFrame = Dataset[Row]` is broken in Java—you must use `Dataset<Row>` instead.

You generally create a `SparkSession` using the singleton[3] builder pattern by calling `getOrCreate()`, which will return an existing session if one is already running. The

2 Sometimes whole-stage codegen can go sideways; you can turn it off by setting `spark.sql.codegen.wholeStage=false`, although we find this most helpful for debugging.

3 Running multiple Spark SQL `Sessions` or even `SparkContexts` in the same JVM can cause errors and is not recommended.

builder can take string-based configuration keys, `config(key, value)`, and shortcuts exist for a number of common parameters. One of the more important common shortcuts is `enableHiveSupport()`, which will give you access to Hive UDFs[4] and *does not require* a Hive Metastore installation—but does require certain extra JARs (discussed in "Spark SQL Dependencies"). Example 5-2 shows how to create a `SparkSession` with Hive support. The `enableHiveSupport()` shortcut not only configures Spark SQL to use these Hive JARs, but it also eagerly checks that they can be loaded—leading to a clearer error message than setting configuration values by hand. In general, using shortcuts listed in the API docs is advised when they are present, since no checking is done in the generic config interface.

Example 5-2. Create a `SparkSession`

```
val session = SparkSession
  .builder()
  .getOrCreate()
// Import the implicits, unlike in core Spark the implicits are defined
// on the session.
import session.implicits._
```



When using `getOrCreate`, if an existing session exists, your configuration values will be ignored, and you will get the existing `SparkSession`. Some options, like `master`, will also only apply if there is no existing `SparkContext` running; otherwise, the existing `SparkContext` will be used for new sessions.

Spark SQL Dependencies

Like the other components in Spark, using Spark SQL requires adding additional dependencies. If you have conflicts with the Hive JARs you can't fix through shading, you can just limit yourself to the *spark-sql* JAR—although you want to have access to the Hive dependencies without also including the *spark-hive* JAR.

To enable Hive support in `SparkSession`, you will need to add both Spark's SQL and Hive components to your dependencies (see Examples 5-3 and 5-4).

Example 5-3. Add Spark SQL and Hive component to "regular" sbt build

```
libraryDependencies ++= Seq(
"org.apache.spark" %% "spark-sql" % "4.1.1",
"org.apache.spark" %% "spark-hive" % "4.1.1")
```

4 UDFs allow us to extend SQL to have additional powers, such as computing the geospatial distance between points.

Depending on how your packaging your project, you might need to mark these and other Spark dependencies as provided to avoid conflicting with cluster provided versions and allow for minor version upgrades on the cluster.

Example 5-4. Add Spark SQL and Hive component to Maven pom file

```
<dependency> <!-- Spark dependency -->
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-sql_2.13</artifactId>
  <version>4.1.1</version>
</dependency>
<dependency> <!-- Spark dependency -->
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-hive_2.13</artifactId>
  <version>4.1.1</version>
</dependency>
```



If you are starting a new Spark project, there is a g8 template, *holdenk/sparkProjectTemplate*, you can use to bootstrap your new project by running `sbt new holdenk/sparkProjectTemplate`.

While it's not required, if you do have an existing Hive Metastore to which you wish to connect with Spark, you can copy your *hive-site.xml* to Spark's *conf/* directory.



The default Hive Metastore version is 2.3.10. For other versions of the Hive Metastore, you will need to set the `spark.sql.hive.metastore.version` property to the desired versions as well as set `spark.sql.hive.metastore.jars` to either "maven" (to have Spark retrieve the JARs) or the system path where the Hive JARs are present.

We will cover the full loading and saving API for JSON in "JSON" on page 90, but to get started, let's load a sample we can use to explore the schema (see Example 5-5).

Example 5-5. Load JSON sample

```
val df1 = session.read.json(path)
```

Feel free to load your own JSON data, but if you don't have any handy to test with, check out the examples in the GitHub resources directory. Now that you've got the JSON data loaded, you can start by exploring what schema Spark has managed to infer for your data.

Basics of Schemas

The schema information, and the optimizations it enables, are one of the core differences between Spark SQL and core Spark. Inspecting the schema is especially useful for DataFrames since you don't have the templated type you do with RDDs or Datasets. Schemas are normally handled automatically by Spark SQL, either inferred when loading the data or computed based on the parent DataFrames and the transformation being applied.

DataFrames expose the schema in either human-readable or programmatic formats. `printSchema()` will show us the schema of a DataFrame and is most commonly used when working in the shell to figure out what you are working with. This is especially useful for data formats like JSON, where the schema may not be immediately visible by looking at only a few records or reading a header. For programmatic usage, you can get the schema by simply calling `schema`, which is often used in ML pipeline transformers. Since you are likely familiar with case classes and JSON, let's examine how the equivalent Spark SQL schema would be represented in Examples 5-6 and 5-7.

Example 5-6. JSON data that would result in an equivalent schema

```
{"name":"mission","pandas":[{"id":1,"zip":"94110","pt":"giant", "happy":true,
                             "attributes":[0.4,0.5]}]}
```

A case class with an equivalent schema is shown in Example 5-7.

Example 5-7. Equivalent case class

```
case class RawPanda(id: Long, zip: String, pt: String,
                    happy: Boolean, attributes: Array[Double])
case class PandaPlace(name: String, pandas: Array[RawPanda])
```



You can do the same thing using JavaBeans in Java. Python dataclasses can also be used but always result in a DataFrame since they don't have Datasets.

Now with the case classes defined, you can create a local instance, turn it into a `Dataset`, and print the schema as shown in Example 5-8, resulting in Example 5-9. The same can be done with the JSON data but requires some configuration as discussed in "JSON" on page 90.

Example 5-8. Create a Dataset with the case class

```
  def createAndPrintSchema() = {
    val damao = RawPanda(1, "M1B 5K7", "giant", true, Array(0.1, 0.1))
    val pandaPlace = PandaPlace("toronto", Array(damao))
    val df = session.createDataFrame(Seq(pandaPlace))
    df.printSchema()
  }
```

Example 5-9. Sample schema information for nested structure (`.printSchema()`)

```
root
 |-- name: string (nullable = true)
 |-- pandas: array (nullable = true)
 | |-- element: struct (containsNull = true)
 | | |-- id: long (nullable = false)
 | | |-- zip: string (nullable = true)
 | | |-- pt: string (nullable = true)
 | | |-- happy: boolean (nullable = false)
 | | |-- attributes: array (nullable = true)
 | | | |-- element: double (containsNull = false)
```

In addition to the human-readable schema, the schema information is also available for you to use programmatically. The programmatic schema is returned as a `StructField`, as shown in Example 5-10.

Example 5-10. `StructField` case class

```
case class StructField(
  name: String,
  dataType: DataType,
  nullable: Boolean = true,
  metadata: Metadata = Metadata.empty)

...
```

Example 5-11 shows the same schema as Example 5-9, in machine-readable format.

Example 5-11. Sample schema information for nested structure (`.schema()`)—manually formatted

```
org.apache.spark.sql.types.StructType = StructType(
  StructField(name,StringType,true),
  StructField(pandas,
    ArrayType(
      StructType(StructField(id,LongType,false),
                 StructField(zip,StringType,true),
                 StructField(pt,StringType,true),
                 StructField(happy,BooleanType,false),
```

```
StructField(attributes,ArrayType(DoubleType,false),true)),
                true),true))
```

From here you can dive into what this schema information means and look at how to construct more complex schemas. The first part is a `StructType`, which contains a list of fields. It's important to note you can nest `StructTypes`, like how a case class can contain additional case classes. The fields in the `StructType` are defined with `StructField`, which specifies the name, type (see the Spark SQL data type page for most Spark types as well as Table 5-1 for some example complex types), and a Boolean indicating if the field may be null/missing.

Table 5-1. Complex Spark SQL types

Scala type	Python type	SQL type	Details	Example
`Array[T]`	`list`	`ArrayType(elementType, containsNull)`	Array of single type of element, `containsNull` true if any null elements.	`Array[Int] ⇒ ArrayType(IntegerType, true)`
`Map[K, V]`	`dict`	`MapType(elementType, valueType, valueContainsNull)`	Key/value map, `valueContainsNull` if any values are null.	`Map[String, Int] ⇒ MapType(StringType, IntegerType, true)`
case class	Dataclass	`StructType(List[StructFields])`	Named fields of possible heterogeneous types, similar to a case class or JavaBean.	`case class Panda(name: String, age: Int) ⇒ StructType(List(StructField("name", StringType, true), StructField("age", IntegerType, true)))`



As you saw in Example 5-11, you can nest `StructFields` and all of the complex Spark SQL types.

Column Metadata

Column metadata in Spark can contain arbitrary key/value pairs and initially was used inside of Spark ML to annotate certain columns during featurization. This original use case has remained and been picked up by some machine learning tools built on top of Spark. In addition to machine learning, some folks use column metadata to pass along "comments" or information about the source of a particular column. Most

metadata is not saved during a write, but when creating a new table with Iceberg, any "comment" or "doc" string metadata will be persisted.



While DataFrame columns have had metadata since the creation, in early versions of Spark, some key transformations would discard metadata.

Now that you've got an idea of how to understand and, if needed, specify schemas for your data, you are ready to start exploring the DataFrame interfaces.



Spark SQL schemas are eagerly evaluated, unlike the data underneath. If you find yourself in the shell and uncertain of what a transformation will do, try it and print the schema. See Example 5-9.

DataFrame API

Spark SQL's DataFrame API allows us to work with DataFrames without having to register temporary tables or generate SQL expressions. The DataFrame API has both transformations and actions. The transformations on DataFrames are more relational in nature, with the Dataset API (covered next) adding a more functional-style API on top.

Transformations

Transformations on DataFrames are similar in concept to RDD transformations, but with a more relational flavor. Instead of specifying arbitrary functions, which the optimizer is unable to introspect, you mostly use a restricted expression syntax so the optimizer can have more information. As with RDDs, we can broadly break down transformations into simple single DataFrame, multiple DataFrame, key/value, and grouped/windowed transformations.



Spark SQL transformations are only partially lazy; the schema is eagerly evaluated. This can make some errors show up earlier than in RDD land, but keep in mind the core transformations will still behave lazily.

Simple DataFrame transformations and SQL expressions

Simple DataFrame transformations allow us to do most of the standard things one can do when working with a row at a time.[5] You can still do many of the same operations defined on RDDs, except using Spark SQL expressions instead of arbitrary functions. To illustrate this, we will start by examining the different kinds of filter operations available on DataFrames.

DataFrame functions, like `filter`, accept Spark SQL expressions instead of lambdas. These expressions allow the optimizer to understand what the condition represents, and with `filter`, it can often be used to skip reading unnecessary records.

To get started, let's look at a SQL expression to filter our data for unhappy pandas using our existing schema. The first step is looking up the column that contains this information. In our case it is `happy`, and for our `DataFrame` (called `df`) we access the column through the apply function in Scala (e.g., `df("happy")`; in Java it's `.col` or in Python column reference `df['happy']`). The `filter` expression requires the expression to return a boolean value, and if you wanted to select happy pandas, the entire expression could be retrieving the column value. However, since we want to find the unhappy pandas, we can check to see that `happy` isn't true using the `!==` operator as shown in Example 5-12.

Example 5-12. Simple filter for unhappy pandas

```
pandaInfo.filter(pandaInfo("happy") !== true)
```



In Scala, to look up the column, we can either provide the column name on the specific `DataFrame` or use the implicit `$` operator for column lookup. This is especially useful when the `DataFrame` is anonymous. The `!` binary negation function can be used together with `$` to simplify our expression from Example 5-12 down to `df.filter(!$("happy"))`. We recommend, and will generally in the rest of the examples use `$` to reference columns, as you can find weird bugs when referencing a column extracted from a previous DataFrame (see SPARK-47001). In non-Scala languages, extracting columns from the DataFrame, as in Example 5-12, is still best practice.

This illustrates how to access a specific column from a DataFrame. For accessing other structures inside of DataFrames, like nested structs, keyed maps, and array elements, use the same apply syntax. So, if the first element in the `attributes` array

5 A row at a time allows for narrow transformations with no shuffle.

represents squishiness, and you only want very squishy pandas, you can access that element by `writing $"attributes"(0) >= 0.5`.

Our expressions need not be limited to a single column. You can compare multiple columns in our "filter" expression. Complex filters like that shown in Example 5-13 are more difficult to push down to the storage layer, so you may not see the same speedup over RDDs that you see with simpler filters.

Example 5-13. More complex filter

```
pandaInfo.filter(
  $"happy".and($"attributes" (0) > $"attributes" (1))
)
```



Spark SQL's column operators are defined on the column class, so a filter containing the expression `0 >= $"friends"` will not compile since Scala will use the `>=` defined on 0. Instead, you would write `$"friend" <= 0` or convert `0` to a column literal with `lit`.[6]

Spark SQL's DataFrame API has a very large set of operators available. You can use all of the standard mathematical operators on floating points, along with the standard logical and bitwise operations (prefix with `bitwise` to distinguish from logical). In Scala, columns use `===` and `!==` for equality to avoid conflict with Scala internals but `==` and `!=` in Python. For columns of strings, `startsWith`/`endsWith`, `substr`, `like`, and `isNull` are all available. The full set of operations is listed in `org.apache.spark.sql.Column` and summarized in Tables 5-2 and 5-3.

Table 5-2. Common Spark SQL Scala and Python operators

Scala operator	Python and Java equivalent	Input column types	Output type	Purpose	Sample	Result
`!==`	`notEqual`	Any	Boolean	Check if expressions not equal	`"hi" !== "bye"`	`true`
`%`	`mod`	Numeric	Numeric	Modulo	`10 % 5`	`0`
`&&`	`and`	Boolean	Boolean	Boolean and	`true && false`	`false`
`*`	`multiply`	Numeric	Numeric	Multiply expressions	`2 * 21`	`42`
`+`	`plus`	Numeric	Numeric	Sum expression	`2 + 2`	`4`
`-`	`minus`	Numeric	Numeric	Subtraction	`2 - 2`	`0`
`-`	`unary_-`	Numeric	Numeric	Unary subtraction	`-42`	`-42`

6 A column literal is a column with a fixed value that doesn't change between rows (i.e., constant).

Scala operator	Python and Java equivalent	Input column types	Output type	Purpose	Sample	Result
/	division	Numeric	Double	Division	43/2	21.5
<	lt	Comparable	Boolean	Less than	"a" < "b"	true
<=	leq	Comparable	Boolean	Less than or equal to	"a" <= "a"	true
===	equals	Any	Any	Equality test (unsafe on null values)	"a" === "a"	true
<=>	eqNullSafe	Any	Any	Equality test (safe on null values)	"a" <=> "a"	true
>	gt	Comparable	Boolean	Greater than	"a" > "b"	false
>=	ge	Comparable	Boolean	Greater than or equal to	"a" >= "b"	false

Table 5-3. Selected Spark SQL expression operators

Operator	Input column types	Output type	Purpose	Sample	Result
apply (JVM) [] (Python)	Complex types	Type of field accessed	Get value from complex type (e.g., structfield/map lookup or array index)	[1,2,3].apply(0) (JVM) / [1, 2, 3][0] (python)	1
bitwiseAND (JVM) bit_and (Python)	Integral Type[a]	Same as input	Computes and bitwise	21.bitwiseAND(11)	1
bitwiseOR (JVM) bit_or (Python)	Integral Type[a]	Same as input	Computes or bitwise	21.bitwiseOR(11)	31
bitwiseXOR (JVM) bit_xor (Python)	Integral Type[a]	Same as input	Computes bitwise exclusive or	21.bitwiseXOR(11)	30

[a] Integral types include ByteType, IntegerType, LongType, and ShortType



Not all Spark SQL expressions can be used in every API call. For example, filter requires that the expression result in a boolean, as do join expressions. Join expressions also require that the join condition be deterministic, as a nondeterministic join does not make sense. Certain types of expressions are partially supported in joins but may force cross-joins—for example, any UDF that cannot be evaluated constantly.

In addition to the operators directly specified on the column, an even larger set of functions on columns exists in org.apache.spark.sql.functions, some of which we cover in Tables 5-4, 5-5, and 5-6. For illustration, this example shows the values for

each column at a specific row, but keep in mind that these functions are called on columns, not on values.

Table 5-4. Spark SQL standard functions

Function name	Purpose	Input types	Example usage	Result
`lit(value)`	Convert a Scala symbol to a column literal	Column and symbol	`lit(1)`	`Column(1)`
`array`	Create a new array column	Must all have the same Spark SQL type	`array(lit(1),lit(2))`	`array(1,2)`
`isNaN`	Check if not a number	Numeric	`isNan(lit(100.0))`	false
`not`	Opposite value	Boolean	`not(lit(true))`	false

Table 5-5. Some Spark SQL common mathematical expressions

Function name	Purpose	Input types	Example usage	Result
`abs`	Absolute value	Numeric	`abs(lit(-1))`	`1`
`sqrt`	Square root	Numeric	`sqrt(lit(4))`	2
`acos`	Inverse cosine	Numeric	`acos(lit(0.5))`	1.04….[a]

[a] Truncated for display purposes. Spark does lots of math.

Table 5-6. Functions for use on Spark SQL arrays

Function name	Complexity	Purpose	Example usage	Result
`array_contains`	`O(N)`	If an array contains a value.	`array_contains(lit(Array(2,3,-1), 3))`	true
`sort_array`	Primitives: Expected `O(N log N)` Worst N^2[a]	Sort an array (ascending default).	`sort_array(lit(Array(2,3,-1)))`	`Array(-1,2,3)`
`explode`	`O(N)`	Create a row for each element in the array—often useful when working with nested JSON records. Either takes a column name or additional function mapping from row to iterator of case classes.	`explode(lit(Array(2,3,-1)), "murh")`	`Row(2)`, `Row(3)`, `Row(-1)`

[a] Uses JDK internal array sort of primitives.

Beyond simply filtering out data, you can also produce a DataFrame with new columns or updated values in old columns. Spark uses the same expression syntax we discussed for `filter`, except instead of having to include a condition (like testing for equality), the results are used as values in the new DataFrame. To see how you can use `select` on complex and regular data types, Example 5-14 uses the Spark SQL `explode` function to turn an input `DataFrame` of `PandaPlaces` into a `DataFrame` of just `PandaInfo` as well as computing the "squishness" to "hardness" ratio of each panda.

Example 5-14. Spark SQL `select` and `explode` operators

```
val pandaInfo = pandaPlace.explode(pandaPlace("pandas")) {
  case Row(pandas: Seq[Row]) =>
    pandas.map {
      case (Row(
            id: Long,
            zip: String,
            pt: String,
            happy: Boolean,
            attrs: Seq[Double]
          )) =>
        RawPanda(id, zip, pt, happy, attrs.toArray)
    }
}
pandaInfo.select(
  ($"attributes" (0) / $"attributes" (1))
    .as("squishyness")
)
```



When you construct a sequence of operations, the generated column names can quickly become unwieldy, so the `as` or `alias` operators are useful to specify resulting column names for simplified accessibility.

While all of these operations are quite powerful, sometimes the logic you wish to express is more easily encoded with if/else semantics. Example 5-15 is a simple example of this, and it encodes the different types of panda as a numeric value.[7] The `when` and `otherwise` functions can be chained together to create the same effect.

7 StringIndexer in the ML pipeline is designed for string index encoding.

Example 5-15. If/else in Spark SQL

```
/** Encodes pandaType to Integer values instead of String values.
  *
  * @param pandaInfo
  *   the input DataFrame
  * @return
  *   Returns a DataFrame of pandaId and integer value for pandaType.
  */
def encodePandaType(pandaInfo: DataFrame): DataFrame = {
  pandaInfo.select(
    $"id",
    (when($"pt" === "giant", 0)
      .when($"pt" === "red", 1)
      .otherwise(2))
      .as("encodedType")
  )
}
```

Specialized DataFrame transformations for missing and noisy data

Spark SQL also provides special tools for handling missing, null, and invalid data. By using `isNan` or `isNull` along with filters, you can create conditions for the rows you want to keep. For example, if you have a number of different columns, perhaps with different levels of precision (some of which may be null), you can use `coalesce(c1, c2, ..)` to return the first non-null column (not to be confused with coalescing/reducing the number of partitions). Similarly, for numeric data, `nanvl` returns the first non-NaN value (e.g., `nanvl(0/0, sqrt(-2), 3)` results in 3). To simplify working with missing data, the `na` function on DataFrame gives us access to some common routines for handling missing data in DataFrameNaFunctions.



Sometimes you know your data is "not null," but Spark doesn't. You can set "check-nullability" to "false" on Iceberg (and some other table sinks) to ignore nullable errors when writing. Generally using coalesce or filtering nulls is better though, as what we "know" and what is true are not always the same. Delta users should check out Chapter 8 of *Delta Lake: The Definitive Guide* (O'Reilly), for information on Delta constraints.

Beyond row-by-row transformations

Sometimes applying a row-by-row decision, as you can with the `filter` function, isn't enough. Spark SQL also allows us to select the unique rows by calling `dropDuplicates`, but as with the similar operation on RDDs (`distinct`), this can require a shuffle, and is often much slower than `filter`. Unlike with RDDs, `dropDuplicates` can optionally drop rows based on only a subset of the columns, such as an ID field, as shown in Example 5-16.

Example 5-16. Drop duplicate panda IDs

```
pandas.dropDuplicates(List("id"))
```

This leads nicely into our next section on aggregates and `groupBy` since often the most expensive component of each is the shuffle.

Aggregates, groupBy, and coGroup

Spark SQL has many powerful aggregates, and thanks to its optimizer, it can be easy to combine many aggregates into one single action/query. Like with pandas' DataFrames, `groupBy` returns special objects on which we can ask for certain aggregations to be performed.

When you group a DataFrame or Dataset by column (or columns), you get back a `RelationalGroupedDataset` object (or `GroupedData` in Python) on which you can specify your desired aggregations. As with RDDs, most of the time, aggregations on DataFrames involve some amount of shuffling, which can be expensive, making aggregations a good place to focus on performance.

`min`, `max`, `avg`, and `sum` are all implemented as convenience functions directly on `RelationalGroupedDataset`, and more can be specified by providing the expressions to agg. Example 5-17 shows how to compute the maximum panda size by zip code. Once you specify the aggregates you want to compute, you can get the results back as a `DataFrame`.



If you're used to RDDs, you might be concerned by `groupBy`, but it is now, mostly,[8] a safe operation on `DataFrames` thanks to the Spark SQL optimizer, which automatically pipelines our reductions, avoiding giant shuffles and mega records.

8 You can still make it behave poorly by using `collect_list` or other aggregates with no reduction possibility.

Example 5-17. Compute the max panda size by zip code

```
def maxPandaSizePerZip(pandas: DataFrame): DataFrame = {
  pandas.groupBy(pandas("zip")).max("pandaSize")
}
```

While Example 5-17 computes the max on a per-key basis, these aggregates can also be applied over the entire `DataFrame` or all numeric columns in a `DataFrame`. This is often useful when trying to collect some summary statistics for the data with which you are working. In fact, there is a built-in `describe` transformation that does just that, although it can also be limited to certain columns, which is used in Example 5-18 and returns Example 5-19.

Example 5-18. Compute some common summary stats, including `count`, `mean`, `stddev`, and more, on the entire DataFrame

```
// Compute the count, mean, stddev, min, max summary stats for all
// of the numeric fields of the provided panda infos. non-numeric
// fields (such as string (name) or array types) are skipped.
val df = pandas.describe()
// Collect the summary back locally
println(df.collect())
```

Example 5-19. Result of `describe` and `collect` on some small sample data (note: summarizes all of the numeric fields)

```
Array([count,3,3], [mean,1.3333333333333333,5.0],
  [stddev,0.5773502691896258,4.358898943540674], [min,1,2], [max,2,10])
```

For computing multiple different aggregations, or more complex aggregations, you should use the `agg` API on the `GroupedData` instead of directly calling `count`, `mean`, or similar convenience functions. For the `agg` API,[9] you either supply a list of aggregate expressions, a string representing the aggregates, or a map of column names to aggregate function names. Once we've called `agg` with the requested aggregates, we get back a regular DataFrame with the aggregated results. As with regular functions, they are listed in the org.apache.spark.sql.functions Scaladoc. Table 5-7 lists some common and useful aggregates. For our example results in these tables, we will consider a `DataFrame` with the schema of a name field (as a string) and age (as an integer), both nullable with values (`{"ikea", null}, \{"tube", 6}, \{"real", 30}`). Example 5-20 shows how to compute both the min and mean for the `pandaSize` column in our running panda example.

9 The magic happens inside the `MutableAggregationBuffer` class.



Computing multiple aggregates with Spark SQL can be much simpler than doing the same tasks with the RDD API.

Example 5-20. Example aggregates using the agg API

```
def minMaxPandaSizePerZip(pandas: DataFrame): DataFrame = {
  pandas.groupBy(pandas("zip")).agg(min("pandaSize"), max("pandaSize"))
}
```

Table 5-7. Sample of Spark SQL aggregate functions for use with agg API

Function name	Purpose	Storage requirement	Input types	Example usage	Example result
`approxCountDistinct`	Count approximate distinct values in column[a]	Configurable through rsd (which controls error rate)	All	`df.agg(approxCountDistinct(df("age"), 0.001))`	2
`avg`	Average	Constant	Numeric	`df.agg(avg(df("age")))`	18
`count`	Count number of items (excluding nulls). Special case of "*" counts number of rows	Constant	All	`df.agg(count(df("age")))`	2
`countDistinct`	Count distinct values in column	O(distinct elems)	All	`df.agg(countDistinct(df("age")))`	2
`first`	Return the first element[b]	Constant	All	`df.agg(first(df("age")))`	6
`last`	Return the last element	Constant	All	`df.agg(last(df("age")))`	30
`stddev`	Sample standard deviation	Constant	Numeric	`df.agg(stddev(df("age")))`	16.97…
`stddev_pop`	Population standard deviation	Constant	Numeric	`df.agg(stddev_pop(df("age")))`	12.0
`sum`	Sum of the values	Constant	Numeric	`df.agg(sum(df("age")))`	36
`sumDistinct`	Sum of the distinct values	O(distinct elems)	Numeric	`df.agg(sumDistinct(df("age")))`	36

Function name	Purpose	Storage requirement	Input types	Example usage	Example result
`min`	Select the minimum value	Constant	Sortable data	`df.agg(min(df("age")))`	`5`
`max`	Select the maximum value	Constant	Sortable data	`df.agg(max(df("age")))`	`30`
`mean`	Select the mean value	Constant	Numeric	`df.agg(mean(df("age")))`	`18`

[a] Implemented with HyperLogLog: *https://oreil.ly/WDhlo*.

[b] This was commonly used in early versions of Spark SQL where the grouping column was not preserved.

Sometimes if you're building a collection that is too large, you'll get an exception about not being able to grow the buffer size. You can work around this in a few different ways; one is excluding the keys with too many entries with an anti join (see Chapter 6)—or using a different kind of aggregate that has a slower (or capped) growth.

Spark also supports "pivot" and "unpivot," converting rows to columns (or vice versa) as part of its aggregations. Functionally (and performance-wise), this is similar to performing a `collect` distinct of the pivot column for its values, then a `collect_list` or explode on the pivot column. Pivots are expensive in Spark in much the same way as `collect_list` can be.



In addition to using aggregates on `groupBy`, you can run the same aggregations on multidimensional cubes with `cube` and rollups with `rollup`.

If the built-in aggregation functions don't meet your needs, you can extend Spark SQL using UDFs as discussed in "Extending with User-Defined Functions (UDFs), Aggregate Functions (UDAFs), and Expressions" on page 83, although things can be more complicated for aggregate functions.

Additionally, in JVM languages, you can provide an arbitrary function and call `groupByKey` which gives back a `KeyValueGroupedDataset`, which offers a more functional API discussed in "Grouped Operations on Datasets" on page 82. This is often simpler but with worse performance than UDAFs.



You can turn an untyped `RelationalGroupedDataset` into a typed `KeyValueGroupedDataset` by calling `.as[KeyType, ValueType]`.

Windowing

Sometimes we don't want to aggregate all values of a key but rather see a sliding window over our data as ordered by some key(s) (often a time, date, or distance-based value). When creating a window, you specify what columns the window is over, the order of the rows within each partition/group, and the size of the window (e.g., K rows before and J rows after *or* a range between values). If it helps to think of this visually, Figure 5-2 shows a sample window and its results. Using this specification, each input row is related to some set of rows, called a frame, that is used to compute the resulting aggregate. Window functions can be very useful for things like computing average speed with noisy data, relative sales, and more. A window for pandas by age is shown in Example 5-21.

Example 5-21. Define a window on the +/-10 closest (by age) pandas in the same zip code

```
val windowSpec = Window
  .orderBy(pandas("age"))
  .partitionBy(pandas("zip"))
  .rowsBetween(
    start = -10,
    end = 10
  ) // can use rangeBetween for range instead
```

Once you've defined a window specification, you can compute a function over it, as shown in Example 5-22. Spark's existing aggregate functions, covered in "Aggregates, groupBy, and coGroup" on page 70, can be computed on an aggregation over the window. Window operations are very useful for things like measuring local volatility or Kalman filtering or many types of relative analysis.



You can create a window without a partition, but this results in shuffling all of the data onto a single machine. There are technical ways Spark could work around this (see Dask's `map_overlap`), but they are not yet implemented.

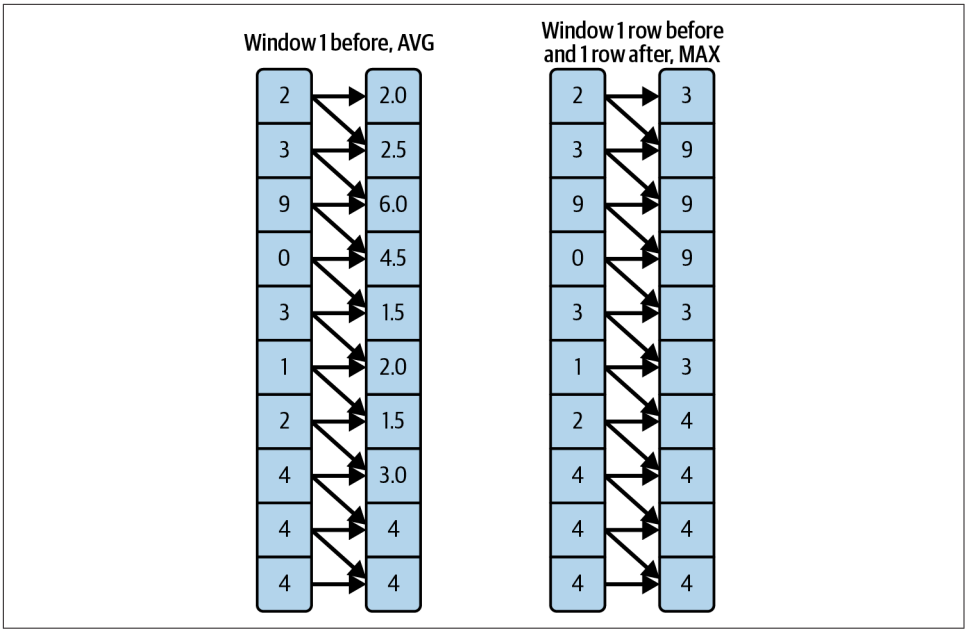


Figure 5-2. Spark SQL windowing

Example 5-22. Compute difference from the average using the window of +/-10 closest (by age) pandas in the same zip code

```
val pandaRelativeSizeCol = pandas("pandaSize") -
  avg(pandas("pandaSize")).over(windowSpec)

pandas.select(
  pandas("name"),
  pandas("zip"),
  pandas("pandaSize"),
  pandas("age"),
  pandaRelativeSizeCol.as("panda_relative_size")
)
```

Windowing in Spark depends on partitions for effective performance and it does not support cross-partition computation. Some other distributed systems, like Dask, allow for computing global windows while spilling values between partitions.[10]

10 Dask has a `map_overlap` primitive that it uses for this functionality.

Sorting

Sorting supports multiple columns in ascending or descending order, with ascending as the default. These sort orders can be intermixed, as shown in Example 5-23. Spark SQL has some extra benefits for sorting, as some serialized data can be compared without deserialization.

Example 5-23. Sort by panda age and size in opposite orders

```
pandas.orderBy(pandas("pandaSize").asc, pandas("age").desc)
```

When limiting results, sorting is often used to only bring back the top or bottom K results. When limiting, you specify the number of rows with `limit(numRows)` to restrict the number of rows in the `DataFrame`. Limits are also sometimes used for debugging without sorting to bring back a small result. If, instead of limiting the number of rows based on a sort order, you want to sample your data, "Sampling" on page 306 covers techniques for Spark SQL sampling as well.

Multi-DataFrame Transformations

Beyond single DataFrame transformations, you can perform operations that depend on multiple DataFrames. The ones that first pop into our heads are most likely the different types of joins, which are covered in Chapter 6, but beyond that, you can also perform a number of set-like operations between DataFrames.

The DataFrame set-like operations allow us to perform many operations that are most commonly thought of as set operations. These operations behave a bit differently than traditional set operations since we don't have the restriction of unique elements. While you are likely already familiar with the results of set-like operations from regular Spark and *Learning Spark*, it's important to review the cost of these operations in Table 5-8.

Table 5-8. Set operations

Operation name	Cost
`unionAll`	Low
`intersect`	Expensive
`except`	Expensive
`distinct`	Expensive

Plain Old SQL Queries and Interacting with Metastore Tables (Hive, Iceberg, etc.)

Sometimes, it's better to use regular SQL queries instead of building up our operations on DataFrames. If you are connected to a metastore, you can directly write SQL queries against the tables and get the results as a DataFrame. If you have a DataFrame you want to write SQL queries against, you can register it as a temporary table, as shown in Example 5-24 (or save it as a managed table if you intend to reuse it between jobs).

Example 5-24. Registering (or saving) and querying tables

```
pandas.registerTempTable("pandas")
val miniPandas = session.sql("SELECT * FROM pandas WHERE pandaSize < 12")
```

Querying a table is the same, regardless of whether it is a temporary table, existing table, or newly saved Spark table; just drop the `WHERE` clause from Example 5-24 or use `session.table`.

The newish `selectExpr` function gives you the ability to mix in some raw SQL while still keeping your code more DataFrame-like. Each parameter takes a SQL expression, although you can also use `expr("your sql here")` inside of a select for the same purpose.

In addition to registering tables, you can also write queries directly against a specific filepath, as shown in Example 5-25.

Example 5-25. Querying a raw file

```
def queryRawFile(): DataFrame = {
  sqlContext.sql("SELECT * FROM parquet.`path_to_parquet_file`")
}
```



You can query views in the same way, but the view will insert a query plan in its place. Different catalogs are in different phases of implementing materialized views, which generally behave like tables but have clear semantics to refresh.

Spark has a default built-in catalog (Derby), but you don't need to limit yourself to Spark's catalog. You can connect Spark to Unity, Polaris, Hive, or DeltaCatalog. Each catalog has its own documentation on how to connect, and note that some of them use Spark's extension APIs to register additional functionality into Spark.

Data Representation in DataFrames and Datasets

DataFrames are more than RDDs of `Row` objects; DataFrames and Datasets have a specialized representation and columnar cache format. The specialized representation is not only more space efficient but also can be much faster to encode than even Kryo serialization. To be clear, like RDDs, DataFrames and Datasets are generally lazily evaluated and build up a lineage of their dependencies (except in DataFrames, where this is called a logical plan and contains more information).

Tungsten

Tungsten is a Spark SQL component that provides more efficient serialization and a few select operations by working directly at the byte level. Looking back on Figure 5-1, we can take a closer look at the space differences between the RDDs and DataFrames when cached in Figure 5-3. Tungsten includes specialized in-memory data structures tuned for the types of operations required by Spark, improved code generation, and a specialized wire protocol.

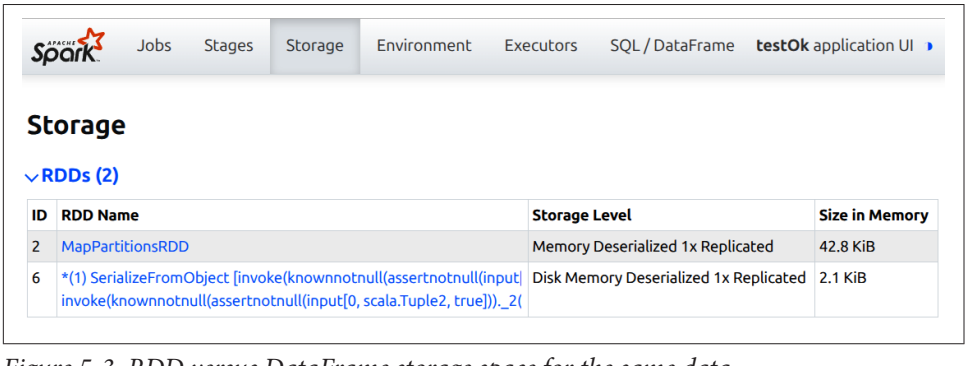


Figure 5-3. RDD versus DataFrame storage space for the same data



For those coming from Hadoop, you can think of Tungsten data types as being `WritableComparable` types on steroids.

Tungsten's representation is substantially smaller than objects serialized using Java or even Kryo serializers. As Tungsten does not depend on Java objects, both on-heap and off-heap allocations are supported. Not only is the format more compact, but serialization times can be substantially faster than with native serialization.



Since Tungsten no longer depends on working with Java objects, you can use either on-heap (in the JVM) or off-heap storage. If you use off-heap storage, it is important to leave enough room in your containers for the off-heap allocations, which you can get an approximate idea for from the web UI.

Tungsten's data structures are also created closely in mind with the kind of processing for which they are used. The classic example of this is with sorting, a common and expensive operation. The in-wire representation is implemented so that sorting can be done without having to deserialize the data again.

By avoiding the memory and GC overhead of regular Java objects, Tungsten is able to process larger Datasets than the same handwritten aggregations.



Some Spark accelerators use Apache Arrow in a similar way to how Tungsten is used in "regular" Spark.

Datasets

Datasets are an exciting extension of Spark SQL that provide additional compile-time type checking. DataFrames are now a specialized version of Datasets that operate on generic `Row` objects and therefore lack the normal compile-time type checking of Datasets. Datasets can be used when your data can be encoded for Spark SQL and you know the type information at compile time. The Dataset API is a strongly typed collection with a mixture of relational (DataFrame) and functional (RDD) transformations. Like DataFrames, Datasets are represented by a logical plan the Catalyst optimizer (see "Query Optimizer" on page 102) can work with, and when cached, the data is stored in Spark SQL's internal encoding format.

While it can be very convenient, using the functional APIs provided by Datasets greatly limits the optimizer. While the types in Datasets can be useful, we generally recommend using the APIs, which are available on DataFrames, along with UDFs/UDAFs when you need to use custom code.

Interoperability with RDDs, DataFrames, and Local Collections

Datasets can be easily converted to/from RDDs; they do not directly extend them. Converting to/from RDDs involves encoding/decoding the data into a different form. Converting to/from DataFrames is almost "free" in that the underlying data does not need to be changed; only extra compile-time type information is added/removed.



The type alias for `DataFrame` is not visible in Java for Spark, so you need to use `Dataset<Row>` in place of `DataFrame`.

To convert a DataFrame to a Dataset, you can use the `as[ElementType]` function on the `DataFrame` to get a `Dataset[ElementType]` back as shown in Example 5-26. The `ElementType` must be a case class or similar, such as a tuple, consisting of types Spark SQL can represent (see "Basics of Schemas" on page 60). To create `Datasets` from local collections, `createDataSet(...)` on the `SQLContext` and the `toDS()` implicit functions are provided on Seqs in the same manner as `createDataFrame(...)` and `toDF()`. For converting from RDD to Dataset, you can first convert from RDD to DataFrame and then convert it to a `Dataset`.



For loading data into a Dataset, unless a special API is provided by your data source, you can first load your data into a DataFrame and then convert it to a Dataset. Since the conversion to the `Dataset` simply adds information, you do not have the problem of eagerly evaluating, and filters and similar operations can still be pushed down to the data store.

Example 5-26. Create a Dataset from a DataFrame

```
def fromDF(df: DataFrame): Dataset[RawPanda] = {
  df.as[RawPanda]
}
```

Converting from a Dataset back to an RDD or DataFrame can be done in similar ways as when converting DataFrames, and both are shown in Example 5-27. The `toDF` simply copies the logical plan used in the Dataset into a DataFrame—so you don't need to do any schema inference or conversion as you do when converting from RDDs. Converting a Dataset of type `T` to an RDD of type `T` can be done by calling `.rdd`, which, unlike calling `toDF`, does involve converting the data from the internal SQL format to the regular types.

Example 5-27. Convert Dataset to DataFrame and RDD

```
/** Illustrate converting a Dataset to an RDD
  */
def toRDD(ds: Dataset[RawPanda]): RDD[RawPanda] = {
  ds.rdd
}

/** Illustrate converting a Dataset to a DataFrame
```

```
  */
def toDF(ds: Dataset[RawPanda]): DataFrame = {
  ds.toDF()
}
```

Converting your Dataset to an RDD can be a way to introduce a "cut" in the optimizer and keep it from reordering anything between that barrier (note: going to/from a DataFrame is not enough for this).

Compile-Time Strong Typing

One of the reasons to use Datasets over traditional DataFrames is their compile-time strong typing. DataFrames have runtime schema information but lack compile-time information about the schema. This strong typing is especially useful when making libraries, because you can more clearly specify the requirements of your inputs and your return types.

Easier Functional (RDD "like") Transformations

One of the key advantages of the Dataset API is easier integration with custom Scala and Java code. `Datasets` expose `filter`, `map`, `mapPartitions`, and `flatMap` with similar function signatures as RDDs, with the notable requirement that your return `ElementType` also be understandable by Spark SQL (such as a tuple or case class of types discussed in "Basics of Schemas" on page 60). Example 5-28 illustrates this using a simple `map` function.

Example 5-28. Functional query on Dataset

```
def funMap(ds: Dataset[RawPanda]): Dataset[Double] = {
  ds.map { rp => rp.attributes.filter(_ > 0).sum }
}
```

Beyond functional transformations, such as `map` and `filter`, you can also intermix relational and grouped/aggregate operations.

Relational Transformations

`Datasets` introduce a typed version of `select` for relational-style transformations. When specifying an expression for this you need to include the type information, as shown in Example 5-29. You can add this information by calling `as[ReturnType]` on the expression/column.

Example 5-29. Simple relational select on Dataset

```
def squishyPandas(ds: Dataset[RawPanda]): Dataset[(Long, Boolean)] = {
  ds.select($"id".as[Long], ($"attributes" (0) > 0.5).as[Boolean])
}
```



Some operations, such as `select`, have both typed and untyped implementations. If you supply a `Column` rather than a `TypedColumn` (e.g., you forget to write `.as[]`), you will get a `DataFrame` back instead of a `Dataset`.

Multi-Dataset Relational Transformations

In addition to single Dataset transformations, there are also transformations for working with multiple Datasets. The standard set operations, namely `intersect`, `union`, and `subtract`, are all available with the same standard caveats as discussed in Table 5-8. Joining Datasets is also supported, but to make the type information easier to work with, the return structure is a bit different than traditional SQL joins.

Grouped Operations on Datasets

Similar to grouped operations on DataFrames (described in "Aggregates, groupBy, and coGroup" on page 70), `groupBy` returns a `RelationalGroupedDataset` when grouped by a column or relational expression or a `KeyValueGroupedDataset` when grouped with a JVM arbitrary function (not an option for Python). You can specify your aggregate functions on all of these, along with a functional mapGroups API. As with the expression in "Relational Transformations" on page 81, you need to use typed expressions so the result can also be a `Dataset`.

Taking our previous example of computing the maximum panda size by zip code in Example 5-17, you would rewrite it to be as shown in Example 5-30.



The convenience functions found on `GroupedData` (e.g., `min`, `max`, etc.) are missing, so all of our aggregate expressions need to be specified through `agg`.

Example 5-30. Compute the max panda size per zip code typed

```
def maxPandaSizePerZip(ds: Dataset[RawPanda]): Dataset[(String, Double)] = {
  ds.map(rp => MiniPandaInfo(rp.zip, rp.attributes(2)))
    .groupByKey(mp => mp.zip)
    .agg(max("size").as[Double])
}
```

Beyond applying typed SQL expressions to aggregated columns, you can also easily use arbitrary Scala code with `mapGroups` on grouped data as shown in Example 5-31. This can save us from having to write custom user-defined aggregate functions (UDAFs) (discussed in "Extending with User-Defined Functions (UDFs), Aggregate Functions (UDAFs), and Expressions" on page 83). However, these generally perform worse than UDAFs.

Example 5-31. Compute the max panda size per zip code using map groups

```
def maxPandaSizePerZipScala(
    ds: Dataset[RawPanda]
): Dataset[(String, Double)] = {
  def groupMapFun(g: String, iter: Iterator[RawPanda]): (String, Double) = {
    (g, iter.map(_.attributes(2)).reduceLeft(Math.max(_, _)))
  }
  ds.groupByKey(rp => rp.zip).mapGroups(groupMapFun)
}
```

While custom UDAFs can be painful to write, they may be able to give better performance than `mapGroups` and can also be used on DataFrames. `mapGroups` requires that all of the data for a single key be shuffled to a single node and does not support any kind of partial aggregation that UDAFs can provide. In the next section you'll see how to build UDAFs for better performance.

Extending with User-Defined Functions (UDFs), Aggregate Functions (UDAFs), and Expressions

User-defined functions and user-defined aggregate functions provide you with ways to extend the DataFrame and SQL APIs with your own custom code while keeping the Catalyst optimizer. The Dataset API (see "Datasets" on page 79) is another performant option for much of what you can do with UDFs and UDAFs. This is quite useful for performance, since otherwise you would need to convert the data to an RDD (and potentially back again) to perform arbitrary functions, which is quite expensive. UDFs and UDAFs can also be accessed from inside of regular SQL expressions, making them accessible to analysts or others more comfortable with SQL.



When using UDFs or UDAFs written in non-JVM languages, such as Python, it is important to note that you lose much of the performance benefit, as the data must still be transferred out of the JVM.



If most of your work is in Python but you want to access some UDFs without the performance penalty, you can write your UDFs in Scala and register them for use in Python (as done in Sparkling-Pandas) or using SparkExtensions to inject your function[s].[11]

Writing nonaggregate UDFs for Spark SQL is incredibly simple: you simply write a regular function and register it using `sqlContext.udf().register`. A simple string length UDF is shown in Example 5-32. If you are registering a Java or Python UDF, you also need to specify your return type.

Example 5-32. String length UDF

```
def setupUDFs(sqlCtx: SQLContext) = {
  sqlCtx.udf.register("strLen", (s: String) => s.length())
}
```



Even with JVM languages, UDFs are generally slower than the equivalent SQL expression would be if it exists.[12]

Aggregate functions (UDAFs) are somewhat trickier to write. Instead of writing a regular Scala function, you extend the `UserDefinedAggregateFunction` and implement a number of different functions, similar to the functions one might write for `aggregateByKey` on an RDD, except working with different data structures. While they can be complex to write, UDAFs can be quite performant compared with options like `mapGroups` on `Datasets` or even a nonoptimizer `aggregateByKey` on RDDs. You can then either use the UDAF directly on columns or add it to the function registry as you did for the regular UDF.

Python user-defined aggregate functions do not currently support partial aggregation and instead require all of the data for one key to be on one machine. As such, we generally don't recommend implementing UDAFs in Python.

Example 5-33 is a simple UDAF for computing the average, although you will likely want to use Spark's built-in `avg` in real life.

11 As of this writing, the SparklingPandas project development is on hold, but early releases still contain some interesting examples of using JVM code from Python.

12 Some early work was being done in SPARK-14083 to parse JVM byte code and generate SQL expressions, but this has been largely abandoned.

Example 5-33. UDAF for computing the average

```
def setupUDAFs(sqlCtx: SQLContext) = {
  class Avg extends UserDefinedAggregateFunction {
    // Input type
    def inputSchema: org.apache.spark.sql.types.StructType =
      StructType(StructField("value", DoubleType) :: Nil)

    def bufferSchema: StructType = StructType(
      StructField("count", LongType) ::
        StructField("sum", DoubleType) :: Nil
    )

    // Return type
    def dataType: DataType = DoubleType

    def deterministic: Boolean = true

    def initialize(buffer: MutableAggregationBuffer): Unit = {
      buffer(0) = 0L
      buffer(1) = 0.0
    }

    def update(buffer: MutableAggregationBuffer, input: Row): Unit = {
      buffer(0) = buffer.getAs[Long](0) + 1
      buffer(1) = buffer.getAs[Double](1) + input.getAs[Double](0)
    }

    def merge(buffer1: MutableAggregationBuffer, buffer2: Row): Unit = {
      buffer1(0) = buffer1.getAs[Long](0) + buffer2.getAs[Long](0)
      buffer1(1) = buffer1.getAs[Double](1) + buffer2.getAs[Double](1)
    }

    def evaluate(buffer: Row): Any = {
      buffer.getDouble(1) / buffer.getLong(0)
    }
  }
  // Optionally register
  val avg = new Avg
  sqlCtx.udf.register("ourAvg", avg)
}
```

This is a little more complicated than our regular UDF, so let's take a look at what the different parts do. You start by specifying what the input type is, then you specify the schema of the buffer you will use for storing the in-progress work. These schemas are specified in the same way as DataFrame and Dataset schemas, discussed in "Basics of Schemas" on page 60.

From there, the rest of the functions are implementing the same functions you use when writing `aggregateByKey` on an RDD, but instead of taking arbitrary Scala

objects, you work with `Row` and `MutableAggregationBuffer`. The final `evaluate` function takes the `Row` representing the aggregation data and returns the final result.

In addition to this rather direct API of working with raw buffers, Spark 3 added a newer, more strongly typed API for aggregates. In this version you still specify the same core functions, but instead of having to deal with buffer elements, you specify the serializer for Spark to use and can use higher-level objects with reduce and merge. An example of this, which collects a unique set of elements up to some limit, is shown in Example 5-34.

Example 5-34. A UDAF collect, which limits how many elements it returns

```
package com.holdenkarau.spark.misc.utils

import scala.collection.mutable.HashSet

import org.apache.spark.sql.expressions.Aggregator
import org.apache.spark.sql.{Encoder, Encoders}

class LimitedCollectSetUDAF(maxElems: Int = 5000) extends Aggregator[Int, Hash
Set[Int], List[Int]] {

  def zero: HashSet[Int] = HashSet.empty[Int]
  def reduce(buffer: HashSet[Int], elem: Int): HashSet[Int] = {
    println(s"Adding $elem with current buffer $buffer")
    if (buffer.size < maxElems) {
      buffer += elem
    }
    println(s"Got buffer $buffer")
    buffer
  }

  def merge(a1: HashSet[Int], a2: HashSet[Int]): HashSet[Int] = {
    println(s"Merging $a1 $a2")
    if (a2.size == maxElems) {
      a2
    } else {
      val elemItr = a2.iterator
      while (a1.size < maxElems && elemItr.hasNext) {
        a1 += elemItr.next()
      }
      a1
    }
  }

  override def finish(a: HashSet[Int]): List[Int] = {
    println(s"Finished $a")
    a.toList
  }
```

```
  override def bufferEncoder: Encoder[HashSet[Int]] = Encoders.kryo[HashSet[Int]]

  // This encoder is "tricky" if we pick the kyro encoder we get "junk" results.
  override def outputEncoder: Encoder[List[Int]] = Encoders.product[List[Int]]
}
```

If you've got especially expensive or complicated logic, this may not be enough, and you may wish to be able to customize code generation.

Custom Expressions

Custom expressions let us go beyond Spark's UDFs' performance at the cost of an unstable API and the difficulty of manipulating raw strings of Java code. User-defined functions generally don't have quite the same performance as Spark's "built-in" operators. This often comes from code generation but additionally can come from Spark having optimizers for specific/common patterns. Custom expressions are only available in the JVM. You can implement your own expression with code generation to make that gap smaller.



Spark's expression and especially its code generation APIs are not intended to be a public interface. It's very much an experts-only choice and is likely to require updating between Spark versions.

The expression abstract base class has a few unimplemented functions, and you'll often want to mix-in some additional traits to implement some of those for you. You'll most likely always need to implement `dataType` and almost always want to implement `doGenCode` as well as a non-codegen eval (which one depends on what you mix in). You can use the `CodegenFallback` if you don't want to implement codegen, but in that case UDFs are often "good enough." Depending on your inputs, you'll want to mix in one of `LeafExpression` (no inputs), `UnaryExpression` (one input), `BinaryExpression` (two inputs), etc. If you can't handle nulls, you can mix in `NullIntolerant` and override `eval` instead of `nullSafeEval`.

The simplest kind of custom expression is a `UnaryExpression` with a "simple" return type. In this case you implement `nullSafeEval` for a non-code-generated version of your expression. `nullSafeEval` is not strongly typed (it takes and returns `Any`), so you'll need to manually check if the input type matches. Similarly for the output, instead of changing the return type of `nullSafeEval`, you define a `dataType` function that returns a Spark SQL data type. Having a correct `nullSafeEval` is important since not every stage will necessarily be code generated (it also can be great for testing). For maximum performance, you'll want to implement code generation as well by overriding `doGenCode`.

Spark SQL's code generation is done by generating Java strings and wrapping them inside a special `ExprCode` object. For `UnaryExpressions` there is a fairly straightforward way to get the Java type of your return value `CodeGenerator.javaType`, get the input value (output of previous expression), and set the result for the next expression.[13] A trivial unary expression with a code generation example is shown in Example 5-35.

Example 5-35. A sample custom expression

```
case class WorkingCodegenExpression(child: Expression) extends UnaryExpression {
  override def prettyName = "workingCodegen"

  override def nullSafeEval(input: Any): Any = {
    if (input == null) {
      return null
    }
    return 1
  }

  override def doGenCode(ctx: CodegenContext, ev: ExprCode): ExprCode = {
    // Code to serialize.
    val input = child.genCode(ctx)
    val javaType = CodeGenerator.javaType(dataType)
    val code = input.code + code"""
        final $javaType ${ev.value} =
          ${input.isNull} ? ${CodeGenerator.defaultValue(dataType)} : 1;
     """
    ev.copy(code = code, isNull = input.isNull)
  }

  override def dataType: DataType = IntegerType

  // New in 3.2
  def withNewChildInternal(newChild: Expression) = {
    copy(child = newChild)
  }
}
```

At runtime the code is compiled with Janino. In some ways this is easier than Scala's macro codegen tools since Java is already well known by most developers, but combining raw strings of Java code can be error-prone. To help reduce some of the common causes of errors, like reusing a variable name in combined code segments, Spark has a `CodeGenContext`, which has many methods like `addBuffered`

13 You might be thinking, "That's a strange way to say *return*," and that's because (most of the time) inside of the code generation you don't return. Instead, multiple expressions are put together in one function to reduce function-calling overhead.

`State`, `addNewFunction`, and so on. Similarly, the `CodeGenerator` also has utils for mutating complex Spark objects (arrays, etc.). When it comes time to debug your code generation, the `explain` method on Datasets can take `codegen` as one of the modes and will print out the generated code for you.

When figuring out how to write custom expressions, your best guide is likely looking at Spark's own expressions, most of which are in the *sql/core/src/main/scala/org/apache/spark/sql/execution* directory. You might also want to take a look at the bebe project by MrPowers or spark-misc-utils by Holden, which are two of the few external OSS implementations of Spark expressions.



Testing code generation can be complicated; however, the spark-testing-base project (covered more in Chapter 11) has three built-in primitives to test custom expressions, namely `testNonCodegen`, `testCodegenOnly`, and `testCombined`, which runs both codegen and non-codegen for the same test case.

Custom expressions, UDFs, UDAFs, and Dataset's functional interfaces all provide ways to intermix arbitrary code with Spark SQL. Sometimes what you want to customize is not how your operations are evaluated but how the data is stored; this is especially common in ML and scientific computing use cases.

Custom Encoders

Custom encoders are very similar to custom expressions in terms of how you implement your code and in many ways are just a special case of `UnaryExpression`. You may wish to consider using custom encoders if you have sparse data or want to use "nonstandard" objects in Spark SQL. Implementing custom encoders is a substantial amount of work, and as with custom expressions, the API is not guaranteed to be stable even between minor versions of Spark.

Assuming that you have created a custom expression that encodes your data, you can then create a Scala implicit so that Spark will use your encoder in place of its internal encoders. For example, to use Holden's compressed binary AltEncoder on arrays, you would add `implicit def encode[Array[Dataset]] = AltEncoder.COMPRESSED_BINARY`.

User-Defined Table Functions

User-defined table functions (UDTFs) are Python's way of generating tables. Generally, you can get better scalable performance by creating a regular UDF and using that over a parallelized range with `mapPartitions`.

Data Loading and Saving Functions

Spark SQL has a different way of loading and saving data than core Spark. To be able to push down certain types of operations to the storage layer, Spark SQL has its own Data Source API. Data sources are able to specify and control which type of operations should be pushed down to the data source. As developers, you don't need to worry too much about the internal activity going on here, unless the data sources you are looking for are not supported.



Data loading in Spark SQL is not quite as lazy as in regular Spark but is still generally lazy. You can verify this by quickly trying to load from a data source that doesn't exist.

DataFrameWriter and DataFrameReader

The DataFrameWriter and the DataFrameReader classes cover writing and reading from external data sources. The `DataFrameWriter` is accessed by calling `write` on a DataFrame or Dataset. The `DataFrameReader` can be accessed through `read` on a `SQLContext`.

Formats

When reading or writing, you specify the format by calling `format(formatName)` on the `DataFrameWriter/DataFrameReader`. Format-specific parameters, such as number of records to be sampled for JSON, are specified by either providing a map of `options` with options or setting option-by-option with `option` on the reader/writer.



The first-party formats `JSON`, `JDBC`, `ORC`, and `Parquet` methods are directly defined on the reader/writers taking the path or connection info. These methods are for convenience only and are wrappers around the more general methods we illustrate in this chapter.

JSON

Loading and writing JSON is supported directly in Spark SQL, and despite the lack of schema information in JSON, Spark SQL is able to infer a schema for us by sampling the records. Loading JSON data is more expensive than loading many data sources, since Spark needs to read some of the records to determine the schema information. If the schema between records varies widely (or the number of records is very small), you can increase the percentage of records read to determine the schema by setting `samplingRatio` to a higher value, as in Example 5-36 where we set the sample ratio to `100%`.

Example 5-36. Load JSON data, using all (100%) of records to determine the schema

```
val df2 = session.read
  .format("json")
  .option("samplingRatio", "1.0")
  .load(path)
```



Spark's schema inference can be a compelling reason to use Spark for processing JSON data, even if the data size could be handled on a single node.

Since our input may contain some invalid JSON records we may wish to filter out, we can also take in an RDD of strings. This allows us to load the input as a standard text file, filter out our invalid records, and then load the data into JSON. This is done by using the built-in `json` function on the `DataFrameReader`, which takes RDDs or paths and is shown in Example 5-37. Methods for converting RDDs of regular objects are covered in "RDDs" on page 95.

Example 5-37. `jsonRDD` load

```
val df3 = session.read.json(session.createDataset(jsonRDD)(Encoders.STRING))
```

JDBC

The JDBC data source represents a natural Spark SQL data source, one that supports many of the same operations. Since different database vendors have slightly different JDBC implementations, you need to add the JAR for your JDBC data sources. Since SQL field types vary as well, Spark uses `JdbcDialects` with built-in dialects for DB2, Derby, MsSQL, MySQL, Oracle, and Postgres.[14] To achieve parallelism with reading from JDBC, Spark requires that a partition column as well as a MIN and MAX range be configured so that Spark can split up which rows each worker is reading.



Spark's approach to parallelism data loads for JDBC data sources does not play well with live datasets that are being modified while a Spark job is running.

While Spark supports many different JDBC sources, it does not ship with the JARs required to talk to all of these databases. If you are submitting your Spark job with

14 Some types may not be correctly implemented for all databases.

`spark-submit`, you can download the required JARs to the host you are launching and include them by specifying `--jars` or supplying the Maven coordinates to `--packages`. Since the Spark shell is also launched this way, the same syntax works, and you can use it to include the MySQL JDBC JAR in Example 5-38.

Example 5-38. Include MySQL JDBC JAR

```
spark-submit --jars ./resources/mysql-connector-java-5.1.38.jar $ASSEMBLY_JAR $CLASS
```

JdbcDialects allow Spark to correctly map the JDBC types to the corresponding Spark SQL types. If there isn't a `JdbcDialect` for your database vendor, the default dialect will be used, which will likely work for many of the types. The dialect is automatically chosen based on the JDBC URL used.



If you find yourself needing to customize the `JdbcDialect` for your database vendor, you can look for a package or `spark-packages` or extend the `JdbcDialect` class and register your own dialect.

As with the other built-in data sources, there exists a convenience wrapper for specifying the properties required to load `JDBC` data, illustrated in Example 5-39. The convenience wrapper JDBC accepts the URL, table, and a `java.util.Properties` object for connection properties (such as authentication information). The properties object is merged with the properties that are set on the reader/writer itself. While the properties object is required, an empty properties object can be provided, and properties instead specified on the reader/writer.

Example 5-39. Create a DataFrame from a JDBC data source

```
session.read
  .format("jdbc")
  .option("url", "jdbc:dialect:serverName")
  .option("dbtable", "table")
  .load()
```

The API for saving a DataFrame is very similar to the API used for loading. The `save()` function needs no path since the information is already specified, as illustrated in Example 5-40, just as with loading.

Example 5-40. Write a DataFrame to a JDBC data source

```
df.write
  .format("jdbc")
  .option("url", "jdbc:dialect:serverName")
```

```
  .option("user", "user")
  .option("password", "pass")
  .option("dbtable", "table")
  .save()
```

In addition to reading and writing JDBC data sources, Spark SQL can also run its own JDBC server (covered in "JDBC/ODBC Server" on page 108).

Parquet

Apache Parquet files are a common format directly supported in Spark SQL, and they are incredibly space-efficient and popular. Apache Parquet's popularity comes from a number of features, including the ability to easily split across multiple files, compression, nested types, and many others discussed in the Parquet documentation. Since Parquet is such a popular format, there are some additional options available in Spark for the reading and writing of Parquet files. These options are listed in Table 5-9. Unlike third-party data sources, these options are mostly configured on the `SQLContext`, although some can be configured on either the `SQLContext` or `DataFrameReader/Writer`.

Table 5-9. Parquet data source options

SQLConf	Default	Purpose
`spark.sql.parquet.mergeSchema`	False	Control if schema should be merged between partitions when reading. Can be expensive, so disabled by default in 1.5.0.
`spark.sql.parquet.binaryAsString`	False	Treat binary data as strings. Old versions of Spark wrote strings as binary data.
`spark.sql.parquet.cacheMetadata`	True	Cache Parquet metadata, normally safe unless underlying data is being modified by another process.
`spark.sql.parquet.compression.codec`	Gzip	Specify the compression codec for use with Parquet data. Valid options are none, uncompressed, snappy, gzip, lzo, brotli, lz4, and zstd.
`spark.sql.parquet.filterPushdown`	True	Push down filters to Parquet (when possible).[a]
`spark.sql.parquet.writeLegacyFormat`	False	Write Parquet metadata in the legacy format.
`spark.sql.parquet.output.committer.class`	`org.apache.parquet.hadoop.ParquetOutputCommitter`	Output committer used by Parquet. If writing to Amazon S3 you may wish to try `org.apache.spark.sql.parquet.DirectParquetOutputCommitter`.

[a] *Push down* means evaluate at the storage, so with Parquet this can often mean skipping reading unnecessary rows or files.

Reading Parquet from an old version of Spark requires some special options, as shown in Example 5-41.

Example 5-41. Read Parquet file written by an old version of Spark

```
def loadParquet(path: String): DataFrame = {
  // Configure Spark to read binary data as string,
  // note: must be configured on session.
  session.conf.set("spark.sql.parquet.binaryAsString", "true")

  // Load parquet data using merge schema (configured through option)
  session.read
    .option("mergeSchema", "true")
    .format("parquet")
    .load(path)
}
```

Writing Parquet with the default options is quite simple, as shown in Example 5-42.

Example 5-42. Write Parquet file with default options

```
def writeParquet(df: DataFrame, path: String) = {
  df.write.format("parquet").save(path)
}
```

Metastore tables

Interacting with metastore tables adds another option beyond the other formats. As covered in "Plain Old SQL Queries and Interacting with Metastore Tables (Hive, Iceberg, etc.)" on page 77, one option for bringing in data from a Hive table is writing a SQL query against it and having the result as a DataFrame. The DataFrame's reader and writer interfaces can also be used with metastore tables, as with the rest of the data sources, as illustrated in Example 5-43.

Example 5-43. Load a table (same for Hive, Iceberg, etc.)

```
def loadHiveTable(): DataFrame = {
  session.read.table("pandas")
}
```

Here we're loading a Hive table, but you could follow the same path for an Iceberg table. You can also connect Spark to multiple metastores, which is super convenient when you need to work with legacy data.

Metastores, like Iceberg and Hive, take over some of the FileScan planning and can often produce more efficient reads. This normally comes from better filter pushdowns.



When loading a Hive table, Spark SQL will convert the metadata and cache the result. If the underlying metadata has changed, you can use `sqlContext.refreshTable("tablename")` to update the metadata, or the caching can be disabled by setting `spark.sql.parquet.cacheMetadata` to `false`.

Saving a managed table is a bit different and is illustrated in Example 5-44.

Example 5-44. Write managed table

```
def saveManagedTable(df: DataFrame): Unit = {
  df.write.saveAsTable("pandas")
}
```

Spark's adaptive query execution (AQE) may use information from the managed table to try and optimize your query. This is most often good, but not always, which we'll cover more about in "Understanding Spark Adaptive Query Execution" on page 104.

RDDs

Spark SQL DataFrames can easily be converted to RDDs of `Row` objects and can also be created from RDDs of `Row` objects as well as JavaBeans, Scala case classes, and tuples. For RDDs of strings in JSON format, you can use the methods discussed in "JSON" on page 90. Datasets of type T can also easily be converted to RDDs of type T, which can provide a useful bridge for DataFrames to RDDs of concrete case classes instead of `Row` objects. RDDs are a special-case data source, since when going to/from RDDs, the data remains inside of Spark without writing out to or reading from an external system.



Converting a DataFrame to an RDD is a transformation (not an action); however, converting an RDD to a DataFrame or Dataset may involve computing (or sampling some of) the input RDD, which makes it a hybrid action/transformation like RDD sorts.

When you create a DataFrame from an RDD, Spark SQL needs to add schema information. If you are creating the DataFrame from an RDD of case classes or plain old Java objects (POJOs), Spark SQL is able to use reflection to automatically determine the schema, as shown in Example 5-45. You can also manually specify the schema for your data using the structure discussed in "Basics of Schemas" on page 60. This can be especially useful if some of your fields are not nullable. You must specify the schema yourself if Spark SQL is unable to determine the schema through reflection, such as an RDD of `Row` objects (perhaps from calling `.rdd` on a `DataFrame` to use a functional transformation, as shown in Example 5-45).



Creating a DataFrame from an RDD is not free in the general case. The data must be converted into Spark SQL's internal format.

Example 5-45. Creating DataFrames from RDDs

```
def createFromCaseClassRDD(input: RDD[PandaPlace]) = {
  // Create DataFrame explicitly using session and schema inference
  val df1 = session.createDataFrame(input)

  // Create DataFrame using session implicits and schema inference
  val df2 = input.toDF()

  // Create a Row RDD from our RDD of case classes
  val rowRDD = input.map(pm =>
    Row(
      pm.name,
      pm.pandas.map(pi => Row(pi.id, pi.zip, pi.happy, pi.attributes))
    )
  )

  val pandasType = ArrayType(
    StructType(
      List(
        StructField("id", LongType, true),
        StructField("zip", StringType, true),
        StructField("happy", BooleanType, true),
        StructField("attributes", ArrayType(FloatType), true)
      )
    )
  )

  // Create DataFrame explicitly with specified schema
  val schema = StructType(
    List(
      StructField("name", StringType, true),
      StructField("pandas", pandasType)
    )
  )

  val df3 = session.createDataFrame(rowRDD, schema)
}
```



Case classes or JavaBeans defined inside another class can sometimes cause problems. If your RDD conversion is failing, make sure the case class being used isn't defined inside another class.

Converting a DataFrame to an RDD is incredibly simple; however, you get an RDD of `Row` objects, as shown in Example 5-46. Since a row can contain anything, you need to specify the type (or cast the result) as you fetch the values for each column in the row. With `Datasets`, you can directly get back an RDD templated on the same type, which can make the conversion back to a useful RDD much simpler.



While Scala has many implicit conversions for different numeric types, these do not generally apply in Spark SQL; instead, we use explicit casting.

Example 5-46. Manually convert a DataFrame

```
def toRDD(input: DataFrame): RDD[RawPanda] = {
  val rdd: RDD[Row] = input.rdd
  rdd.map(row =>
    RawPanda(
      row.getAs[Long](0),
      row.getAs[String](1),
      row.getAs[String](2),
      row.getAs[Boolean](3),
      row.getAs[Array[Double]](4)
    )
  )
}
```



If you know that the schema of your DataFrame matches that of another, you can use the existing schema when constructing your new DataFrame. One common place where this occurs is when an input DataFrame has been converted to an RDD for functional filtering and then back.

Local collections

Much like with RDDs, you can also create DataFrames from local collections and bring them back as local collections, as illustrated in Example 5-47. The same memory requirements apply; namely, the entire contents of the DataFrame will be in-memory in the driver program. As such, distributing local collections is normally limited to unit tests or joining small Datasets with larger distributed Datasets.

Example 5-47. Creating from a local collection

```
def createFromLocal(input: Seq[PandaPlace]) = {
  session.createDataFrame(input)
}
```



The `LocalRelation`'s API we used here allows us to specify a schema in the same manner as when we are converting an RDD to a DataFrame.

Collecting data back as a local collection is more common and often done postaggregation or filtering on the data. For example, with ML pipelines collecting the coefficients or (as discussed in our Goldilocks example in Chapter 8) collecting the quantiles to the driver. Example 5-48 shows how to collect a DataFrame back locally. For larger Datasets, saving to an external storage system (such as an object store, database, or HDFS) is recommended.



Just as with RDDs, do not collect large DataFrames back to the driver. For Python users, it is important to remember that `toPandas()` collects the data locally.

Example 5-48. Collecting the result locally

```
def collectDF(df: DataFrame) = {
  val result: Array[Row] = df.collect()
  result
}
```

Additional formats and tables

As with core Spark, the data formats that ship directly with Spark only begin to scratch the surface of the types of systems with which you can interact. Some vendors publish their own implementations, and some are published on Spark Packages. As of this writing there are over twenty formats listed on the Data Source's page, with some of the most popular being Avro, Redshift, and a unified wrapper around 6+ databases called deep-spark. In addition to the file formats, additional tables like Iceberg and Delta Lake can also be added in a similar manner but often require specifying Spark extensions as well as loading the package.

Once you've included the package with your Spark job, you need to specify the file or table format, as you did with the Spark-provided ones. The name should be mentioned in the package's documentation.

Going beyond existing options

There are a few options if the data format you are looking for isn't directly supported in either Spark or one of the libraries. Since many formats are available as Hadoop input formats, you can try to load your data as a Hadoop input format and convert

the resulting RDD as discussed in "RDDs" on page 95. This approach is relatively simple but means Spark SQL is unable to push down operations to our data store.[15]

For a deeper integration, you can implement your data source using the Data Source API. Depending on which operations you wish to support operator pushdown for, in your base relation you will need to implement additional traits from the `org.apache.spark.sql.sources` package. The details of implementing a new Spark SQL data source are beyond the scope of this book, but if you are interested, the Scaladoc for `org.apache.spark.sql.sources` and `spark-csv`'s CsvRelation can be good ways to get started.

Save Modes

In core Spark, saving RDDs always requires that the target directory does not exist, which can make appending to existing tables challenging. With Spark SQL, you can specify the desired behavior when writing out to a path that may already have data. The default behavior is `SaveMode.ErrorIfExists`; matching the behavior of RDDs, Spark will throw an exception if the target already exists. The different save modes and their behaviors are listed in Table 5-10. Example 5-49 illustrates how to configure an alternative save mode.

Table 5-10. Save modes

Save mode	Behavior
`ErrorIfExists`	Throws an exception if the target already exists. If the target doesn't exist, write the data out.
`Append`	If the target already exists, append the data to it. If the data doesn't exist, write the data out.
`Overwrite`	Overwrites the relevant partitions with the new data. When `spark.sql.sources.partitionOverwriteMode` is static (default) the partitions are determined by explicit partitioning (or all data if none present) and in dynamic mode partitions are determined based on what partitions have data in the DataFrame.
`Ignore`	If the target already exists, silently skip writing out. Otherwise, write out the data.

Example 5-49. Specify save mode of append

```
def writeAppend(input: DataFrame): Unit = {
  input.write.mode(SaveMode.Append).save("output/")
}
```

In addition to the write API and its save modes, Spark 4 has added a new way to store the contents of a DataFrame into a table.

15 For example, only reading the required partitions when a filter matches one of our partitioning schemes.

Merge

Spark 4 also adds a "merge" API, which allows you to perform operations that more closely match what folks expect upsert to be and more. The merge API works by having you specify possibly multiple but at least one operation to perform when there are rows that match or are distinct based on the conditions of your merge operation. You can specify how to handle the lack of matches in either the target, where there is data in your DataFrame matching the condition but not the target table, or the source, where you have data in the source table matching the condition but not your DataFrame. The different match/nonmatch conditions have different operations you can perform, with match supporting merge or delete and nonmatch supporting source being new in Spark 4. The different catalog implementations of the merge API may reveal unique performance implications.

It's best practice to write your match conditions as narrowly as possible and ensure your data is partitioned on them. We believe it is good practice to assume that any loading match or nonmatch condition will require loading the data that it corresponds to. In practice, this may be able to be avoided for deletes in certain cases.



The names *source* and *target* are what are used in the code, but we find it easier to think of *source* as your DataFrame/Dataset and *target* as the table you are writing into.

To illustrate Spark 4's merge into API, we show upsert in Example 5-50.

Example 5-50. Spark 4 upsert using the new Scala merge APIs

```
    input
      .mergeInto("pandaInfo", $"source.id" === $"target.id")
      .whenMatched() // Note you can override the general match condition above if
desired
      .updateAll()
      .whenNotMatched()
      .insertAll()
```

Upserts with file-backed formats can be more expensive than traditional databases, so you'll need to make sure they are batched and used appropriately.



Frequent mutation (upserts or merges) with file-backed formats (like Iceberg, Delta, Hive, Hudi, etc.) can result in tables that are slow to query and require offline compaction. If you find yourself depending on frequent upserts or merges, you may be better served by a database rather than a data lake/warehouse.

Partitions (Discovery and Writing)

Partition data is an important part of Spark SQL since it powers one of the key optimizations to allow reading only the required data, discussed more in "Logical and Physical Plans" on page 102. If you know how your downstream consumers may access your data (e.g., reading data based on zip code), when you write your data, it is beneficial to use that information to partition your output. When reading the data, it's useful to understand how partition discovery functions so you can have a better understanding of whether your filter can be pushed down.



Filter pushdown can make a huge difference when working with large datasets by allowing Spark to only access the subset of data required for your computation instead of doing effectively a full table scan.

When reading partitioned data, you point Spark to the root path of your data, and it will automatically discover the different partitions. Not all data types can be used as partition keys; currently only strings and numeric data are the supported types. Some file types (like Iceberg or Delta) can add more flexibility for partitioning.

If your data is all in a single DataFrame, the `DataFrameWriter` API makes it easy to specify the partition information while you are writing the data out. The `partitionBy` function takes a list of columns to partition the output on, as shown in Example 5-51. You can also manually save out separate DataFrames (say, if you are writing from different jobs) with individual `save` calls.

Example 5-51. Save partitioned by zip code

```
def writeOutByZip(input: DataFrame): Unit = {
  input.write.partitionBy("zipcode").format("json").save("output/")
}
```

You can also specify `partitionBy` when creating a new table, but when writing to an existing table, specifying how the data is partitioned will cause an error.

In addition to splitting the data by a partition key, it can be useful to make sure the resulting file sizes are reasonable, especially if the results will be used downstream by another Spark job.



Partition statistics can also be used to filter out reading large files when filters are looking for specific values. The number of files able to be pruned will be lower, but you're less likely to encounter the small files problem (also conceptually the too many files problem). For example, with Iceberg, min/max values on specific columns can be used to skip reading files. You can read more on how Delta Lake statistics work and Iceberg stats.

The next section will look at query optimization in Spark, which, starting in Spark 3, has become increasingly connected with Spark SQL's output operations, allowing partitioning and sorting information for the output to "bubble up" from the write step into the query itself. This is often a win, but not always.

Query Optimizer

Catalyst is the Spark SQL query optimizer, which is used to take the query plan and transform it into an execution plan that Spark can run. Much as our transformations on RDDs build up a DAG, as we apply relational and functional transformations on DataFrames/Datasets, Spark SQL builds up a tree representing our query plan, called a *logical plan*. Spark is able to apply a number of optimizations on the logical plan and can also choose between multiple physical plans for the same logical plan using a cost-based model. Many folks have spent long periods of time manually tuning partitioning configuration in Spark, and Spark's AQE (adaptive query execution) aims to solve some of this.

The final phase of most Spark SQL logical and physical plans is either some kind of write/output operation or an aggregation. In either case these final phases are increasingly impactful on earlier stages of the query through Spark's adaptive query execution.

Logical and Physical Plans

When using transformations on DataFrames/Datasets (or SQL queries), you implicitly construct a logical plan; the first phase of which is an unresolved logical plan. Much like a compiler, the Spark optimizer is multiphased, and before many optimizations can be performed, it needs to resolve the references and types of the expressions.

This resolved plan is referred to as the logical plan, and Spark applies a number of optimizers directly on the logical plan, producing an optimized logical plan.

These optimizers can be written using pattern matching on the tree, such as the rule for simplifying additions between two literals. The optimizer is not limited to pattern matching, and rules can also include arbitrary Scala code.

Once the logical plan has been optimized, Spark will produce a physical plan. The physical plan stage has both rule-based and cost-based optimizations to produce the optimal physical plan. You can use `explain` on a Dataset much the same way SQL developers use explain to understand their query plans. One of the most important optimizations at this stage is predicate pushdown to the data source level.[16]

Static Optimizer Rules You Should Understand (and Working Around Them)

Spark SQL has a large number of static optimizer rules, which can allow you to focus more on the *what* and less on the *how*. However, for maximum performance, it's sometimes important to take control of the *how* when the general-case optimizers no longer meet our needs.

Static optimizers in Spark SQL broadly fall into three different categories. Reordering or pushdown rules, which try to take anything that can reduce the size of data being processed and push them down as far as possible so they are evaluated sooner. Operator combinators/collapsers merge adjacent operators; think multiple filters or multiple projections. Finally, there are simplifiers, which will do things like fill in constants, remove unreachable possibilities, and so on. Of course not all optimizers fall neatly into one of these categories, but having this rough idea of the different optimization types Spark is likely to perform gives you an idea of issues to keep an eye out for.

While these optimizers can greatly improve your queries, it's good practice to not be too overly dependent on them. For example, while Spark *could* optimize a `slice` over top of a `collect_list` it does not presently.[17] There are times when the logic of an optimization is not implemented because of potential corner cases (like certain filter pushdowns). As a result, if you follow the same good practices you're used to in regular Spark (filtering data early, etc.), your queries may be able to outperform a naively written query.



You can use the `spark.sql.planChangeLog.level` property to change the log level message of the SQL optimizer rules for debugging (by default it's logged at the trace level, which often has too much noise) and see how the plan is transformed.

16 General predicate pushdown occurs earlier and will push predicates through joins, unions, etc., when possible.

17 Although it's on Holden's radar to improve.

Sometimes you don't want the optimizers to work on a specific part of logic. For example, you may not want a given filter pushed down, perhaps because it's expensive and you have an alias to reduce the cost. In general most optimizers avoid touching any elements that are marked as "nondeterministic" and often will not move anything above (or below) them. You can mark your UDFs as nondeterministic by calling `asNondeterministic` or mixing in `Nondeterministic` on a custom expression. Another common technique to prevent the optimizer from going "too far" is to cache/persist or checkpoint a DataFrame (or temporary table in SQL)—the optimizer won't push any operations "through" one side of a cache/persist to the other. This can have more overhead, but if you are going to reuse some of the data later anyway, it is an excellent option. Conversely, adding unnecessary or ill-placed cache/persist operations can preclude more optimal performance.

The static optimizers operate on the query plan and don't know the data sizes involved. Instead, optimizations dependent on data size are handled through adaptive query execution, discussed next.

Understanding Spark Adaptive Query Execution

Spark's adaptive query execution (AQE) aims to use runtime information about the data it is processing along with the target output to go beyond static optimizations. During the initial start of a query, Spark may delegate to the data source to determine what the representation looks like, but after that, it uses information from the query as it runs. Two of the biggest areas AQE impacts are the partitioning of your data as well as the join strategies used. We'll discuss what the different join execution strategies are and how they are chosen in the next chapter focused on joins.

AQE optimizations are not visible using Spark's explain command, since AQE depends on runtime statistics. To debug AQE issues, you'll need to run the actual query, and the Spark UI will show you the finalized plan.

When query statistics indicate that the partitions are becoming "too small," controlled by `spark.sql.adaptive.advisoryPartitionSizeInBytes`, AQE will attempt to coalesce the partitions down. It will do this "postevaluation" so you don't normally have to worry about this part of the coalesce impacting the wider previous transformation, but if you've got data that is expensive or slow to compute[18] but small, the default `spark.sql.adaptive.advisoryPartitionSizeInBytes` may be suboptimal. You can also control the minimum number of partitions, which is the minimum parallelism, with `spark.sql.adaptive.coalescePartitions.minPartitionNum`.

An effective, but potentially confusing, optimization from AQE is its empty relation propagation. If you've got a subquery that is effectively empty (impossible join

18 Like a UDF that talks to a slow external service.

condition, empty unions, etc.) Spark will replace this with a dummy `LocalRelation` that is empty. The join replacement logic can be especially complicated to debug when the optimization is introduced due to missing data and can result in seemingly early termination of the Spark program compared to its "regular" runs.[19]



While AQE generally does better than unoptimized code, there are cases, especially when targeting Iceberg tables, where AQE does more harm than good.

One feature of AQE that is generally beneficial but sometimes can cause severe performance regressions is the matching of partitioning to target tables.[20] In general this is a good idea, since if your data is partitioned or sorted in the same way it is written, the write phase will be more efficient and you'll generally have "better" file layouts. However, this is not always worth the price, especially when there is a large amount of key skew. In those cases, whatever smart partitioning work you've done to avoid skew can be undone by the target table partitioning. In Iceberg the easy way to work around this is to set the table property (or write time property) of `write.distribution-mode` to `none`.

This does, however, increase the chance of having a large number of small files and many writers from each partition unless you manually sort/hash the data to match the desired on-disk partitioning beforehand.

Code Generation

Spark may also apply code generation for the components before executing a given stage. Code generation is done using Janino to compile Java code. Earlier versions used Scala's quasiquotes,[21] but the overhead was too high to enable code generation for small datasets. In some TPCDS queries, code generation can result in >10× improvement in performance.

Large Query Plans and Iterative Algorithms

While the Catalyst optimizer is quite powerful, one of the cases where it currently runs into challenges is with very large query plans. These query plans tend to be the result of iterative algorithms, like graph algorithms or machine learning algorithms.

19 This can result in seemingly impossible behavior, like queries referencing nonexistent columns succeeding. This can happen when a subquery is pruned by AQE.

20 In a case one of the coauthors encountered while working on this chapter, there was a roughly 4× performance regression from the automatic attempted matching.

21 Scala quasiquotes are part of Scala 2's macro system.

One simple workaround for this is converting the data to an RDD and back to DataFrame/Dataset at the end of each iteration, as shown in Example 5-52. Although if you're in Python, be sure to use the underlying Java RDD rather than round-tripping through Python (see Example 9-10 for how to do this). Another, somewhat more heavy option, is to write the data to storage and continue from there.

Example 5-52. Round trip through RDD to cut query plan

```
val rdd = df.rdd
rdd.cache()
session.createDataFrame(rdd, df.schema)
```

SparkSession Extensions and Custom Optimizers

Many of the components in Spark can be extended, including the optimizer. SparkSession Extensions are most frequently used by catalogs (like Iceberg or Delta Lake) to add additional functionality. You can have multiple extensions registered at the same time, which is important not only for multicatalog support but also if you want to add your own extensions while still using your existing catalog.

As with the custom expression API, session extensions are experimental and subject to change between versions. In fact, if you look at projects like Iceberg, you'll see that they have separate directories for building support for each minor version of Spark—in part because of these API changes.

While you can use extensions to add many kinds of rules, probably the easiest (and thankfully most impactful) are the regular optimizers, which work on logical plans. You can geek out and use tree transformations to make your SQL queries more efficient. This is potentially important if you start making extensive use of custom expressions. For example, if you wrote your own accelerated filter expression, Spark would not know to push your filter down in the same way it does with its built-in filter expressions.

For demonstration, we will create an optimizer rule in Example 5-53, which filters out any null data on columns on which we perform a null-intolerant projection. This is similar to an internal optimizer rule that filters out null values early when you have an explicit comparison to a non-null value.

Example 5-53. Optimizer rule to dynamically filter nulls

```
object NullabilityFilterOptimizer extends Rule[LogicalPlan] {

  def apply(plan: LogicalPlan): LogicalPlan = {
    plan.transform { case p @ Project(projectList, projChild) =>
      val children = projectList.flatMap(_.children)
      // If there are no null intolerant children don't worry about it
      if (children.isEmpty) {
        p
      } else {
        val filterCond = children.map(IsNotNull(_)).reduceLeft(And)
        Project(projectList, Filter(filterCond, projChild))
      }
    }
  }
}
```

Then, to add this optimizer rule, we created a SQLExtension (not so creatively called `com.highperformancespark.examples.dataframe.SQLExtension`) shown in Example 5-54 and added it to the `spark.sql.extensions` config.

Example 5-54. SQL extension to register a new optimizer rule

```
class SQLExtension extends SparkSessionExtensionsProvider {
  override def apply(extensions: SparkSessionExtensions): Unit = {
    // There are _many different_ types of rules you can inject, here we're focused
on
    // making things go fast so our sample is an optimizer rule (AQE rules could
also make sense).
    extensions.injectOptimizerRule(session => NullabilityFilterOptimizer)
  }
}
```

In our experience, the best way to learn how to write a new optimizer rule for Spark SQL is to look at the existing optimizers in `Optimizer.scala` and find ones that do similar things to what you want.



When developing and debugging optimizers, it's important to note that almost all operations are pushed to the local relation level, so your optimizer rules may not run.

Debugging Spark SQL Queries

While Spark SQL's query optimizer has access to much more information, we still sometimes need to take a peek under the hood and to make sure it's working as we expect. Similar to `toDebugString` on RDDs, we have `explain` and `printSchema` functions on DataFrames.

One thing that can make a big difference is figuring out if Spark SQL was able to push down a filter. Looking at the final query plan, you can see if the filters were able to be pushed down "next to" the data source. Some data sources will only "partially" handle a filter (leaving Spark to reevaluate the filter) as a safety mechanism when they might not filter out everything. These are still generally "good" filter pushdowns as the amount of data being read has been reduced.

JDBC/ODBC Server

Spark SQL provides a JDBC server to allow external tools, such as business intelligence GUIs like Tableau, to work with data accessible in Spark and to share resources. Spark SQL's JDBC server requires that Spark be built with Hive support. If you're building new applications, Spark Connect offers much of the same functionality while supporting more than just SQL, but existing JDBC applications don't work with it.



Since the server tends to be long-lived and runs on a single context, it can also be a good way to share cached tables, although Spark Connect is taking over this role somewhat.

Spark SQL's JDBC server is based on the HiveServer2 from Hive, and most corresponding connectors designed for HiveServer2 can be used directly with Spark SQL's JDBC server. Simba also offers specific drivers for Spark SQL.



Spark JDBC connections allow arbitrary code execution, so be careful who you open it up to.

The server can either be started from the command line or started using an existing `HiveContext`. The command-line start and stop commands are `./sbin/start-thriftserver.sh` and `./sbin/stop-thriftserver.sh`. When starting from the command line, you can configure the different Spark SQL properties by specifying `--hiveconf property=value` on the command line. Many of the rest of the

command-line parameters match those of `spark-submit`. The default host and port is `localhost:10000` and can be configured with `hive.server2.thrift.port` and `hive.server2.thrift.bind.host`.



When starting the JDBC server using an existing `HiveContext`, you can simply update the config properties on the context instead of specifying command-line parameters.

Examples 5-55 and 5-56 illustrate two different ways to configure the port used by the thrift server.

Example 5-55. Start JDBC server on a different port

```
./sbin/start-thriftserver.sh --hiveconf hive.server2.thrift.port=9090
```

Example 5-56. Start JDBC server on a different port in Scala

```
hiveContext.setConf("hive.server2.thrift.port", "9090")
HiveThriftServer2.startWithContext(hiveContext)
```



When starting the JDBC server on an existing `HiveContext`, make sure to shut down the JDBC server when exiting.

Conclusion

The considerations for using DataFrames/Datasets over RDDs are complex and changing with the rapid development of Spark SQL. One of the cases where Spark SQL can be difficult to use is when the number of partitions needed for different parts of your pipeline changes, or if you otherwise wish to control the partitioner. While RDDs lack the Catalyst optimizer and relational style queries, they are able to work with a wider variety of data types and provide more direct control over certain types of operations. DataFrames and Datasets also only work with a restricted subset of data types—but when your data is in one of these supported classes, the performance improvements of using the Catalyst optimizer provide a compelling case for accepting those restrictions.

DataFrames can be used when you have primarily relational transformations, which can be extended with UDFs when necessary. Compared to RDDs, DataFrames benefit from the efficient storage format of Spark SQL, the Catalyst optimizer, and the ability to perform certain operations directly on the serialized data. One drawback to

working with DataFrames is that they are not strongly typed at compile time, which can lead to errors with incorrect column access and other simple mistakes.

Datasets can be used when you want a mix of functional and relational transformations while benefiting from the optimizations for DataFrames and are, therefore, a great alternative to RDDs in many cases. As with RDDs, Datasets are parameterized on the type of data contained in them, which allows for strong compile-time type checking but requires that you know your data type at compile time (although `Row` or another generic type can be used). The additional type safety of Datasets can be beneficial even for applications that do not need the specific functionality of DataFrames.

Pure RDDs work well for data that does not fit into the Catalyst optimizer. RDDs have an extensive and stable functional API, and upgrades to newer versions of Spark are less likely to require substantial code changes, and performance is more likely to remain consistent. RDDs also make it easy to control partitioning, which can be very useful for many distributed algorithms. Some types of operations, such as multicolumn aggregates, complex joins, and windowed operations, can be daunting to express with the RDD API. RDDs can work with any Java or Kryo serializable data, although the serialization is often more expensive and less space efficient than the equivalent in DataFrames/Datasets.

Now that you have a good understanding of Spark SQL, it's time to continue on to joins, for both RDDs and Spark SQL.

CHAPTER 6

Joins (SQL and Core)

Joining data is an important part of many of our pipelines, and both Spark Core and SQL support the same fundamental types of joins. They do, however, have very different execution options and performance. While joins are very common and powerful, they warrant special performance consideration, as they may require large network transfers or even create datasets beyond our capability to handle.[1] In core Spark, it can be more important to think about the ordering of operations, since the DAG optimizer, unlike the SQL optimizer, isn't able to reorder or push down filters.

To understand the performance of joins in Spark, it's important to distinguish two similar-sounding concepts: join types and join execution techniques. Join types impact the result and are things like left and right inner joins, outer joins, full self joins, etc. Join execution techniques refer to how the engine, Spark, computes the result and can be things like broadcast, hash, sort, etc. The join type, as well as the data sizes, conditions, configuration, and hints, all influence what join technique Spark chooses. Another way of thinking of this is the *join type* is the logical operation, and the *join execution* is the method Spark uses to implement a join.

Core Spark Joins

In this section, we will go over the RDD-type joins. Joins, in general, are expensive since they require that corresponding keys from each RDD be located on the same machine so that they can be combined locally. If the RDDs do not have known partitioners, they must be shuffled so that both RDDs share a partitioner (except for broadcast joins) and data with the same keys lives in the same partitions, as shown in

1 As the saying goes, the cross product of big data and big data is an out-of-memory exception.

Figure 6-1. If they have the same partitioner, data may still need to be transferred but not shuffled.

The partitioning of your data, and if Spark is aware of it, has a large impact on joins. If the RDDs you are joining have the same known partitioners, there can be narrow dependencies on both RDDs. If there is one known and one unknown, or they have different known partitioners,[2] there will be a narrow dependency on one RDD and a shuffle/wide dependency on the other, as in Figure 6-2. In the best case with the same partitioner materialized in the same operation, the data may be co-located, as in Figure 6-3, so as to avoid network transfer entirely.

As with most key/value operations, the cost of the join increases with the number of keys and the distance the records have to travel in order to get to their correct partition.

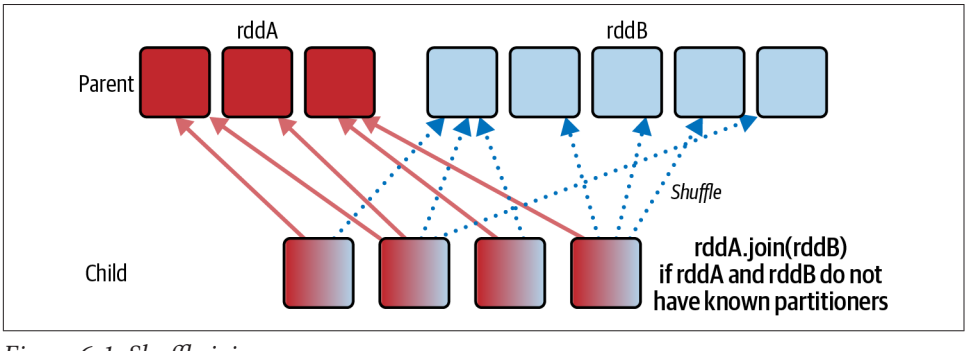


Figure 6-1. Shuffle join

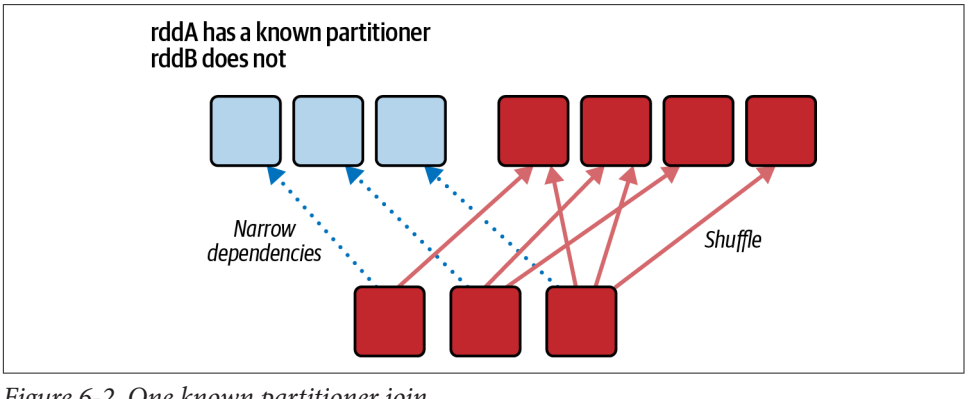


Figure 6-2. One known partitioner join

2 Spark picks the partitioner with the highest number of partitions regardless of the actual amount of data. Alternatively you can specify which partitioner to choose as part of the join.

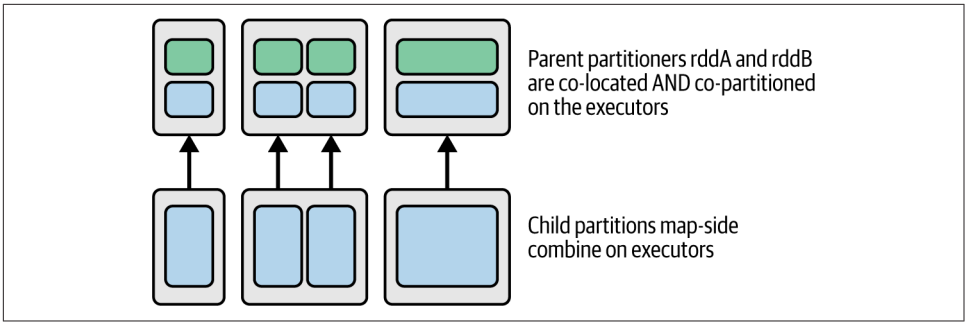


Figure 6-3. Co-located join



Two RDDs are co-located if they have the same partitioner and are shuffled as part of the same action.

Core Spark joins are implemented using the `cogroup` function. We discuss `cogroup` in "Co-Grouping" on page 184.

Choosing a Join Type

The default join operation in Spark is an inner join and so includes only values for keys present in both RDDs and, in the case of multiple values per key, provides all permutations of the key/value pair. The best scenario for a standard join is when both RDDs contain the same set of distinct keys. With duplicate keys, the size of the data may expand dramatically, causing performance issues, and if one key is not present in both RDDs, you will lose that row of data. Here are a few guidelines:

- When both RDDs have duplicate keys, the join can cause the size of the data to expand dramatically. It may be better to perform a `distinct` or `combineByKey` operation to reduce the key space or to use `cogroup` to handle duplicate keys instead of producing the full cross product. By using smart partitioning during the combine step, it is possible to prevent a second shuffle in the join (we will discuss this in detail later).
- If keys are not present in both RDDs, you risk losing your data unexpectedly. It can be safer to use an outer join so that you are guaranteed to keep all the data in either the left or the right RDD and then filter the data after the join.
- If one RDD has some easy-to-define subset of the keys, you may be better off filtering or reducing the second RDD before the join to reduce the size of shuffle data, which you will ultimately throw away anyway.



Join is one of the most expensive operations you will commonly use in Spark, so it is worth doing what you can to shrink your data before performing a join.

For example, suppose you have one RDD with some data in the form `(Panda id, score)` and another RDD with `(Panda id, address)`, and you want to send each panda some mail with her best score. You could join the RDDs on `id` and then compute the best score for each `address`, as shown in Example 6-1.

Example 6-1. Basic RDD join

```
def joinScoresWithAddress1(
    scoreRDD: RDD[(Long, Double)],
    addressRDD: RDD[(Long, String)]
): RDD[(Long, (Double, String))] = {
  val joinedRDD = scoreRDD.join(addressRDD)
  joinedRDD.reduceByKey((x, y) => if (x._1 > y._1) x else y)
}
```

However, this is not as fast as first reducing the score data, so that the first dataset contains only one row for each panda with her best score, and then joining that data with the address data (as shown in Example 6-2).

Example 6-2. Prefilter before join

```
def joinScoresWithAddress2(
    scoreRDD: RDD[(Long, Double)],
    addressRDD: RDD[(Long, String)]
): RDD[(Long, (Double, String))] = {
  val bestScoreData = scoreRDD.reduceByKey((x, y) => if (x > y) x else y)
  bestScoreData.join(addressRDD)
}
```

If each panda had 1,000 different scores, then the size of the shuffle we did in the first approach was 1,000 times the size of the shuffle we did with this approach!

If we wanted to, we could also perform a left outer join to keep all keys for processing, even those missing in the right RDD, by using `leftOuterJoin` in place of `join`, as in Example 6-3. Spark also has `fullOuterJoin` and `rightOuterJoin` depending on which records we wish to keep. Any missing values are `None`, and present values are `Some('x')`.

Example 6-3. Basic RDD left outer join

```
def outerJoinScoresWithAddress(
    scoreRDD: RDD[(Long, Double)],
    addressRDD: RDD[(Long, String)]
): RDD[(Long, (Double, Option[String]))] = {
  val joinedRDD = scoreRDD.leftOuterJoin(addressRDD)
  joinedRDD.reduceByKey((x, y) => if (x._1 > y._1) x else y)
}
```

Choosing an Execution Plan

In order to join data, Spark needs the data that is to be joined (i.e., the data based on each key) to live on the same machine. The default implementation of a join in Spark is a *shuffle hash join*. The shuffle hash join ensures that data on each partition will contain the same keys by partitioning the second dataset with the same partitioner as the first, so that the keys with the same hash value from both datasets are in the same partition. While this approach always works, it can be more expensive than necessary because it requires a shuffle. The shuffle can be avoided if:

1. Both RDDs share a known partitioner.
2. One of the datasets is small enough to fit in memory, in which case we can do a broadcast hash join (we will explain what this is later).

Note that if the RDDs are co-located, the network transfer can be avoided, along with the shuffle.

Speeding up joins by assigning a known partitioner

If you have to do an operation before the join that requires a shuffle, such as `aggregateByKey` or `reduceByKey`, you can prevent the additional shuffle for the join by adding a hash partitioner with the same number of partitions as an explicit argument to the first operation before the join. You could make the example in the previous section even faster by using the partitioner for the address data as an argument for the `reduceByKey` step, as in Example 6-4 and Figure 6-4.

Example 6-4. Known partitioner join

```
def joinScoresWithAddress3(
    scoreRDD: RDD[(Long, Double)],
    addressRDD: RDD[(Long, String)]
): RDD[(Long, (Double, String))] = {
  // If addressRDD has a known partitioner we should use that,
  // otherwise it has a default hash parttioner, which we can reconstruct by
  // getting the number of partitions.
  val addressDataPartitioner = addressRDD.partitioner match {
    case (Some(p)) => p
    case (None)    => new HashPartitioner(addressRDD.partitions.length)
  }
  val bestScoreData = scoreRDD.reduceByKey(
    addressDataPartitioner,
    (x, y) => if (x > y) x else y
  )
  bestScoreData.join(addressRDD)
}
```

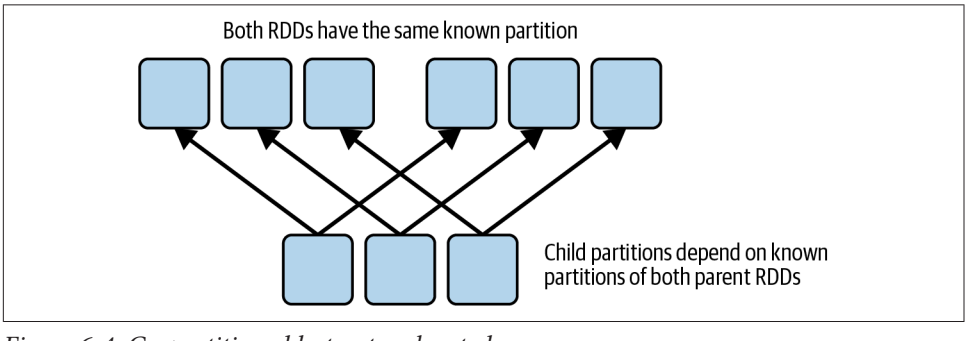


Figure 6-4. Co-partitioned but not co-located



If the RDDs sharing the same partitioner are materialized by the same action, they will end up being co-located (which can even reduce network traffic). One way to do this is by unioning them and calling `count()`.

Speeding up joins using a broadcast hash join

A broadcast hash join collects all of the data from the smaller RDD onto the driver and then pushes a copy to each worker node. Then it does a map-side combine with each partition of the larger RDD. If one of your RDDs can fit in memory or can be made to fit in memory, it is always beneficial to do a broadcast hash join, since it doesn't require a shuffle. This is illustrated in Figure 6-5.

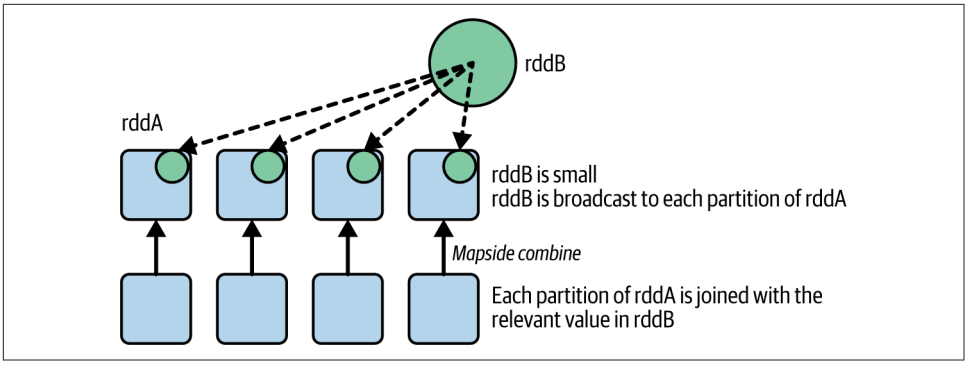


Figure 6-5. Broadcast hash join

Spark Core does not have an implementation of the broadcast hash join. Instead, we can manually implement a version of the broadcast hash join by collecting the smaller RDD to the driver as a map, then broadcasting the result, and using `mapPartitions` to combine the elements.

Example 6-5 is a general function that could be used to join a larger and smaller RDD. Its behavior mirrors the default "join" operation in Spark. We exclude elements whose keys do not appear in both RDDs.

Example 6-5. Manual broadcast hash join

```
def manualBroadcastHashJoin[
    K: Ordering: ClassTag,
    V1: ClassTag,
    V2: ClassTag
](bigRDD: RDD[(K, V1)], smallRDD: RDD[(K, V2)]) = {
  val smallRDDLocal: Map[K, V2] = smallRDD.collectAsMap()
  val smallRDDLocalBcast = bigRDD.sparkContext.broadcast(smallRDDLocal)
  bigRDD.mapPartitions(
    iter => {
      iter.flatMap { case (k, v1) =>
        smallRDDLocalBcast.value.get(k) match {
          // Note: You could switch this to a left join by changing the empty seq
          // to instead return Seq(k, Seq.empty[(V1, V2)])
          case None     => Seq.empty[(K, (V1, V2))]
          case Some(v2) => Seq((k, (v1, v2)))
        }
      }
    },
    preservesPartitioning = true
  )
}
```

Partial manual broadcast hash join

Sometimes not all of our smaller RDDs will fit into memory, but some keys are so overrepresented in the large dataset that you want to broadcast just the most common keys. This is especially useful if one key is so large that it can't fit on a single partition. In this case you can use `countByKeyApprox` on the large RDD to get an approximate idea of which keys would most benefit from a broadcast.[3] You then filter the smaller RDD for only these keys, collecting the result locally in a `HashMap`. Using `sc.broadcast` you can broadcast the `HashMap` so that each worker only has one copy and manually perform the join against the `HashMap`. Using the same `HashMap`, you can then filter your large RDD down to not include the large number of duplicate keys and perform your standard join, unioning it with the result of your manual join. This approach is quite convoluted but may allow you to handle highly skewed data you couldn't otherwise process.

Spark SQL Joins

Spark SQL supports the same basic join types as core Spark, but the optimizer can do more of the heavy lifting for you—although you also give up some of your control. For example, Spark SQL can sometimes push down or reorder operations to make your joins more efficient. On the other hand, you don't control the partitioner for the RDDs making up the DataFrames or Datasets, so you can't manually avoid shuffles as you did with core Spark joins.

DataFrame Joins

Joining data between `DataFrames` is one of the most common multi-`DataFrame` transformations. The standard SQL join types are all supported and can be specified as the `joinType` in `df.join(otherDf, sqlCondition, joinType)` when performing a join. As with joins between RDDs, joining with nonunique keys will result in the cross product (so if the left table has rows `R1` and `R2` with `key1` and the right table has rows `R3` and `R5` with `key1` you will get `(R1, R3), (R1, R5), (R2, R3), (R2, R5))` in the output. While we explore Spark SQL joins, we will use two example tables of pandas, Tables 6-1 and 6-2.

Once we've covered the kinds of joins you can do with DataFrames/Spark SQL, we'll look at the different ways these joins can be executed, their performance characteristics, and how the Spark SQL optimizer chooses the join type. While some of these are similar to the RDD joins discussed previously, there are important differences and new options for join execution.

3 If the number of distinct keys is too high, you can also use `reduceByKey`, sort on the value, and take the top k.



While self-joins are supported, you must first alias the relevant fields to distinct names to access them.

Table 6-1. Table of pandas and sizes (our left DataFrame)

Name	Size
Happy	1.0
Sad	0.9
Happy	1.5
Coffee	3.0

Table 6-2. Table of pandas and zip codes (our right DataFrame)

Name	Zip
Happy	94110
Happy	94103
Coffee	10504
Tea	07012

Spark's supported join types are *inner*, *left_outer* (aliased as *outer*), *left_anti*, *right_outer*, *full_outer*, and *left_semi*. With the exception of left_semi and left_anti, these join types all join the two tables, but they behave differently when handling rows that do not have keys in both tables.

The inner join is both the default and likely what you think of when you think of joining tables. It requires that the key be present in both tables, or the result is dropped as shown in Example 6-6 and Table 6-3.

Example 6-6. Simple inner join

```
// Inner join implicit
df1.join(df2, df1("name") === df2("name"))
// Inner join explicit
df1.join(df2, df1("name") === df2("name"), "inner")
```

Table 6-3. Inner join of df1, df2 on name

Name	Size	Name	Zip
Coffee	3.0	Coffee	10504
Happy	1.5	Happy	94110
Happy	1.5	Happy	94103
Happy	1.0	Happy	94110

Name	Size	Name	Zip
Happy	1.0	Happy	94103

Left outer joins will produce a table with all of the keys from the left table, and any rows without matching keys in the right table will have null values in the fields that would be populated by the right table. Right outer joins are the same, but with the requirements reversed. A sample left outer join is in Example 6-7, and the result is shown in Table 6-4.

Example 6-7. Left outer join

```
// Left outer join explicit
df1.join(df2, df1("name") === df2("name"), "left_outer")
```

Table 6-4. Left outer join df1, df2 on name

Name	Size	Name	Zip
Sad	0.9	null	null
Coffee	3.0	Coffee	10504
Happy	1.0	Happy	94110
Happy	1.0	Happy	94103
Happy	1.5	Happy	94110
Happy	1.5	Happy	94103

A sample right outer join is in Example 6-8, and the result is shown in Table 6-5.

Example 6-8. Right outer join

```
// Right outer join explicit
df1.join(df2, df1("name") === df2("name"), "right_outer")
```

Table 6-5. Right outer join df1, df2 on name

Name	Size	Name	Zip
Coffee	3.0	Coffee	10504
Happy	1.0	Happy	94110
Happy	1.0	Happy	94103
Happy	1.5	Happy	94110
Happy	1.5	Happy	94103
null	null	Tea	07012

To keep all records from both tables you can use the full outer join, which results in Table 6-6.

Table 6-6. Full outer join df1, df2 on name

Name	Size	Name	Zip
Sad	0.9	null	null
Coffee	3.0	Coffee	10504
Happy	1.0	Happy	94110
Happy	1.0	Happy	94103
Happy	1.5	Happy	94110
Happy	1.5	Happy	94103
null	null	Tea	07012

Left semi joins (as in Example 6-9 and Table 6-7) and left anti joins (as in Table 6-8) are the only kinds of joins that only have values from the left table. A left semi join is the same as filtering the left table for only rows with keys present in the right table. The left anti join also only returns data from the left table, but instead only returns records that are not present in the right table.

Example 6-9. Left semi join

```
// Left semi join explicit.
// Here we're explicit about which DF which col comes from given
// the shared name.
df1.join(df2, df1("name") === df2("name"), "left_semi")
```

Table 6-7. Left semi join

Name	Size
Coffee	3.0
Happy	1.0
Happy	1.5

Table 6-8. Left anti join

Name	Size
Sad	0.9

Self joins

Self joins are supported on DataFrames, but we end up with duplicate column names. So that you can access the results, you need to alias the DataFrames to different names—otherwise, you will be unable to select the columns due to name collision (see Example 6-10). Once you've aliased each DataFrame, in the result, you can access the individual columns for each DataFrame with `dfName.colName`.

Example 6-10. Self join

```
val joined = df.as("a").join(df.as("b")).where($"a.name" === $"b.name")
```

Cross/cartesian joins

Cross joins are a special case that is even more likely to create large results. Every row in the first DataFrame is joined with every row in the second DataFrame, which can quickly blow up the results. Cross joins are best used when one of the two DataFrames is small, like a list of provinces.

You need to be careful when using cross joins, as the size of the data naturally grows rather quickly and can cause significantly degraded performance in successive operations.

Complex join conditions

Efficient joins in Spark depend on Spark's (re)partitioning the data so that equality can be computed between two partitions. Certain functions (e.g., a UDF, regexes, >=, etc.) will work as join conditions but force something similar to cartesian joins. This is because there is no static way for Spark to ensure the matching data is in the same partitions. Joins with an equality condition are sometimes called *equi-joins*, and others are called *non-equi-joins*.

Understanding equi-joins can be a bit confusing since not all UDF usage in a join function results in this fallback. If the core join condition is still a "simple" operation performed on the results of complex operations, like equality on UDF results, Spark can still perform its regular join. It's simplest to see with two example uses of UDFs for joins, one "cheap" and one expensive (see Example 6-11).

Example 6-11. Bad complex/UDF join—forces all-to-all comparison

```
def badJoin(
    df1: Dataset[Pandas],
    df2: Dataset[Pandas]
): Dataset[(Pandas, Pandas)] = {
  val session = df1.sparkSession
  val sle = session.udf.register(
    "strLenEq",
    (s: String, s2: String) => s.length() == s2.length()
  )
  df1.joinWith(df2, sle(df1("name"), df2("name"))).alias("strlenEqJoin")
}
```

When possible, instead of this, separate out your equality checking and write your UDF to return the per-value results as in Example 6-12.

Example 6-12. Safe join with UDFs—follows regular join operations

```
def okJoin(
    df1: Dataset[Pandas],
    df2: Dataset[Pandas]
): Dataset[(Pandas, Pandas)] = {
  val session = df1.sparkSession
  val sl = session.udf.register("strLen", (s: String) => s.length())
  df1.joinWith(df2, sl(df1("name")) === sl(df2("name"))).alias("strlenJoin")
}
```

While intuitively, all-to-all comparisons are more expensive, the next section will look at the different join execution operators' performance in more detail.

Concrete SQL Join Execution Operators

Spark SQL has a few key join execution techniques for different data sizes, skews, join conditions, and join types. Understanding the join techniques and their trade-offs is useful so you can provide correct join hints when necessary and structure your code to support the desired join technique.

The core Spark SQL join techniques are broadcast hash join, broadcast nested loop, shuffle-hash, shuffle-sort-merge, and shuffle and replicate nested loop. Review Table 6-9 for a brief summary, and then we'll dive into each in more detail.

Table 6-9. Join execution summary

Join name	Join types	Join conditions	Input partitioning[a]	Output partitioning	Performance	Narrow or wide
Broadcast hash	All except full outer	equi-join	Any	Same as larger DataFrame	Fast when one DataFrame is small	Narrow (non-broadcast)
Broadcast nested loop	All	equi and non-equi	Any	Often same as larger	Depends on one DataFrame being small and being able to do minimal passes	Narrow (non-broadcast)
Shuffle and replicate (also known as cartesian)	Inner and cartesian	equi and non-equi	Any	Undefined	High chance of data explosion	Narrow
Shuffle hash	All	equi join	No skew: Any skew: Clustered	Depends on join type	Requires high cardinality	Wide
Shuffle-sort-merge	All	equi join on sortable	No skew: Any skew: Clustered	Depends on join type	Requires high cardinality, less likely to OOM	Wide

[a] Spark will automatically insert a shuffle if needed to match the desired input partitioning.

Broadcast hash avoids shuffling and is immensely performant if one of the two DataFrames is small enough. They support all join types except for full-outer joins. Broadcast joins require equi-joins, although the similar broadcast nested loop join supports non-equi-joins. Broadcast joins are most often selected by `spark.sql.autoBroadcastJoinThreshold`, which has a default value of ~10 MB. The output partitioning is the same as the input partitioning of the nonbroadcasted DataFrame. While they can be performant when used correctly, they can also often cause too-large task result errors and out-of-memory issues when the broadcast DataFrame is too large. Since this is such a high performance operation, when possible, Spark will use this strategy.

Broadcast nested loop joins are similar to broadcast hash joins but with fewer restrictions. The nested loop does not refer to iteratively broadcasting parts of the Dataset; instead, it refers to iterating over the Dataset multiple times to compute the join conditions. They support all join types and both equi and non-equi join conditions. As with broadcast join, there is a chance of out-of-memory errors when the broadcast DataFrame is too large. It also performs poorly when multiple iterations are required to compute the join. The ideal cases, with single iterations over the Datasets, are left broadcast with right outer join, right broadcast with left outer, left semi, left anti, or existence join, and either side broadcast in an inner join. Because this join is more likely to be slow or out of memory, it is normally only created as a last resort.

Shuffle and replicate joins, also known as cartesian product joins, are the most likely to cause an explosion in terms of data size. They are implemented by computing each partition with every other partition. For inner joins, the result is filtered on the join condition, which is pipelined as part of the evaluation. Uniquely, shuffle and replicate joins have the advantage of creating comparatively narrow dependencies where each output partition depends only on one partition from each of its parents.

The next two shuffle-based joins, shuffle hash and shuffle sort, are similar to the RDD-based joins discussed earlier in this chapter.

Shuffle hash joins are one of Spark's earliest forms and offer reasonable performance in many cases. It supports the majority of join conditions and data types and has fairly reasonable performance. Shuffle hash joins require that one of the two RDDs have partitions that are small enough to create a hash map of the keys.

Shuffle sort joins are very similar to shuffle hash joins but use sorting instead of hashing. It requires that the keys you are joining on are sortable, which the majority of primitive types are. Complex types, like maps, however, are not sortable.

On top of the core join techniques, Spark AQE adds an optimized skew join, which, while not exactly another join technique, splits large partitions in shuffle sort and hash-based joins. Spark's AQE can also add additional join hints dynamically to suggest a better strategy using runtime data size statistics.

In Scala Spark SQL, you can see the type of join being performed by calling `queryExecution.executedPlan`, and more generally, `explain` will show the type of join Spark has chosen. As with core Spark, if one of the tables is much smaller than the other, you may want a broadcast hash join. You can hint to Spark SQL that a given DF should be broadcast for a join by calling `broadcast` on the `DataFrame` before joining it (e.g., `df1.join(broadcast(df2), "key")`). Spark also automatically uses the `spark.sql.conf.autoBroadcastJoinThreshold` to determine if a table should be broadcast.



Join hints can help Spark choose the "correct" sorting operation, broadcast the desired side of a join, or target a specific output partitioning. You can specify these hints with `/* [name of hint], [next hint], etc */` in SQL. These hints are only *hints* and Spark may still choose a different operator.

Dataset Joins

You can also join Datasets with `joinWith`, and this behaves similarly to a regular relational join, except the result is a tuple of the different record types as shown in Example 6-13. This is somewhat more awkward to work with after the join but also does make self joins, as shown in Example 6-14, much easier, as you don't need to alias the columns first.

Example 6-13. Joining two Datasets

```
val result: Dataset[(RawPanda, CoffeeShop)] =
  pandas.joinWith(coffeeShops, pandas("zip") === coffeeShops("zip"))
```

Example 6-14. Self join a Dataset

```
val result: Dataset[(RawPanda, RawPanda)] =
  pandas.as("l").joinWith(pandas.as("r"), $"l.zip" === $"r.zip")
```



Using a self join and a `lit(true)`, you can produce the cartesian product of your Dataset, which can be useful but also illustrates how joins (especially self joins) can easily result in unworkable data sizes.

As with DataFrames, you can specify the type of join desired (e.g., inner, left outer, right outer, left semi), changing how records present only in one Dataset are handled. Missing records are represented by null values, so be careful.

Conclusion

Now that you have explored joins, it's time to focus on transformations and their associated performance considerations.

CHAPTER 7

Effective Transformations

Most commonly, Spark programs are structured on RDDs and Datasets. Most Spark programs using the RDD APIs involve reading data from stable storage into the RDD format, performing a number of computations and data transformations on the RDD, and writing the resulting RDD to stable storage or collecting it to the driver. Even Dataset operations are largely compiled down into RDD transformations. Thus, most of the power of Spark comes from its transformations: operations that are defined on RDDs and return RDDs.

At present, Spark contains specialized functionality for about a half-dozen types of RDDs, each with its own properties and scores of different transformation functions. In this section, we hope to give you the tools to think about how your RDD transformation, or series of transformations, will be evaluated. In particular: what kinds of RDDs these transformations return, whether persisting or checkpointing RDDs between transformations will make your computation more efficient, and how a given series of transformations could be executed in the most performant way possible.



The transformations in this section are those associated with the RDD object used in Spark Core. RDDs are also used inside of DStreams with legacy Spark Streaming, but they have different functionality and performance properties. Datasets, DataFrames, and Spark SQL (streaming or regular) all compile down to RDDs, with an additional optimizer in between. It is still important to understand RDD operation performance when working at these higher abstraction layers.



As Spark moves forward, more RDD transformations will become available on Datasets, which can be used in Spark SQL and which are discussed in "Datasets" on page 79.

Narrow Versus Wide Transformations

In Chapter 2, we introduced one important distinction between types of transformations: those with *wide dependencies* and those with *narrow dependencies*. This distinction is important because it has strong implications for how transformations are evaluated and, consequently, for their performance. In this section, we will more precisely define the wide and narrow transformations, demonstrate how to determine whether a transformation is wide or narrow, and explain why this distinction matters for evaluation and performance.



Recall that Spark is lazily evaluated, meaning that a transformation is not executed until an action that depends on that transformation is called. This, as we discussed in detail in "Lazy Evaluation" on page 13, has important consequences for fault tolerance, performance, and debugging. If the information in this tip is confusing, please refer back to Chapter 2, which will give you the basic understanding of the Spark execution engine needed for this chapter.

To summarize what we covered in Chapter 2: wide transformations are those that require a shuffle, while narrow transformations are those that do not. In "Wide Versus Narrow Dependencies" on page 21, we explained that in narrow transformations, the child partitions (the partitions in the resulting RDD) depend on a known subset of the parent partitions. While this definition is correct, it is less precise than the formal definition of narrow transformations.

The 2012 paper that first presented the evaluation semantics for Spark defines transformations with narrow dependencies as those in which "each partition of the parent RDD is used by at most one partition of the child RDD."[1] The creators define transformations with wide dependencies as transformations in which "multiple child partitions may depend on [each partition in the parent]." This definition states the analogue of what we explained in Chapter 2, in which we defined narrow and wide dependencies in relation to the child RDD's dependencies. In contrast, the creators' definition defined narrow and wide dependencies in terms of the dependencies on the parent RDD, rather than those on the child RDD.

1 Matei Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," 9th USENIX Symposium on Networked Systems Design and Implementation, April 25–27, 2012.

We think the definition presented in Chapter 2 is easier to conceptualize since one usually designs a program by thinking from the input data (parent RDD) to the output data (child RDD). However, the Spark evaluation engine (the "DAG") builds an execution plan in reverse: from the output (the last action) to the input RDD. Thus, the Spark creators' definition mirrors the way that Spark is evaluated, and consequently, it is more precise in two important ways. First, the founders' definition rules out the case of one parent partition having multiple children in a narrow dependency. It explains why coalesce is only a narrow transformation when it is reducing rather than increasing the number of partitions. Second, the founders' definition clarifies why the number of tasks used to complete a computation corresponds to each *output partition* rather than each *input partition*—when RDDs are evaluated, the tasks needed to compute a transformation are computed on the child partitions.

Figure 7-1 shows dependencies between parent and child partitions for narrow and wide transformations for the Spark program in Example 7-1. Assume `rdd1` is an RDD of integers.

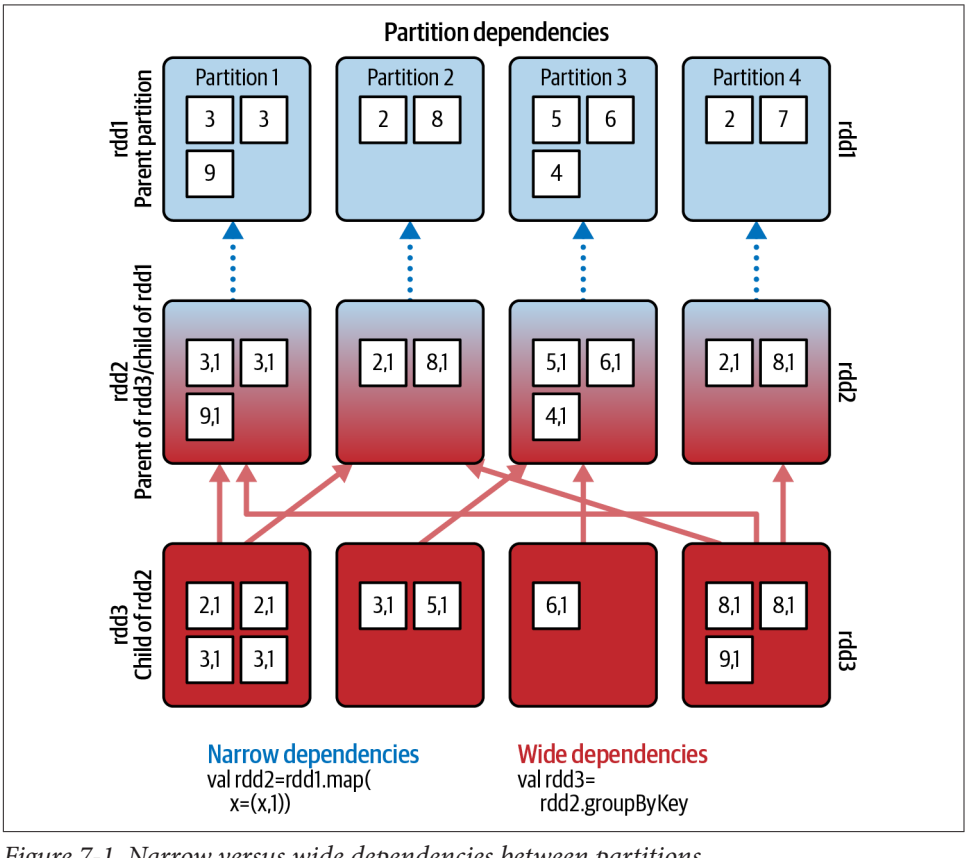


Figure 7-1. Narrow versus wide dependencies between partitions

Example 7-1. Narrow versus wide example

```
// Narrow dependency. Map the rdd to tuples  of (x, 1)
val rdd2 = rdd1.map(x => (x, 1))
// wide dependency groupByKey
val rdd3 = rdd2.groupByKey()
```

We use the same structure as the diagrams presented in Figures 2-3 and 2-4. Arrows represent partition dependencies. Each child partition has arrows pointing to the parent partitions upon which it depends; if an arrow points from partition y to partition x, that means that x depends on y. Dotted blue arrows represent narrow dependencies, and solid red arrows represent wide dependencies.

We assume the RDD has four partitions. Unlike the diagrams presented in Chapter 2, here we will show how actual records in a very small RDD might be distributed among the partitions. In this case we show how `rdd1`, `rdd2`, and `rdd3` would be partitioned if `rdd1` were an RDD of the integers `3, 3, 9, 2, 8, 5, 6, 7`.

As you can see, to compute the `map` step, each child partition depends on just one parent, since the data doesn't need to be moved between partitions for the operation to be computed. However, in the `groupByKey` step, Spark needs to move some number of the records so that those with the same key are in the same partition in order to combine records corresponding to a single key into one iterator (recall that an iterator is a local, rather than a distributed, collection). Thus, the child partitions depend on many partitions in the parent RDD.



Figure 7-1 is intended to show how partitions, an abstract concept used in Spark evaluation, depend on each other, rather than any physical data movement across machines. Each line of squares in the diagram represents the same executors at different points in time. The arrows denote dependencies between partitions. In fact repartitioning data does not necessarily require data movement across machines, since partitions may reside on the same executor.

Implications for Performance

In "Wide Versus Narrow Dependencies" on page 21, we asserted that transformations with narrow dependencies are faster to execute partly because narrow transformations can be combined and executed in one pass of the data. In this section we hope to explain why this is from an evaluation perspective.

Narrow dependencies do not require data to be moved across partitions. Consequently, narrow transformations don't require communication with the driver node, and an arbitrary number of narrow transformations can be executed on any subset of the records (any partition) given one set of instructions from the driver. In Spark

terminology, we say that each series of narrow transformations can be computed in the same "stage" of the query execution plan.

In contrast, as we stated in "The Anatomy of a Spark Job" on page 26, a shuffle associated with a wide dependency marks a new stage in the RDD's evaluation. Because tasks must be computed on a single partition and the data needed to compute each partition of a wide dependency may be spread across machines, transformations with wide dependencies may require data to be moved across partitions. Thus, the downstream computations cannot be computed before the shuffle finishes.

For example, it should be intuitive that sorting cannot be accomplished with narrow transformations because sorting requires an order to be defined on all of the records —not just within each partition. Indeed, the `sortByKey` function has wide dependencies. It requires the data to be partitioned so all the keys within a certain range live on the same partition. That way, sorting the data on each partition leads to a sorted result. Any narrow transformations following the sort cannot be done until after the shuffle completes because the records on each partition may change.

Stage boundaries have important performance consequences. Except in the case of multiple RDD operations like `join`, the stages associated with one RDD must be executed in sequence (see Chapter 6). Thus, not only are shuffles expensive since they require data movement and potential disk I/O (for the shuffle files), they also limit parallelization.

Implications for Fault Tolerance

The cost of failure for a partition with wide dependencies is much higher than for one with narrow dependencies, since it requires more partitions to be recomputed. If one partition in the parent of a `mappedRDD` (the resulting RDD type of a `map` operation) fails, for example, only one of its children must be recomputed, and the tasks needed to recompute that child partition can be distributed across the executors to make this recomputation faster. In contrast, if the parent of the sorted RDD loses a partition, it is possible (in the worst case) that all the child partitions will need to be recomputed. For this reason, the cost of recomputing a partition in the case of failure for a partition with wide dependencies is much higher than for a partition with narrow dependencies.

Chaining together transformations with wide dependencies only increases the risk of a very expensive recomputation, particularly if any of the wide transformations have a high probability of causing memory errors. In some instances, the cost of recomputation may be high enough that it is worth checkpointing an RDD, so the intermediate results are saved. We will discuss `checkpointing` in detail in "Reusing RDDs" on page 155.

The Special Case of coalesce

The `coalesce` operation is used to change the number of partitions in an RDD or Dataset. As shown by the diagram in Figure 2-3 in Chapter 2, when `coalesce` reduces the number of output partitions, each parent partition is used in exactly one child partition since the child partitions are the union of several parents. Thus, according to our definition of narrow dependencies, coalesce is a narrow transformation even though it changes the number of partitions in the RDD. Since tasks are executed on the child partition, the number of tasks executed in a stage that includes a `coalesce` operation is equivalent to the number of partitions in the result RDD of the coalesce transformation.



Using `coalesce`, the number of partitions can decrease in one stage without causing a shuffle. However, `coalesce` causes the upstream partitions in the entire stage to execute with the level of parallelism assigned by `coalesce`, which may be undesirable in some cases. Avoid this behavior at the cost of a shuffle by using the `repartition` function instead.

In RDD land you can make `coalesce` behave like `repartition`, but we recommend instead just using `repartition` when you want a shuffle and `coalesce` when you don't; it makes the code easier to read.

What Type of RDD Does Your Transformation Return?

RDDs are an abstracted concept in two ways: they can be of almost any arbitrary type of record (e.g., `String`, `Row`, `Tuple`), and they can also be members of one of several implementations of the `RDD` interface with varying properties. Both distinctions are important for performance and evaluation. The first is important, because some transformations can only be applied to RDDs with certain record types. The second is important because each transformation returns one of the several implementations of the RDD interface, and therefore the same transformation called on two different RDD implementations (such as a `mappedRDD` versus a `CoGroupedRDD`) may be evaluated differently. In particular, some RDD implementations retain information about the ordering or locality of the records in the RDD from previous transformations. Understanding the data locality and partitioning information associated with the resulting RDD of a transformation can help avoid unnecessary shuffles. We will save a more detailed discussion of this for "Preserving Partitioning Information Across Transformations" on page 188 since it is most relevant to pair RDDs. In this section, we will discuss preserving record type information because it can be important for performance and surprisingly difficult as Spark programs get complicated.

The RDD is a collection type which, much like collection types in Scala, Java, and most other strongly typed languages, is instantiated with a type parameter indicating the type of the members in the collection.



In Scala, the syntax for a type parameter is brackets (e.g., `List[String]`), which indicates a sequence of String objects. It is equivalent to the Java syntax < > (e.g., `List<String>`).

RDDs are similarly typed. For example, if you use `sc.textfile` to read in your RDD, you will end up with an `RDD` of String type (denoted `RDD[String]` in Scala and `RDD<String>` in Java).

The RDD's record type information is important because many transformations are only defined on RDDs that are of a particular type, so trying to use methods on an RDD of a generic type will return compile-time or runtime errors. For example, if an RDD of tuples has lost its type information and is interpreted by the compiler to be of type `RDD[Any]` or even `RDD[(Any, Int)]`, calling `sortByKey` will not compile. The compilation error occurs because `sortByKey` can only be called on RDDs of key/value pairs where the keys have some implicit ordering. Similarly, numeric functions such as `max`, `min`, and `sum` can only be called on RDDs of `Long`, `Int`, or `Double`.

Record type information is one of many places in the Spark API where implicit conversions are likely to cause difficulties. If you are writing subroutines to be used in RDD transformation, it is often best to specify the helper function's input and return types concretely and avoid writing them on a generic type.

One instance that often leads to problems losing type information is when working with DataFrames as RDDs. DataFrames can be implicitly converted to RDDs of `Rows`. However, since the Spark SQL `Row` object is not strongly typed (it can be created from sequences of any value type), the Scala compiler cannot "remember" the type of value used to create the row. Indexing a row will return a value of type `Any`, which must be cast to a more specific type, such as a `String` or an `Int`, to perform most calculations. The type information for the rows is stored in the schema. However, converting to an RDD throws away a DataFrame's schema information, so it is important to bind the DataFrame schema to a variable. One of the advantages of the Dataset API is that it is strongly typed, so the values in each row will retain their type information even after conversion to an RDD.

Minimizing Object Creation

Garbage collection is the process of freeing up the memory allocated for an object once that object is no longer needed. Since Spark runs in the JVM, which has automatic memory management and large data structures, garbage collection can quickly become an expensive part of our Spark job. Garbage collection, or *GC*, errors are a common cause of failure. Even if garbage collection overhead doesn't prohibit a job from running, garbage collection creates additional serialization time, which can significantly slow it down. We can minimize the GC cost by reducing the number of objects and the size of those objects. We can reduce the size and number of our objects by reusing existing objects and by using data structures (such as primitive types) that take up less space in memory.

Reusing Existing Objects

Some RDD transformations allow us to modify the parameters in the lambda expression rather than returning a new object. For example, in the sequence function of the aggregation function for `aggregateByKey` and `aggregate`, we can modify the original accumulator argument and define the combine function in such a way that the combination is created by modifying the first of the two accumulators. A common and effective paradigm for complicated aggregations is to define a Scala class with sequence and combine operations that return the existing object using the `this.type` annotations.

For example, suppose that we wanted to do some custom aggregation that is not already defined in Spark. Let's say that we have an RDD of key/value pairs where the keys are the pandas' instructors and the values are the pupils' panda report cards. For each instructor we want to know the length of the longest word used, the average number of words used per report card, and the number of instances of the word "happy." One valid, easy-to-read approach would be to use the `aggregateByKey` function, which takes three arguments: a zero value that represents an empty accumulator, a sequence function that takes the accumulator and a value and adds the value to the accumulator, and a combine operator that defines how the accumulators should be combined. In this instance we could define our accumulator to be an object with four fields: the total count of all the words, the total number of reports, the longest word seen so far, and the total number of mentions of the word *happy*.

For clarity, we can define this as its own object with methods for sequence and combine. We have named this object `MetricsCalculator`, and it might be coded as shown in Example 7-2.

Example 7-2. Custom aggregation object

```
class MetricsCalculator(
    val totalWords: Int,
    val longestWord: Int,
    val happyMentions: Int,
    val numberReportCards: Int
) extends Serializable {

  def sequenceOp(reportCardContent: String): MetricsCalculator = {
    val words = reportCardContent.split(" ")
    val tW = words.length
    val lW = words.map(w => w.length).max
    val hM = words.count(w => w.toLowerCase.equals("happy"))

    new MetricsCalculator(
      tW + totalWords,
      Math.max(longestWord, lW),
      hM + happyMentions,
      numberReportCards + 1
    )
  }

  def compOp(other: MetricsCalculator): MetricsCalculator = {
    new MetricsCalculator(
      this.totalWords + other.totalWords,
      Math.max(this.longestWord, other.longestWord),
      this.happyMentions + other.happyMentions,
      this.numberReportCards + other.numberReportCards
    )
  }

  def toReportCardMetrics =
    ReportCardMetrics(
      longestWord,
      happyMentions,
      totalWords.toDouble / numberReportCards
    )
}
```

We could then use this object in the arguments to our aggregation function, as shown in Example 7-4, in a routine that maps the RDD of instructors and report text to a case class with the three metrics we care about in Example 7-3.

Example 7-3. Case class for aggregations

```
case class ReportCardMetrics(
    longestWord: Int,
    happyMentions: Int,
    averageWords: Double
)
```

Example 7-4. Aggregation example without object reuse

```
  /** Given an RDD of (PandaInstructor, ReportCardText) aggregate by instructor
    * to an RDD of distinct keys of (PandaInstructor, ReportCardStatistics)
    * where ReportCardMetrics is a case class with
    *
    * longestWord -> The longest word in all of the reports written by this
    * instructor happyMentions -> The number of times this instructor was
    * mentioned the word happy averageWords -> The average number of words per
    * report card for this instructor
    */
  def calculateReportCardStatistics(
      rdd: RDD[(String, String)]
  ): RDD[(String, ReportCardMetrics)] = {

    rdd
      .aggregateByKey(
        new MetricsCalculator(
          totalWords = 0,
          longestWord = 0,
          happyMentions = 0,
          numberReportCards = 0
        )
      )(
        seqOp = (
            (reportCardMetrics, reportCardText) =>
              reportCardMetrics.sequenceOp(reportCardText)
        ),
        combOp = (x, y) => x.compOp(y)
      )
      .mapValues(_.toReportCardMetrics)
  }
```

This method is superior to using a two `map` and one `reduceByKey` method. The aggregate function combines each partition locally, then does a shuffle to perform the cross-partition reduction. However, it has the disadvantage of creating a new instance of our custom object for each record in the Dataset and for each combine step. A very simple way to reduce the cost of object creation would be to modify our `MetricsCalculator` to use Scala's `this.type` design paradigm so that the sequence operation modifies the original accumulator and the combine operation modifies the first accumulator rather than returning a new one, as shown in Example 7-5.

Example 7-5. Aggregation example with object reuse

```
class MetricsCalculatorReuseObjects(
    var totalWords: Int,
    var longestWord: Int,
    var happyMentions: Int,
    var numberReportCards: Int
) extends Serializable {
```

```
  def sequenceOp(reportCardContent: String): this.type = {
    val words = reportCardContent.split(" ")
    totalWords += words.length
    longestWord = Math.max(longestWord, words.map(w => w.length).max)
    happyMentions += words.count(w => w.toLowerCase.equals("happy"))
    numberReportCards += 1
    this
  }

  def compOp(other: MetricsCalculatorReuseObjects): this.type = {
    totalWords += other.totalWords
    longestWord = Math.max(this.longestWord, other.longestWord)
    happyMentions += other.happyMentions
    numberReportCards += other.numberReportCards
    this
  }

  def toReportCardMetrics =
    ReportCardMetrics(
      longestWord,
      happyMentions,
      totalWords.toDouble / numberReportCards
    )
}
```

Our aggregation routine will remain the same.



It should be obvious that the Scala code within the sequence operator is slower than it needs to be. Rather than performing three different functional calls on the `words` array, we ought to go through the string as a string buffer, counting the words, keeping track of the longest word, and counting the occurrence of the word *happy* (or at least use a while loop to parse the `words` array rather than three recursive calls). We have left this solution since we think it is easier to read, and the primary intention of the example is to show how to optimize the `aggregateByKey` Spark routine.

Reduce (which calls aggregate) and the fold operations (`foldLeft`, `fold`, `foldRight`) can also benefit from object reuse. However, these aggregation functions are unique. It is best to avoid mutable data structures in Spark code (and Scala code in general) because they can lead to serialization errors and may have inaccurate results. For many other RDD functions, particularly narrow transformations, modifying the first value of the argument is not safe because the transformations may be chained together with lazy evaluation and may be evaluated multiple times. For example, if you have an RDD of mutable objects, modifying the arrays with a map function

may lead to inaccurate results since the objects may be reused more times than you expect—especially if the RDD is recomputed.

Using Smaller Data Structures

Spark can be a memory hog. An important way to optimize Spark jobs for both time and space is to stick to primitive types rather than custom classes. Although it may make code less readable, using arrays rather than case classes or tuples can reduce GC overhead. Scala arrays, which are exactly Java arrays under the hood, are the most memory-efficient of the Scala collection types. Scala tuples are objects, so in some instances it might be better to use a two- or three-element array rather than a tuple for expensive operations. The Scala collection types in general incur a higher GC overhead than arrays.

Notice that our `ReportCardMetrics` object is just a wrapper for a few numeric values. Although it is less readable and less object-oriented, it is more space-efficient to use a four-element array of integers. We can maintain the same readable code paradigm by using a Scala `object` instead of a `class` and defining the sequence, and combining operations as functions on strings and arrays as shown in Example 7-6.

Example 7-6. Using an array as the aggregation object

```
object MetricsCalculator_Arrays extends Serializable {
  val totalWordIndex = 0
  val longestWordIndex = 1
  val happyMentionsIndex = 2
  val numberReportCardsIndex = 3

  def sequenceOp(
      reportCardMetrics: Array[Int],
      reportCardContent: String
  ): Array[Int] = {

    val words = reportCardContent.split(" ")
    // modify each of the elements in the array
    reportCardMetrics(totalWordIndex) += words.length
    reportCardMetrics(longestWordIndex) = Math.max(
      reportCardMetrics(longestWordIndex),
      words.map(w => w.length).max
    )
    reportCardMetrics(happyMentionsIndex) += words.count(w =>
      w.toLowerCase.equals("happy")
    )
    reportCardMetrics(numberReportCardsIndex) += 1
    reportCardMetrics
  }

  def compOp(x: Array[Int], y: Array[Int]): Array[Int] = {
```

```
    // combine the first and second arrays by modifying the elements
    // in the first array
    x(totalWordIndex) += y(totalWordIndex)
    x(longestWordIndex) = Math.max(x(longestWordIndex), y(longestWordIndex))
    x(happyMentionsIndex) += y(happyMentionsIndex)
    x(numberReportCardsIndex) += y(numberReportCardsIndex)
    x
  }

  def toReportCardMetrics(ar: Array[Int]): ReportCardMetrics =
    ReportCardMetrics(
      ar(longestWordIndex),
      ar(happyMentionsIndex),
      ar(totalWordIndex) / ar(numberReportCardsIndex)
    )
}
```

We would then need to modify our aggregation code slightly. We are not using the same custom aggregation object, and the zero value has changed. This is shown in Example 7-7.

Example 7-7. Aggregation with arrays to minimize expensive object creation

```
  def calculateReportCardStatisticsWithArrays(
      rdd: RDD[(String, String)]
  ): RDD[(String, ReportCardMetrics)] = {

    rdd
      .aggregateByKey(
        // the zero value is a four element array of zeros
        Array.fill[Int](4)(0)
      )(
        // seqOp adds the relevant values to the array
        seqOp = (reportCardMetrics, reportCardText) =>
          MetricsCalculator_Arrays
            .sequenceOp(reportCardMetrics, reportCardText),
        // combo defines how the arrays should be combined
        combOp = (x, y) => MetricsCalculator_Arrays.compOp(x, y)
      )
      .mapValues(MetricsCalculator_Arrays.toReportCardMetrics)
  }
```

Within a function, it is often beneficial to avoid intermediate object creation. It is important to remember that converting between types (such as between different flavors of Scala collections) creates intermediate objects. This is yet another place in which implicit conversions may have unfortunate performance implications.

For example, suppose that, observing our note in the previous section, you wanted to speed up the sequence function of the `MetricsCalculator_ReuseObjects` object. Then, you realized that your coworker had written a general-purpose utility that

finds the instances of the word *happy* and the longest word in a collection of strings (shown in Example 7-8).

Example 7-8. Function with implicit sequence conversions

```
def findWordMetrics[T <: Seq[String]](collection: T): (Int, Int) = {
  val iterator = collection.toIterator
  var mentionsOfHappy = 0
  var longestWordSoFar = 0
  while (iterator.hasNext) {
    val n = iterator.next()
    if (n.toLowerCase == "happy") {
      mentionsOfHappy += 1
    }
    val length = n.length
    if (length > longestWordSoFar) {
      longestWordSoFar = length
    }

  }
  (longestWordSoFar, mentionsOfHappy)
}
```

Your coworker helpfully defined her function on any type that extends a Scala `Traversable` index. Thus, you won't need to convert the array of words at all and can happily write the code shown in Example 7-9.

Example 7-9. Aggregation with bad implicit conversions

```
val totalWordIndex = 0
val longestWordIndex = 1
val happyMentionsIndex = 2
val numberReportCardsIndex = 3
def fasterSeqOp(
    reportCardMetrics: Array[Int],
    content: String
): Array[Int] = {
  val words: Seq[String] = content.split(" ")
  val (longestWord, happyMentions) = CollectionRoutines.findWordMetrics(words)
  reportCardMetrics(totalWordIndex) += words.length
  reportCardMetrics(longestWordIndex) = longestWord
  reportCardMetrics(happyMentionsIndex) += happyMentions
  reportCardMetrics(numberReportCardsIndex) += 1
  reportCardMetrics
}
```



Modifying a value passed into your transformation is not always safe, so double-check the documentation for the function you are using.

Unfortunately, in terms of object creation, this new implementation is actually worse than the previous one. It creates two extra objects containing the collection with the words each time a sequence operation is called! First, when the `findWordMetrics` routine is called, the input array has to be implicitly converted to a `Traversable` object (creating a new object of the same size), and again when your coworker's code casts the `Traversable` object to an `Iterator`.



Beyond reducing the objects that are directly allocated, Scala's implicit conversions can sometimes cause additional allocations in the process of converting.

Iterator-to-Iterator Transformations with mapPartitions

The RDD `mapPartitions` function takes as its argument a function from an `iterator` of records (representing the records on one partition) to another `iterator` of records (representing the output partition).

The `mapPartitions` transformation is one of the most powerful in Spark since it lets the user define an arbitrary routine on one partition of data. The `mapPartitions` transformation can be used for very simple data transformations like string parsing, but it can also be used for complex, expensive data-processing work to solve problems such as secondary sort or highly custom aggregations. Many of Spark's other transformations, like `filter`, `map`, and `flatMap`, can be built using `mapPartitions`. Optimizing the `mapPartitions` routines is an important part of writing complicated and performant Spark code, as we will see in Chapter 8. To allow Spark the flexibility to spill some records to disk, it is important to represent your functions inside of `mapPartitions` in such a way that your functions do not force loading the entire partition in memory (e.g., implicitly converting to a list). Iterators have many methods we can use to write functional-style transformations. You may also construct your own custom iterator extending the `Iterator` interface. When a transformation directly takes and returns an iterator without forcing it through another collection, we call it an *iterator-to-iterator* transformation.

What Is an Iterator-to-Iterator Transformation?

A Scala `iterator` object is not actually a collection but a function that defines a process of accessing the elements in a collection one by one. Not only are iterators immutable, but the same element in an iterator can only be accessed one time. In other words, iterators can only be traversed once, and they extend the Scala interface `TraversableOnce`. Iterators have some of the same methods defined on them as other immutable Scala collections, such as mappings (`map` and `flatMap`), additions (`++`), folds (`foldLeft`, `reduceRight`, `reduce`), element conditions (`forall` and `exists`), and traversals (`next` and `foreach`). In some instances, these methods behave differently than other Scala collections. Since the iterator can only be traversed once, any of the iterator methods that require looking at all the elements in the iterator will leave the original iterator empty.



Java has its own implementation of iterators, `java.util.Iterator`, which have the same benefits as Scala iterators for Spark's evaluation.



Beware of your function calls. It is easy to accidentally consume an iterator, by calling an object that traverses through the iterator such as `size`, or to trigger an implicit conversion. Iterators can be converted to any other Scala collection type. However, converting them requires accessing each of the elements. Thus, after it has been converted to a new collection type, an iterator will be at its last element (empty).

In some ways it can be helpful to conceptualize iterator methods as we would RDD methods—as either *transformations* or *actions*—because like an RDD, an iterator is actually a set of evaluation instructions rather than a stored state. Some iterator methods, like `next`, `size`, and `foreach`, traverse the iterator and evaluate it (more like an action). Others, like `map` and `flatMap`, return a new iterator—which is really a set of evaluation instructions—much like RDD transformations return a new RDD. However, in contrast to Spark transformations, iterator transformations are executed linearly, one element at time, rather than in parallel. This makes iterators slower but much easier to use than if they could be executed in parallel. For example, if we needed to store some information about the records we have seen, we could do that in a `filter` or a `map` function on the iterator, since the map/filter routine would be applied to each element sequentially. (See Example 7-12 at the end of this section.) One-to-one functions are also not chained together in iterator operations, so using three `map` calls still requires looking at each element in the iterator three times.

By "iterator-to-iterator transformation" we mean using one of these iterator "transformations" to return a new iterator rather than a) converting the iterator to a different collection or b) evaluating the iterator with one of the iterator "actions" and building a new collection. To reiterate: using a while loop to traverse the elements of an iterator and build a new collection (even a new iterator) *does not* qualify as an iterator-to-iterator transformation. Converting an iterator to a more intuitive collection type, manipulating it, and converting back to an iterator is not an iterator-to-iterator transformation. Indeed, converting the iterator argument in `mapPartitions` to a collection object eliminates all the benefits of iterator-to-iterator transformations.

Space and Time Advantages

The primary advantage of using iterator-to-iterator transformations in Spark routines is that their transformations allow Spark to selectively spill data to disk. Conceptually, an iterator-to-iterator transformation means defining a process for evaluating elements one at a time. Thus, Spark can apply that procedure to batches of records rather than reading an entire partition into memory or creating a collection with all of the output records in memory and then returning it. Consequently, iterator-to-iterator transformations allow Spark to manipulate partitions that are too large to fit in memory on a single executor without out-of-memory (OOM) errors.

Furthermore, keeping the partition as an iterator allows Spark to use disk space more selectively. Rather than spilling an entire partition when it doesn't fit in memory, the iterator-to-iterator transformation allows Spark to spill only those records that do not fit in memory, thereby saving disk I/O and the cost of recomputation. Lastly, using methods defined on iterators avoids defining intermediary data structures. Reducing the number of large intermediate data structures is a way to avoid unnecessary object creation, which can slow down garbage collection, as we talked about in "Minimizing Object Creation" on page 134.

An Example

For all their advantages, iterators can be a much harder abstraction to conceptualize and use than collection types such as arrays and hash maps, with which users may be more familiar from other languages. Here we provide an example of a complicated `mapPartitions` routine, which, given a sorted RDD of `(value, columnIndex), count)` tuples and a list of rank statistics on this partition, returns the `(value, columnIndex)` pairs that represent rank statistics, shown in Example 7-10. This method is part of the optimal solution to the "Goldilocks problem," which is presented in full in "Goldilocks Version 4: Reduce to Distinct on Each Partition" on page 210 and introduced in "The Goldilocks Example" on page 171.

Example 7-10. Example `mapPartitions`

```
  private def findTargetRanksIteratively(
      sortedValueColumnPairs: RDD[(Double, Int)],
      ranksLocations: Array[(Int, List[(Int, Long)])]
  ): RDD[(Int, Double)] = {

    sortedValueColumnPairs.mapPartitionsWithIndex(
      (partitionIndex: Int, valueColumnPairs: Iterator[(Double, Int)]) => {
        val targetsInThisPart: List[(Int, Long)] =
          ranksLocations(partitionIndex)._2
        if (targetsInThisPart.nonEmpty) {
          val columnsRelativeIndex: collection.MapView[Int, List[Long]] =
            targetsInThisPart.groupBy(_._1).view.mapValues(_.map(_._2))
          val columnsInThisPart = targetsInThisPart.map(_._1).distinct

          val runningTotals: mutable.HashMap[Int, Long] = new mutable.HashMap()
          runningTotals ++= columnsInThisPart
            .map(columnIndex => (columnIndex, 0L))
            .toMap

          // filter this iterator, so that it contains only those (value, columnIn
dex)
          // that are the ranks statistics on this partition
          // Keep track of the number of elements we have seen for each columnIndex
using the
          // running total hashMap.
          valueColumnPairs
            .filter { case (value, colIndex) =>
              lazy val thisPairIsTheRankStatistic: Boolean = {
                val total = runningTotals(colIndex) + 1L
                runningTotals.update(colIndex, total)
                columnsRelativeIndex(colIndex).contains(total)
              }
              (runningTotals contains colIndex) && thisPairIsTheRankStatistic
            }
            .map(_.swap)
        } else {
          Iterator.empty
        }
      }
    )
  }
```

This routine is a good example of a place where we are likely to see performance gains from an iterator-to-iterator transformation, since it is a complicated routine performed on partitions that we anticipate will be too large to fit in memory. However, it is an instance where using iterators is, from a design perspective, a nonobvious choice because we have to keep a map of running totals with the number of elements for each column we have seen so far. A more straightforward way to

design this routine would be as follows: loop through the iterator, store the running totals in a `HashMap`, and build a new collection of the elements we want to keep using an array buffer—then convert the array buffer to an iterator, shown in Example 7-11.

Example 7-11. `mapPartitions` example without an iterator-to-iterator transformation

```
def withArrayBuffer(
    valueColumnPairsIter: Iterator[((Double, Int), Long)],
    targetsInThisPart: List[(Int, Long)]
): Iterator[(Int, Double)] = {

  val columnsRelativeIndex: collection.MapView[Int, List[Long]] =
    targetsInThisPart.groupBy(_._1).view.mapValues(_.map(_._2))

  // The column indices of the pairs that are desired rank statistics that live in
  // this partition.
  val columnsInThisPart: List[Int] = targetsInThisPart.map(_._1).distinct

  // A HashMap with the running totals of each column index. As we loop through
  // the iterator. We will update the hashmap as we see elements of each
  // column index.
  val runningTotals: mutable.HashMap[Int, Long] = new mutable.HashMap()
  runningTotals ++= columnsInThisPart
    .map(columnIndex => (columnIndex, 0L))
    .toMap

  // we use an array buffer to build the resulting iterator
  val result: ArrayBuffer[(Int, Double)] =
    new scala.collection.mutable.ArrayBuffer()

  valueColumnPairsIter.foreach { case ((value, colIndex), count) =>

    if (columnsInThisPart contains colIndex) {

      val total = runningTotals(colIndex)
      // the ranks that are contains by this element of the input iterator.
      // get by filtering the
      val ranksPresent = columnsRelativeIndex(colIndex)
        .filter(index => (index <= count + total) && (index > total))
      ranksPresent.foreach(r => result += ((colIndex, value)))
      // update the running totals.
      runningTotals.update(colIndex, total + count)
    }
  }
  // convert
  result.toIterator
}
```

At first this looks like an okay solution since we are estimating that the number of elements we are returning is small and because array buffers are usually a relatively

performant way to build up Scala collections. However, if the input data is very large relative to the cluster size, we still see out-of-memory errors and failures in this step. A more efficient solution would be to use an iterator-to-iterator transformation. We can convert this subroutine to an iterator-to-iterator transformation, although our routine is not parallelizable (it requires keeping a list of running totals). We can do this because the subroutine we need can be completed on one element of the iterator without any information about the other elements. The final solution uses the `filter` function of iterators—to eliminate any elements that are not in the final data—and a `flatMap` to build the new iterator of elements in the resulting partitions, as shown in Example 7-12.

Example 7-12. `mapPartitions` with iterator-to-iterator transformations

```
  def asIteratorToIteratorTransformation(
      valueColumnPairsIter: Iterator[((Double, Int), Long)],
      targetsInThisPart: List[(Int, Long)]
  ): Iterator[(Int, Double)] = {

    val columnsRelativeIndex =
      targetsInThisPart.groupBy(_._1).mapValues(_.map(_._2))
    val columnsInThisPart = targetsInThisPart.map(_._1).distinct

    val runningTotals: mutable.HashMap[Int, Long] = new mutable.HashMap()
    runningTotals ++= columnsInThisPart
      .map(columnIndex => (columnIndex, 0L))
      .toMap

    // filter out the pairs that don't have a column index that is in this part
    val pairsWithRanksInThisPart = valueColumnPairsIter.filter {
      case (((value, colIndex), count)) =>
        columnsInThisPart contains colIndex
    }

    // map the valueColumn pairs to a list of (colIndex, value) pairs that corre
spond
    // to one of the desired rank statistics on this partition.
    pairsWithRanksInThisPart.flatMap { case (((value, colIndex), count)) =>

      val total = runningTotals(colIndex)
      val ranksPresent: List[Long] = columnsRelativeIndex(colIndex)
        .filter(index =>
          (index <= count + total)
            && (index > total)
        )

      val nextElems: Iterator[(Int, Double)] =
        ranksPresent.map(r => (colIndex, value)).toIterator

      // update the running totals
```

```
      runningTotals.update(colIndex, total + count)
      nextElems
    }
  }
```

This approach allows the function to spill to disk selectively by working with each element in the iterator one at a time. This implementation saves space by incrementally building the result rather than storing the new collection type in memory as an array buffer. It saves a penny on garbage collection by not creating the array buffer as an intermediate step.



If you are using an `ArrayBuffer` to build a new collection for `mapPartitions`, it is always possible (and likely more performant) to use a `map` or `flatMap` on the iterator to incrementally add new elements.

Set Operations

Spark has a variety of set-like operations, some of which are expensive and some of which have different behavior than the mathematical definitions of the equivalent operations. In this section we hope to explain how to use these operations safely and effectively.

Since RDDs aren't distinct, they mainly differ from mathematical set operations in how they handle duplicates. For example, `union` merely combines its arguments, so the result of union will always have the size of both RDDs combined. `intersection` and `subtract` are defined similarly to their set-theoretic counterparts, but since the input RDDs (unlikely mathematical sets) can have duplicates, the results may be unexpected. Subtracting will remove all of the elements in the first RDD that have a key present in the second RDD. Thus, it is possible that by subtracting, the result will be smaller than the size of the first RDD minus the size of the second, breaking one of the laws of set theory.

For example, the simple unit test in Example 7-13 will pass.

Example 7-13. Subtract example

```
val a = Array(1, 2, 3, 4, 4, 4, 4)
val b = Array(3, 4)
val rddA = sc.parallelize(a)
val rddB = sc.parallelize(b)
val rddC = rddA.subtract(rddB)
assert(rddC.count() < rddA.count() - rddB.count())
```

In Spark, `intersection` co-groups the argument RDDs using their values as keys and filters out those elements that don't appear in both. Consequently, the result of RDD `intersection` contains no duplicates. Although this is the expected behavior for `intersection`, using several set operations on RDDs containing duplicates can lead to unexpected behavior. The union of the two RDDs in Example 7-13 is an RDD containing two elements, `1` and `2`. Thus, as the unit test in Example 7-14 demonstrates, we cannot always "re-create" `rddA` as the union of the intersection and the subtraction.[2]

Example 7-14. Intersection example

```
val a = Array(1, 2, 3, 4, 4, 4, 4)
val b = Array(3, 4)
val rddA = sc.parallelize(a)
val rddB = sc.parallelize(b)
val intersection = rddA.intersection(rddB)
val subtraction = rddA.subtract(rddB)
val union = intersection.union(subtraction)
assert(!rddA.collect().sorted.sameElements(union.collect().sorted))
```



To make an RDD more like a set, you can use `distinct` prior to computing any set operations. However, calling `distinct` will cause a shuffle, and if the data is not already partitioned, this can be expensive.

Reducing Setup Overhead

Some operations require setup work per-worker or per-partition, like creating a database connection or setting up a random number generator. For transformations you can use `mapPartitions`, do the setup work per partition in the map function, and then perform your desired transformation on the iterator for the partition. We will illustrate doing this with a pseudorandom number generator in Example 7-15.

Example 7-15. Create one random number generator per partition using broadcast variable

```
rdd.mapPartitions { itr =>
  // Only create once RNG per partitions
  val r = new Random()
  itr.filter(x => r.nextInt(10) == 0)
}
```

2 If A and B are sets, (A - B) ∪ (B ∩ A) = A in all cases. This is not true in Spark. If `rddA` or `rddB` have duplicate keys or if `rddA` and `rddB` have overlapping keys, then (A - B) ∪ (B ∩ A) is a *subset* of A.



It is important to remember to use an iterator-to-iterator transformation to allow spilling to disk selectively, as discussed in "Iterator-to-Iterator Transformations with mapPartitions" on page 141.

Beyond using this pattern to reduce setup overhead in transformations, another common pattern is to create a connection inside of an action to save the data. If your work is writing out the data, you can use the same pattern as with `mapPartitions` except with `foreachPartition`.

If the setup work can be serialized, a broadcast variable can distribute the object that we cover next. If the setup work can't be serialized, a broadcast variable with a `transient lazy val` can be used as well. See Example 7-17 in the next section.

Shared Variables

Spark has two types of shared variables—broadcast variables and accumulators—each of which can only be written in one context (driver or worker, respectively) and read in the other. Broadcast variables can be written in the driver program and read on the executors, whereas accumulators are written onto the executors and read on the driver.

Broadcast Variables

Broadcast variables give us a way to take a local value on the driver and distribute a read-only copy to each machine rather than shipping a new copy with each task. Broadcast variables might not seem especially useful, since we can just capture a local variable in our closure to transfer data from the driver to the workers; however, the savings of only sending one copy per machine versus sending one copy per task can make a huge difference, especially when the same broadcast variable is used in additional transformations. Two common examples of using broadcast variables are a) broadcasting a small table to join against and b) broadcasting a machine learning model to be able to run the predictions on our data.

Creating a broadcast variable is done by calling `broadcast` on the `SparkContext`. This distributes the value to the workers and gives us back a wrapper that allows us to access the value of the workers by calling `value`, as shown in Examples 7-16 and 7-17. If a `broadcast` variable is created with a variable input, the input should not be modified after the variable has been created since existing workers will not see the updates, and new workers may see the new value.

Example 7-16. Sample broadcast of a hashset of invalid panda locations to filter out

```
val invalid: HashSet[Long] = HashSet() ++ invalidPandas
val invalidBroadcast = sc.broadcast(invalid)
input.filter { panda => !invalidBroadcast.value.contains(panda.id) }
```

Example 7-17. Create one random number generator per worker

```
class LazyPrng {
  @transient lazy val r = new Random()
}
def customSampleBroadcast[T: ClassTag](
    sc: SparkContext,
    rdd: RDD[T]
): RDD[T] = {
  val bcastprng = sc.broadcast(new LazyPrng())
  rdd.filter(x => bcastprng.value.r.nextInt(10) == 0)
}
```



The value for a broadcast variable must be a local, serializable value: no RDDs or other distributed data structures.

Internally, Spark uses broadcast variables for the Hadoop job configuration objects and large blocks of Python code for UDFs. If a broadcast variable is no longer needed, you can explicitly remove it by calling `unpersist()` on the broadcast variable.

Accumulators

Accumulators are the second type of Spark's shared variables, allowing us to collect by-product information from a transformation or action on the workers and then bring the result back to the driver. With Spark's execution model, Spark adds to accumulators only once the computation has been triggered (e.g., by an action). If the computation happens multiple times, Spark will update the accumulator each time. This multiple counting can be desirable for process-level information, like computing the entire time spent parsing records. However, it can be disastrous for data-related information like counting the number of invalid records.



Accumulators can be unpredictable. In their current state, they are best used where potential multiple counting is the desired behavior.

Accumulators have a number of built-in types that make it easy to create an accumulator for common use cases. Accumulators are not intended for collecting large amounts of information, so if you find yourself adding a large number of elements to a collection or appending to a string, you may wish to consider a separate action instead of an accumulator. The default operation for numeric accumulators is the + operation, so we could use this to sum the fuzziness of all of the pandas as shown in Example 7-18 (now with names).

Example 7-18. Compute fuzziness of pandas with accumulators (legacy)

```
def computeTotalFuzzyNess(
    sc: SparkContext,
    rdd: RDD[RawPanda]
): (RDD[(String, Long)], Double) = {
  // Create an named accumulator for doubles
  val acc = sc.doubleAccumulator("fuzzyNess")
  val transformed = rdd.map { x => acc.add(x.attributes(0)); (x.zip, x.id) }
  // accumulator still has zero value
  // Note: This example is dangerous since the transformation may be
  // evaluated multiple times.
  transformed.count() // force evaluation
  (transformed, acc.value)
}
```

Additionally, accumulators support a wide variety of data types provided the operation is associative, but some are easier to get in trouble with than others. To use an accumulator of a different type, you need to implement the `AccumulatorV2[InputType, ValueType]` interface and provide `reset`, `copy`, `isZero`, `value`, `merge`, and `add` methods. You are responsible for the specifics of the class that keeps track of the accumulated values. In general, a simple `var` or two will do the trick. In addition to the required method, override `resetAndCopy` to improve performance in certain cases.

Generally, the `reset` and `copy` methods are used together with the `resetAndCopy` method, which can often be more efficiently implemented to avoid the copy stage (as is done in both of the custom accumulator examples, Examples 7-19 and 7-20). The `reset` method resets the value of the current accumulator back to zero so that `isZero`, if called, will return true. The `copy` method needs to create a copy of the provided accumulator, with the new accumulator having the same value as the current accumulator. This is called when copying the value to the workers so that Spark can avoid the expense (and confusion) of copying any of the previously accumulated work to the drivers.

The type parameters of the `AccumulatorV2` interface specify the type being accumulated over (`add`) and the final return type (`value`). Importantly, this does *not* constrain or specify the type used to hold the accumulation itself. A single variable is used to

keep track of values in the following examples. However, you need not limit yourself to one variable. Inside of many of Spark's numeric accumulators, two `vars` are used.

The `merge` method for the accumulator API's type signature takes the same base `AccumulatorV2` type. Since the `AccumulatorV2` trait doesn't specify anything about how workers should keep track of the values as they are evaluated, you will need to cast the accumulator you receive to the expected type so you can access your own internal accumulation field(s). A basic implementation of this is shown in Example 7-19.

Example 7-19. Compute maximum panda ID

```
def computeMaxFuzzyNess(
    sc: SparkContext,
    rdd: RDD[RawPanda]
): (RDD[(String, Long)], Option[Double]) = {
  class MaxDoubleAccumulator extends AccumulatorV2[Double, Option[Double]] {
    // Here is the var we will accumulate our value in to.
    var currentVal: Option[Double] = None
    override def isZero = currentVal.isEmpty

    // Reset the current accumulator to zero - used when sending over the wire
    // to the workers.
    override def reset() = {
      currentVal = None
    }

    // Copy the current accumulator - this is only realy used in context of
    // copy and reset - but since its part of the public API lets be safe.
    def copy() = {
      val newCopy = new MaxDoubleAccumulator()
      newCopy.currentVal = currentVal
      newCopy
    }

    // We override copy and reset for "speed" - no need to copy the value if
    // we care going to zero it right away. This doesn't make much difference
    // for Option[Double] but for something like Array[X] could be huge.

    override def copyAndReset() = {
      new MaxDoubleAccumulator()
    }

    // Add a new value (called on the worker side)
    override def add(value: Double) = {
      currentVal = Some(
        // If the value is present compare it to the new value - otherwise
        // just store the new value as the current max.
        currentVal.map(acc => Math.max(acc, value)).getOrElse(value)
      )
    }
```

```
    override def merge(other: AccumulatorV2[Double, Option[Double]]) = {
      other match {
        case otherFuzzy: MaxDoubleAccumulator =>
          // If the other accumulator has the option set merge it in with
          // the standard add procedure. If the other accumulator isn't set
          // do nothing.
          otherFuzzy.currentVal.foreach(value => add(value))
        case _ =>
          // This should never happen, Spark will only call merge with
          // the correct type - but that won't stop someone else from calling
          // merge so throw an exception just in case.
          throw new Exception(
            "Unexpected merge with unsupported type" + other
          )
      }
    }
    // Return the accumulated value.
    override def value = currentVal
  }
  // Create a new custom accumulator
  val acc = new MaxDoubleAccumulator()
  sc.register(acc)
  val transformed = rdd.map { x => acc.add(x.attributes(0)); (x.zip, x.id) }
  // accumulator still has None value.
  // Note: This example is dangerous since the transformation may be
  // evaluated multiple times.
  transformed.count() // force evaluation
  (transformed, acc.value)
}
```

This still requires that the result be the same as the type we are accumulating. If we wanted to collect all of the distinct elements, we would likely want to collect a set, and the types would be different. This is shown in Example 7-20.

Example 7-20. Compute unique panda IDs

```
def uniquePandas(sc: SparkContext, rdd: RDD[RawPanda]): HashSet[Long] = {
  class UniqParam extends AccumulatorV2[Long, HashSet[Long]] {
    var accValue: HashSet[Long] = new HashSet[Long]()

    def value = accValue

    override def copy() = {
      val newCopy = new UniqParam()
      newCopy.accValue = accValue.clone
      newCopy
    }
    override def reset() = {
      this.accValue = new HashSet[Long]()
    }
```

```
    override def isZero() = {
      accValue.isEmpty
    }

    // We override copy and reset for speed - no need to copy the value if
    // we care going to zero it right away.
    override def copyAndReset() = {
      new UniqParam()
    }
    // For adding new values
    override def add(value: Long) = {
      accValue += value
    }
    // For merging accumulators
    override def merge(other: AccumulatorV2[Long, HashSet[Long]]) = {
      other match {
        case otherUniq: UniqParam =>
          accValue = accValue ++ otherUniq.accValue
        case _ =>
          throw new Exception("only support merging with same type")
      }
    }
  }
  // Create an accumulator for keeping track of unique values
  val acc = new UniqParam()
  // Register with a name
  sc.register(acc, "Unique values")
  val transformed = rdd.map { x => acc.add(x.id); (x.zip, x.id) }
  // accumulator still has Double.MinValue
  transformed.count() // force evaluation
  acc.value
}
```



The `value` function can perform complex work and return a different type than the input type or internal accumulated value. For example, if you were computing the average, you might have a `value` function that divides two longs, returning a double.

When working with cached data, our accumulators can seem almost consistent, but as discussed in "Interaction with Accumulators" on page 167 this is only true until machine loss or other recompute.



You may provide a name for accumulators in Scala so they show up in the web UI. Simply add a name as the second param. This does involve calling `toString` on the accumulator, though—so if that is an expensive operation, leave your accumulator unnamed.

Reusing RDDs

Spark offers several options for RDD reuse, including persisting, caching, and checkpointing. However, Spark does not perform any of these automatically[3] because storing RDD for reuse breaks some pipelining, which can be a waste if the RDD is only used once or if the transformation is inexpensive to recompute. All kinds of persistence (of which caching is one type) and checkpointing have some cost and are unlikely to improve performance for operations that are performed only once. Furthermore, on large datasets, the cost of persisting or checkpointing can be so high that recomputing is more desirable. However, for some specific kinds of Spark programs, reusing an RDD can lead to huge performance gains, both in terms of speed and reducing failures.

Cases for Reuse

In this section we cover some instances when persisting or checkpointing RDDs may foster performance gains. Broadly speaking, the most important cases for reuse are using an RDD many times; performing multiple actions on the same RDD; and for long chains of (or very expensive) transformations.

Iterative computations

For transformations that use the same parent RDD multiple times, reusing an RDD forces evaluation of that RDD and so can help avoid repeated computations. For example, if you were performing a loop of joins to the same dataset, persisting that dataset could lead to huge performance improvements since it ensures that the partitions of that RDD will be available in-memory to do each join.

In Example 7-21 we are computing the root mean squared error (RMSE) on a number of different RDDs representing predictions from different models. To do this we have to join each RDD of predictions to an RDD of the data in the validation set.



In this example, we use `persist()`, which persists the RDD in memory. As we will explain in "Types of Reuse: Cache, Persist, Checkpoint, Shuffle Files" on page 159, `cache()` is equivalent to `persist()`, which is equivalent to `persist("MEMORY_ONLY")`.

Example 7-21. A function with iterative computations

```
val testSet: Array[RDD[(Double, Int)]] =
  Array(
```

3 Some notable exceptions are inside of certain ML algorithms, which, if passed in an unpersisted RDD, will automatically persist and unpersist the RDD.

```
        validationSet.mapValues(_ + 1),
        validationSet.mapValues(_ + 2),
        validationSet)
    validationSet.persist() //persist since we are using this RDD several times
    val errors = testSet.map( rdd => {
        rmse(rdd.join(validationSet).values)
    })
```

Without persisting, Spark would have to reload and repartition the training dataset RDD to complete the `join`. However, with persistence, the training RDD will stay loaded in memory on the executors with each run of the algorithm. We discuss performance considerations with different kinds of joins in detail in "Core Spark Joins" on page 111.

Checkpointing, another form of RDD reuse that writes an RDD to external storage, will also break the RDD's lineage. However, checkpointing will keep the partitions loaded on the executors.

Multiple actions on the same RDD

If you do not reuse an RDD, each action called on an RDD will launch its own Spark job with the full lineage of RDD transformations. Persisting and checkpointing breaks the RDD's lineage, so the same series of transformations preceding the `persist` or `checkpoint` call will be executed only once. Because persisting or checkpointing an RDD lasts for the duration of a Spark application (although it *may* be evicted by subsequent cached/persisted data), an RDD persisted during one Spark job will be available in a subsequent job executed with the same `SparkContext`. For example, suppose that we wanted to collect the first 10% of the records in an RDD. We could use the code in Example 7-22, which calls `sortByKey`, then `count`, then `take`.

Example 7-22. An example of two actions without a persist step

```
val sorted = rddA.sortByKey()
val count = sorted.count() // sorted Action 1
val sample: Long = count / 10
val sampled = sorted.take(sample.toInt) // sorted Action 2
```

The `sortByKey` (and presumably the read operation) needed to create the RDD, `sorted`, will occur twice if we do not store the RDD: once in the job called by `count` and again in the job called by `take`. We can't test this element of the execution programmatically, but if you were to run this application and view the web UI, you would see that this code launches two jobs, and each one includes a sort stage. However, if we add a persist or checkpoint call before the actions (as shown in Example 7-23), the transformation will only be executed once since Spark builds a lineage graph from either an RDD's creation or a persisted/checkpointed RDD.

Example 7-23. Two actions with a persist step

```
val sorted = rddA.sortByKey()
sorted.persist()
val count = sorted.count() // sorted Action 1
val sample: Long = count / 10
val sampled = sorted.take(sample.toInt) // sorted Action 2
```



Persisted RDDs only survive for the duration of a Spark application. To reuse data between Spark applications, use checkpointing with the same directory.

If the cost to compute each partition is very high

Even if a program does not use the same RDD multiple times, persisting and checkpointing can speed up a routine and reduce the cost of failures by storing intermediary results. Persisting or checkpointing can be particularly useful if the cost of computing one partition is very high because they ensure that the entire expensive operation will not need to be recomputed in the case of downstream failures.

For example, if your program requires a long series of one-to-one transformations, those transformations will all be combined into very computationally intensive tasks. While this is good so long as the tasks succeed and fit in memory, it does mean that if one of the downstream transformations fails, then the cost to recompute a single partition may be enormous. If all of the narrow transformations together create more GC overhead or memory strain than your cluster's executors can handle, then checkpointing or persisting `off_heap` can be particularly useful. Both persisting `off_heap` and checkpointing allow the RDD to be stored outside of the Spark executor memory, leaving space to compute. These options are also the only way to prevent recomputation if the entire Spark worker fails. Sometimes breaking up a long lineage graph for its own sake can help a job succeed since it means each of the tasks will be smaller.

Narrow transformations are generally faster than wide ones. However, some individual narrow transformations, such as training a model per partition or working with very wide rows, can be expensive. In these cases, reusing an RDD after the expensive computation so it is not recomputed may improve performance.

Deciding if Recompute Is Inexpensive Enough

Although persisting in memory is a flagship feature of Spark, it is not free. It is space-intensive to store data in memory and will take time to serialize and deserialize. Persisting in memory and in-memory computations are both done in the Spark executor JVM. Thus, persisting in memory may take space that could be used for

downstream computations or increase the risk of memory failures. Caching with Java-based memory structures (any of Spark's options besides using `off_heap` storage options) will incur a much higher garbage collecting cost than will recomputing.

Persisting to disk or *checkpointing* (writing the RDD to an external filesystem) has the disadvantages of MapReduce, causing expensive write and read operations. If the RDD is checkpointed or persisted to disk, we must factor in not only the disk space used on the cluster to write the RDD but also the computational cost on the Spark executors of the additional disk I/O. In most cases, checkpointing a large RDD can be used to reduce failures in high-traffic clusters but rarely leads to performance improvements, even if the RDD has to be recomputed due to the high cost of checkpointing.



Our experience has been that it is easy to underestimate just how expensive storing and reading an RDD is relative to recomputing. We have also found that for relatively simple operations the cost of the read operation needed to load the RDD far outweighs the others, so persisting is most useful when it prevents triggering another read operation or in the case of many iterative computations.

Furthermore, breaking an RDD's lineage by forcing evaluation through persisting or checkpointing prevents transformations with narrow dependencies from being combined into a single task. Consequently, we lose some of the narrow transformations' value when they cannot be combined and executed in one task. For instance, persisting or checkpointing between a simple `map` and `filter` step will break pipelining so that the previously intermediate data can be persisted, causing Spark to do two passes through the data rather than just one, since the transformation has to be evaluated in order to materialize the RDD after the `map`. Breaking lineage between narrow transformations is only desirable in the most extreme cases.

The preceding guidelines are good heuristics for when reuse will provide significant benefits. In general, it is worth reusing an RDD rather than recomputing it if the computation is large relative to your cluster and the rest of your job. The best way to tell if you need to reuse your RDDs is to run a job. If your job runs very slowly, see if persisting the RDDs may help before attempting to rewrite the program since persisting and checkpointing will help reduce the cost of recomputing data in the case of a failure or eliminate it altogether. If a job is failing with GC or out-of-memory errors, checkpointing or persisting `off_heap` may allow the job to complete, particularly if the cluster is noisy. On the other hand, if you were already persisting with the options that use in-memory persistence, consider removing the persist call or switching to checkpointing or `off_heap` persistence.



If you are testing some code before putting it into production, consider creating the persistence level with a variable so that you can pass in a persistence level to try as a command-line argument. The function presented in Example 7-24 uses this paradigm; it contains a `storageLevel` argument (which could be `NONE`).

Types of Reuse: Cache, Persist, Checkpoint, Shuffle Files

If you decide that you need to reuse your RDD, Spark provides a multitude of options for how to store the RDD. Thus, it is important to understand when to use the various types of persistence. There are three primary operations that you can use to store your RDD: cache, persist, and checkpoint. In general, caching (equivalent to persisting with the in-memory storage) and persisting are most useful to avoid recomputation during one Spark job or to break RDDs with long lineages, since they keep an RDD on the executors during a Spark job. Checkpointing is most useful to prevent failures and a high cost of recomputation by saving intermediate results. Like persisting, checkpointing helps avoid computation, thus minimizing the cost of failure, and avoids recomputation by breaking the lineage graph.

Persist and cache

Persisting an RDD means materializing an RDD (usually by storing it in memory on the executors) for reuse *during the current job*. Spark remembers a persisted RDD's lineage so that it can recompute it for the duration of a Spark job if one of the persisted partitions is lost. After the job ends, the `persist` function takes a `StorageLevel` argument that specifies how the RDD should be stored. Spark provides a number of different storage levels as constants, but each one is created based on five attributes of how to store the RDD: `useDisk`, `useMemory`, `useOffHeap`, `deserialized`, and `replication`. Calling `toString` on a storage level will reveal what options it contains. The Spark documentation about persistence includes a fairly comprehensive list of the out-of-the-box storage options that are exposed to you.

Still, we think it may be useful to provide some more information about each of the five properties that compose each storage option. This should give you a deeper understanding of which option to choose:

`useDisk`
: If set, partitions that do not fit in memory will be written to disk.

  The storage-level flags containing `DISK` (such as `MEMORY_AND_DISK`) enable this. By default, if partitions do not fit in memory, they will simply be evicted and will need to be recomputed when the persisted RDD is used (see "LRU Caching" on page 163). Therefore, persisting to disk can ensure that recomputation of those additional large partitions is avoided. However, reading from disk can be time-

intensive, so persistence to disk is only important if the cost of recomputation is particularly high.



It may be beneficial to allow writing to disk if you expect that an RDD cannot fit in memory. However, if the cost of recomputing the partitions is not high (they are simple mappings and don't reduce the size of the data), it may actually be faster to recompute some partitions rather than read from disk.

`useMemory`
: If set, the RDD will be stored in memory or be directly written to disk.

  The `DISK_ONLY` storage levels are the only options that mark this as false. Most of the speed benefits of caching come from keeping RDDs in memory, so if the motivation for reuse is fast access for repeated computations, it is probably a good idea to choose a storage option that stores partitions in memory. However, there are some cases where disk-only persistence makes sense, e.g., when the computation is more expensive than reading from the local disk or the network filesystem is especially slow (such as with certain object stores).

`useOffHeap`
: If set, the RDD will be stored outside of the Spark executor in an external system such as Tachyon.

  The storage option `off_heap` enables this property. If memory is a serious issue, or a cluster is noisy and partitions are evicted, this option may be compelling. We will talk more about the benefits of Tachyon in "Alluxio (nee Tachyon)" on page 163.

`deserialized`
: If set, the RDD will be stored as deserialized Java objects.

  This can make storing RDDs more space efficient, especially when using a faster serializer—but incurs some performance overhead. Storage options that include the "`_SER`" suffix such as `MEMORY_ONLY_SER` enable serialization.



If your RDD is too large to persist in-memory, first try to serialize it with the `MEMORY_ONLY_SER` option. This will keep the RDD fast to access, but will decrease the memory needed to store it.

`replication`
: Replication is an integer that controls the number of copies of the persisted data to be stored in the cluster.

By default this is set to 1; however, serialization options that end in `_2` such as `DISK_ONLY_2` replicate each partition across two nodes. Use this option to ensure faster fault tolerance. However, be aware that persistence with replication incurs double the space and speed costs of persistence without replication. Replication is usually only necessary in an instance of a noisy cluster or bad connection where failures are unusually likely. It might also be useful if you do not have time to recompute in case of failure, such as when serving a live web application.



The RDD operation `cache()` is equivalent to the persist operation with no storage-level argument, i.e., `persist()`. Both `cache()` and `persist()` persist the RDD with the default storage-level `MEMORY_ONLY`, which is equivalent to `StorageLevel(false, true, false, true)`, which stores RDDs in-memory as deserialized Java objects, does not write to disk as partitions get evicted, and doesn't replicate partitions.

Checkpointing

Checkpointing writes the RDD to an external storage system such as HDFS or Amazon S3, and—in contrast to persisting—forgets the RDD's lineage. Since checkpointing requires writing the RDD outside of Spark, checkpointed information survives beyond the duration of a single Spark application and forces evaluation of an RDD. Checkpointing takes up more space in external storage and may be slower than persisting since it requires potentially costly write operations. However, it does not use any Spark memory and will not incur recomputation if a Spark worker fails.

Figure 7-2 illustrates the difference between in-memory persistence and checkpointing and RDD. Persisting stores the RDD's partitions in-memory or on disk in the caching layer of each executor. Checkpointing writes each partition to some external system, which should be persistent (like HDFS, etc.).

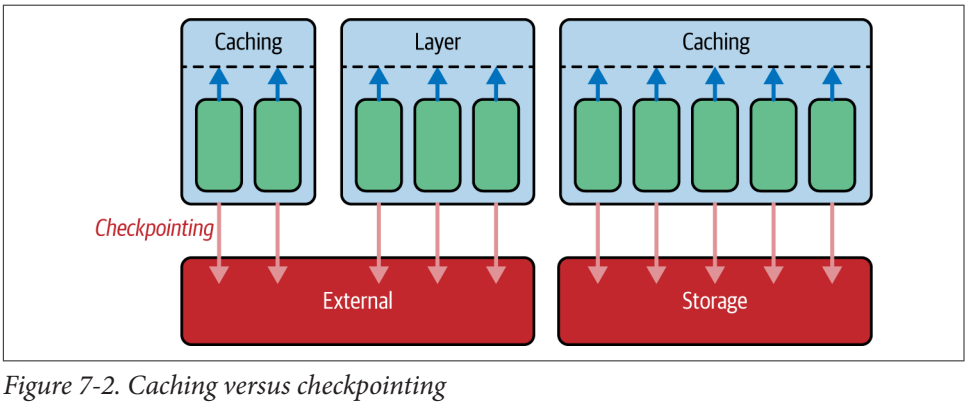


Figure 7-2. Caching versus checkpointing

It is best to use checkpointing when the cost of failure and recomputation is of more concern than additional space in external storage. Broadly speaking, we advise persisting when jobs are slow and checkpointing when they are failing. If a Spark job is failing due to out-of-memory errors, checkpointing will reduce the cost and likelihood of failure without using up memory on the executors. If your jobs are failing due to network errors or preemption on a noisy cluster, checkpointing can reduce the likelihood of failure by breaking up a long-running job into smaller segments. To call `checkpoint`, call `setCheckpointDir(directory: String)` from the `SparkContext` object and pass in a path to a location on HDFS to write the intermediate results. Then, in the Spark job, call `.checkpoint()` from the RDD.

Checkpointing example

Example 7-24 makes use of custom storage levels and checkpointing options. The function is used in the Goldilocks example, which we describe in detail in "Goldilocks Version 4: Reduce to Distinct on Each Partition" on page 210, that makes use of custom storage level and checkpointing options. In this case we are doing several very expensive transformations: first a sort and then two very substantial map partition routines. When running on a noisy cluster, we found it advantageous to checkpoint this function after the sort. The value of the `directory` parameter is the checkpoint directory. The `sorted` value is a sorted RDD or key/value pairs.

Example 7-24. Checkpoint example

```
def findQuantilesWithCustomStorage(
    valPairs: RDD[((Double, Int), Long)],
    colIndexList: List[Int],
    targetRanks: List[Long],
    storageLevel: StorageLevel = StorageLevel.MEMORY_AND_DISK,
    checkPoint: Boolean,
    directory: String = ""
): Map[Int, Iterable[Double]] = {

  val n = colIndexList.last + 1
  val sorted = valPairs.sortByKey()
  if (storageLevel != StorageLevel.NONE) {
    sorted.persist(storageLevel)
  }

  if (checkPoint) {
    sorted.sparkContext.setCheckpointDir(directory)
    sorted.checkpoint()
  }

  val partitionColumnsFreq = getColumnsFreqPerPartition(sorted, n)
  val ranksLocations =
    getRanksLocationsWithinEachPart(targetRanks, partitionColumnsFreq, n)
```

```
  val targetRanksValues = findTargetRanksIteratively(sorted, ranksLocations)
  targetRanksValues.groupByKey().collectAsMap()
}
```



Spark includes a *local checkpointing* option that truncates the RDD's lineage graph but doesn't persist to stable storage. This is not suitable for clusters that may experience failures, preemption, or dynamic scale-downs during the time the RDD may be referenced.

Alluxio (nee Tachyon)

Tachyon is a distributed, in-memory storage system that is developed separately from Spark. It sits above a storage system, such as Amazon S3 or HDFS, and can be used on its own or with an external computational framework such as Spark or MapReduce. Like Spark, Tachyon can be used in standalone cluster mode or with Mesos or YARN. Read more about Tachyon's architecture and how to integrate it with Spark in the Tachyon documentation.

Tachyon can be used as an input or output source for Spark applications (data stored with Tachyon can be used to create RDDs) or for `off_heap` persistence during a Spark application. Using Tachyon for persistence has several advantages. First, it reduces garbage collection overhead, since data is not stored as Java objects. Second, it allows multiple executors to share the same external memory pool in Tachyon. Third, since the data is stored in memory outside of Spark, it is not lost if individual executors crash. It can be particularly useful if you want to reuse an RDD but are running out of memory or seeing garbage collection errors. It is also the best way to reuse a very large RDD between multiple applications.



Tachyon's developer and user communities are very strong in China, so part of its documentation may be stronger in Mandarin than in English.

LRU Caching

RDDs that are stored in memory and/or on disk in Spark are not automatically unpersisted when they are no longer going to be used downstream. Instead, RDDs stay in memory for the duration of a Spark application, until the driver program calls the function `unpersist`, or memory/storage pressure causes their eviction. Spark uses *Least Recently Used* (LRU) caching to determine which partitions to evict if the executors begin to run out of memory.

LRU caching dictates that the data structure that was least recently accessed will be evicted. However, because of lazy evaluation, it may be a bit tricky to predict which partitions will be evicted first. Generally, Spark evicts the oldest partitions; those that were created or used in the earliest Spark job or in the earliest stage within a given job. LRU caching behaves differently for different persistence options. For memory-only persistence operations configured with LRU caching, Spark will recompute the evicted partition each time it is needed. For memory and disk options, LRU caching will write the evicted partition to disk. If you want to take a persisted RDD out of memory to free up space, use `unpersist`.

Shuffle Files

Regardless of a `persist` or `checkpoint` call, Spark does write some data to disk during a shuffle. These files are called *shuffle files*, and they usually contain all of the records in each input partition sorted by mapper. Usually shuffle files remain in the local directory of the workers for the duration of an application. Thus, if the driver program reuses an RDD that has already been shuffled, Spark may be able to avoid recomputing that RDD up to the point of shuffle by using the shuffle files on the mapper.

Unlike the other caches, we can't determine if a given RDD still has its shuffle files present (e.g., there is no equivalent of the `isCheckPointed command`), which returns true if that RDD has been checkpointed. In general, though, shuffle files aren't explicitly cleaned up until an RDD goes out of scope. However, the web UI can be helpful in determining if stages are being skipped this way, as shown in Figure 7-3.

The performance of reusing shuffle files is similar to the performance of an RDD that is cached at the level of disk only.



Shuffle files can be large, and Spark has no explicit cache management for them. Keeping references to RDDs depending on shuffled output can lead to out-of-disk errors if the RDDs are not garbage collected on the driver.

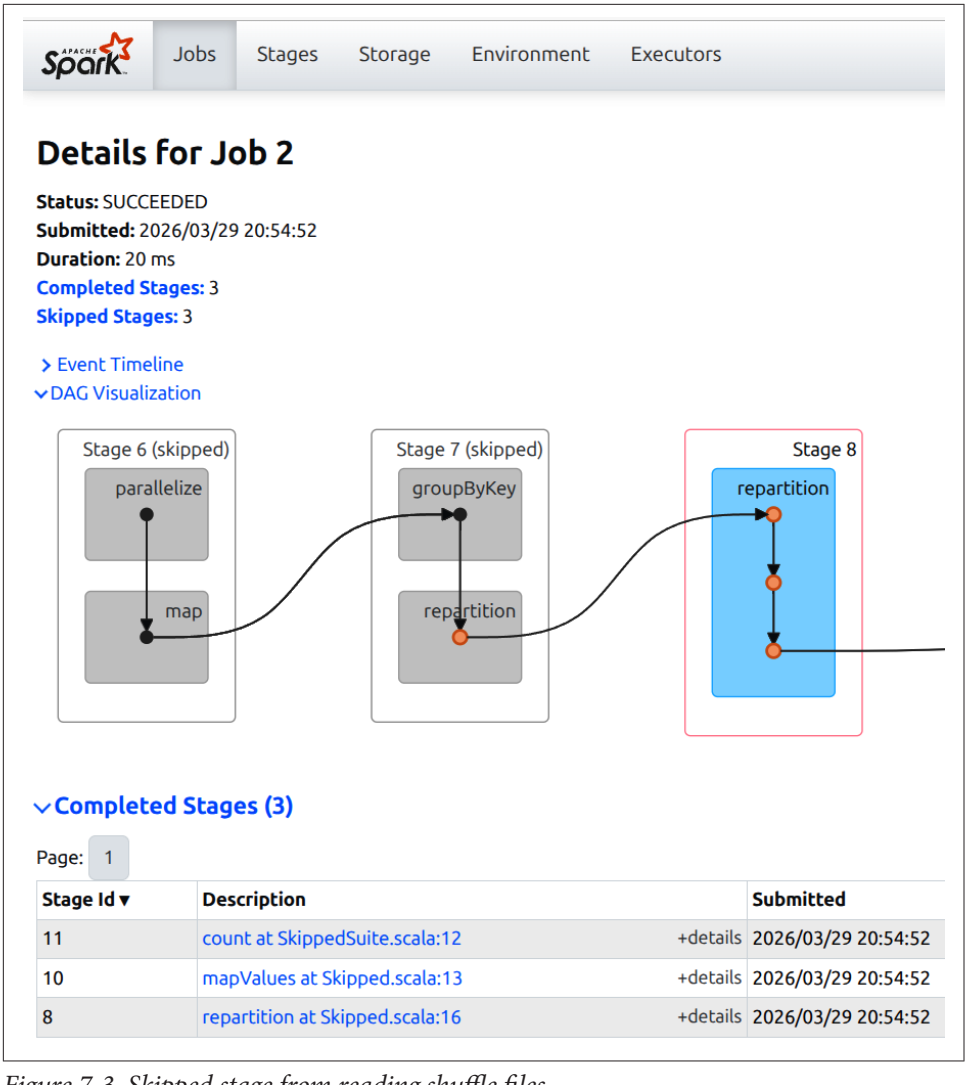


Figure 7-3. Skipped stage from reading shuffle files

Out-of-disk-space error

Out-of-disk-space errors can be unexpected, but in clusters with small amounts of disk space, they are surprisingly common. Disk space errors can be caused by long-running shell environments in which RDDs created at the top scope are never garbage collected. Spark writes the output of its shuffle operations to files on the disk of the workers in the Spark local dir. These files are only cleaned up when an RDD is garbage collected, which, if the amount of memory assigned to the driver program is large, can occur infrequently. One solution is to explicitly trigger garbage

collection (assuming the RDDs have gone out of scope)—if the DAG is getting too long, checkpointing can help make the RDDs available for garbage collection.

Noisy Cluster Considerations

Noisy clusters, or those with a high volume of unpredictable traffic, pose a fundamental challenge to Spark's evaluation. By default, Spark doesn't save most intermediate results (besides in a shuffle step). Thus, in the case of VM preemptions, Spark will have to recompute the calculation in the job up to the point of failure. In a noisy cluster, where long-running jobs are often interrupted, this poses a huge challenge. Checkpointing can be especially helpful to get jobs to run at all. Checkpointing breaks an RDD's lineage, therefore reducing the cost to recompute downstream transformations. Checkpointing also persists to external storage so that unexpected failures do not lead to data loss. If failures are common but not fatal, it may be worth configuring your job to persist to multiple machines using a storage option like `MEMORY_AND_DISK_2`, which replicates data on two machines. That way, failures on one node will not require a recompute. This can be especially important with wide transformations, which are very expensive.

By default, Spark uses a first-in, first-out (FIFO) paradigm to queue jobs within a system. This means that the first job submitted will run in its entirety, getting priority on all the available resources. However, if a job doesn't need the whole cluster, the next job may start. FIFO scheduling can be useful to ensure that space-intensive jobs are able to use the resources that they need. However, if you launch a job a few seconds behind a many-hour process, the FIFO strategy can be frustrating. Spark offers a fair scheduler, modeled after the Hadoop fair scheduler, to allow high-traffic clusters to share resources more evenly. The fair scheduler allocates the tasks from different jobs to the executors in a "round-robin fashion" (i.e., parsing out a few tasks to the executors from each job). With the fair scheduler, a short, small job can be launched before an earlier long-running job is completed.

The fair scheduler also supports putting jobs into pools and allocating different priorities (weight) to those pools. Jobs within a pool are allocated the same number of resources, and the pools are allocated resources according to their weight. Using pools can be a good way to ensure that high-priority jobs or very expensive jobs are completed. The fair scheduler also ensures that users are allocated resources evenly regardless of how many jobs they submit. You can read more about using and configuring a fair scheduler in the Spark job scheduling documentation.

Interaction with Accumulators

The interaction of caching and accumulators can make reasoning about accumulators more difficult. As we mentioned, if part of an RDD has to be recomputed, Spark may continue to add values to the accumulator as it recomputes, causing the values in the recomputed part to be double counted. Furthermore, not all computations will always compute the entirety of a partition. Surprisingly, caching does not prevent either double counting or problems that arise from partially evaluated partitions. Cached partitions may be evicted, so double counting may still arise if the machine with the cached data fails or if the partition is evicted to make space for a more recently cached partition. Unfortunately, caching with accumulators may cause a job that appears to compute the correct value on small data to later compute the incorrect value on large data.

Conclusion

Now that you have explored how to get the most out of your standard RDD transformations, as well as joins, it's time to explore the concerns associated with the most important and complicated subset of RDD transformations, key/value pair operations. Not all of the techniques you will have learned need to be applied in every Spark program, and some of the takeaways from this chapter are more about when certain tools are not a good fit (see "Accumulators" on page 150). Many of the same techniques and considerations for standard RDD transformations apply when working with key/value data: if your transformation doesn't depend on the key, the techniques from this chapter may even be more relevant.

CHAPTER 8

Working with Key/Value Data

Like many data parallel distributed computing tools, Spark relies heavily on the key/value pair paradigm to define and parallelize operations, particularly wide transformations that require the data to be redistributed between machines. Anytime we want to perform grouped operations in parallel or change the ordering of records among machines—be it computing an aggregation statistic or merging customer records—the key/value functionality of Spark is useful as it allows us to easily parallelize our work.

In Scala, Spark has its own `PairRDDFunctions` class containing operations defined on RDDs of tuples. The `PairRDDFunctions` class, made available through implicit conversion, contains most of Spark's methods for joins and custom aggregations. The `OrderedRDDFunctions` class contains the methods for sorting. The `OrderedRDDFunctions` are available to RDDs of tuples in which the first element (the key) has an implicit ordering. In Python these are instead defined directly on the RDD base class.



Similar operations are available on Datasets as discussed in "Grouped Operations on Datasets" on page 82.

Despite their utility, key/value operations can lead to a number of performance issues. In fact, most expensive operations in Spark fit into the key/value pair paradigm because most wide transformations are key/value transformations, and most require some fine-tuning and care to be performant. These performance considerations will be the focus of this chapter. We hope to provide not just a guide to using the functions in the `PairRDDFunctions` and `OrderedRDDFunctions` classes, but to build on the architecture lessons of Chapters 2 and 7 to present a thorough guide to

thinking about how Spark evaluates wide transformations and how to redesign the logic of a program to make tasks that require ordering data most efficient.

In particular, operations on key/value pairs can cause:

- Out-of-memory errors in the driver
- Out-of-memory errors on the executor nodes
- Shuffle failures
- "Straggler tasks" or partitions, which are especially slow to compute

The first problem, memory errors in the driver, is usually caused by actions. We will discuss the performance problems associated with actions on key/value pairs in "Actions on Key/Value Pairs" on page 175. The last three performance issues—out of memory on the executors, shuffles, and straggler tasks—are all most often caused by shuffles associated with the wide transformations in the `PairRDDFunctions` and `OrderedRDDFunctions` classes. Throughout this chapter, we will focus on two primary techniques to avoid performance problems associated with shuffles, which we call "shuffle less often" and "shuffle better":

Shuffle less often
: We will provide techniques to minimize the number of shuffles needed to complete a complex computation. One way to minimize the number of shuffles in a computation that requires several transformations is to make sure to preserve partitioning across narrow transformations to avoid reshuffling data (see "Preserving Partitioning Information Across Transformations" on page 188). In some instances, we can use the same partitioner on a sequence of wide transformations. This can be particularly useful to avoid shuffles during joins and to reduce the number of shuffles required to compute a sequence of wide transformations (see "Co-Grouping" on page 184 and "Leveraging Co-Located and Co-Partitioned RDDs" on page 188). We will also discuss leveraging custom partitioners (see "Custom Partitioning" on page 187) to distribute the data most effectively for downstream computations as well as how to push computational work into the shuffle stage to make a complicated computation more efficient (see "Secondary Sort and repartitionAndSortWithinPartitions" on page 193).

Shuffle better
: Sometimes, computation cannot be completed without a shuffle. However, not all wide transformations and not all shuffles are equally expensive or prone to failure. By using wide transformations such as `reduceByKey` and `aggregateByKey` that can perform map-side reductions and that do not require loading all the records for one key into memory, you can prevent memory errors on the executors and speed up wide transformations, particularly for aggregation operations (see "What's So Dangerous About the groupByKey Function?" on page 176 and

"Preventing Out-Of-Memory Errors with Aggregation Operations" on page 182). Lastly, shuffling data in which records are distributed evenly throughout the keys and which contain a high number of distinct keys prevents out-of-memory errors on the executors and "straggler tasks" (see "Straggler Detection and Unbalanced Data" on page 208).

The Goldilocks Example

Throughout this chapter, we will refer to a project that the authors worked on that required finding arbitrary rank statistics in high-dimensionality and high-volume data as an example of a complex key/value transformation.

The client—we will call her Goldilocks—had data representing thousands of different metrics for hundreds of millions of pandas. Her data looked something like Table 8-1.

Table 8-1. Goldilocks example data

Panda name	Happiness	Niceness	Softness	Sweetness
Mama Panda	15.0	0.25	2467.0	0.0
Papa Panda	2.0	1000.0	35.4	0.0
Baby Panda	10.0	2.0	50.0	0.0
Baby Panda's toy panda	3.0	8.5	0.2	98.0

The attributes for each panda are represented as doubles.

Goldilocks wanted us to design an application that would let her input an arbitrary list of integers `1,2,...,n,...,k` and return the `n`th best element in each column. For example, if Goldilocks input 8, 1000, and 20 million, our function would need to return the 8th, 1000th, and 20 millionth best-ranking panda for each attribute column.

To illustrate this example, suppose that Goldilocks wanted to find the 2nd and 4th elements from Table 8-1. We would want our function to return something like Table 8-2.

Table 8-2. Goldilocks example result

Column name	Column index	Rank statistics
Happiness	1	List(3.0, 15.0)
Niceness	2	List(2.0, 1000.0)
Softness	3	List(35.4, 2467.0)
Sweetness	4	List(0.0, 98.0)

We call this candidate "Goldilocks" because she was very picky and her house (i.e., in-house cluster) was crowded with other users (bears). In this case, Goldilocks would not accept approximate quantile boundaries but required the output of our function to be values in the original Dataset. Thus, this task is inherently expensive since it requires sorting all the values in each column in some way.

Because the data is columnar, we could consider Spark SQL to solve this problem. However, early Spark SQL did not have any support for rank statistics. It may be possible to write a UDF/UDAF to solve the problem, but it would be quite cumbersome because our use case is complicated and cannot be computed on each row. Thus, our solution has to leverage Spark Core.[1]

Goldilocks Version 0: Iterative Solution

One intuitive solution to the Goldilocks problem is to loop through each column, mapping each row to a single value, then use Spark's `sortBy` and `zipWithIndex` functions on each column, and then filter for the indices that correspond to the desired rank statistics.



For simplicity, we will assume that the columnar data was read in from stable storage as a DataFrame, that the rows are all well-formed, and that the string column with each panda's name was dropped. Consequently, our function takes a DataFrame of all double columns (representing the panda data) and a list of long types representing the positions of the elements to find for each column (e.g., 1st, 100th). The function should return a map from the column index to a list of the rank statistics in that column.

Example 8-1 is an implementation of this first solution to the Goldilocks problem in which we loop through each column and sort each one using Spark's distributed sort.

Example 8-1. Goldilocks version 0, iterative solution

```
def findRankStatistics(
    dataFrame: DataFrame,
    ranks: List[Long]
): Map[Int, Iterable[Double]] = {
  require(ranks.forall(_ > 0))
  val numberOfColumns = dataFrame.schema.length
```

1 As we hope to demonstrate in this chapter, even in cases where out-of-the-box functionality in Spark SQL may cover your use case, the creative control offered by Spark Core can often allow us to develop a better routine for a very specific use case. Indeed, the final solution we present in this chapter is able to do some clever reductions that probably make it faster than even a well-implemented rank statistics implementation in Spark SQL.

```
    var i = 0
    var result = Map[Int, Iterable[Double]]()

    while (i < numberOfColumns) {
      val col = dataFrame.rdd.map(row => row.getDouble(i))
      val sortedCol: RDD[(Double, Long)] = col.sortBy(v => v).zipWithIndex()
      val ranksOnly = sortedCol.filter {
        // rank statistics are indexed from one. e.g. first element is 0
        case (colValue, index) => ranks.contains(index + 1)
      }.keys
      val list = ranksOnly.collect()
      result += (i -> list)
      i += 1
    }
    result
  }
```

This solution works and is relatively robust, but it is very slow since it has to sort the data once for each column and does so iteratively. In other words, if we have 8,000 columns, we have to do 8,000 sorts!

So how can we do better?

Since each sort can be done without knowledge of the other sorts, our intuition should be that it is possible to parallelize this computation using each column as the unit of parallelization. We can represent the data as one long list of key/value pairs where the keys represent the column indices. Then, we can perform our computation in parallel for each key.

We would map the data in Table 8-1 to the following list of key/value pairs:

```
(key, value)
(1, 15.0)
(2, 0.25)
(3, 2467.0)
(4, 0.0)
(1, 2.0)
(2, 1000.0)
(3, 35.4)
(4, 0.0)
(1, 10.0)
(2, 2.0)
(3, 50.0)
(4, 0.0)
(1, 3.0)
(2, 8.5)
(3, 0.2)
(4, 98.0)
```

If we read in our data as a DataFrame, we can do this mapping with a simple function like the one shown in Example 8-2.

Example 8-2. Goldilocks version 1, mapping to column index/value pairs

```
def mapToKeyValuePairs(dataFrame: DataFrame): RDD[(Int, Double)] = {
  val rowLength = dataFrame.schema.length
  dataFrame.rdd.flatMap(row =>
    Range(0, rowLength).map(i => (i, row.getDouble(i)))
  )
}
```



Spark's `flatMap` operation mimics the behavior of the `flatMap` operation that is defined on iterators and collections in Scala. `flatMap` is a very versatile narrow transformation, but for those new to Scala, it can be a bit confusing. `flatMap` lets the user define a mapping from each record to a collection of elements and then combines the resulting collections together. In this case the mapping is defined from a Spark SQL `Row` object to a sequence of elements, in this case `(columnIndex, value)` pairs. The resulting RDD will have more records than the previous RDD, and each will be of `(columnIndex, value)` pairs. Increasing the total number of records is not a requirement for the `flatMap` operation. In fact, `flatMap` can be particularly useful because, unlike `map`, it allows us to return an empty collection for one of the records. Thus, the operator can be used to both filter and transform the elements in one pass. In other words, a `map` and `filter` on the same RDD or collection can always be combined into one `flatMap` step.

After applying this function, we can perform this computation in parallel by column index (in this case, column index is the key for each record). Framed in this way, the Goldilocks problem is a key/value pair problem. Specifically:

> Design a function that takes an input RDD of integer/double pairs and a list of longs, 1 … , k, …j and returns a map of key to a list of k doubles that are the n1th, n2th .. nkth elements for that column index (key).

How to Use PairRDDFunctions and OrderedRDDFunctions

If you have been using Spark for a while, you are probably familiar with the `PairRDDFunctions` and `OrderedRDDFunctions` classes. However, we will still provide a brief introduction about how to use them. The Spark RDD class makes use of Scala implicits, and the `PairRDDFunctions` will be available on any RDD of type `(K,V)`. For `PairRDDFunctions`, `K` and `V` can be of any type, but for the `OrderedRDDFunctions` (`sortByKey`, `repartitionAndSortWithinPartitions`, `filterByRange`), `K` must have some implicit ordering. Most common types, like the numeric types or strings, have their ordering already defined in Scala. To use a custom type, you may have to define the ordering yourself. Spark uses implicit conversion to convert an RDD that meets

the PairRDD or OrderedRDD requirements from a generic type to the PairRDD or OrderedRDD type. This implicit conversion requires that the correct library already be imported. Thus, to use Spark's pairRDDFunctions, you need to have imported the SparkContext; i.e., imports must include import org.apache.spark.SparkContext._.



When writing a function that uses OrderedRDDFunctions of a generic key type, you may need to include code defining an implicit val of type ordering. In the secondary sort example, which we will discuss in "Secondary Sort and repartitionAndSortWithinPartitions" on page 193, we define an ordering of an object called "Panda Keys" as shown in Example 8-3.

Example 8-3. Defining an implicit ordering to work with OrderedRDDFunctions

```
implicit def orderByLocationAndName[A <: PandaKey]: Ordering[A] = {
  Ordering.by(pandaKey => (pandaKey.city, pandaKey.zip, pandaKey.name))
}

implicit val ordering: Ordering[(K, S)] = Ordering.Tuple2
```

Actions on Key/Value Pairs

In "Functions on RDDs: Transformations Versus Actions" on page 20, we discussed how transformations are computed on the Spark executors when an action is called. We also explained that actions usually move data out of the Spark executors either by collecting it to the driver or by writing to stable storage. In general, we advised you to be very cautious about actions that return unbounded data to the driver, as they can cause out-of-memory errors in the driver. Most key/value actions (including countByKey, countByValue, lookUp, and collectAsMap) return data to the driver. In most instances, they return unbounded data since the number of keys and the number of values are unknown. For example, countByKey returns a data point for each key, and thus it may cause memory errors if there are more distinct keys than fit in memory on the driver. Conversely, lookUp returns all the values for each key, so it will cause memory problems if one key has more data than will fit in memory on the driver.



The lookUp operation is also expensive because it triggers a shuffle if the RDD doesn't have a known partitioner.

In addition to the number of records, the size of each record is an important factor in causing memory errors. For example, if each record is a custom object or a collection type, an action that succeeded in collecting the same number of records in an RDD of bytes may still fail. In general, we want to try to design key/value problems so that the keys fit into memory on the driver. The values should be at least well distributed by key and at best distributed so that each key has no more records than can fit in memory on each executor. We will discuss the effects of bad key distribution in "Straggler Detection and Unbalanced Data" on page 208, and will provide some suggestions for working around skewed data. As with all Spark programs, we should try to perform transformations that reduce the size of the data before calling actions that move results to the driver.

Key/value transformations can also cause memory errors, most often in the executors, if they require all the data associated with one key to be kept in memory on one partition. Avoiding memory errors and optimizing transformations for fewer shuffles is a bit more complicated than avoiding problems with actions. Thus, key/value transformations will be the focus of the rest of this chapter.

What's So Dangerous About the groupByKey Function?

Many sources—including the Spark documentation—warn against the scalability of the `groupByKey` function, which returns an iterator of each element by key. This section attempts to explain the cases in which `groupByKey` causes problems at scale. We also hope to offer some advice about alternatives to using `groupByKey`. First, we want to revisit the Goldilocks case, because our first solution to the Goldilocks problem was to use `groupByKey`.

Goldilocks Version 1: groupByKey Solution

One simple solution to the Goldilocks problem is to use `groupByKey` to group the elements in each column. `groupByKey` returns an iterator of elements by each key, so to sort the elements by key we have to convert the iterator to an array and then sort the array.[2] After converting the iterator to an array, we can sort the array and filter for the elements that correspond to our rank statistics.

Example 8-4 is an implementation of the `groupByKey` solution. For consistency, this function also takes a DataFrame and a list of element positions as long values. It calls the function that creates key/value pairs that we described in Example 8-2.

2 Iterators can only be traversed once. Thus, iterators cannot be used without conversion because sorting requires multiple passes through the data. They must be converted to another data structure first. For more about the iterator type and its advantages and limitations, see "Iterator-to-Iterator Transformations with mapPartitions" on page 141.

Example 8-4. Goldilocks version 1, `groupByKey` solution

```
def findRankStatistics(
    dataFrame: DataFrame,
    ranks: List[Long]
): Map[Int, Iterable[Double]] = {
  require(ranks.forall(_ > 0))
  // Map to column index, value pairs
  val pairRDD: RDD[(Int, Double)] = mapToKeyValuePairs(dataFrame)

  val groupColumns: RDD[(Int, Iterable[Double])] = pairRDD.groupByKey()
  groupColumns
    .mapValues(iter => {
      // convert to an array and sort
      val sortedIter = iter.toArray.sorted

      sortedIter.toIterable.zipWithIndex.flatMap({ case (colValue, index) =>
        if (ranks.contains(index + 1)) {
          Iterator(colValue)
        } else {
          Iterator.empty
        }
      })
    })
    .collectAsMap()
}

def findRankStatistics(
    pairRDD: RDD[(Int, Double)],
    ranks: List[Long]
): Map[Int, Iterable[Double]] = {
  assert(ranks.forall(_ > 0))
  pairRDD
    .groupByKey()
    .mapValues(iter => {
      val sortedIter = iter.toArray.sorted
      sortedIter.zipWithIndex
        .flatMap(
          { case (colValue, index) =>
            if (ranks.contains(index + 1)) {
              // this is one of the desired rank statistics
              Iterator(colValue)
            } else {
              Iterator.empty
            }
          }
        )
        .toIterable // convert to more generic iterable type to match out spec
    })
    .collectAsMap()
}
```

This solution has several advantages. First, it gives the correct answer. Second, it is very short and easy to understand. It leverages out-of-the-box Spark and Scala functions, and so it introduces few edge cases and is relatively easy to test. On small data, particularly if the input data has many columns but few records, it is actually relatively efficient because it only requires one shuffle in the `groupByKey` step and because the sorting step can be computed as a narrow transformation on the executors.



In this function, we do use `collectAsMap`, which can have the same issues as `collect` that we warned you about earlier. In this instance, however, the danger of memory errors is minimal because at the point of collecting, we know the number of keys and the number of values per each key. The number of keys is exactly the number of columns, which we have assumed to be no larger than a few thousand. The number of values is equal to the length of the rank statistics list we used as input for the function, which was not originally stored in a distributed way. However, it might be good practice to add a limit to the size of the input list to prevent failures in the collect step.

In the environment that we were using and on data with 10,000 rows and a few thousand columns, this solution was orders of magnitude faster than the one presented in "The Goldilocks Example" on page 171 in which we looped through the columns iteratively and sorted each one. However, on data with a few million rows, we found that the solution failed consistently with memory exceptions even on a many-node cluster.

Why groupByKey fails

If you have read *Learning Spark* or spent much time working with Spark at scale, the results of the `groupByKey` approach to solving the Goldilocks problem shouldn't surprise you, as `groupByKey` on RDDs is known to cause memory errors at scale. The reason is that the "groups" created by `groupByKey` are always iterators, which can't be distributed. This causes an expensive "shuffled read" step in which Spark has to read all of the shuffled data from disk and into memory.

Figure 8-1 is a screenshot taken from the Spark web UI that illustrates the high cost of `groupByKey`.

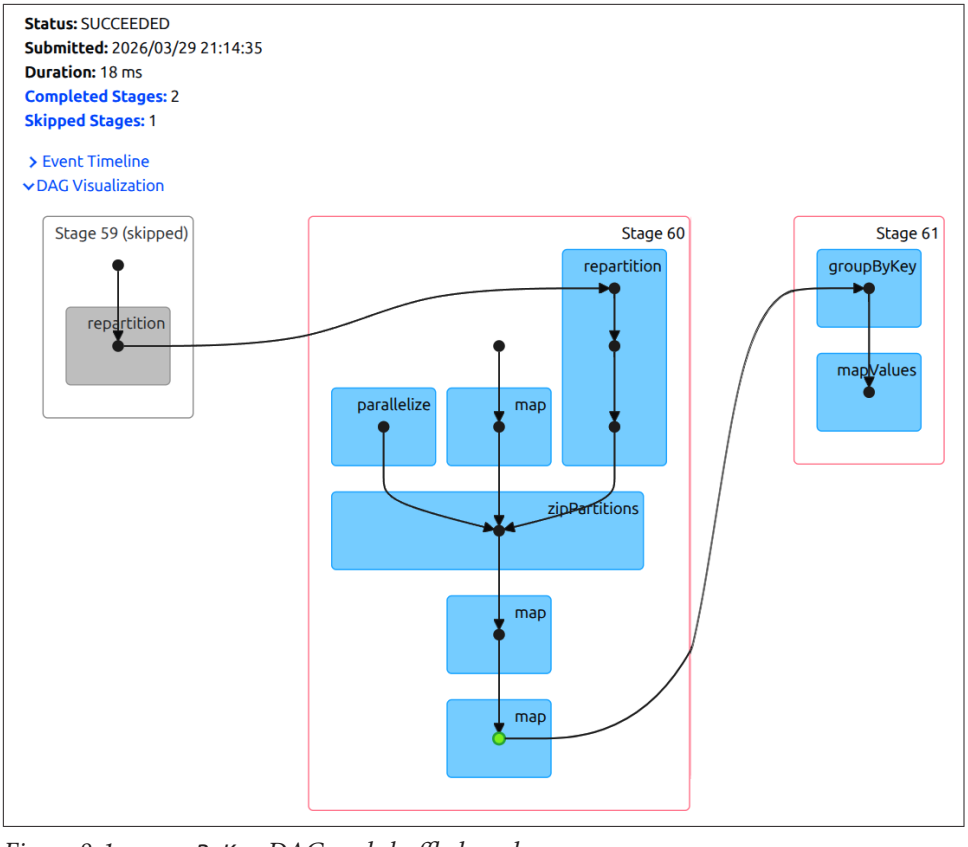


Figure 8-1. `groupByKey` DAG and shuffled read

Notice that in this computation, the shuffled read is 86 MB even though the input data is about 200 MB. In other words, Spark has to read almost all of the shuffled data into memory.

As a consequence of partitioning by the hash value of the keys and pulling the result into memory to group as iterators, `groupByKey` often leads to out-of-memory errors on the executors if there are many duplicate records per key. Each record whose key has the same hash value must live in memory on a single machine. Thus, if just one of your keys contains too many records to fit in memory on one executor, the entire operation will fail.

Figure 8-2 illustrates a `groupByKey` operation of data about handlebar mustaches. As you can see, there are more records corresponding to the 94110 zip code, which is the zip code for the Mission District in San Francisco. Even if the records fit on the executors when evenly distributed, all the records associated with the 94110 zip code will not fit on a single executor after the `groupByKey` step.

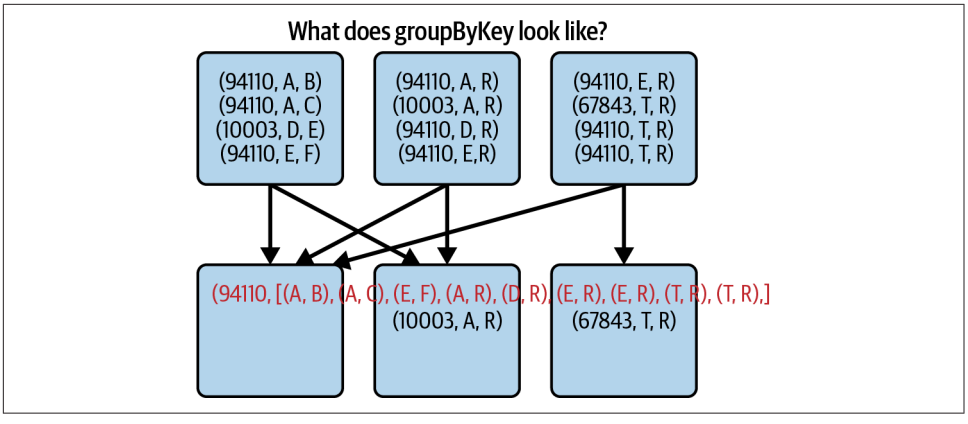


Figure 8-2. `groupByKey` tip over

In general, it is better to choose aggregation operations that can do some map-side reduction to decrease the number of records by key before shuffling (e.g., `aggregateByKey` or `reduceByKey`). If this is not possible, using a wide transformation that does not require all the values associated with one key to be kept in memory as we discuss in "Secondary Sort and repartitionAndSortWithinPartitions" on page 193 is a good alternative to `groupByKey`. If you must use `groupByKey`, it is best if the next operation is an iterator-to-iterator transformation as discussed in "Iterator-to-Iterator Transformations with mapPartitions" on page 141.

Choosing an Aggregation Operation

Shuffling the records to combine those with the same key is a common use case for key/value Spark operations, and Spark provides a number of such aggregation operations. Most of them are built atop the generic `combineByKey` operation, but they differ widely in performance. In this section we will detail these operations and some of the performance considerations associated with them.

Dictionary of Aggregation Operations with Performance Considerations

Aggregation operations have specific performance considerations, which we summarize in Table 8-3.

Table 8-3. A dictionary of Spark's key/value aggregation operations

Function	Purpose	Key restriction	Runs out of memory when	Slow when	Output partitioner
`groupByKey`	Group values with the same key into a single iterator.	Cannot have array keys with the default `HashPartitioner`. To use array keys, use a custom partitioner.	If all of the records associated with any single key take up too much space in memory to be read from disk on one executor.	If there is not a known partitioner, this causes a shuffle. The shuffle gets more expensive as the number of distinct keys increases, the number of records per key increases, or the records are not evenly distributed across the keys.	`HashPartitioner` by default but supports custom partitioning.
`combineByKey`	Combine values with the same key using a different result type.	Same as above.	The "combine by" routine uses too much memory or creates too much garbage collection overhead, or the accumulator for one key becomes too large (this is the problem in `groupByKey`).	Same as above, but *can* be faster than `groupByKey` if the combine operation is a reduction.	Same as above.
`aggregateByKey`	Same as `combineByKey`, but uses one zero value for all accumulators.	Same as above.	Same as above, but if implemented well, less likely to cause garbage collection errors (see "Minimizing Object Creation" on page 134).	See above, but generally faster than `combineByKey` since it will perform the merging map-side before sending to a combiner.	Same as above.
`reduceByKey`	Combine values with the same key. Reduction must be to same type as original values.	See above, but often less expensive than `combineByKey` since `aggregateByKey` supports reusing the accumulator object to avoid object creation.	See above, but the type restriction makes memory errors unlikely. So long as the combine is not to a collection type, the function is probably reducing. Garbage collection is less than `aggregateByKey` since no additional accumulator object is created.	Same as `aggregateByKey`.	Same as above.

Function	Purpose	Key restriction	Runs out of memory when	Slow when	Output partitioner
`foldByKey`	Combine values with the same key using an associative combine function and a zero value, which can be added to the result an arbitrary number of times. Use instead of `reduceByKey` when a natural 0 exists.	See above.	See above.	See above. Performance is nearly identical to `reduceByKey`.	See above.



To avoid memory allocation in `aggregateByKey`, modify the accumulator rather than return a new one. See "Minimizing Object Creation" on page 134.

Preventing Out-Of-Memory Errors with Aggregation Operations

`CombineByKey` and all of the aggregation operators built on top of it (`reduceByKey`, `foldLeft`, `foldRight`, `aggregateByKey`) are no better than `groupByKey` in terms of memory errors *if* they cause the accumulator to become too large for one key. In fact, if you look up the implementation of `groupByKey`, you can see that it is actually implemented using `combineByKey`, where the accumulator is an iterator with all the data. Thus, the accumulator is the size of all the data for that key. In other words, these operations are unlikely to cause memory errors as long as the combining steps make the data smaller. However, if the accumulator gets larger with the addition of each new record, it will eventually cause memory errors if there are many records associated with one key.

Imagine doing a back-of-the-envelope memory calculation on your sequence and combine operators.

Given the sequence operation:

```
SeqOp(acc, v) => acc'
```

…and the combine operation:

```
combOp(acc1, acc2) = acc3.
```

Calculate whether:

```
memory(acc') < memory(acc) + memory(v)
and memory(acc3) < memory(acc1) + memory(acc2).
```

If so, the function is likely a reduction. If not, as is the case in `groupByKey`, you may need to consider a different strategy.

Beyond being less likely to run out of memory than `groupByKey`, the following four functions—`reduceByKey`, `treeAggregate`, `aggregateByKey`, and `foldByKey`—are implemented to use map-side combinations, meaning that records with the same key are combined before they are shuffled. This can greatly reduce the shuffled read. Compare the shuffled read in our original `groupByKey` (Figure 8-1) to the amount in `reduceByKey` in Figure 8-3. Recall that in the `groupByKey` case, our shuffled read was close to the size input. However, applying `reduceByKey` to the same input data reduces that number to a few hundred kilobytes!

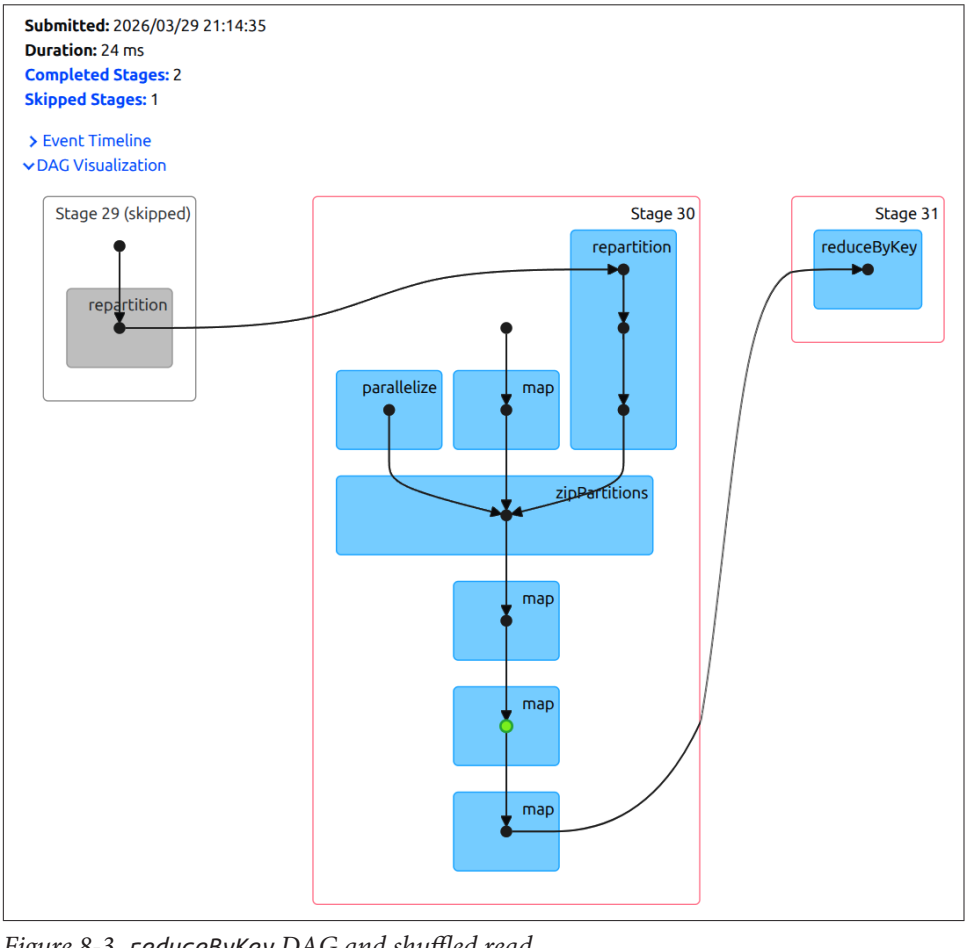


Figure 8-3. `reduceByKey` DAG and shuffled read

Multiple RDD Operations

Some transformations can operate on multiple RDD inputs. The most obvious of these are join-type operations, but they are far from the only ones.

Co-Grouping

Much in the same way all of the accumulator operations (`reduceByKey`, `aggregateByKey`, `foldByKey`) are implemented using `combineByKey`, all of the join operations are implemented using the `cogroup` function, which uses the `CoGroupedRDD` type. A `CoGroupedRDD` is created from a sequence of key/value RDDs, each with the same key type. `cogroup` shuffles each of the RDDs so that the items with the same value from each of the RDDs will end up on the same partition and into a single RDD by key.

The `PairRDDFunctions` class provides several signatures for `cogroup`. `cogroup` and its alias `groupWith` can take one, two, or three RDDs with the same key type as arguments (regardless of value type), return an RDD with each key, and then return a tuple of `Iterable` objects where each `Iterable` is all the values of the RDDs for that key.

Suppose, for example, that we had two datasets of information about each panda: one with the scores in a series of games and one with their favorite foods. We could use `cogroup` to associate each panda's ID with an iterator of their scores and another iterator of their favorite foods, as in Example 8-5.

Example 8-5. `cogroup` example

```
val cogroupedRDD: RDD[(Long, (Iterable[Double], Iterable[String]))] =
  scoreRDD.cogroup(foodRDD)
```

`cogroup` can be useful as an alternative to join when joining with multiple RDDs. Rather than doing joins on multiple RDDs with one RDD, it is more performant to co-partition the RDDs since that will prevent Spark from shuffling the RDD being repeatedly joined.

For example, if we needed to join the panda score data with both address and favorite foods, it would be better to use `cogroup` than two join operations, as shown in Example 8-6.

Example 8-6. Cogroup to avoid multiple joins on the same RDD

```
val addressScoreFood = addressRDD.cogroup(scoreRDD, foodRDD)
```

Despite its advantages, `cogroup` will cause memory errors for the same reason as `groupByKeys` if one key in either RDD or both combined is associated with more data

than will fit on a single partition. In particular, `cogroup` requires that all the records in all of the co-grouped RDDs for one key be able to fit on one partition. For more information on joins, see Chapter 4.

Partitioners and Key/Value Data

As we covered in Chapter 2, a partition in Spark represents a unit of parallel execution that corresponds to one task. An RDD without a known partitioner will assign data to partitions according only to the data size and partition size.[3] The partitioner object defines a mapping from the records in an RDD to a partition index. By assigning a partitioner to an RDD, we can guarantee something about the records on each partition—for example, that it falls within a given range (range partitioner) or includes only elements whose keys have the same hash code (hash partitioner).

There are three methods that exist exclusively to change the way an RDD is partitioned. For RDDs of a generic record type, `repartition` and `coalesce` can be used to simply change the number of partitions that the RDD uses, irrespective of the value of the records in the RDD. As we discussed in "The Special Case of coalesce" on page 132, `repartition` shuffles the RDD with a hash partitioner and the given number of partitions. (We will explain in more detail how hash partitioning works in "Hash Partitioning" on page 186.) `coalesce`, on the other hand, is an optimized version of `repartition` that avoids a full shuffle if the desired number of partitions is less than the current number of partitions. Recall that when `coalesce` reduces the number of partitions, it does so by merely combining partitions—and thus `coalesce` is not a wide transformation since the partition can be determined at design time. When `coalesce` increases the number of partitions, it has the same behavior as `repartition`. For RDDs of key/value pairs, we can use a function called `partitionBy`, which takes a partition object rather than a number of partitions and shuffles the RDD with the new partitioner. `partitionBy` allows for much more control in the way that the records are partitioned since the partitioner supports defining a function that assigns a partition to a record based on the value of that key.

In all cases, `repartition` and `coalesce` do not assign a known partitioner to the RDD. In contrast, using `partitionBy` (and most other key/value functions that cause a shuffle) results in an RDD with a known partitioner. In some instances, when an RDD has a known partitioner, Spark can rely on the information about data locality

3 Often RDDs without known partitioners can be RDDs loaded from storage. In this case the RDD's data is most often effectively partitioned in the same way as the underlying storage (e.g., the splits in Hadoop). However, once the data is read into Spark, Spark does not know what the underlying partitioning is and consequently cannot take advantage of this information. This is one way in which checkpointing is different from simply saving an RDD to stable storage and then reading it manually. In the checkpointing case, Spark saves some metadata about the RDD, including, if applicable, its partitioner.

provided by the partitioner to avoid doing a shuffle even if the transformation has wide dependencies. The rest of the sections in this chapter will be about ways to leverage knowledge about partitioning or use custom partitioning to "shuffle less" and "shuffle better." Specifically, we aim to use this information to minimize the number of times a program causes a shuffle, the distance the data has to travel in a shuffle, and the likelihood that a disproportionate amount of the data will be sent to one partition, thus causing out-of-memory errors or straggler tasks.

Using the Spark Partitioner Object

Conceptually, the partitioner defines how records will be distributed and thus which records will be completed by each task. Practically, a partitioner is actually an interface with two methods—`numPartitions` and `getPartition`. `numPartitions` defines the number of partitions in the RDD after partitioning. `getPartition` defines a mapping from a key to the integer index of the partition where records with that key should be sent. There are two implementations for the partitioner object provided by Spark: the `HashPartitioner` and `RangePartitioner`. If neither of these suits your needs, it is possible to define a custom partitioner.

Hash Partitioning

The default partitioner for pair RDD operations (not ordered RDD operations) is a `HashPartitioner`. A `HashPartitioner` determines the index of the child partition based on the hash value of the key. The hash partitioner requires a `partitions` parameter, which determines the number of partitions in the output RDD and the number of bins used in the hashing function. If unspecified, Spark uses the value of the `spark.default.parallelism` value in the `SparkConf` to determine the number of partitions. If the default parallelism value is unset, Spark defaults to the largest number of partitions that the RDD has had in its lineage. In wide transformations that use hash partitioning, such as `aggregateByKey`, if you provide the optional parameter of the target partition count, that is used as an argument to the hash partitioner.

Range Partitioning

Range partitioning assigns records whose keys are in the same range to a given partition. Range partitioning is required for sorting since it ensures that by sorting records within a given partition, the entire RDD will be sorted. The range partitioner first determines the range bounds for each partition by sampling: optimizing for an equal distribution of records across partitions. Then, each record in the RDD will be shuffled to the partition whose range bounds include the key. Highly unbalanced data (i.e., lots of values for some keys and not others, and if the distribution of the keys is not uniform) makes the sampling less accurate—and, as we have discussed,

uneven partitioning may cause downstream tasks to be slower than others, causing "straggler" tasks. If there are too many duplicate keys for all the records associated with one key to fit on one executor, then range partitioning, like hash partitioning, may cause memory errors. Performance problems associated with sorting are usually caused by these problems with the range partitioning step.

Creating a `RangePartitioner` with Spark requires not only a number of partitions, but also the actual RDD to sample. The RDD must be a tuple, and the keys must have an ordering defined.

Sampling actually requires partially evaluating the RDD, causing a break in the execution graph. Thus, range partitioning is actually both a transformation and an action. The cost of sampling means that, in general, range partitioning is more expensive than hash partitioning. The requirement that keys be ordered means that range partitioning cannot be done on all RDDs of tuples. Therefore, key/value operations (such as aggregations), which require records with each key to be on the same machine but not ordered in a particular way, use a `HashPartitioner` as the default. However, these methods can also be performed with a custom partitioner or range partitioner.

Custom Partitioning

To define a unique function for partitioning the data other than by the key's hash value or ordering, Spark allows the user to define a custom partitioner. In order to define a partitioner, you must implement the following methods:

`numPartitions`
: A method that returns an integer number of partitions. Expect that this number is greater than zero.

`getPartition`
: A method that takes a `key` (of the same type as the RDD being partitioned) and returns an integer representing the index of the partition that specifies where records with that key belong. The integer must be between zero and the number of partitions (defined in the `numPartitions` method).

`equals`
: An (optional) method to define equality between partitioners. The equality method for a `HashPartitioner` returns true if the number of partitions is equal. The range partitioner does so only if the range bounds are equal. The equality of partitioners can be particularly important for joins and co-grouping because, in some instances, if an RDD is already partitioned according to a partitioner, Spark is smart enough not to shuffle again with the same partitioner. We will discuss this in more detail in "Leveraging Co-Located and Co-Partitioned RDDs" on page 188.

`hashcode`
: This method is required only if the equals method has been overridden. The hashcode of a `HashPartitioner` is simply its number of partitions. The hashcode of a `RangePartitioner` is a hash function derived from the range bounds.

Preserving Partitioning Information Across Transformations

Some wide transformations change the partitioning of an RDD, as will be shown in Table 8-4. Spark remembers this information by updating the partitioner property of the RDD. When doing a series of transformations, it is important to understand how an RDD is partitioned because, in some instances, we can use this information to avoid doing future shuffles.

Using narrow transformations that preserve partitioning

Some narrow transformations, such as `mapValues`, preserve the partitioning of an RDD if it exists. Unless a transformation is known to only change the value part of the key/value pair in Spark, the resulting RDD will not have a known partitioner (even if the partitioning has not changed). It's important to note that common transformations like `map` and `flatMap` can change the key, so even if your function does not change the key, the resulting RDD will not have a known partitioner. Instead, if we don't want to modify the keys, we can call the `mapValues` function (defined only on pair RDDs) because it keeps the keys, and therefore the partitioner, exactly the same. The `mapPartitions` function will also preserve the partition if the `preservesPartitioning` flag is set to true. Assuming we have some RDD `data` of type `RDD[(Double, Int)]`, we could write the test in Example 8-7 to illustrate this property.

Example 8-7. Maintaining partitioning information with `mapValues`

```
val sortedData = data.sortByKey()
val mapValues: RDD[(Double, String)] = sortedData.mapValues(_.toString)
assert(mapValues.partitioner.isDefined,
  "Using Map Values preserves partitioning")

val map = sortedData.map( pair => (pair._1, pair._2.toString))
assert(map.partitioner.isEmpty, "Using map does not preserve partitioning")
```

Leveraging Co-Located and Co-Partitioned RDDs

Co-located RDDs are RDDs with the same partitioner that reside in the same physical location in memory. Co-location is important because all of the `CoGroupedRDD` functions—a category that includes the `cogroup` operations and all of the join operations—require the RDDs being grouped to have all of their partitions co-located.

RDDs can only be combined without any network transfer if they have the same partitioner and if each of the corresponding partitions in memory are on the same executor. Partitions will be in memory on the same executor if they were partitioned in the lineage associated with the same job.

Co-partitioning is related to but distinct from partition co-location. We say that multiple RDDs are *co-partitioned* if they are partitioned by the same known partitioner. We say that partitions are *co-located* if they are both loaded into memory on the same machine. RDDs are only guaranteed to be co-located if they are put into memory by the same job and the same partitioner: if one action contains the partitioning of both RDDs in its lineage. RDDs will be co-partitioned if their partitioner objects are equal but if the corresponding partitions for each RDD are not in the same physical location. Recall that "same partitioner" means the partitioner objects are equal according to the equality function defined in the partitioner class.

In Example 8-8, both `rddA` and `rddB` will be co-located.

Example 8-8. An example of co-located RDDs

```
val rddA = a.partitionBy(partitionerX)
rddA.cache()
val rddB = b.partitionBy(partitionerY)
rddB.cache()
val rddC = a.cogroup(b)
rddC.count()
```

Before Spark evaluates `rddC.count()`, neither RDD is actually loaded into memory due to Spark's lazily evaluated nature. When Spark launches the associated `rddC.count()` job, both RDDs are pulled into memory since their lineages are merged by the `cogroup` operation. In this case the join won't cause any network traffic because both RDDs are loaded into memory in the same location.

In contrast, if we were to call an action on `rddA` and `rddB` before the action on the co-grouped RDD, the `cogroup` may cause some network traffic since the RDDs are not co-located (Example 8-9).

Example 8-9. RDDs co-partitioned but not co-located

```
val rddA = a.partitionBy(partitionerX)
rddA.cache()
val rddB = b.partitionBy(partitionerY)
rddB.cache()
val rddC = a.cogroup(b)
rddA.count()
rddB.count()
rddC.count()
```

In this case, `rddA` and `rddB` are loaded into memory from different actions. They are co-partitioned, but there is no guarantee that their partitions will all be co-located. Thus, although the repartition calls prevent the join operator from triggering shuffles in both RDDs, there may still be some network traffic to line up the partitions and load both RDDs into memory. Although the design of your program may require calling actions in this order, it is often worth thinking about the lineage of an RDD before calling an action on it so as to minimize network traffic.

Dictionary of Mapping and Partitioning Functions PairRDDFunctions

The performance of the different mapping and partitioning functions available on `PairRDDs` is shown in Table 8-4.

Table 8-4. Dictionary of mapping and sampling operations for RDDs or key/value pairs

Function	Purpose	Key restriction	Runs out of memory when	Slow when	Output partitioner
`mapValues`	Apply a mapping function to each value of a pair RDD without changing the key.	None	Almost never	Slow when the mapping operation is very complicated or expands the size of each record.	In contrast to `map`, this preserves the partitioning of the data for use in future operations. If the input RDD has a known partitioner, the output RDD will have the same partitioner. If you can perform your mapping on just the values, it is almost always beneficial to do so.
`flatMapValues`	Perform the `flatMap` function on just the values of a key/value RDD. See our discussion of the `flatMap` paradigm near the end of "Goldilocks Version 0: Iterative Solution" on page 172.	None	Unlikely unless the function applied to each value is very expensive or the result iterator is very large, causing a dramatic expansion of the number of records for each key.	Slow when the mapping routine is very complicated, creates many new objects, or dramatically increases the number of records (such as tokenization).	Preserves partitioner associated with the input RDD. However, the distribution of duplicate values in the keys may change. If many duplicated values are created, this may slow down downstream shuffles.
`keys`	Return an RDD of just the keys (not distinct).	None	Almost never	Essentially free	Preserves the partitioning.

Function	Purpose	Key restriction	Runs out of memory when	Slow when	Output partitioner
`values`	Return an RDD of just the values.	None	Almost never	Essentially free	Does not preserve partitioning. Future wide transformations will cause a shuffle even if they have the same partitioner as the input RDD.
`sampleByKey`	Given a map from the keys to the percent of each key to sample, returns a stratified sample of the input RDD. The function is implemented with `mapPartitions` and uses a random number to determine whether each record will be kept. Thus, the size of the resulting sample may not exactly correspond to the percentage specified.	None	Almost never, unless the key map is too large to be broadcast to one of the worker nodes.	Same as `mapPartitions`, the function completes one pass through the data and does not require a shuffle.	Preserves partitioning of the input data.

In addition to sampling and mapping operations specific to PairRDDs, Table 8-5 shows the partitioning functions.

Table 8-5. Partitioning functions

Function	Purpose	Key restriction	Runs out of memory when	Slow when	Output partitioner
`partitionBy`	Takes a partitioner object and partitions the RDD accordingly. The partitioner object defines target partition index for records based on key.	Depends on which partitioner is used. See "Using the Spark Partitioner Object" on page 186.	When there are many duplicate values for each key, regardless of partitioner.	Always causes a shuffle. Range partitioners are generally slower than hash partitioners since they require the data to be partially evaluated in order to sample it.	Partitioned according to the partition argument.

The `PairRDDFunctions` class also includes all of the multiple RDD operations, including `join` and `cogroup`, which we covered in detail in Chapter 6.

Dictionary of OrderedRDDOperations

In addition to special functions for RDDs of key/value pairs, RDDs with an ordering on the key have even more functions, summarized in Table 8-6.

Table 8-6. Dictionary of operations in the `OrderedRDDFunctions` class

Function	Purpose	Key restriction	Runs out of memory when	Slow when	Output partitioner
`sortByKey`	Return an RDD sorted by the key.	Key must have implicit ordering.	When data is very unevenly distributed both in terms of repeated keys and distribution of keys. In particular, if the data associated with one key can't fit in memory.	Slows down with many duplicate keys. If the keys are not uniformly distributed, the data won't be partitioned evenly, which can also cause straggler tasks. Note: One of the few nonpure transformations requires an action to sample the input to create the range partitioner.	Range partitioner. Default number of partitions is the same as the input RDD.
`repartitionAndSortWithinPartitions`	Takes a partitioner and an implicit ordering. Partitions the RDD according to the partitioner and then sorts all the records on each partition according to the implicit ordering.	Key must have implicit ordering.	Depends on the partitioner. See `partitionBy`.	Slow with key skew because partitioner creates straggler tasks and because sorting each key will be more expensive. However, this is much faster than partitioning using `partitionBy` and then sorting within each key using `mapPartitions` because the sort is pushed into the shuffle step. See "Secondary Sort and repartitionAndSortWithinPartitions" on page 193.	Same as partitioner argument.
`filterByRange`	Takes a lower and an upper bound for the keys and returns an RDD of just the records whose key falls in that range.	See above.	Almost never.	When the RDD has already been partitioned by a range partitioner, this is cheaper than a generic `filter` because it scans only the partitions whose keys are in the desired range. Otherwise, performance is the same as the generic `filter` operation.	Preserves partitioning.

Sorting by Two Keys with sortByKey

Spark's `sortByKey` does allow sorting by tuples of keys for tuples with two elements. Thus, it is relatively easy to sort by two values in Spark by making use of Scala's implicit tuple ordering and then sort by a composite key described by `(Key1, Key2)`. For example, suppose that rather than finding the nth item, Goldilocks just wanted us to create a directory of `(columnIndex, value)` pairs sorted first by column index then by value. Assuming an RDD `indexValuePairs` of type `RDD[(Int, Double)]`, we could solve this problem with:

```
indexValuePairs.map((_, null)).sortByKey()
```

Then `sortByKey` will use the implicit ordering on an `(Int, Double)` tuple, which simply compares the first value and then compares the second.



`sortByKey` does not support implicit ordering on product types other than `Tuple2`.

Secondary Sort and repartitionAndSortWithinPartitions

Sorting in Spark could be implemented by partitioning an RDD with a `RangePartitioner` and then sorting within each partition using `mapPartitions`, much as we did in Example 8-4. However, this approach to sorting is slower than Spark's actual implementation of `sortByKey`. Instead of partitioning and then sorting, Spark leverages a technique called *secondary sort*, which pushes some of the work of sorting on the individual machine into the shuffle stage.



Secondary sort is a performant way of ordering data both among machines and within a single machine. The term comes from the MapReduce paradigm and describes a technique by which the programmer maps with one function but defines a different order for the elements to be used in the reduce call. The effect of this in Spark is that some of the sorting work that must be done locally can be accomplished during the shuffle stage rather than in the next stage, after the shuffle has completed.

Spark has a built-in function to perform a secondary sort called `repartitionAndSortWithinPartitions`. The `repartitionAndSortWithinPartitions` function is a wide transformation that takes a partitioner—defined on the argument RDD—and an implicit ordering, which must be defined on the keys of the RDD. The function partitions the data according to the partitioner argument and then sorts the records on each partition according to the ordering.



We do not need to directly pass an implicit ordering to the `repartitionAndSortWithinPartitions` function. If the function is called on an RDD whose keys have an ordering, Spark can infer that ordering and will sort accordingly. To use `repartitionAndSortWithinPartitions` to order on types that either do not have an implicit ordering or have an ordering other than the one we want, we need to define the implicit ordering on the keys of the RDD in our program before calling the `repartitionAndSortWithinPartitions` function. See Example 8-12.

If we look up the implementation of `sortByKey`, we can see that it calls the `repartitionAndSortWithinPartitions` function with a `RangePartitioner` and uses implicit ordering defined on the keys. As we discussed in "Range Partitioning" on page 186, the `RangePartitioner` will sample the data and assign a range of values for each partition based on the inferred distribution of the keys (for example, keys with values between 0 and 10 shall be placed on partition index two). Then `repartitionAndSortWithinPartitions` will sort the values on each partition (each range of data), and thus the entire result will be sorted by key. The secondary sort paradigm and the `repartitionAndSortWithinPartitions` can be used not only to do a performant sort on one key but also to define two kinds of ordering on the data: one that governs partitioning and another that governs the ordering of elements on the child partitions. The rest of this section will focus on the second use case, in which we want to organize the data first by one ordering and next by another.

Leveraging repartitionAndSortWithinPartitions for a Group by Key and Sort Values Function

The best way to order data with two orderings is to use the `repartitionAndSortWithinPartitions` function. One common use case for this is to define a function, which we might call "group by key and sort values," that returns an RDD grouped by key with the values in each group sorted. Unlike sorting by tuple keys, which we discussed in "Sorting by Two Keys with sortByKey" on page 193, this approach could be generalized to any partitioning defined on any key type and any custom ordering. We use `repartitionAndSortWithinPartitions` to repartition the RDD by one ordering on the keys and then define an implicit ordering for the records on a given partition.

With a little searching you can find numerous `groupByKeyAndSortValues` functions, although none of them have been merged into Spark. The presentation in *Advanced Analytics with Spark* (O'Reilly, 2015) by Sandy Ryza et al. is particularly good, and the following implementation closely mirrors his. This `groupByKeyAndSortValues` function assumes data in the form `((k, s), v)`, where `s` is the secondary key (perhaps derived from the value). It partitions the RDD by the first part of the key, then sorts by the second part of the key. Then, it combines all the values associated with one key into a sorted iterator.

The function has four steps:

1. Define a custom partitioner that partitions records according to the first element of the key.
2. Define an implicit ordering on the values. This is only necessary because the function is generic. The implicit ordering on tuples is first value, second value. We just have to tell Spark to use that tuple ordering.
3. Use `repartitionAndSortWithinPartitions` on the input RDD with the custom partitioner defined in step 1.
4. Coalesce the items using a `mapPartitions` routine. We can leverage the fact that items with the same first key are on the same partition and that the elements within each partition are sorted first by the first ordering and then by the second ordering.



Because we are using hash partitioning, this function does not actually sort values by the first key. Rather, it groups keys with the same hash value on the same machine. Thus, if we run the function of the values one through five and use four partitions, the first partition will contain one and five. To force the keys to appear in true sorted order, we would need to define a range partitioner. However, using the hash value is good enough if our goal is simply to group like keys.

Example 8-10 is the code for the custom partitioner. As you can see, we order only on the first part of the key. If we define the ordering on both parts of the key, the partitioner will group by the hash value of the entire tuple key. Consequently, the function may put elements with the same primary key but different secondary keys on two different partitions.

Example 8-10. Step 1 of secondary sort: defining a custom partitioner

```
class PrimaryKeyPartitioner[K, S](partitions: Int) extends Partitioner {

  /** We create a hash partitioner and use it with the first set of keys.
    */
  val delegatePartitioner = new HashPartitioner(partitions)

  override def numPartitions = delegatePartitioner.numPartitions

  /** Partition according to the hash value of the first key
    */
  override def getPartition(key: Any): Int = {
    val k = key.asInstanceOf[(K, S)]
    delegatePartitioner.getPartition(k._1)
  }
}
```

Next, we define the implicit ordering, as shown in Example 8-11. Recall from the specification of the function that both parts of the key already have an ordering defined. Spark already has an ordering for tuples of two elements with orderings, so we can simply tell Spark to use the generic Tuple2 ordering.

Example 8-11. Step 2 of secondary sort: define the implicit ordering

```
    implicit def orderByLocationAndName[A <: PandaKey]: Ordering[A] = {
      Ordering.by(pandaKey => (pandaKey.city, pandaKey.zip, pandaKey.name))
    }

    implicit val ordering: Ordering[(K, S)] = Ordering.Tuple2
```

Now, incorporating these two subroutines, we can define a function `groupByKeyAndSortBySecondaryKey`, as in Example 8-12. The new function will partition according to the partitioner defined in step 1, sort by the order defined in step 2, and then use a `groupSorted` function to combine the elements with the same first key into a single iterator.

Example 8-12. A general example of a "group by key and sort by secondary key" function

```
def groupByKeyAndSortBySecondaryKey[
    K: Ordering: ClassTag,
    S: Ordering: ClassTag,
    V: ClassTag
](pairRDD: RDD[((K, S), V)], partitions: Int): RDD[(K, List[(S, V)])] = {
  // Create an instance of our custom partitioner
  val colValuePartitioner = new PrimaryKeyPartitioner[Double, Int](partitions)

  // define an implicit ordering, to order by the second key the ordering will
  // be used even though not explicitly called
  implicit val ordering: Ordering[(K, S)] = Ordering.Tuple2

  // use repartitionAndSortWithinPartitions
  val sortedWithinParts =
    pairRDD.repartitionAndSortWithinPartitions(colValuePartitioner)

  sortedWithinParts.mapPartitions(iter => groupSorted[K, S, V](iter))
}

def groupSorted[K, S, V](
    it: Iterator[((K, S), V)]
): Iterator[(K, List[(S, V)])] = {
  val res = List[(K, ArrayBuffer[(S, V)])]()
  it.foldLeft(res)((list, next) =>
    list match {
      case Nil =>
        val ((firstKey, secondKey), value) = next
        List((firstKey, ArrayBuffer((secondKey, value))))

      case head :: rest =>
        val (curKey, valueBuf) = head
        val ((firstKey, secondKey), value) = next
        if (!firstKey.equals(curKey)) {
          (firstKey, ArrayBuffer((secondKey, value))) :: list
        } else {
          valueBuf.append((secondKey, value))
          list
        }

    }
  ).map { case (key, buf) => (key, buf.toList) }
```

```
    .iterator
}
```



When developing a function like this one that relies heavily on partitioning, make sure that your unit tests are for data that spans more than one partition. Make sure to test on different numbers of partitions and different data, because there is some randomness in partitioning (especially range partitioning). See Chapter 11 for more about running good distributed tests.

How Not to Sort by Two Orderings

It's important to note that several other seemingly obvious approaches to this problem are not guaranteed to give the correct result. For example, even regardless of performance issues, using `groupBykey` does not maintain the order of the values within the groups. Thus, the following implementation may not give the correct results:

```
indexValuePairs.sortByKey().groupByKey()
```

Spark sorting is also not guaranteed to be stable (preserve the original order of elements with the same value). Hence, repeated sorting is not a viable option:

```
indexValuePairs.sortByKey.map(_.swap()).sortByKey
```

In this case, the second `sortByKey` may not preserve the ordering generated in the first sort.

Goldilocks Version 2: Secondary Sort

The logic of secondary sort generalizes well beyond simply ordering data. It applies to any use case that requires the records to be arranged according to two different keys. The original Goldilocks example is related to secondary sort since it requires us to shuffle on one key (the column index) and then order the data within each key by value (by the value in the cells). Thus, rather than using `groupByKey` to ensure that the values associated with each key are coalesced and then sorting the elements associated with each key as a separate step, we can use `repartitionAndSortWithin Partitions`. Using `repartitionAndSortWithinPartitions` we can partition on the column index and sort on the value in each column. We are then guaranteed that all the values associated with each column will be on one partition and that they will be in sorted order of the values. We can simply loop through the elements on each partition, filter for the desired rank statistics in one pass through the data, and use the `groupSorted` function to combine the rank statistics associated with our column.

Defining the custom partitioner

The ordering and partition in `repartitionAndSortWithinPartitions` must be defined on the keys of the RDD, and thus we need to use the (column index, value) pairs as keys. We can map to a dummy value (like 1 or null) so that Spark will interpret the RDD as key/value pairs where the keys are a tuple of `(column index, value)`. We will then need to define a custom partitioner that partitions the keys based on the hash value of the first part of the key (the column index), as shown in Example 8-13.

Example 8-13. Goldilocks version 2, defining a custom partitioner to partition on column index

```
class ColumnIndexPartition(override val numPartitions: Int)
    extends Partitioner {
  require(
    numPartitions >= 0,
    s"Number of partitions " +
      s"($numPartitions) cannot be negative."
  )

  override def getPartition(key: Any): Int = {
    val k = key.asInstanceOf[(Int, Double)]
    Math.abs(k._1) % numPartitions // hashcode of column index
  }
}
```

Filtering on each partition

On each partition, we want the elements to be ordered first by column index, and second by value. The former ensures that all the records associated with one key are on the same partition. The latter ensures that the elements that are adjacent will be those with the same column indices and will be sorted so that we can find the rank statistics.

Ordering by the first value of a tuple and then the second is the existing implicit ordering on tuples. Thus, we do not have to specify an ordering for our data. After the `repartitionAndSortWithinPartitions` call, we know that the data will be partitioned according to column index and sorted by column index and value. For example, suppose that we were using the DataFrame described in Table 8-1 and that we were using three partitions. The first partition would contain the following values:

```
((1, 2.0), 1)
((1, 3.0), 1)
((1, 10.0), 1)
((1, 15.0), 1)
((4, 0.0), 1)
((4, 0.0), 1)
```

```
((4, 0.0), 1)
((4, 98.0), 1)
```

We can use the `filter` operation to loop through the elements of the iterator even though the filter requires us to keep track of global data. Recall from our discussion of iterator-to-iterator transformations in "Iterator-to-Iterator Transformations with mapPartitions" on page 141 that the `map`, `filter`, and `flatMap` operations defined on iterators transform the elements in the iterator in order. Thus, because the elements are sorted and grouped by key, we can keep track of a running total for the column index. If the element is one of the ones that corresponds to the target ranks statistic, then we can keep it. We then have to map the iterator to the first half of the tuple to remove the 1 dummy value. Note that we could combine these `map` and `filter` steps into one `flatMap` operation. We have chosen to present them separately in Example 8-14 since we think that the `filter` operation is easier to interpret.

Example 8-14. Goldilocks version 2, leveraging `repartitionAndSortWithinPartitions`

```
def findRankStatistics(
    dataFrame: DataFrame,
    targetRanks: List[Long],
    partitions: Int
) = {

  val pairRDD: RDD[((Int, Double), Int)] =
    GoldilocksGroupByKey.mapToKeyValuePairs(dataFrame).map((_, 1))

  val partitioner = new ColumnIndexPartition(partitions)
  // sort by the existing implicit ordering on tuples first key, second key
  val sorted = pairRDD.repartitionAndSortWithinPartitions(partitioner)

  // filter for target ranks
  val filterForTargetIndex: RDD[(Int, Double)] =
    sorted.mapPartitions(
      iter =>
        {
          var currentColumnIndex = -1
          var runningTotal = 0
          iter.filter({ case (((colIndex, value), _)) =>
            if (colIndex != currentColumnIndex) {
              currentColumnIndex = colIndex // reset to the new column index
              runningTotal = 1
            } else {
              runningTotal += 1
            }
            // if the running total corresponds to one of the rank statistics.
            // keep this ((colIndex, value)) pair.
            targetRanks.contains(runningTotal)
          })
```

```
        }.map(_._1),
      preservesPartitioning = true
    )
  groupSorted(filterForTargetIndex.collect())
}
```

Combine the elements associated with one key

After the `mapPartitions` step, we have to do one last local transformation to group the elements associated with one column index into a map. The code for the `groupSorted` function is presented in Example 8-15.

Example 8-15. Goldilocks version 2, group the elements associated with one key

```
private def groupSorted(
    it: Array[(Int, Double)]
): Map[Int, Iterable[Double]] = {
  val res = List[(Int, ArrayBuffer[Double])]()
  it.foldLeft(res)((list, next) =>
    list match {
      case Nil =>
        val (firstKey, value) = next
        List((firstKey, ArrayBuffer(value)))
      case head :: rest =>
        val (curKey, valueBuf) = head
        val (firstKey, value) = next
        if (!firstKey.equals(curKey)) {
          (firstKey, ArrayBuffer(value)) :: list
        } else {
          valueBuf.append(value)
          list
        }
    }
  ).map { case (key, buf) => (key, buf.toIterable) }
    .toMap
}
```

Notice that this code is very similar to the grouping function presented in Example 8-12.

Performance

This solution is considerably faster than the version 1 `groupByKey` solution on any shape of data. By using `repartitionAndSortWithinPartitions`, we are able to push the work to sort each column into the shuffle stage. Since the elements are sorted after the shuffle, we are able to use iterator-to-iterator transformations to filter the data and avoid forcing all the values associated with one partition into memory.

However, if the columns are relatively long, the `repartitionAndSortWithinPartitions` step may still lead to failures since it still requires one executor to be able to store all of the values associated with all of the columns that have the same hash value. Indeed, in our case we still saw failures in the shuffle stage using this approach at scale. In fact, a viable solution to the Goldilocks problem required taking an entirely different approach.

A Different Approach to Goldilocks

Unfortunately, none of the existing key/value transformations provided a magic bullet for the Goldilocks problem. None of the other aggregation operations that we might use as an alternative to `groupByKey` help us since the operation that we want to perform for each key—a sort—won't reduce the size of the data by key. As we discussed in the previous section, even rewriting our `groupByKey` approach using sophisticated secondary sort techniques was leading to failures. In the end, the secondary sort approach still required partitioning by the column index, which was not granular enough for the size of our data and the resources we had available. Instead, a performant solution to this problem required entirely rethinking how we parallelized the computational work.

Before we dive into the solution, let's review some of the methods we have learned to make transformations more performant:

- Narrow transformations on key/value data are quick and easy to parallelize relative to wide transformations that cause a shuffle.
- Partition locality can be retained across some narrow transformations following a shuffle. This applies to `mapPartitions` if we use `preservePartitioning=true` or `mapValues`.
- Wide transformations are best with many unique keys. This prevents shuffles from directing a large proportion of the data to reside on one executor.
- `sortByKey` is a particularly good way to partition data and sort within partitions since it pushes the ordering of data on local machines into the shuffle stage.
- Using iterator-to-iterator transforms in `mapPartitions` prevents whole partitions from being loaded into memory.
- We can sometimes rely on shuffle files to prevent recomputation of wide transformations even if we call several actions on the same RDD.

Using these insights, we were able to construct a solution to the Goldilocks problem using only one `sortByKey` and three `mapPartitions` operations. The critical insight is that the unit of parallelization for this problem does not need to be the columns. We can essentially solve the problem for each range of values. If the cell values are sorted and we know how many elements are on each partition from each column (which is

easy to calculate using a performant `mapPartitions` routine), then we can determine the location of the nth element.

Our solution can be enumerated in five steps:

1. Map the rows of data to pairs of `(cell value, index)`.
2. Perform a `sortByKey` operation on all tuples defined in step 1.
3. Using `mapPartitions`, determine how many elements in each column are on each partition and collect that information to the driver.
4. Perform a local computation on the result of step 3 to determine the location of each desired rank statistic. For example, suppose that we are looking for the 13th element. Suppose also that in step 3 we determined that the first partition had 10 elements from column six. In this case, we can conclude that the 13th element will be the third largest element in column six on the second partition.
5. Using the result of step 4, use another `mapPartitions` transformation to filter for the elements that correspond to the desired rank statistics. Collect this information back to the driver.

Map to (cell value, column index) pairs

Example 8-16 is the code for step 1 of our solution: mapping to the `(cell value, column index)` pairs. We use Spark's `flatMap` function to transform each row into a sequence of tuples.

Example 8-16. Goldilocks algorithm version 3, map to `(cell value, column index)` pairs

```
private def getValueColumnPairs(dataFrame: DataFrame): RDD[(Double, Int)] = {
  dataFrame.rdd.flatMap { row: Row =>
    row.toSeq.zipWithIndex
      .map { case (v, index) =>
        (v.toString.toDouble, index)
      }
  }
}
```

Sort and count values on each partition

Once we have mapped the rows so that they are keyed on cell value, use `sortByKey`. After the sort, we will have calculated the number of elements on each partition. Example 8-17 is a function that takes a sorted RDD of `(double, column index)` pairs and the number of columns in the original DataFrame, then returns an array where each element corresponds to a partition. Each element of the array contains

the partition index and an array of the counts of elements on that partition in each column. The length of each subarray will correspond to the number of columns in the original dataset.

Example 8-17. Goldilocks algorithm version 3, count values by column on each partition

```
private def getColumnsFreqPerPartition(
    sortedValueColumnPairs: RDD[(Double, Int)],
    numOfColumns: Int
): Array[(Int, Array[Long])] = {

  val zero = Array.fill[Long](numOfColumns)(0)

  def aggregateColumnFrequencies(
      partitionIndex: Int,
      valueColumnPairs: Iterator[(Double, Int)]
  ) = {
    val columnsFreq: Array[Long] = valueColumnPairs.aggregate(zero)(
      (a: Array[Long], v: (Double, Int)) => {
        val (value, colIndex) = v
        // increment the cell in the zero array corresponding to this column index
        a(colIndex) = a(colIndex) + 1L
        a
      },
      (a: Array[Long], b: Array[Long]) => {
        a.zip(b).map { case (aVal, bVal) => aVal + bVal }
      }
    )

    Iterator((partitionIndex, columnsFreq))
  }

  sortedValueColumnPairs
    .mapPartitionsWithIndex(aggregateColumnFrequencies)
    .collect()
}
```

The subfunction `aggregateColumnFrequencies` is applied to the records on each partition. It uses the aggregate operation defined on iterators. The zero value is an array the length of the original column of zeros. For each pair in the iterator, the sequence operation of the aggregation operation increments the cell that corresponds to that column index in the zero array. The combine operation adds the values in two of these arrays. Thus, the result is an array of the counts for the corresponding column index. For example, assume that there were three columns and the first two partitions contained the following key/value pairs:

```
Partition 1: (1.5, 0) (1.25, 1) (2.0, 2) (5.25, 0)
Partition 2: (7.5, 1) (9.5, 2)
```

The output would be:

```
[(0, [2, 1, 1]), (1, [0, 1, 1])]
```

We expect this step to be a relatively inexpensive operation. The `mapPartitions` step is a narrow transformation since it requires traversing the iterator just once. Thus, this operation will not incur a shuffle and can spill to disk selectively. We use arrays to aggregate because, as we discussed in "Using Smaller Data Structures" on page 138, they should create the least garbage collection overhead. After this `mapPartitions` step, we collect the results into an array.



We are not using the result of this `mapPartitions` operation in a distributed way since we are collecting it to the driver. Thus, we actually do not need to set the preserves partitioning function to false.

Determine location of rank statistics on each partition

Once we have the results of the `getColumnsFreqPerPartition` function, we have to use that information to determine where the rank statistics are on each partition, as shown in Example 8-18. This computation is done locally with the results of the previous function. In order to determine the location of each rank statistic, we loop through the (sorted) result of the previous function while keeping a running total of the elements in each column across the partitions. If, for any of the columns, a rank statistic is between the previous and updated value of the running total, we know that the rank statistics can be found on that partition. If this is the case, we increment the `relevantIndexList` with the column index and the rank statistic—the previous running total. We can then return an array, where each element is the partition index, and a list object of pairs. Each pair represents a column index for that rank statistic and the index of that rank statistics in that column.

For example, if the inputs to the function were:

```
targetRanks: [5]
partitionColumnsFreq: [(0, [2, 3]), (1, [4, 1]), (2, [5, 2])]
numOfColumns: 2
```

The output will be:

```
(0, []), (1, [(0, 3)]), (2, [(1, 1)])]
```

Example 8-18. Goldilocks algorithm version 3, determine location of rank statistics on each partition

```
private def getRanksLocationsWithinEachPart(
    targetRanks: List[Long],
    partitionColumnsFreq: Array[(Int, Array[Long])],
    numOfColumns: Int
): Array[(Int, List[(Int, Long)])] = {
```

```
    val runningTotal = Array.fill[Long](numOfColumns)(0)
    // The partition indices are not necessarily in sorted order, so we need
    // to sort the partitionsColumnsFreq array by the partition index (the
    // first value in the tuple).
    partitionColumnsFreq.sortBy(_._1).map {
      case (partitionIndex, columnsFreq) =>
        val relevantIndexList = new mutable.ListBuffer[(Int, Long)]()

        columnsFreq.zipWithIndex.foreach { case (colCount, colIndex) =>
          val runningTotalCol = runningTotal(colIndex)
          val ranksHere: List[Long] = targetRanks.filter(rank =>
            runningTotalCol < rank && runningTotalCol + colCount >= rank
          )

          // For each of the rank statistics present add this column index and the
          // index it will be at on this partition (the rank - the running total).
          relevantIndexList ++= ranksHere
            .map(rank => (colIndex, rank - runningTotalCol))

          runningTotal(colIndex) += colCount
        }

        (partitionIndex, relevantIndexList.toList)
    }
  }
```

Filter for rank statistics

Now, armed with the location (partition index and position within each partition) of the rank statistics for each column, we have to pass through the sorted data again to filter for the correct rank statistics. This task is accomplished with the function in Example 8-19, called `findTargetRanksIteratively`, which uses the original sorted tuples of `(value, column index)` pairs and the results of the previous function. We use an iterator-to-iterator transformation with a `filter` and a `map` step. (Note that these could be replaced by `flatMap`.) This produces the final result, an RDD of `(columnIndex, rank statistic)` pairs, which we can then collect back to the driver.

Example 8-19. Goldilocks algorithm version 3, filter for the desired rank statistics

```
  private def findTargetRanksIteratively(
      sortedValueColumnPairs: RDD[(Double, Int)],
      ranksLocations: Array[(Int, List[(Int, Long)])]
  ): RDD[(Int, Double)] = {

    sortedValueColumnPairs.mapPartitionsWithIndex(
      (partitionIndex: Int, valueColumnPairs: Iterator[(Double, Int)]) => {
        val targetsInThisPart: List[(Int, Long)] =
          ranksLocations(partitionIndex)._2
        if (targetsInThisPart.nonEmpty) {
```

```
          val columnsRelativeIndex: collection.MapView[Int, List[Long]] =
            targetsInThisPart.groupBy(_._1).view.mapValues(_.map(_._2))
          val columnsInThisPart = targetsInThisPart.map(_._1).distinct

          val runningTotals: mutable.HashMap[Int, Long] = new mutable.HashMap()
          runningTotals ++= columnsInThisPart
            .map(columnIndex => (columnIndex, 0L))
            .toMap

          // filter this iterator, so that it contains only those (value, columnIn
dex)
          // that are the ranks statistics on this partition
          // Keep track of the number of elements we have seen for each columnIndex
using the
          // running total hashMap.
          valueColumnPairs
            .filter { case (value, colIndex) =>
              lazy val thisPairIsTheRankStatistic: Boolean = {
                val total = runningTotals(colIndex) + 1L
                runningTotals.update(colIndex, total)
                columnsRelativeIndex(colIndex).contains(total)
              }
              (runningTotals contains colIndex) && thisPairIsTheRankStatistic
            }
            .map(_.swap)
        } else {
          Iterator.empty
        }
      }
    )
  }
```

Goldilocks Version 3: Sort on Cell Values

Combining all of these functions, we get a full solution to the Goldilocks problem (Example 8-20).

Example 8-20. Goldilocks version 3, sort on values

```
  def findRankStatistics(
      dataFrame: DataFrame,
      targetRanks: List[Long]
  ): Map[Int, Iterable[Double]] = {

    val valueColumnPairs: RDD[(Double, Int)] = getValueColumnPairs(dataFrame)
    val sortedValueColumnPairs = valueColumnPairs.sortByKey()
    sortedValueColumnPairs.persist(StorageLevel.MEMORY_AND_DISK)

    val numOfColumns = dataFrame.schema.length
    val partitionColumnsFreq =
      getColumnsFreqPerPartition(sortedValueColumnPairs, numOfColumns)
```

```
    val ranksLocations = getRanksLocationsWithinEachPart(
      targetRanks,
      partitionColumnsFreq,
      numOfColumns
    )

    val targetRanksValues =
      findTargetRanksIteratively(sortedValueColumnPairs, ranksLocations)
    targetRanksValues.groupByKey().collectAsMap()
  }
```

From a code readability perspective, this solution is ugly. It requires dozens of lines of code and four passes through the data. However, we expect that it will avoid memory errors on the executors and complete faster than the `groupByKey` or secondary sort solutions. This is because the data on each column should be mostly distinct doubles, and thus the shuffle should be fairly efficient. The final two `mapPartitions` routines involve reducing the data and can be achieved through iterator-to-iterator transformations, so we expect them to scale well. Indeed, on randomly distributed test data with many records, this solution outperforms the other implementations of the Goldilocks problem by orders of magnitude.

Straggler Detection and Unbalanced Data

"Stragglers" are those tasks within a stage that take much longer to execute than the other tasks in that stage. Recall from our discussion in "Spark Job Scheduling" on page 23 that a new stage begins after each wide transformation. When wide transformations are called on the same RDD, stages must usually be executed in sequence, so straggler tasks may hold up an entire job. Stragglers occur when Spark has not allocated resources correctly, and in particular if the data has not been partitioned evenly. Stragglers are a good indication of unbalanced keys because the distribution of tasks depends on partitioning, which in turn depends on the keys. The Spark web UI allows you to monitor tasks as they are executed in real time.

If during a wide transformation you notice that some partitions take much longer than others or show more retries, it is likely that the data is not being partitioned evenly. This usually happens because some keys have many more values than others. In this case, it will speed up shuffle operations to either use something else as keys or add random "noise" to your keys to create more distinct keys. Sometimes, you can even perform a map-side reduction to combine or filter records with duplicates on each partition before shuffling all the data.



While `sortByKey` is less likely to cause memory errors at scale than `groupByKey`, it is still quite possible. Think back to Figure 8-2 and see how we can still tip over with a `sortByKey` in Figure 8-4.



One workaround to unbalanced keys can be adding "junk" to the end of the key, such as a random number. That way, Spark can recognize the keys as distinct and spread them across partitions. In our case, this could mean spreading the zeros across machines, which should not affect the accuracy.

The following images are meant to illustrate an imbalanced `sortByKey` (see Figure 8-4) and a balanced `sortByKey` (see Figure 8-5).

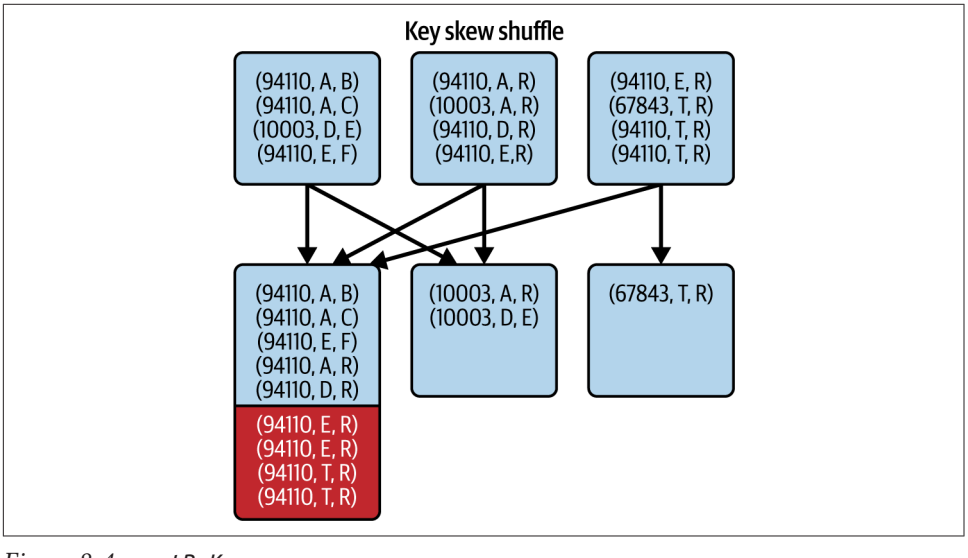


Figure 8-4. `sortByKey` memory errors

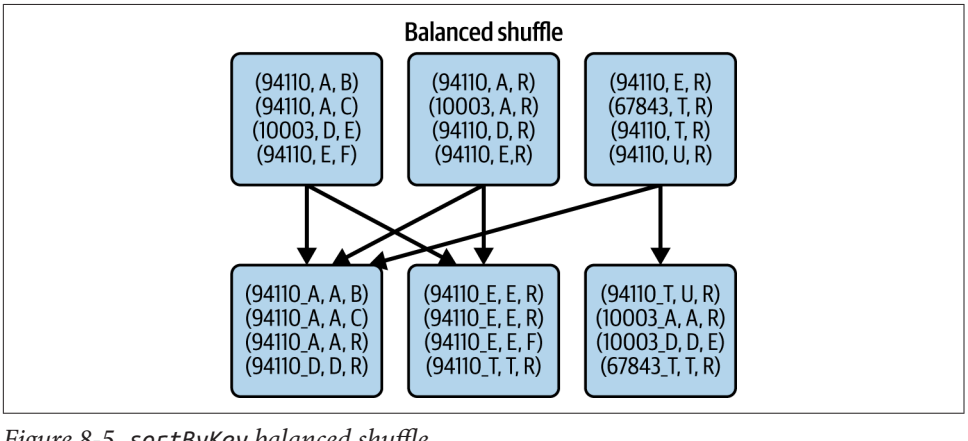


Figure 8-5. `sortByKey` balanced shuffle

Back to Goldilocks (Again)

Goldilocks has told us that her data was evenly distributed and that all the columns had "lots" of distinct values. However, when she tried to run the algorithm on her data, it failed. When we observed the algorithm running in the web UI, we noticed that some partitions were running much slower than the others and sometimes running out of memory. This indicated that we had too many duplicate keys. This was surprising because in our implementation, the keys were the values of the data (doubles). However, further analysis revealed that in most of the columns, about 25% of the values were zeros. Thus, nearly one in four keys in our sort was a zero value. This meant that 1/4 of the records in the RDD we were sorting clustered around zero. Consequently, no matter how many partitions we used, a quarter of the records were being shuffled to the first few partitions.

Goldilocks Version 4: Reduce to Distinct on Each Partition

Rather than trying to partition those pairs differently, we realized that the last four steps of the algorithm could be modified to work on input of tuples of `((cell value, column index), count)`. Then, in the first step, rather than mapping the records on each partition to `(cell value, column index)` pairs, we could map each partition to distinct pairs, keeping track of the number of times each `((cell value, column index))` pair appeared on that partition. By mapping to `distinct` on each partition, we reduce the number of duplicate keys without incurring a shuffle (as would a distributed `distinct` call on the entire RDD).

After creating these `((value, columnIndex), count)` tuples, we know that although we might have some repeated keys, we introduce a theoretical limit on the number of duplicate keys. Specifically, if the same value is present in a column on each partition, the maximum number of duplicate keys is (the number of columns * the number of partitions).[4] Not only does this input step balance the data so it can be partitioned more effectively, it also dramatically reduces the total number of records to be shuffled. In the Goldilocks case, where 25% of the rows were zero, we now have roughly 75% of the number of tuples to sort as the previous iteration of our solution.

Aggregate to ((cell value, column index), count) on each partition

Example 8-21 is the code for this first step. Rather than mapping to `(cell_value, column_index)` pairs, we mapped to `((cell value, column index), count)` on each partition. We were transforming the columnar data into pairs by using a `mapPartitions` step. Now, rather than doing a `flatMap` of each row in the original data, we

4 Using a distributed `reduceByKey` could have reduced this to simply the number of columns at the cost of a shuffle.

update a `HashMap` whose keys are the `(value, index)` pairs and whose values are the number of times that `(cell value, column index)` pair appears in that partition. Even if each column on a given partition has all unique values, we expect that this solution will not cause out-of-memory errors. We are making the `HashMap` on each partition, and so the size of the `HashMap` should still be smaller than the iterator of all the records. However, if we did have all distinct values, this might not be the case, because a hash map is a much less memory-efficient data structure than an iterator.

Example 8-21. Goldilocks version 4, aggregate on each partition

```
def getAggregatedValueColumnPairs(
    dataFrame: DataFrame
): RDD[((Double, Int), Long)] = {

  val aggregatedValueColumnRDD = dataFrame.rdd.mapPartitions(rows => {
    val valueColumnMap = new mutable.HashMap[(Double, Int), Long]()
    rows.foreach(row => {
      row.toSeq.zipWithIndex.foreach { case (value, columnIndex) =>
        val key = (value.toString.toDouble, columnIndex)
        val count = valueColumnMap.getOrElseUpdate(key, 0)
        valueColumnMap.update(key, count + 1)
      }
    })

    valueColumnMap.toIterator
  })

  aggregatedValueColumnRDD
}
```

Sort and find rank statistics

The rest of the function is similar to the original version. We simply adjust for keeping track of the number of times the pair appears rather than assuming that each pair in the sorted RDD occurred once. The updated code is shown in Examples 8-22, 8-23, 8-24, and 8-25, with Example 8-26 bringing everything together.

Example 8-22. Goldilocks version 4, count values per column on each partition

```
private def getColumnsFreqPerPartition(
    sortedAggregatedValueColumnPairs: RDD[((Double, Int), Long)],
    numOfColumns: Int
): Array[(Int, Array[Long])] = {

  val zero = Array.fill[Long](numOfColumns)(0)

  def aggregateColumnFrequencies(
      partitionIndex: Int,
```

```
      pairs: Iterator[((Double, Int), Long)]
  ) = {
    val columnsFreq: Array[Long] = pairs.aggregate(zero)(
      (a: Array[Long], v: ((Double, Int), Long)) => {
        val ((value, colIndex), count) = v
        a(colIndex) = a(colIndex) + count
        a
      },
      (a: Array[Long], b: Array[Long]) => {
        a.zip(b).map { case (aVal, bVal) => aVal + bVal }
      }
    )

    Iterator((partitionIndex, columnsFreq))
  }

  sortedAggregatedValueColumnPairs
    .mapPartitionsWithIndex(aggregateColumnFrequencies)
    .collect()
}
```

Example 8-23. Goldilocks version 4, determine locations of rank statistics on each partition

```
  private def getRanksLocationsWithinEachPart(
      targetRanks: List[Long],
      partitionColumnsFreq: Array[(Int, Array[Long])],
      numOfColumns: Int
  ): Array[(Int, List[(Int, Long)])] = {

    val runningTotal = Array.fill[Long](numOfColumns)(0)

    partitionColumnsFreq.sortBy(_._1).map {
      case (partitionIndex, columnsFreq) =>
        val relevantIndexList = new mutable.ListBuffer[(Int, Long)]()

        columnsFreq.zipWithIndex.foreach { case (colCount, colIndex) =>
          val runningTotalCol = runningTotal(colIndex)

          val ranksHere: List[Long] = targetRanks.filter(rank =>
            runningTotalCol < rank && runningTotalCol + colCount >= rank
          )
          relevantIndexList ++= ranksHere
            .map(rank => (colIndex, rank - runningTotalCol))

          runningTotal(colIndex) += colCount
        }

        (partitionIndex, relevantIndexList.toList)
    }
  }
```

Example 8-24. Goldilocks version 4, filter for rank statistics

```
  private def findTargetRanksIteratively(
      sortedAggregatedValueColumnPairs: RDD[((Double, Int), Long)],
      ranksLocations: Array[(Int, List[(Int, Long)])]
  ): RDD[(Int, Double)] = {

    sortedAggregatedValueColumnPairs.mapPartitionsWithIndex(
      (
          partitionIndex: Int,
          aggregatedValueColumnPairs: Iterator[((Double, Int), Long)]
      ) => {

        val targetsInThisPart: List[(Int, Long)] =
          ranksLocations(partitionIndex)._2
        if (targetsInThisPart.nonEmpty) {
          FindTargetsSubRoutine.asIteratorToIteratorTransformation(
            aggregatedValueColumnPairs,
            targetsInThisPart
          )
        } else {
          Iterator.empty
        }
      }
    )
  }
```

Example 8-25. Goldilocks version 4, iterator-to-iterator transformation to filter for the rank statistics

```
  def asIteratorToIteratorTransformation(
      valueColumnPairsIter: Iterator[((Double, Int), Long)],
      targetsInThisPart: List[(Int, Long)]
  ): Iterator[(Int, Double)] = {

    val columnsRelativeIndex =
      targetsInThisPart.groupBy(_._1).mapValues(_.map(_._2))
    val columnsInThisPart = targetsInThisPart.map(_._1).distinct

    val runningTotals: mutable.HashMap[Int, Long] = new mutable.HashMap()
    runningTotals ++= columnsInThisPart
      .map(columnIndex => (columnIndex, 0L))
      .toMap

    // filter out the pairs that don't have a column index that is in this part
    val pairsWithRanksInThisPart = valueColumnPairsIter.filter {
      case (((value, colIndex), count)) =>
        columnsInThisPart contains colIndex
    }

    // map the valueColumn pairs to a list of (colIndex, value) pairs that corre
```

```
spond
    // to one of the desired rank statistics on this partition.
    pairsWithRanksInThisPart.flatMap { case (((value, colIndex), count)) =>

      val total = runningTotals(colIndex)
      val ranksPresent: List[Long] = columnsRelativeIndex(colIndex)
        .filter(index =>
          (index <= count + total)
            && (index > total)
        )

      val nextElems: Iterator[(Int, Double)] =
        ranksPresent.map(r => (colIndex, value)).toIterator

      // update the running totals
      runningTotals.update(colIndex, total + count)
      nextElems
    }
  }
```

Putting all the code together, we have a final solution to the Goldilocks problem (Example 8-26).

Example 8-26. Goldilocks with hash map

```
  def findRankStatistics(
      dataFrame: DataFrame,
      targetRanks: List[Long]
  ): Map[Int, Iterable[Double]] = {

    val aggregatedValueColumnPairs: RDD[((Double, Int), Long)] =
      getAggregatedValueColumnPairs(dataFrame)
    val sortedAggregatedValueColumnPairs =
      aggregatedValueColumnPairs.sortByKey()
    sortedAggregatedValueColumnPairs.persist(StorageLevel.MEMORY_AND_DISK)

    val numOfColumns = dataFrame.schema.length
    val partitionColumnsFreq =
      getColumnsFreqPerPartition(sortedAggregatedValueColumnPairs, numOfColumns)
    val ranksLocations =
      getRanksLocationsWithinEachPart(
        targetRanks,
        partitionColumnsFreq,
        numOfColumns
      )

    val targetRanksValues =
      findTargetRanksIteratively(
        sortedAggregatedValueColumnPairs,
        ranksLocations
      )
```

```
    targetRanksValues.groupByKey().collectAsMap()
  }
```

As expected, this solution led to a 4× to 5× speed up of an empty cluster.

Goldilocks postmortem

Given the task of finding an arbitrary number of rank statistics by group (column), we have presented five solutions:

- "Goldilocks Version 0: Iterative Solution" on page 172. Our first solution iteratively looped through each group and performed a distributed sort, resulting in one stage and one expensive distributed sort per group.
- "Goldilocks Version 1: groupByKey Solution" on page 176. The next solution used `groupByKey` shuffle records associated with the same group onto the same partition. Then we were able to sort each group in one stage by using `mapPartitions` to sort the values in each group.
- "Goldilocks Version 2: Secondary Sort" on page 198. Using the secondary sort technique, we improved our `groupByKey` solution by replacing the `groupByKey` and sorting steps with the `repartitionAndSortWithinPartitions` function to push the work of sorting each group into the shuffle stage.
- "Goldilocks Version 3: Sort on Cell Values" on page 207. Next, we realized that it was possible to solve the problem using only one sort on the value of each record, rather than on the group. We developed a solution that keyed the records by value (rather than by group/column index), sorted all the records, and then performed a series of narrow transformations to collect the results. We expected the new sorting keys (the values in the columns) to contain fewer duplicates than the size of each group, which we used as a key in version 2.
- "Goldilocks Version 4: Reduce to Distinct on Each Partition" on page 210. Finally, upon realizing that we had a high number of duplicate records in each group, we modified the previous solution to perform a map-side reduction before sorting the data. This solution had better results with our client's skewed data.

We found that performance was dependent mostly on three characteristics of the input data: 1) the number of original records, 2) the number of groups (columns in this case) to compute the metrics on, and 3) the percentage of duplicate records by group. Note that these metrics are all relative to the size and shape of the hardware you are computing on. A small number of records really means that the records fit comfortably in the computational memory of all of the executors.

"Goldilocks Version 4: Reduce to Distinct on Each Partition" on page 210 is not the best solution for all input data. The only one of these five solutions that is always "bad" is the `groupByKey` solution (version 1). We would expect that version 1 should

perform worse than the `repartitionAndSortWithinPartitions` (version 2) solution in all cases due to the limitations of `groupByKey` and the advantages of secondary sort (see "Secondary Sort and repartitionAndSortWithinPartitions" on page 193). The remaining four versions are all desirable in some cases.

Version 0 uses the value in each group to sort (as do solutions 3 and 4). Thus, it is optimal that there is only one group, because it doesn't require extra passes through the data.

Version 2 partitions by group. Thus it is optimal if there are many groups, and each group fits easily in memory on one executor. If the groups are sufficiently small, it would outperform versions 3 and 4, which sort on value and require three narrow transformations to determine the correct results, because this solution requires fewer passes through the data. With enough groups and rank statistics, this solution may also be more likely to succeed than versions 3 and 4, both of which require keeping a hash map of rank statistics for each group in memory.

Version 3 partitions by values across all groups. Thus, it is optimal if each group is very large and there are few duplicate values across the columns. Unlike version 2, only the records with duplicate values need to be on one partition. Assuming that the number of duplicate values is less than the maximum group size, this will lead to better parallelization.

Version 4 uses a narrow transformation to aggregate distinct values by group on the input partitions, then partitions by distinct values (like solution 3). Solution 4 is better than 3 only when there are many duplicate values in the columns. However, if all values are unique, this operation provides no benefit and risks causing memory errors by creating a hash map of all the values on each partition.

In our use case, we had input data containing several thousand groups with 300 million records each. In each column (group) we saw that about 25% of records were identical. Versions 0, 1, and 2 did not complete on a 10-node, crowded cluster. Version 3 did complete, but version 4 led to a 4× improvement in production over version 3.

Conclusion

In this chapter, we have seen how to use functions in the `PairRDDFunctions` and `OrderedRDDFunctions` classes in ways that are more likely to succeed at scale. Much of the chapter has focused on specific techniques for working with wide transformations. We focused on some of the causes of memory errors during the shuffle stage, such as avoiding aggregation operations like `groupByKey` that do not reduce the space needed to store all the records associated with each key. We have learned about partitioning: how thinking ahead toward the next key/value transformation and doing smart partitioning can reduce the number of shuffles we need. We have

focused on some strategies to do fewer shuffles: using smart partitioning, maintaining partition information with narrow transformations, and leveraging co-location for joins. When shuffles are required, we have covered some techniques for reducing the cost of those shuffles. In particular, we have shown that unbalanced data, particularly a high number of duplicate values per key, is likely to slow down shuffles and cause memory errors.

We hope that in addition to presenting some tricks for improving the performance of RDD transformations, this chapter provides some tools for reconceptualizing problem solving with Spark. By focusing on the Goldilocks use case, we have tried to show that writing performant Spark code sometimes requires a new kind of thinking. Simply stringing together the set of API calls that most obviously describe what you are trying to do often leads to a solution such as the one presented in "Goldilocks Version 1: groupByKey Solution" on page 176—an implementation that succeeds on relatively small, clean inputs and fails in the wild west of real-world data. Instead, we hope to show that despite the relative simplicity of writing distributed code with Spark, the best results can be achieved only with a close eye to how those distributed computations will be performed. We hope to encourage you to think of keys in Spark not as the index or category for the records, but as the axis of parallelization for the computational load. We encourage you wherever possible to design Spark routines with as much knowledge as possible about the size, distribution, and complexity of the data they are processing. Philosophically, we want to demonstrate how high-performance code often requires big rather than small changes—and that the most performant code is not always the cleanest.

CHAPTER 9
Going Beyond Scala

Working in Spark doesn't mean limiting yourself to Scala or even limiting yourself to the JVM or languages that Spark explicitly supports. There are many ways to use Spark with different languages, including remote procedure call systems like Spark Connect, JVM interop options like Java Native Interface (JNI), Java Native Access (JNA), or Java Native Runtime (JNR), and prebuilt wrappers over these (like PySpark or SparklyR). This chapter will discuss the performance considerations of using other languages in Spark and how to work with existing libraries effectively. You will learn the performance trade-offs of using different languages and how you can use common Spark accelerators to improve your existing pipelines with minimal changes.

Spark's language interoperability can be thought of in two groups: one is the worker code inside of your transformations (e.g., the lambdas inside of your maps), and the second is being able to specify the transformations on RDDs/Datasets (e.g., the driver program).

Spark has first-party APIs, or built-in support for APIs, to write driver programs and worker code in R, Python, Scala, and Java. The first-party languages share much of the same design, making it easier to reason about their performance. However, even if support is first-party, it does not mean it will be better. In some cases, third-party bindings have taken interesting work[1] to minimize overhead that has not

1 This includes exploring shared memory, some of which may be present in some of the closed sourced optimizers as well.

been implemented in the first-party languages, including JavaScript.[2] Others follow a more traditional implementation, like Julia, C#, and F#. Spark Connect extends this support to a wide variety of languages, albeit with a smaller API scope coverage.



There is a proposal and active discussion on how to extend Spark UDF support to more languages using interprocess communication (IPC). This proposal is in the early stages and has already failed its first vote, but it is likely that a future iteration of it will be adopted. If you want to write UDFs in COBOL, Go, or Rust without dealing with foreign function interfaces (FFIs), this has solid potential.

Often, the language you will choose to specify the code inside of your transformations will be the same as the language for writing the driver program, but when working with specialized libraries or tools (such as CUDA[3]), Spark supports a range of languages on the driver, and you can use an even more comprehensive range of languages inside your transformations on the workers. We will discuss the design behind language support and how the performance difference can impact your work. On the worker side, the Spark worker is always running in the JVM and, if necessary, will start another process for the target and copy the required data and result. This copying can be expensive, but Spark's DAG of dependencies and clever pipelining minimize the amount of data copying that needs to occur. The techniques that the different language APIs use for interfacing their worker code are similar to the same methods you can use to call your custom code regardless of the language of your driver.

There are many ways to go outside the JVM, ranging from the JNI, Unix pipes, or interfacing with long-running companion servers over sockets. These are the same techniques used inside Spark's internals when interfacing with other languages. For example, JNI is used to call some linear algebra libraries, and Unix pipes are used to interface with Python code on the workers. The most efficient solution often depends on whether multiple transformations will need to be evaluated, the environment and language setup costs, and the computational complexity of the transformations. Regardless of which specific approach you choose to integrate other languages outside the JVM, these all require copying your data from the JVM to the runtime of your target language.

2 The demand for JavaScript big data did not emerge as hoped, but the EclairJS engine demonstrates an interesting and unique approach using different JS evaluations, allowing for faster execution along the ideas of using Py4J, which was not successful due to native libraries. It's a fascinating design, albeit not a project that is currently maintained.

3 CUDA is a specialized language for parallel GPU programming from NVIDIA. ROCm is a similar option from AMD.



Regardless of how you go outside of the JVM, when you introduce additional languages or processes, there is a greater chance for container out-of-memory exceptions as the JVM may have a partial view of the memory used. These container out-of-memory exceptions can be difficult to debug and may only be visible through exit code 137.

New Spark accelerators, which primarily execute outside of the JVM, exist to run faster than Spark's built-in tools. Some of these accelerators can reduce the "in and out of the JVM" cost by also implementing the read and write stages, so the data is first loaded outside of the JVM or written without even touching the JVM. In most cases, these optimizers will still have to transfer some of the records to and from the JVM. Still, the general performance improvement of using specialized instructions or hardware (like GPUs or ASICs like Speedata's) can dwarf this relatively small cost.

Languages with JVM implementations, like Kotlin or Java, can avoid the expensive copy of the data from the Spark worker to the target language. Some languages take a mixed approach, which executes the worker inside of the JVM but leaves the driver program outside of the JVM. While there is, of course, some overhead in having the driver program outside of the JVM, the amount of data that needs to be passed between the Scala driver and target driver is much smaller compared to the amount of data processed by even just one of the workers.

Beyond Scala Within the JVM

This section will look at how to access the Spark APIs from different languages within the JVM and some of the performance considerations of going outside of Scala. Even if you are going outside of the JVM, it is helpful to understand this section since the non-JVM languages often depend on the Java APIs rather than the Scala APIs.

Working in other languages doesn't always mean moving beyond the JVM, and staying within the JVM can have many performance benefits—mostly from not having to copy data. While you don't necessarily need special bindings or wrappers to access Spark outside of Scala, calling Scala code can be difficult in other languages. Spark supports Java 8 lambdas for use within transformations, and users with older versions of the JDK can implement the corresponding interface from `org.apache.spark.api.java.function`. Even when data doesn't need to be copied, working in a different language can have small yet important performance considerations.

The difficulty with accessing the Scala APIs is especially true for accessing functions with class tags[4] or using functionality provided through implicit conversions (such as all of the `Double` and `Tuple-specific` functionality on RDDs). For functionality that depends on implicit conversions, equivalent classes are often provided along with explicit transformations to these concrete classes. For functions that rely on class tags, "fake" class tags (e.g., `AnyRef`) can be supplied. These are normally provided automatically by wrappers for the Java API. Using the concrete class instead of the implicit conversion generally doesn't add any overhead, but the fake class tags can limit some of the compiler optimizations.

The Java API is kept quite close to the Scala API in terms of features, with only the occasional functionality or Developer API not being available. Support for other JVM languages, like Clojure's Sparkling, is done using the Java APIs instead of calling the Scala APIs directly. Since most of the language bindings, even non-JVM languages like Python and R, go through the Java APIs, it is helpful to understand the Java APIs.

The Java APIs closely resemble the Scala APIs, while avoiding depending on class tags or implicit conversions. The lack of implicit conversions means that rather than automatically converting RDDs containing `Tuples` or `Doubles` to special classes with additional functions, explicit function conversions must be used (such as `mapToDouble` and `mapToPair`). These functions are only defined on Java RDDs, meaning these special types are simply wrappers of Scala RDDs, which maintains interoperability. These special functions also return different types, such as `JavaDoubleRDD` and `JavaPairRDD`, which have the functionality that is provided by the implicit conversions in Scala.

Let's revisit the canonical word count example using the Java APIs shown in Example 9-1. Since calling Scala APIs from Java can sometimes be convoluted, Spark's Java APIs are primarily implemented in Scala while hiding the non-Java-compatible components (namely, class tags and Scala-specific types). This allows the Java wrappers to be a very thin layer, consisting of only a few lines on average, with very little reimplementation required.

Example 9-1. Java Word count example

```
import scala.Tuple2;

import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaSparkContext;
```

4 Used to store type information that is erased or not known at compile time. The most common example for usage of class tags is constructing Arrays.

```
import java.util.regex.Pattern;
import java.util.Arrays;

public final class WordCount {
  private static final Pattern pattern = Pattern.compile(" ");

  public static void main(String[] args) throws Exception {
    JavaSparkContext jsc = new JavaSparkContext();
    JavaRDD<String> lines = jsc.textFile(args[0]);
    JavaRDD<String> words = lines.flatMap(e -> Arrays.asList(
                                          pattern.split(e)).iterator());
    JavaPairRDD<String, Integer> wordsIntial = words.mapToPair(
      e -> new Tuple2<String, Integer>(e, 1));
  }
}
```

Sometimes, you may want to convert your Java RDDs to Scala RDDs or vice versa. Most commonly, this is for libraries that require or return Scala RDDs. Still, sometimes core Spark functionality may not yet be available in the Java API, and converting your RDD to a Scala RDD is an easy way to access the new functionality.

If you have a Java RDD you want to pass to a Scala library expecting a regular Spark RDD, you can access the underlying Scala RDD with `rdd()`. Most often, this is sufficient to pass the resulting RDD to whichever Scala library you need to call. (Some notable exceptions are Scala libraries that depend on implicit conversions of the contents of the RDD or class tag information.) In this case, writing a small wrapper in Scala can be the simplest way to access the implicit conversions. If a Scala shim is out of the question, explicitly call the corresponding function on the JavaConverters object and construct a fake class tag.

To construct a fake class tag, you can use `scala.reflect.ClassTag$.MODULE$.AnyRef()` or get the actual class tag with `scala.reflect.ClassTag$.MODULE$.apply(CLASS)` as illustrated in Examples 9-2 and 9-3.

Moving from a Scala RDD to a Java RDD often requires more class tag information than most Spark libraries. This is because, while the different JavaRDDs expose public constructors that take Scala RDDs as arguments, these are intended to be called from within Scala and, therefore, expect class tag information.

If you are in a mixed-language project or library, consider constructing the Java RDD in Scala, where the class tag information is more easily available.

Fake class tags are most commonly used in generic or templated code where you don't know the exact types at compile time. Using fake class tags often works, although some specialization may be lost in the Scala side; very occasionally, the Scala code depends on correct class tag information. In this case, you *must* use an accurate class tag. In most cases, using a real class tag is not substantially more effort and can offer performance advantages, so use them when possible.

Example 9-2. Java/Scala RDD interoperability with a fake class tag

```
  public static JavaPairRDD wrapPairRDDFakeCt(
    RDD<Tuple2<String, Object>> rdd) {
    // Construct the class tags by casting AnyRef - this would be more commonly done
    // with generic or templated code where we can't explicitly construct the cor
rect
    // class tag as using fake class tags may result in degraded performance.
    ClassTag<Object> fake = ClassTag$.MODULE$.AnyRef();
    return new JavaPairRDD(rdd, fake, fake);
  }
```

Example 9-3. Java/Scala RDD interoperability

```
  public static JavaPairRDD wrapPairRDD(
    RDD<Tuple2<String, Object>> rdd) {
    // Construct the class tags
    ClassTag<String> strCt = ClassTag$.MODULE$.apply(String.class);
    ClassTag<Long> longCt = ClassTag$.MODULE$.apply(scala.Long.class);
    return new JavaPairRDD(rdd, strCt, longCt);
  }
```

Both the Spark SQL and the ML pipeline APIs are mostly unified between Scala and Java. There are still Java-specific helper functions in which the equivalent Scala function is complex to call. Some examples of this are the various numeric functions, like `plus`, `minus`, etc., on `Column` as the overloaded `Scala` equivalents (`+`, `-`) cannot be easily accessed. Rather than having `JavaDataFrame` and a `JavaSQLContext`, the methods required for Java access are available on the regular `DataFrame` and `SQLContext`. This can be somewhat confusing, as some of the methods that will appear in the JavaDoc may not be usable from Java, but in those cases, similarly named functions will be provided to be called from Java. Java UDFs, and by extension most other non-Scala languages, require specifying the return type of your function, as it can't be inferred in the same way it is done in Scala. A simple Java UDF is shown in Example 9-4.

Example 9-4. Simple Java UDF

```
    sqlContext.udf()
      .register("strlen",
                (String s) -> s.length(), DataTypes.StringType);
```

While the types required by the Scala and Java APIs are different, for the most part, the Java collection types can be wrapped without requiring an extra copy. For iterators, the wrap conversion can be done lazily as the elements are accessed, allowing Spark to spill the data as needed (as discussed in "Iterator-to-Iterator Transformations with mapPartitions" on page 141). This is especially important since, for

many simple operations, the cost of copying the data can quickly dominate the actual computation required.

Custom Code Beyond Scala and Beyond the JVM

Going beyond the JVM greatly opens up the scope of different languages you can work in. However, in its current architecture, going outside of the JVM in Spark—especially on the workers—can involve a substantial performance cost of copying data on worker nodes between the JVM and the target language. For complex operations, the cost of copying the data is relatively low, but for simpler operations, the cost of copying the data can easily double the computation cost. The first non-JVM language to be directly supported inside of Spark is Python, and its API and interface have become a model that other non-JVM languages have based their implementations on.

How PySpark works

PySpark connects to JVM Spark using a mixture of pipes on the workers and Py4J, a specialized library for Python/Java interoperability, on the driver. This relatively simple architecture, shown in Figure 9-1, hides many complexities involved in making PySpark work. One of the bigger challenges is that even once the data has been copied from the Python worker to the JVM, it isn't in a form the JVM can easily parse. This requires special handling on both the Python worker and Java to ensure sufficient information for things like partitioning is available in the JVM.

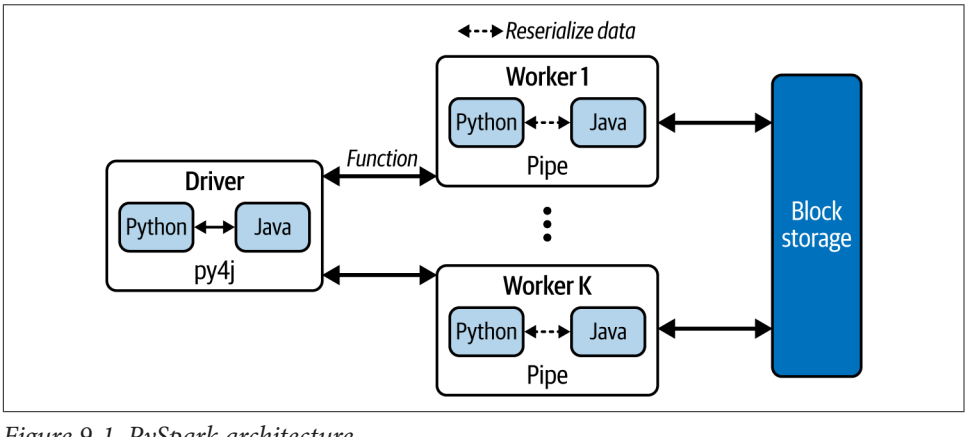


Figure 9-1. PySpark architecture

After the initial reading from persistent storage (like HDFS or Amazon S3) and between any shuffle, the data on the workers needs to be passed between the JVM and Python.

PySpark RDDs

Transferring the data to and from the JVM and starting the Python executor has significant overhead. Using the DataFrame/Dataset API avoids many of the performance challenges with the PySpark RDD API by keeping the data inside the JVM for as long as possible.

Copying the data from the JVM to Python is done using sockets and pickled or arrow-encoded objects. A more general version of this, for talking to programs in other languages, is available through the `PipedRDD` interface illustrated in "Using Pipe and Friends" on page 236.

Since piping the data back and forth for each transformation would be expensive, PySpark pipelines Python transformations inside of the Python interpreter when possible, so a `filter` and then a `map` will be chained together on the iterator of Python objects using a specialized `PipelinedRDD`. Even when the data has to be shuffled and PySpark is unable to chain our transformations inside of a single worker VM, the Python interpreter is capable of being reused so the interpreter startup overhead doesn't further slow us down.

This is only part of the puzzle. Normal `PipedRDDs` work on `Strings`, which can't easily be shuffled since there is no inherent key. The approach taken in PySpark, and mirrored in many other language bindings, is a special `PairwiseRDD` in which the key must be a `long` and is only deserialized with custom Scala code to parse the Python value. This deserialization is not overly expensive but does serve to illustrate that for the most part, Spark Scala treats the results of Python as opaque byte arrays.

DataFrames and Datasets generally use some similar variation of ArrowPythonRunner, which, as the name implies, uses Arrow to move the data back and forth.

Since there is some overhead associated with serialization and deserialization, PySpark uses a batch serializer, and this can occasionally result in unexpected effects (like when repartitioning PySpark will not split up things in the same batch). For all its simplicity, this approach to integrating works surprisingly well, with the majority of operations on Scala RDDs available in Python. Some of the more difficult places are interacting with libraries, such as MLlib, and loading and saving from different sources.

Interacting with different formats is another restriction, as much of Spark's load/save code is based on Hadoop's Java interfaces. This means that any data loaded is initially loaded into the JVM and then transferred to Python.

For interacting with MLlib, generally two approaches have been taken: either a specialized data type is used in PySpark with equivalent Scala decoders, or the algorithm is reimplemented in Python. These problems are avoided with Spark ML, which uses the DataFrame/Dataset interface (now considered the primary API for MLlib) that generally keeps the data stored in the JVM.

PySpark DataFrames and Datasets

DataFrames and Datasets avoid many of the performance downsides of the Python RDD API by keeping the data inside the JVM for as long as possible. The same benchmark we did to illustrate DataFrames' general improvement over RDDs (Figure 5-1) shows a greater difference when rerun in Python (Figure 9-2).

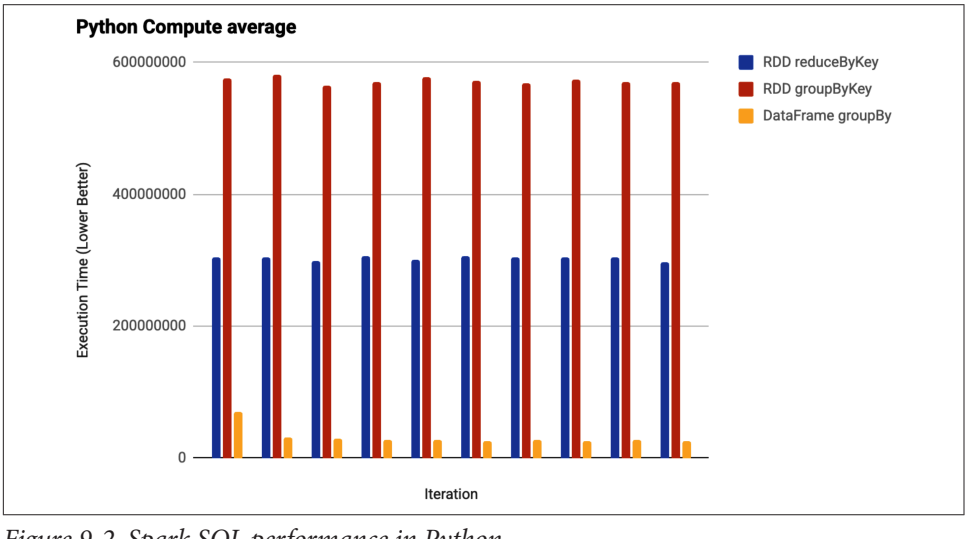


Figure 9-2. Spark SQL performance in Python

For many operations on DataFrames and Datasets, the data may never actually need to leave the JVM, although using Python UDFs, UDAFs, or lambdas naturally requires transferring some of the data to the JVM. This results in a simplified architecture diagram for many operations, which, instead of Figure 9-1, looks like Figure 9-3.

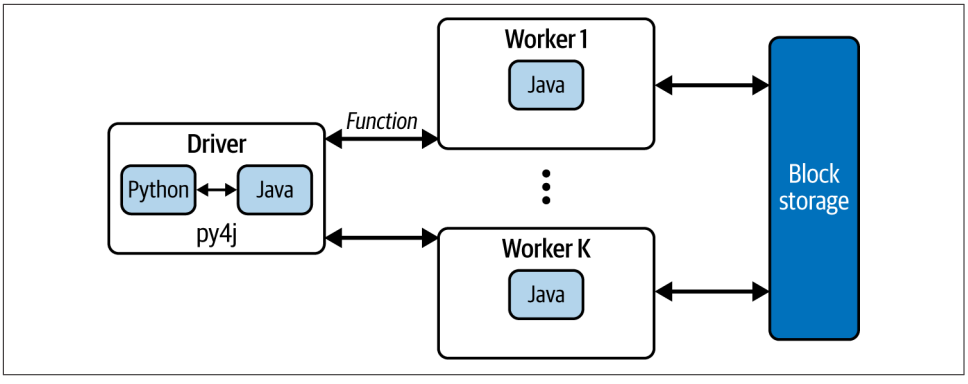


Figure 9-3. PySpark SQL architecture

PySpark doesn't use Jython as most Python users need access to libraries like NumPy, SciPy, and pandas, which do not work well in Jython.[5]

Pandas API on Spark

If you are more familiar with the pandas API or need to port existing pandas code to Spark, you can consider using Spark's implementation of the pandas APIs. Under the hood, pandas on Spark is implemented in Spark, so everything you've learned about Spark performance will generally apply to pandas on Spark.

Not all of panda's APIs are supported in the Spark version. Some of these are intentional, like not implementing iter functions, which would require serial processing, and others are missing because pandas is also under active development. Additionally your intuition around which functions are available on local collections and not on distributed collections can serve as a guide for what functions might be intentionally left out of pandas API on Spark.

User-defined functions in PySpark with and without Arrow

You can still use custom Python code with DataFrames by creating user-defined functions. Arrow is generally faster for serialization and deserialization in both Java and Python. Spark's Arrow accelerated UDFs also offer the ability to perform vectorized operations instead of row-by-row.

The classic "row-by-row" UDF is shown in Example 9-5.

5 Some early work is being investigated to see if Jython can be used to accelerate Python UDFs, which don't depend on C extensions, but this work has been shelved. See SPARK-15369 for details.

Example 9-5. Simple Python UDF

```
@udf("long")
def classic_add1(e: int) -> int:
    return e + 1
```

When possible, you should use the Arrow-accelerated UDFs, including the `pandas_udfs`, as the serialization and deserialization cost is much lower. To take the most advantage of Arrow-accelerated UDFs, you switch from working on individual rows/elements to working on batches, which can be pandas Series or DataFrames.

Most regular UDFs (applied as part of select, filter, etc.) take and return pandas Series (see Example 9-6).

Example 9-6. Arrow Python UDF

```
@pandas_udf("long")
def pandas_add1(s: pd.Series) -> pd.Series:
    # Vectorized operation on all of the elems in series at once
    return s + 1
```

On the other hand, if your UDF takes or returns nested structures, that column (or return value) will be a pandas DataFrame (see Example 9-7).

Example 9-7. Python UDF with nested structures

```
@pandas_udf("long")
def pandas_nested_add1(d: pd.DataFrame) -> pd.Series:
    # Takes a struct and returns the age elem + 1, if we wanted
    # to update (e.g. return struct) we could update d and return it instead.
    return d["age"] + 1
```

UDAFs (user-defined aggregate functions), which are those used with windows, `groupBy`, and so on, take series and return scalar values (think long/string). See Example 9-8.

Example 9-8. Python user-defined aggregate function

```
@pandas_udf("long")
def pandas_sum(s: pd.Series) -> int:
    return s.sum()
```

It's important to note that these batches that your UDF will be passed may be smaller than the entire partition. This means that even your batched UDF may be invoked multiple times per partition. If your UDF has some state that needs to be initialized (think DB connection, table, etc.), you can also work on "batches of batches" by taking iterators of the underlying type (see Example 9-9).

Example 9-9. Batches of batches Arrow Python UDF

```
@pandas_udf("long")
def pandas_batches_of_batches(t: Iterator[pd.Series]) -> Iterator[pd.Series]:
    my_db_connection = None  # Expensive setup logic goes here
    for s in t:
        # Do something with your setup logic
        if my_db_connection is None:
            # Vectorized operation on all of the elems in series at once
            yield s + 1
```

These UDFs allow you to use your custom Python code (and any libraries you need) in Spark.

It's also technically possible to call Python UDFs from Scala or even JDBC by launching PySpark and then having PySpark start your Scala's main function post registration. This is kind of hacky but potentially useful for accessing unique Python libraries. The next section will dig into how to more broadly call into Java and Scala from Python.

Accessing the backing Java objects and mixing Scala code

An important implication of the PySpark architecture is that many of Spark's Python classes simply exist as wrappers to translate your Python calls to the JVM.

If you work with Scala/Java developers and you wish to collaborate, preexisting wrappers won't exist to call your own code—but you can register Java/Scala UDFs and then use them from Python. You can do this through the `registerJavaFunction` utility on the `sqlContext`.

Sometimes these wrappers don't do everything you need, and since Python doesn't have strong protections around accessing private methods, you can jump directly into the JVM. The same techniques can be used to call your own JVM code and, with a bit of work, translate the results into Python objects.



While the Py4J API is accessible, these techniques depend on implementation details of PySpark, and these implementation details may change between releases. Additionally, this does not work when using Spark Connect.

Thinking back to "Large Query Plans and Iterative Algorithms" on page 105, we suggested that it was important to use the JVM version of DataFrames and RDDs to cut the query plan. This is a workaround for when a query plan becomes too large for the Spark SQL optimizer to process. By putting an RDD in the middle, the SQL optimizer can't see back past the point where the data is in an RDD. While you could accomplish the same thing using public Python APIs, you would lose much of the

advantage of DataFrames, as the entire data would need to be round-tripped through the Python workers. Instead, by using some of the internal APIs, you can cut the lineage from Python while keeping the data in the JVM (as shown in Example 9-10).

Example 9-10. Cut large DataFrame query plan with Python

```
def cutLineage(df):
    """
    Cut the lineage of a DataFrame - used for iterative algorithms

    .. Note: This uses internal members and may break between versions
    >>> df = rdd.toDF()
    >>> cutDf = cutLineage(df)
    >>> cutDf.count()
    3
    """
    jRDD = df._jdf.toJavaRDD()
    jSchema = df._jdf.schema()
    jRDD.cache()
    session = df.sparkSession
    javaSparkSession = session._jsparkSession
    newJavaDF = javaSparkSession.createDataFrame(jRDD, jSchema)
    newDF = DataFrame(newJavaDF, session)
    return newDF
```

In general, the convention for most Python objects is `_j[shortname]` to access the underlying Java version. So, for example, the `SparkContext` has `_jsc` to get at the underlying Java `SparkContext`. This is only available on the driver program, so if any PySpark objects are sent to the workers, you won't be able to access the underlying Java component, and large parts of the API will not work.

The Python APIs generally wrap Java versions of the API rather than directly wrapping the Scala versions. If you want to access a JVM Spark class that does not already have a Python wrapper, you can directly use the Py4J gateway on the driver. The `SparkContext` contains a reference to the gateway in `_gateway`. Arbitrary Java objects can be accessed with `sc._gateway.jvm.[fulljvmclassname]`.

Py4J depends heavily on reflection to determine which methods to call. This is normally not a problem but can become confusing with numeric types. Attempting to call a Scala function expecting a `Long` with an `Integer` will result in an error message about not being able to find the method, even though in Python the distinction normally would not matter.

The same technique works for your own Scala classes, provided they are on the classpath. You can add JARs to the classpath with `spark-submit` with `--jars` or by setting the `spark.driver.extraClassPath` configuration property. Example 9-11,

which we used to generate Figure 9-2, is intentionally structured to use the existing Scala code to generate the performance testing data.

Example 9-11. Calling non-Spark JVM classes with Py4J

```
    sc = sqlCtx._sc
    javaSparkSession = sqlCtx._jsparkSession
    jsc = sc._jsc
    scalasc = jsc.sc()
    gateway = sc._gateway
    # Call a java method that gives us back an RDD of JVM Rows (Int, Double)
    # While Python RDDs are wrapped Java RDDs (even of Rows) the contents are
    # different, so we can't directly wrap this.
    # This returns a Java RDD of Rows - normally it would better to
    # return a DataFrame directly, but for illustration we will work
    # with an RDD of Rows.
    java_rdd = gateway.jvm.com.highperformancespark.examples.tools.GenerateScaling
Data.generateMiniScaleRows(
        scalasc, rows, numCols
    )
    # Schemas are serialized to JSON and sent back and forth
    # Construct a Python Schema and turn it into a Java Schema
    schema = StructType(
        [StructField("zip", IntegerType()), StructField("fuzzyness", DoubleType())]
    )
    jschema = javaSparkSession.parseDataType(schema.json())
    # Convert the Java RDD to Java DataFrame
    java_dataframe = javaSparkSession.createDataFrame(java_rdd, jschema)
    # Wrap the Java DataFrame into a Python DataFrame
    python_dataframe = DataFrame(java_dataframe, sqlCtx)
    # Convert the Python DataFrame into an RDD
    pairRDD = python_dataframe.rdd.map(lambda row: (row[0], row[1]))
    return (python_dataframe, pairRDD)
```

Attempting to use the Py4J bridge inside of your transformations will fail at runtime.

While many of the Python classes are simply wrappers of Java objects, not all Java objects can directly be wrapped into Python objects and then used in Spark. For example, objects in PySpark RDDs are represented as pickled strings, which can only be easily parsed in Python. Thankfully, DataFrames are standardized between the languages, so provided you can convert your data into a DataFrame, you can then wrap it in Python and use it directly as a Python DataFrame or convert the Python DataFrame to a Python RDD.

Scala UDFs and UDAFs can be used from Python without having to go through the Py4J API.



With the increased popularity of machine learning and Spark, special logic has been added to more quickly get data into PyTorch; this will be covered more in the Chapter 10 on using Spark for machine learning.

PySpark dependency management

Often a large part of the reason one wants to use a language other than Scala is for the libraries that are available with that language. In addition to language-specific libraries, you may need to include libraries for Spark itself to use, especially when working with different data formats. There are a few different options for using both Spark-specific and language-specific libraries in PySpark.

Spark Packages is a system that allows us to easily include JVM dependencies with Spark. A common reason for wanting additional JVM libraries in PySpark is support for additional data formats.

If you are working in the Scala shell, you can use the `--packages` command-line argument to specify the Maven coordinates of a package you want in the shell. If you are building a Scala package, you can also add any requirements to your assembly *.jar*.

For Python, you can create a Java or Scala project with your JVM dependencies and add the *.jar(s)* with `--jars`. If you've launched Python and then imported PySpark, you can instead specify the `spark.jars.packages` configuration variable.

When using Spark Packages, the dependencies are automatically fetched from Maven and distributed to the cluster. If your JVM dependency is not available in Maven, you can use the same technique we discuss next for adding local Python dependencies or add it to your local Maven.

Adding local dependencies with PySpark can be done at both job submission time and dynamically using the SparkContext. Local dependencies can be *.jar* files for JVM requirements or *.py*, *.zip*, and *.egg* for Python dependencies, which are automatically added to the `PYTHONPATH`.[6]

Work is currently underway to allow Python Spark programs to specify required pip packages and have them auto-installed, but the proposal has not yet been accepted. See the pull request and Spark-5929 for the status of this proposal.

6 Some providers also support wheel files.



For a while many vendors used custom solutions built on Anaconda, but these have largely decreased in popularity for new deployments.

For those using Kubernetes, another option can be to create a Docker container with your desired Python libraries installed. This is a great improvement over the Hadoop ecosystem days, as it allows for additional customization, including native code that your Python packages may depend on.

If none of these work for your cluster configuration, there are a few remaining options, but these are less ideal. The simplest, but very hacky, approach is to have your transformations explicitly import the package and, on failure, perform a pip installation. Similar approaches can be done with broadcast variables or a setup map at the start of the program. Failing that, you can ask your cluster administrator to install the package systemwide.

Installing PySpark

First-party languages for Spark don't require any separate installation, but as mentioned for Python packages, Python has its own mechanisms for dealing with package management.

PySpark is now published on PyPI, so you can install it with a simple `pip install pyspark==3.5.0`. Alternatively, you can also `pip install` from the Python directory of a regular Spark distribution, which is preferable for folks who work with a customized Spark distribution. Once you have PySpark pip installed, you can then start your favorite Python interpreter and import PySpark like any other package or start the PySpark shell with `pyspark`.

It's important to note that pip installing Spark is optional. If you wish, you can run PySpark from a regular Spark setup without pip installation (although then you must use `spark-submit` or `pyspark` from the Spark bin directory).

How SparklyR Works

SparkR was initially added into core Apache Spark but has been subsumed by SparklyR from RStudio, and the integrated version is now deprecated. SparklyR, SparkR, and PySpark all have similar design decisions, and your understanding of PySpark should roughly transfer.

To give you an idea of what the SparkR interface looks like, the standard word count example has been rewritten in R in Example 9-12.

Example 9-12. SparkR word count

```
library(SparkR)

# Setup SparkContext & SQLContext
sc <- sparkR.init(appName="high-performance-spark-wordcount-example")

# Initialize SQLContext
sqlContext <- sparkRSQL.init(sc)

# Load some simple data

df <- read.text(fileName)

# Split the words
words <- selectExpr(df, "split(value, \" \") as words")

# Compute the count
explodedWords <- select(words, alias(explode(words$words), "words"))
wc <- agg(groupBy(explodedWords, "words"), "words" = "count")


# Attempting to push an array back fails
# resultingSchema <- structType(structField("words", "array<string>"))
# words <- dapply(df, function(line) {
#   y <- list()
#   y[[1]] <- strsplit(line[[1]], " ")
# }, resultingSchema)
# Also attempting even the identity transformation on a DF from read.text fails
# in Spark 2.0-preview (although works fine on other DFs).

# Display the result
showDF(wc)
```

To execute your own custom R code, you can use the `dapply` method on DataFrames as illustrated in Example 9-13. SparkR's custom code execution support has a long way to go, as illustrated by the difficulty of attempting to perform a word count with `dapply` in Example 9-12.

Example 9-13. SparkR arbitrary code with DataFrames

```
library(SparkR)

# Setup SparkContext & SQLContext
sc <- sparkR.init(appName="high-performance-spark-wordcount-example")

# Initialize SQLContext
sqlContext <- sparkRSQL.init(sc)


# Count the number of characters - note this fails on the text DF due to a bug.
```

```
df <- createDataFrame (sqlContext,
  list(list(1L, 1, "1"),
  list(2L, 2, "22"),
  list(3L, 3, "333")),
  c("a", "b", "c"))
resultingSchema <- structType(structField("length", "integer"))
result <- dapply(df, function(row) {
  y <- list()
  y <- cbind(y, nchar(row[[3]]))
}, resultingSchema)
showDF(result)
```

Internally `dapply` is implemented in a similar way to Python's UDF support. As with PySpark, arbitrary non-JVM code execution with SparkR is slower than traditional Scala Spark code.

Spark on the Common Language Runtime (CLR)—C# and Friends

Microsoft's .NET for Apache Spark provides C# bindings for working with Apache Spark. The general design is similar to that of PySpark, with the internals of `PythonRDD` instead communicating with the CLR. As with PySpark, RDD transformations involve copying the data from the JVM, and DataFrame transformations that don't use UDFs in C# don't require copying the data on the workers (or even launching the CLR). If you are curious about using CLR languages with Apache Spark, you can check out the design documents and examples.

Calling Other Languages from Spark

In addition to using other languages to call Spark, we can call other languages from Spark.

Using Pipe and Friends

If there aren't existing wrappers for the language you are working with, one of the simplest options is using Spark's pipe interface. To use the pipe interface, you start by converting your RDDs into a format in which they can be sent over a Unix pipe. Often simple formats like JSON or CSV are used for communicating, as lightweight libraries exist for generating and parsing these records in many languages.

Let's return to the Goldilocks example from "The Goldilocks Example" on page 171. Suppose that in addition to optimal panda porridge temperature, you also wanted to find out which pandas had been commenting on Spark PRs;[7] you might create/cook

7 This is somewhat of a stretch as far as the relationship to Goldilocks goes, but you know.

up a quick little Perl script, as in Example 9-14. Later on, if you want to use this script in Spark, you can use the `pipe` command to call your Perl script from the workers.

Example 9-14. Perl script to be called from pipe

```
#!/usr/bin/perl
use strict;
use warnings;

use Pithub;
use Data::Dumper;

# Find all of the commentors on an issue
my $user = $ENV{'user'};
my $repo = $ENV{'repo'};
my $p = Pithub->new(user => $user, repo => $repo);
while (my $id = <>) {
    chomp ($id);
    my $issue_comments = $p->issues->comments->list(issue_id => $id);
    print $id;
    while (my $comment = $issue_comments->next) {
        print " ".$comment->{"user"}->{"login"};
    }
    print "\n";
}
```

Since pipe only works with strings, you will need to format your inputs as a string and parse the result string back into the correct data type, as in Example 9-15. Then you can call your external program using the pipe interface as shown below:

Example 9-15. Using pipe (from Scala Spark) to talk to a Perl program on the workers

```
  def lookupUserPRS(
      sc: SparkContext,
      input: RDD[Int]
  ): RDD[(Int, List[String])] = {
    // Copy our script to the worker nodes with sc.addFile
    // Add file requires absolute paths
    val distScriptName = "ghinfo.pl"
    val userDir = System.getProperty("user.dir")
    val localScript = s"${userDir}/src/main/perl/${distScriptName}"
    val addedFile = sc.addFile(localScript)

    // Pass enviroment variables to our worker
    val enviromentVars = Map("user" -> "apache", "repo" -> "spark")
    val result = input
      .map(x => x.toString)
      .pipe(SparkFiles.get(distScriptName), enviromentVars)
    // Parse the results
    result.map { record =>
```

```
    val elems: Array[String] = record.split(" ")
    (elems(0).toInt, elems.slice(1, elems.size).sorted.distinct.toList)
  }
}
```



Spark will not automatically copy your script to the worker machines, so if you are calling a custom program, you can use the `sc.addFile` interface as in Example 9-15. Otherwise (e.g., if you are calling a systemwide program), just skip that part.

PySpark and SparkR both use specialized versions of the piped RDDs for communication with the workers.

It is essential that you write your code to handle empty partitions since your program will be called even for empty partitions.

Java Native Interface

The JNI is another option for interfacing with other languages. JNI can work well for calling certain C/C+\+ libraries, as well as other statically compiled languages like Fortran. While JNI doesn't exactly suffer from double serialization in the same way calling PySpark or using `pipe` does, you still need to copy your data out of the JVM and back.

This is why some libraries, such as JBLAS, implement some components inside of the JVM, since once the copy cost is added, the performance benefit of native code can go away.

To illustrate how to use JNI with Spark, consider calling a very simple C function that sums all of the nonzero inputs. Its function signature is shown in Example 9-16.

Example 9-16. Simple C header

```
#ifndef _SUM_H
#define _SUM_H

int sum(int input[], int num_elem);

#endif /* _SUM_H */
```

You can write the JNI specification to call this in either Java (Example 9-17) or Scala (Example 9-18). Although the tooling for Java can be a bit simpler, there is no significant difference between them.

Example 9-17. Simple Java JNI

```
class SumJNIJava {
  public static native Integer sum(Integer[] array);
}
```

Example 9-18. Simple Scala JNI

```
class SumJNI {
  @native def sum(n: Array[Int]): Int
}
```

Manually writing wrappers takes effort. Check out SWIG to automatically generate parts of your bindings.

Once you have your C function and your JNI class specification, you need to generate your class files and, from them, generate the binder heading (see Example 9-19). The `javah` command will take the class files and generate headers that are then used to create a C-side wrapper.

Example 9-19. Generate header with the command-line interface

```
javah -classpath ./target/examples-0.0.1.jar \
com.highperformancespark.examples.ffi.SumJNI
```

For those of you building with SBT, Jakob Odersky's `sbt-jni` package makes it easy to integrate your native code with your Scala project. `sbt-jni` is published as an SBT plugin like `spark-packages-sbt` and is included by adding an entry to *project/plugins.sbt* as shown in Example 9-20.

Example 9-20. Add `sbt-jni` plugin to project/plugins.sbt

```
addSbtPlugin("com.github.sbt" %% "sbt-jni" % "1.7.1")
```

`sbt-jni` simplifies generating the header file by adding the `javah` target to sbt, which will generate the header files and place them in *./target/native/include/*.

Once we have our header file, we need to write a wrapper in C. The generated header file shouldn't be modified but rather imported into our shim as shown in Example 9-21.

Example 9-21. JNI C shim

```
#include "sum.h"
#include "include/com_highperformancespark_examples_ffi_SumJNI.h"
#include <ctype.h>
#include <jni.h>
```

```
/*
 * Class:     com_highperformancespark_examples_ffi_SumJNI
 * Method:    sum
 * Signature: ([I)I
 */
JNIEXPORT jint JNICALL Java_com_highperformancespark_examples_ffi_SumJNI_sum
(JNIEnv *env, jobject obj, jintArray ja) {
  jsize size = (*env)->GetArrayLength(env, ja);
  jint *a = (*env)->GetIntArrayElements(env, ja, 0);
  return sum(a, size);
}
```

`sbt-jni` also simplifies building and packaging native code, adding `nativeCompile`, `javah`, and `packageBin` to allow you to easily build an assembly JAR with both your native files and Java artifacts. For `sbit-jni` to build your native code (in addition to the JVM code) as well, you need to provide a Makefile. If you are starting with a new project, the `nativeInit` CMake target will generate a skeleton *CMakeLists.txt* file you can use as a basis for your native build.

In our example project, we've built the native code along with the Scala code. Alternatively, especially if you plan to support multiple architectures, you may wish to create a separate package for your native code.

If your artifact is built with `sbt-jni`, you can use the `nativeLoader` decorator from `ch.jodersky.jni.nativeLoader` to automatically load your native code as needed. In the example we've been working on, our library is called `libhigh-performance-spark0`, so we can have it automatically loaded by adding the decorator to our SumJNI class, as in Example 9-22.

Example 9-22. Native loader decorator

```
@nativeLoader("high-performance-spark0")
```

If you are working in Java or just want more control, you can use `System.loadLibrary`, which takes a library name and searches `java.library.path` or `System.load` with an absolute path.



Leave off the "lib" prefix, which `loadLibrary` (and `sbt-jni`) automatically appends, or you will get confusing runtime linking errors.



The Oracle JNI specification can be a useful reference.

If your native library likely isn't packaged in your JAR, you need to make sure the JVM running the Spark worker is able to call it. If your library is already installed on the workers, you can add `-Djava.library.path=...` to your `spark.executor.extraJavaOptions`.

The JVM is expanding its options for accessing non-JVM code beyond the traditional JNI interface. These new options can be faster but require modern versions of the JVM (like 22 and higher), and while many of the APIs have been finalized, a few remain subject to change. If you need to access non-JVM code, it may be worth looking into what the latest options are.

Java Native Access (JNA)

JNA is a community-driven alternative to JNI to allow calling of native code, ideally without all of the boilerplate required by JNI. Although JNA is a community package, this does not mean it is low quality; it is used by a variety of mature projects and has been used by Spark application developers. We can use JNA to call our previous example in both Scala (Example 9-23) and Java.

Example 9-23. Scala simple JNA

```
import com.sun.jna._
object SumJNA {
  Native.register("high-performance-spark0")
  @native def sum(n: Array[Int], size: Int): Int
}
```

It's important to note that these JNA examples skip the requirement for writing the JNI wrapper (as in Example 9-21) and instead directly call the C function for us. While SWIG can do a good job of generating much of the JNI wrappers, for some this is a compelling reason to use JNA over JNI.

When using JNA, `jna.boot.library.path` allows you to add libraries to the search path *before* the system library path.

Project Panama

Comparatively new in the JDK, Project Panama offers automatic wrapper generation with jextract, as well as columnar type interfaces and shared memory, which is ideal for large datasets. Project Panama is still incubating, and its APIs may change throughout the JDK Enhancement Process, so while it can bring substantial benefits, it's more likely to require code changes than using "old-fashioned" JNI.

Underneath Everything Is Fortran

A surprising number of numeric computing libraries still have Fortran implementations. Thankfully many of these libraries already have Java or Python wrappers, which greatly simplify our access. These libraries often can make intelligent decisions about what operations are worth the overhead of copying our data into Fortran and what operations make more sense to be implemented in the host language. Not all Fortran code already has wrappers, and you may find yourself in a place with which you want to interface.

The general process is to first create a C/C++ wrapper that exposes the Fortran code for Java to call and then link the C/C++ code together with the Fortran code. Continuing the sum example in Fortran (Example 9-24), you would create a C wrapper like Example 9-25 and then follow the existing steps for calling a C library in "Java Native Interface" on page 238.

Example 9-24. Fortran sum function

```
INTEGER FUNCTION SUMF(N,A) BIND(C, NAME='sumf')
INTEGER A(N)
SUMF=SUM(A)
END
```

Example 9-25. C wrapper for Fortran sum function

```
// Fortran routine
extern int sumf(int *, int[]);

// Call the fortran code which expects by reference size
int wrap_sum(int input[], int size) {
  return sumf(&size, input);
}
```



If you like `sbt-jni`, you can extend the generated `CMake` file to also compile your Fortran code.

These wrappers can also be automatically generated with programs like FortWrap, or skipped entirely with JNA. Calling the Fortran function with JNA is very similar to calling the C function, as shown in Example 9-26.

Example 9-26. Fortran SUMF through JNA

```
import com.sun.jna._
import com.sun.jna.ptr._
object SumFJNA {
  Native.register("high-performance-spark0")
  @native def sumf(n: IntByReference, a: Array[Int]): Int
  def easySum(size: Int, a: Array[Int]): Int = {
    val ns = new IntByReference(size)
    sumf(ns, a)
  }
}
```

Calling Fortran code from the JVM is more difficult than calling C code. If available, it's often better to use existing wrappers, as they can make intelligent decisions about which components to execute in Fortran rather than in the JVM.

Getting to the GPU

GPUs are another great way of working with parallel, numeric computing problems. They are particularly effective at certain machine learning problems. One of the best options for getting to the GPU comes from using NVIDIA's Spark RAPIDS accelerator, which has the ability to leave your data on the GPU for calling other libraries.

Alternatively, when working with RDDs, you can have Spark assign GPUs through resource profiles and then use your GPU-compatible libraries. Some people, including people at DeepMind, have also used Aparapi to automate the compilation of Java code to OpenCL.

When working with DataFrames, or if you're working with GPUs consistently throughout your job, you can configure Spark to assign GPU resources with `spark.task.resource.gpu.amount` with any decimal value, for example, `0.25` meaning each task uses a quarter of a GPU.

Going Beyond the JVM with Spark Accelerators

As Spark has grown in popularity, both open source and closed source accelerators have been developed to allow for improving the running speed of your existing Spark code without rewriting it. Parts of these accelerators often require sticking within a limited range of supported operations, mostly Datasets and SQL without user-defined functions. The most common accelerator is Databricks Photon, followed by Spark

RAPIDS for (GPU acceleration) and Gluten, and Blaze for CPU-based acceleration. There is also a relatively new project, Apache Arrow DataFusion Comet, which has some important and unique benefits. Each may require a bit more nuanced attention to the specifics of cluster configurations than core Spark does.



Accelerators often involve some slight constant increase in overhead and are therefore not well suited to frequent small queries.

At their core, all of the accelerators work by replacing how Spark's executors process data using code outside of the JVM. The accelerators generally either use Apache Arrow or a similar format, suited for vectorized operations that apply the same logic to many elements at once instead of one at a time. Each accelerator works by inserting an optimizer step to Spark SQL, which replaces those components of the SQL plan the optimizer can handle with a node to trigger the executor side accelerator (commonly through JNI). Each of the three leading accelerators uses a custom shuffle implementation and data readers/writers, allowing them to avoid copying data into and out of the JVM when executing queries entirely supported by the optimizer. For queries where some parts do not have a native implementation, they also have code to copy data to/from the JVM with some additional overhead involved.



As with all of the techniques in this chapter, there is a penalty to be paid whenever you have to move data into or out of the JVM. These new optimizers, mostly, transparently fall back to the JVM when an operation (or data source/sink) is not implemented inside of the optimizer—but this also means that changing data source formats, or even just using a new not yet fully supported function, can result in huge performance degradation.

You can get a better understanding of both how much work the optimizer is able to handle and where the data copies are occurring by looking at the Spark query plan. Each optimizer has internal alternatives for the components of the Spark plan they handle, as well as names for the inserted stages that copy data to/from the JVM.

Databricks Photon and Apache Arrow DataFusion Comet are currently the only two accelerators that do not require static allocation between the JVM and accelerator. This is especially important since many Spark users have to spend a substantial amount of time with out-of-memory exceptions.



At present, accelerators largely either ignore UDFs or require manual translation, with NVIDIA's RAPIDS being the one exception. There is a new proposal to make UDFs more accessible via transpilation, and while it has been accepted in principle and there is a proof-of-concept, it is not yet complete.

Databricks Photon

While Databricks Photon is the most popular of Spark accelerators, it is also closed source, so it is more difficult to ascertain how it currently functions, but there is a paper on how Databricks implemented it.[8] In general, Photon works by converting parts of your Spark SQL (or Dataset/DataFrame) operation into native code with vectorized operations.



As closed source, if Databricks makes changes, we may not know, so be careful about depending on any of the details discussed here.

Every time you see a transition node in your Spark SQL plan with Photon enabled, it means that the data is having to be copied to/from the JVM. While the optimizer does a good job organizing transformations to minimize these, it can be worth investigating if you can manually move any of them together to reduce the number of copies (for example, automatic reordering between join conditions is limited automatically but can often be done safely by humans).

Databricks Photon has some important memory management features that help it stand apart from the other optimizers. Photon does not require statically allocated off-heap memory; instead, it shares memory with Spark's existing memory manager. This greatly reduces the change of container out-of-memory exceptions. In addition, Photon is integrated with Spark's existing "spill-to-disk" infrastructure, making processing partitions too large to fit in memory possible (provided that each columnar batch can fit in memory). Given how difficult memory problems can be to debug in Spark, as well as their frequency, these benefits are difficult to overstate.

An important drawback of Photon over the other accelerators is that it is impossible to add your own functions.[9] This means if your work is largely executed inside of complex non-built-in operations, Photon is unlikely to be able to offer much performance improvement.

8 Alexander Behm et al., "Photon: A Fast Query Engine for Lakehouse Systems," ACM SIGMOD Conference, June, 2022.

9 Unless you get a job at Databricks.

As a commercial offering, it's difficult to determine the project's "health," but it appears to continue to be a core element of what Databricks offers, and being created by many of the same people developing Spark means it is likely to continue to be updated to new versions of Spark.

Apache Arrow DataFusion Comet

Apache Arrow DataFusion Comet (which we'll call Comet from here on out) was initially contributed to the Apache Arrow project by Apple in early 2024. At the time of the writing, the early contribution lacked a number of important features and could only accelerate a few operations—but it had some of the most promising roadmaps of any of the open source projects. Comet's unified memory management with Apache Spark makes it, while perhaps not the choice of today, a likely top contender in the future for open source Spark acceleration.

As of this writing, Comet has not yet had a release, so to try out Comet, you'll need to build it from source. In practice, since most of the accelerators are not statically linked on modern systems, you'd have to do this regardless (see Example 9-27).

Example 9-27. Building Comet

```
# If we don't have fusion checked out do it
if [ ! -d arrow-datafusion-comet ]; then
  git clone https://github.com/apache/arrow-datafusion-comet.git
fi

# Build JAR if not present
if [ -z "$(ls arrow-datafusion-comet/spark/target/comet-spark-spark*.jar)" ]; then
  cd arrow-datafusion-comet
  make clean release PROFILES="-Pspark-${SPARK_MAJOR} -Pscala-2.13"
  cd ..
fi
COMET_JAR="$(pwd)/$(ls arrow-datafusion-comet/spark/target/comet-spark-spark*SNAP-
SHOT.jar)"
export COMET_JAR
```

Once you've got the JARs, you can then enable Comet with Spark with the command-line options in Example 9-28.

Example 9-28. Command-line flags to run with Comet

```
--jars ${COMET_JAR} \
--driver-class-path ${COMET_JAR} \
--conf spark.comet.enabled=true \
--conf spark.comet.exec.enabled=true \
--conf spark.comet.exec.all.enabled=true \
--conf spark.shuffle.manager=org.apache.spark.sql.comet.execution.shuffle.CometShuf-
```

```
fleManager \
--conf spark.comet.exec.shuffle.enabled=true \
--conf spark.comet.columnar.shuffle.enabled=true \
--conf spark.sql.extensions=org.apache.comet.CometSparkSessionExtensions
```

The previous example enables all of Comet's functionality, but you can use Comet without the shuffle manager. Doing this means that the data will have to flow back and forth between the JVMs more frequently, but it can be useful for debugging or when you are working in an environment with its own shuffle manager.



If you get a class loader error with Comet shuffle manager, double-check you've specified `-driver-class-path` as well as `-jars`, or you can copy the Comet JAR into your Spark distribution's regular classpath.

Comet does not yet have support for custom user-defined functions, although it is part of the roadmap.

As of the writing of this chapter (2026), Comet + Spark is in its early stages; its unique approach to memory management is very promising. Since it is built on top of the DataFusion library and based on an existing in-production accelerator at Apple, it will hopefully develop quickly.

Project Gluten

Project Gluten uses Apache Arrow for data representation and supports multiple native backends of execution. The three open source backends it supports out of the box are Arrow Compute, Facebook's Velox, and Kyligence's ClickHouse. In our experience, as of the writing of this book, Project Gluten feels rougher around the edges than the other optimizers, with more fiddling required for install. That being said, it also can be run on commodity hardware without any special license and supports a wide array of operations.

Don't assume just because the Gluten website or repo shows a backend as available it will actually work; some of the diagrams are more references to future work than implemented today.



It is very important to make sure the native build target you choose matches your setup, as errors (or even corruption) may occur with native library mismatches.

Getting started

There are prebuilt JARs for Gluten, but they are only built against select versions of Ubuntu and Spark. You can check out the releases page to see if there is a prebuilt JAR for your configuration; otherwise, you'll need to build either the Velox or ClickHouse-enabled JAR you want.[10] Since our examples in this book run on more than just Ubuntu 18.04, we build Gluten + Velox as part of the book's CI testing, and our build is shown in Example 9-29.

Example 9-29. Gluten + Velox build

```
sudo apt-get install -y locales wget tar tzdata git ccache cmake ninja-build build-
essential \
     llvm-dev clang libiberty-dev libdwarf-dev libre2-dev libz-dev libssl-dev
libboost-all-dev \
     libcurl4-openssl-dev maven rapidjson-dev libdouble-conversion-dev libgflags-
dev \
     libsodium-dev libsnappy-dev nasm
sudo apt install -y libunwind-dev
sudo apt-get install -y libgoogle-glog-dev
sudo apt-get -y install docker-compose
sudo apt-get install -y libre2-9 || sudo apt-get install -y libre2-10
  if [ ! -d incubator-gluten ]; then
    git clone https://github.com/apache/incubator-gluten.git
  fi
  cd incubator-gluten
  sudo ./dev/builddeps-veloxbe.sh --enable_s3=ON
  mvn clean package -Pbackends-velox -Pspark-3.4 -DskipTests
  GLUTEN_JAR_PATH="$(pwd)/package/target/gluten-package-*-SNAPSHOT-${SPARK_MAJOR_VER
SION}.jar"
```



As of the writing, building Gluten is a flaky process of trial and error outside of the presupported base systems.

Here we will focus on Velox, but the general principles are the same between Velox and ClickHouse.

10 In our experience, the build instructions are generally accurate, but may be missing various libraries. If you see a CMake library error during the build you likely need to install that package on your build machine (searching for the error message tends to tell you the package names on different Linux systems).

Velox + Gluten

Enabling Gluten involves making the JAR available to all of the executors and drivers and some minor configuration. A sample set of Spark configs enabling Gluten is shown in Example 9-30.

Example 9-30. Enable Gluten acceleration

```
spark.plugins=io.glutenproject.GlutenPlugin
spark.memory.offHeap.enabled=true
spark.shuffle.manager=org.apache.spark.shuffle.sort.ColumnarShuffleManager
# This static allocation is one of the hardest part of using Gluten
spark.memory.offHeap.size=20g
```

The trickiest part is figuring out what the correct value is for `spark.memory.offHeap.size` since if you overprovision the value you're "wasting" memory, and if you underprovision, you may get a container OOM error.

You can also, manually, make your user-defined functions in C++ (and from there call other ABI-compatible languages), although this process is much more involved than writing "regular" Spark UDFs. The exact details of building Gluten + Velox UDFs are beyond the scope of this book, but if you're familiar with C++, you can see the documentation on creating UDFs as part of the Gluten project.



You must ensure that any native UDFs you build are suitable for deployment on your cluster. If you have a different local system than your deployment cluster, like an ARM Mac and x86 Ubuntu cluster, you'll need to cross-compile (or remotely build) your library. Cross-compiling native libraries is beyond the scope of this book, but thankfully there are a great many resources, including Docker's faster cross-platform builds, on how to set this up.

ClickHouse + Gluten

ClickHouse is an alternative backend for Gluten, however they do not currently distribute prebuilt ClickHouse + Gluten JARs, which may indicate a reduced amount of testing. ClickHouse and Gluten also do not, as present, have a built-in story for handling user-defined functions, so any UDFs will result in a copy to/from the JVM.

Gluten's future

Gluten seems to be rather actively developed but lags substantially behind upstream. Even several months after Spark 3.5's Scala 2.13 release, the latest version supported was Spark 3.3 with Scala 2.12 on Ubuntu 18.04, which was already past the standard support end of life. Getting fixes, even small ones, upstreamed into Gluten appears challenging for a mixture of reasons, from an incomplete local development experi-

ence to challenges with reviewer engagement. While these challenges are not unique to Gluten, many projects lag behind the latest and can be difficult to get fixes into; it's important to keep this in mind when considering a new dependency.

Gluten recently graduated from the ASF incubator, which is a good sign for future openness to collaboration.

Gluten also has some important performance improvements on its roadmap. All optimizers perform best when they can handle the initial reading and writing stages, as this avoids large data copies. At present, Gluten delegates this to Spark for many of the most popular metastores, including Iceberg and Delta Lake. While Gluten could be used with GPUs, we do not see any current open source implementations. The next accelerator we'll dive into uses GPUs for its speed-up.[11]

Spark RAPIDS

While we cover GPU usage in Spark in more detail in Chapter 10, this section focuses on Spark using RAPIDS to accelerate analytic workloads, which stands out from the other accelerators because it achieves its speedups by relying on specialized hardware (GPUs). Beyond that requirement, the underlying acceleration pattern is familiar: selected workloads are executed outside the JVM, with native file and memory representations, and an emphasis on columnar processing rather than row-oriented execution.

Like Gluten and Photon, Spark RAPIDS also includes native implementations of common file and table formats. And, just like the others, it can gracefully fall back to Spark's JVM-based implementations when it encounters an unsupported format for reading or writing.

Setting up and configuring Spark RAPIDS

Spark RAPIDS is released as a JAR with broad support for different Spark versions,[12] which you can download. From there, launching Spark w/RAPIDS is incredibly simple, and you can, at its simplest, launch with `--jars ./accelerators/rapids-4-spark_2.12-24.02.0.jar --conf spark.plugins=com.nvidia.spark.SQLPlugin`.

While you can use our general guidance with optimizers, most Spark built-ins can benefit from acceleration; the RAPIDS library also gives you a handy way to check by setting the `spark.rapids.sql.mode` property to `explainOnly`. In explain mode, the driver logs will log which parts of your query plan can be executed by RAPIDS.

11 Given many of their similarities, we'd love it if Spark RAPIDS and Project Gluten combined, but we don't see that on either of their roadmaps.

12 Doing this inside of a single JAR is extremely difficult, so thank you to the NVIDIA team.



If you end up using RAPIDS extensively, you can also include it on your global Hadoop classpath or Docker base containers for faster worker startup time.

Once you've verified you can benefit from RAPIDS, you need to ensure there are GPU resources available on your Spark cluster. Each resource manager and cloud is a little different, and the cloud APIs are likely to change, so take a look at the correct guide for your cluster:[13] Yarn GPUs, NVIDIA's GPU discovery on Kubernetes, or NVIDIA's cloud guide. In addition to having the GPU resources present and labeled, remember Spark can request executors of different sizes and types so not every node needs to have GPUs.



While RAPIDS needs to be loaded at start, you can enable and disable it by setting `spark.rapids.sql.mode` to true or false. You can combine this with a resource profile to selectively use GPU resources for relevant parts of your workloads.

RAPIDS has some extra configuration parameters you may wish to fine-tune, and they publish their own fine-tuning guide.

13 Or have your cluster administrator take a look at.

UDFs

Spark RAPIDS is unique in its approach to UDFs; instead of not supporting them or requiring custom code, it has built-in transpiling for select UDFs. It supports select pandas UDFs and even some Scala and Java code. In practice, the Scala/Java UDF compilation is largely limited to operations that can be expressed without a UDF anyway, but it can make the code easier to read for those of us who like lambdas.



The UDF transpiling is experimental and may produce slightly different results than nontranspiled UDFs. Always validate accelerated jobs.

Going beyond RAPIDS on the GPU

One of the understated benefits of using Spark RAPIDS is that your data is already on the GPU for machine learning. Chapter 10 will look more at how to use Spark for machine learning, and Spark RAPIDS is one of the ways you can ensure your data is both in the correct format and location (GPU).

Project health

Overall, Spark RAPIDS appears to be an important offering from NVIDIA, and with the core accelerators (CUDA kernels) shared between many projects, it's likely to continue to be updated. That being said, given that the majority of the development comes from NVIDIA, we are unlikely to see features like ROCm support added.

In Chapter 10, we discuss machine learning capabilities, where using RAPIDS and everything you learned here become key for success.

Application-Specific Integrated Circuits (ASICs)

While they are not production-ready at the time of this writing, new ASICs have the potential for greatly accelerating Spark Dataset/SQL workflows. One of the primary vendors, Speedata, claims a "100x faster and 90% cost saving," and while you will likely not experience the idealized results (unless you're all pure SQL), they are sufficiently fast that it may be worth investigating, especially if you control your hardware deployments.

Managing Memory Outside of the JVM

The more languages involved, the more chance there is for something to go wrong and accidentally exceed memory limits, resulting in an out-of-memory exception (or OOM kill with exit code 137). To an extent, Python memory is managed with `spark.executor.memoryOverhead` and `spark.executor.pyspark.memory`, although since Python often depends on native libraries that may do their own memory allocations, this is not complete. You'll need to carefully manage the amount of "overhead" assigned to non-JVM tasks in all cases except Arrow DataFusion Comet and Photon.

The Future (from ~2026)

Apache Arrow is supported by a growing number of languages, and it has the potential to serve as a semistandardized bridge between languages. In the future, we hope that the accelerators and Spark are able to standardize on shared formats as well as interfaces. Some variant of Apache Arrow has the potential to be this format. While standardizing here would benefit the users, it would also reduce the "moat" for each of the accelerators, so it may not happen for business reasons.

Conclusion

Writing high performance Spark code need not be limited to Scala, let alone the JVM (although it can certainly make things easier). Spark has a wide variety of language bindings, both built-in and third-party, and can interface with even more languages using JNI, JNA, pipes, or sockets. For some operations, the cost of copying the data outside of the JVM and back can be more expensive than just doing the operation in the JVM—even with specialized libraries—so it is important to consider the complexity of your transformations before going outside of the JVM.[14] In the next chapter you'll see even more of how Spark can work beyond the JVM with AI libraries.

14 One relatively frequent issue that arises with processes moving in and out of the JVM is that different languages can rely on different floating-point representation precisions or default types, which can lead to slightly different result sets in some situations (which would be flagged in regression testing or similar validation exercises). This can be difficult (with the difficulty increasing alongside code complexity) to identify in context, so we wished to highlight this to hopefully save some readers precious time debugging. This also occurs with some database sources as data enters the Spark ecosystem.

CHAPTER 10

Spark: A Thoughtful Step into Generative AI

If you've built data pipelines in Spark, you already know the hard part of GenAI: *getting the right data to the right place, repeatedly, under real constraints*.

GenAI didn't replace pipelines—it made them more valuable. Training and inference are only as good as the workflows around them: dataset preparation, filtering, governance, evaluation, and scalable execution. That is Spark's home turf. Over time, Spark has grown into something more. It's steadily carving out a role in the world of generative AI, not through flashy promises but through quiet, deliberate evolution.

Imagine training a deep generative model using Spark. Not on specialized, expensive systems but on a Spark cluster that grows with your data. Spark allows this with careful scalability and thoughtful integration, turning ambitious AI projects into manageable realities.

We've seen Spark unlock possibilities like:

- Generating product descriptions for ecommerce, tailored to customer needs
- Creating visuals based on user prompts, streamlining creative workflows
- Summarizing complex legal documents into digestible, actionable insights

But let's not gloss over the challenges. GenAI thrives on scale, and that scale brings complexity. You might find yourself working with billions of rows of text, images, or structured data. Without the right tools, this kind of workload can quickly spiral out of control.

This is where Spark comes in. Companies like Meta, Netflix, Anthropic, Apple, and OpenAI are already leveraging Spark for their machine learning platforms (MLPs). Spark's distributed nature makes it a strong fit for these demanding AI workflows, provided you approach it with a clear plan.

Now, you might be cautious about thinking of Spark as a generative AI tool. We were, too. At first glance, Spark doesn't scream "AI innovator." But its distributed processing capabilities make it uniquely suited for building pipelines that integrate seamlessly with tools like PyTorch and TensorFlow.

In this chapter, you will be able to:

- Decide when Spark is the right backbone for a GenAI workload (and when it's not).
- Build an end-to-end pipeline that covers *preprocessing → training/fine-tuning → inference*, using Spark-native distributed execution.
- Choose between three practical architectures: *external API*, *distributed training*, and *batch inference at scale*, with trade-offs you can defend.

This isn't about overstating Spark's capabilities. It's about showing you where Spark fits and where it doesn't in the evolving AI landscape. The goal is to give you the tools and knowledge to make informed decisions, whether you're scaling your data, fine-tuning a model, or setting up AI-driven pipelines.

Generative AI is complex, and integrating it with Spark requires careful planning. But with the right approach, Spark can help you take measured, effective steps into this exciting space.

This chapter assumes you're not trying to turn Spark into "a GPU supercomputer." You're using Spark for what it's best at: *distributed data processing and orchestration* and connecting it cleanly to deep learning frameworks when training or inference needs to scale.

By the end, you'll be able to design Spark-based GenAI pipelines for training and inference, choose the right distributed approach (TorchDistributor, TensorFlowDistributor, or API-based), and avoid common production traps.

So, What Is This Generative AI Thing?

Imagine you're in a workshop surrounded by materials: blocks of wood, bolts, wires, and tools of every kind. You can build almost anything, but every creation is a manual process—each bolt and board is meticulously assembled by hand. Now, picture that workshop with a twist: instead of merely assembling pieces, your tools come to life, sculpting new parts and structures entirely on their own, based on examples they've seen before. This transformation in your workshop mirrors the leap from traditional AI to *generative AI*.

Generative AI isn't just about following static instructions or identifying patterns in data; it's about creating something new. Where traditional AI analyzes existing information to draw conclusions or automate repetitive tasks, generative AI learns

to generate content from scratch, producing text, images, music, or even synthetic data that didn't exist before. In a way, it's like having a creative partner in your digital workshop, capable of generating fresh content by understanding the examples you give it.

How Generative AI Works: Training the Creator

Imagine teaching an apprentice in your workshop. First, you show them countless examples of past creations—say, landscapes if you're working with images or paragraphs of writing if you're working with text. Over time, this apprentice doesn't just memorize each example; they start recognizing patterns and structures. Generative AI models are like this apprentice, learning from large datasets to capture the essence of what they've seen.

Once trained, these models enter the generation phase. Now, if you ask the model to produce a new landscape or draft a paragraph, it can produce something unique that still feels like it belongs with the original examples. Each model type has its own "personality" in how it generates outcomes. The personality is based on the model architecture directly. Here is a list of potential architectures you can follow to build a generative model:

Generative adversarial networks (GANs)
: Realistic image generation

Variational autoencoders (VAEs)
: Latent-space variations

Transformers
: Context-aware text

Diffusion models
: Iterative denoising image generation

Let's dive into each one of them:

GANs
: Imagine two rival apprentices, one creating new pieces and the other critiquing them. One apprentice tries to make the best possible creation, while the other tries to spot flaws. Through this playful competition, the creator learns to improve until their work is almost indistinguishable from the originals—also known as the training dataset.

  In research these two competing neural networks are named the *generator* and the *discriminator*. GANs excel in generating realistic images and are often the engines behind AI-generated art and synthetic photography.

VAEs
: VAEs take a different approach. Instead of directly creating, they first learn a compressed "blueprint" of the examples and then reconstruct them. When asked to make something new, they can tweak this blueprint to create a variation. VAEs are handy for generating data that's not too specific but still respects the general form and style of the original data, like creating a unique but generic landscape. VAEs are often used to generate images.

Transformers
: If GANs and VAEs are focused on images, transformers are your masters of language and sequential data. Picture an apprentice who learns the rhythm and flow of language, following sequences and structures closely. Transformers, the architecture behind models like GPT, can write coherent essays, hold conversations, or even code. They're not just good at learning patterns but also at using them to generate logically connected and context-aware responses.

Diffusion models
: If transformers are your masters of language, diffusion models are your master painters and restorers. Picture an artist who starts with a canvas full of static noise and then, stroke by stroke, learns how to reveal a real image hidden inside it. That's diffusion: it doesn't "draw" an image in one shot. It refines it through a sequence of denoising steps, gradually turning randomness into something coherent, detailed, and surprisingly lifelike.

In the end, these architectures aren't just different "personalities." They're different philosophies of creation. GANs learn by tension, VAEs by compression, Transformers by attention, and Diffusion by cleaning noisy, corrupted data. Each reflects a belief about how information should be represented, transformed, and ultimately expressed.

What unites them is simple: learn the underlying structure of the world, then generate something new without breaking that structure. That's the quiet power of generative AI. It doesn't copy. It abstracts. It compresses complexity into something reusable. And when it works, it gives us outputs that look less like replicas and more like possibilities.

A World of Applications: Generative AI at Work

Once you have this generative "apprentice," the question becomes, what can it actually *do*? A lot more than autocomplete your sentences. Generative AI expands what we can build because it doesn't just analyze existing data; it creates new data, new artifacts, and new possibilities:

Text creation
: Models like GPT can draft articles, summarize long reports, and even write runnable code. In customer service, they can hold context-aware conversations that feel closer to guided assistance than scripted replies.

Visual and art generation
: DALL·E and GAN-based systems can turn text prompts into images—marketing mockups, design variations, and even photorealistic faces. For creative teams, they function less like tools and more like collaborators that never get tired.

Synthetic data for privacy
: When real data is sensitive, scarce, or messy, generative models can produce synthetic datasets that preserve statistical patterns without exposing private details. This is especially useful in domains like healthcare, finance, and security, where privacy is a feature, not an afterthought.

Personalization
: Generative systems can tailor outputs—songs, product descriptions, recommendations—based on a user's preferences in real time. Instead of one-size-fits-all content, we get experiences shaped to each person's context and intent.

Taken together, these applications point to the same shift: generative AI isn't just another automation tool. It's a new interface for creation—fast, adaptive, and increasingly woven into how modern systems learn, design, and interact.

How Is GenAI Related to Deep Learning?

Deep learning is the foundation of GenAI. Deep learning provides a multilayered neural network that allows a model to learn complex patterns from a large dataset. By processing these patterns and going over them, GenAI predicts the next pattern, word, sound, or pixel and constructs a coherent response piece by piece.

If you say that you train a GenAI model, you are training a neural network.

If you are using a GenAI model for inference, you are leveraging the neural network as follows:

Prompting the model
: Inference starts with an instruction, what we now casually call a *prompt*. It can be as simple as "write a story about a brave Spark engineer" or as complex as a multimodal input combining text and images. The prompt doesn't just ask for an output; it sets the model's direction, scope, and constraints.

Generating content from learned structure
: From there, the model reaches back into everything it learned during training. Text models generate token by token, choosing the next word based on the

statistical and semantic patterns they've internalized. Image models often begin with noise and iteratively reshape it into something that matches the prompt. In both cases, generation is less about retrieving an answer and more about reconstructing one from learned structures.

Sampling and refinement
: Most generative models don't produce a single deterministic output. They sample by exploring possible continuations of outputs and selecting among them. Some architectures perform iterative refinement, improving the draft over multiple steps the way a human might revise a paragraph or sketch.

Producing the final output
: All of this resolves into a final artifact: a paragraph, an image, a melody, a design. Inference is the moment where the model's compressed knowledge and your instruction meet, producing something new in real time, guided by patterns but not bound by any single example.

Together, these steps show that inference isn't magic; it's a controlled unfolding of what the model learned during training. You provide direction, the model applies structure, and through sampling and refinement, it converges on an output that balances creativity with constraint. Prompting becomes less about telling the model *what* to say and more about steering *how* it thinks.

Can You Spark It?

In a digital lab, Gen, the generative AI model, has two missions: to *learn* and to *create*. Spark, the lab's powerful orchestrator, divides tasks across multiple stations, allowing GenAI to process massive amounts of data quickly.

First, during *training*, Spark feeds GenAI vast amounts of text, images, and audio, splitting the workload among different nodes. GenAI learns patterns from every corner of data, refining its skills with each pass. Spark keeps everything running smoothly, so GenAI becomes a master creator in record time.

Then comes *inference*. People bring prompts like "Write me a poem about building an AI pipeline" or "Generate a futuristic computer architecture." Spark organizes these requests, distributing them to nodes where GenAI responds instantly, drawing on its training to generate text, images, and more. With Spark handling the heavy lifting, GenAI creates on demand, transforming ideas into reality at scale.

Phew, that was a simple way to distinguish between training and inference for a model. But here's the real question: *How do you make it work in practice?*

Effortless TensorFlow Scaling with Spark's TensorFlowDistributor

Imagine you've got a powerful TensorFlow model, all set to take on a huge dataset. You're excited to push it to a Spark cluster and let it flex its muscles across multiple nodes, maybe even GPUs. But then, reality hits—scaling up TensorFlow is like trying to build a networked city from scratch. Workers need to talk to each other, data needs to be perfectly divided, and you need every GPU to fire on all cylinders.

That's where Spark's new *Distributor* enters, ready to play the hero. Think of it as the skilled city planner who takes your training code and sets up a bustling, organized network.

The great connector

Each TensorFlow worker in your cluster needs a reliable way to connect with others. Without it, they'd be stranded, unable to share knowledge and updates. Imagine a city without bridges or roads—chaos! *TensorFlowDistributor builds these "bridges" automatically, ensuring every node has a clear path to every other*. It even sets up protocols, so data flows smoothly from node to node, like Spark's invisible highway system, keeping your training aligned and in sync.

The data alchemist

When you're training a massive model, you can't have every worker look at the same piece of data simultaneously. That would be like a library where every librarian tries to check out the same book. So, *TensorFlowDistributor becomes the master alchemist, splitting and shuffling data*, giving each worker a unique batch to train on. And every time the workers ask for data, they get a fresh, unique mix—no repetitive loops, just efficient learning!

The GPU commander

Ah, the GPUs—the high-powered engines that turn big data into models. But managing them across nodes can be tricky, like assigning engines to a fleet of race cars with no central command. *TensorFlowDistributor steps in as the GPU commander, strategically placing each worker on the right GPUs*, ensuring no resource is wasted, and every machine gets exactly what it needs to keep your training sprinting forward, whether it's a single GPU or multi-GPU setup.

Your time to innovate

With all these systems in place, TensorFlowDistributor's last gift is one of simplicity: *you don't have to worry about any of it*. You write your model and training steps just once, as if you were running them on a single node. TensorFlowDistributor then lifts that code, distributes it across the cluster, and lets Spark handle the rest. Or do you?

TensorFlowDistributor: Show me how!

This tool turns Spark into a TensorFlow enabler, handling distribution of model training across multiple nodes and GPUs.

How it works. Using Spark, TensorFlowDistributor launches TensorFlow jobs on clusters, managing worker communication, data parallelism, and GPU handling. No more heavy setup; just focus on your model.

Getting started. A couple of lines, and you're good to go:

```
    # Run in distributed mode
    self._check_encryption()
    self._logger.info('Distributed training in progress...')
    self._logger.info( 'View Spark executor stderr logs to inspect   training...')
    result = self.sc.parallelize(range(self._num_tasks),self._num_tasks) \
               .barrier() \
               .mapPartitions(spark_task_program) \
               .collect()[0]
    self._logger.info(f'Training with {self._num_slots} slots is complete!')
    return result
```

```
from pyspark.ml.tensorflow import TensorFlowDistributor

def train_step(learning_rate, use_gpu):
    # TensorFlow setup: initialize, load data, define model, etc.
    # Standard distributed training loop

distributor = TensorFlowDistributor(num_proces ses=4, local_mode=False,
use_gpu=True)
distributor.run(train_step, learning_rate=0.001, use_gpu=True)
```

Under the hood. `num_processes` is your concurrency control, while `local_mode` lets you choose whether the driver node joins the training. And `use_gpu=True`? You guessed it—full GPU power! With the `run` method, TensorFlow training seamlessly scales across nodes.

Why it's a game changer. Distributed TensorFlow usually requires complex setups, but `TensorFlowDistributor` takes Spark's distributed computing know-how and combines it with TensorFlow's deep learning prowess. It's like having your data cake and eating it too.

Scaling PyTorch with Spark's TorchDistributor

Spark isn't just about scaling TensorFlow! With the `TorchDistributor`, Spark 3.4+ now gives a powerful, *simple* way to scale PyTorch training across clusters. Let's break it down into some sparks ttttttttand flames of awesomeness.

What is `TorchDistributor`? Think of it as Spark's way of handling the heavy lifting in training PyTorch models. Imagine training your model across multiple machines or GPUs without sweating the complex setup!

How it works. `TorchDistributor` uses PyTorch's `torch.distributed.run` to handle all the worker connections, data splits, and model parallelism across nodes. You can set it to multinode GPU, single-node multi-GPU—whichever matches your setup.

Get up and running. With just a few lines of code, you're scaling like a pro:

```
from pyspark.ml.torch.distributor import TorchDistributor

def train(learning_rate, use_gpu):
    # Standard PyTorch setup
    # Initialize distributed settings
    # Define model, data loader, etc.

distributor = TorchDistributor(num_processes=2, local_mode=False, use_gpu=True)
distributor.run(train, learning_rate=1e-3, use_gpu=True)
```

What's going on here? `num_processes` controls concurrent training tasks. `local_mode` decides if the driver node joins in, while `use_gpu=True` harnesses GPU power. And just like that, Spark distributes PyTorch across nodes!

Why it's a game changer. Distributed ML traditionally takes complex orchestration. With Spark's `TorchDistributor`, you're blending Spark's distributed computing expertise with PyTorch's deep learning chops, getting scale without the sweat.

Supercharging Distributed Training with DeepSpeed

DeepSpeed is an open source deep learning optimization library developed by Microsoft to make large-scale model training faster, more efficient, and less resource-intensive. It's specifically designed to address the challenges of training massive AI models, such as large language models and transformers, by optimizing memory usage, computation speed, and cost.

Here's a breakdown of what makes DeepSpeed powerful:

Memory optimization with ZeRO (zero redundancy optimizer)
: ZeRO is a unique optimization technique within DeepSpeed that splits model parameters, gradients, and optimizer states across multiple devices (such as GPUs), rather than storing all of them on every device. Picture this as a technique for sharing the load, where instead of each worker bearing the full weight of the model's components, they divide it up. With ZeRO, DeepSpeed ensures that no single GPU is overloaded—each one only needs to handle a slice of the whole. Suddenly, models that were too big to fit on a single device can be distributed seamlessly across multiple GPUs.

ZeRO has multiple stages, each offering a different balance between memory savings and communication overhead. This allows users to fine-tune DeepSpeed for specific hardware configurations and performance needs.

Mixed-precision training
: DeepSpeed supports *Automatic Mixed Precision (AMP)*, which enables models to use a combination of lower precision (e.g., FP16) and standard precision (FP32) calculations. Traditionally, models would calculate everything in high precision, but that's slow and heavy. DeepSpeed finds a middle ground: it uses high precision only where it's absolutely necessary and lowers precision where it won't impact the final result. This technique reduces memory usage and speeds up computations while preserving model accuracy, making it ideal for large-scale models that require faster training and lower resource costs.

Pipeline and model parallelism
: For extremely large models, DeepSpeed offers *pipeline parallelism* and *model parallelism*. Pipeline parallelism splits the model across multiple GPUs, allowing each GPU to process a part of the model sequentially, while model parallelism splits the model into segments that are processed in parallel.

  These techniques allow developers to efficiently use resources, maximizing the use of multiple GPUs or nodes for very large models that wouldn't fit on a single GPU.

Efficient data and gradient management
: DeepSpeed optimizes data and gradient handling by supporting features like *gradient accumulation* (which enables larger effective batch sizes without increasing memory demands) and *gradient checkpointing* (which reduces memory usage by recalculating some gradients instead of storing them).

  These optimizations further improve memory efficiency, allowing for larger batch sizes and more complex models.

Seamless integration with PyTorch
: DeepSpeed is fully integrated with PyTorch, making it accessible to PyTorch users without major changes to codebases. Through the use of APIs and configuration files, DeepSpeed adds distributed and optimized training capabilities to existing PyTorch models. For large-scale distributed training, tools like `DeepSpeedTorchDistributor` can be used to integrate DeepSpeed with Spark distributed computing environments.

Larger models
: DeepSpeed enables the training of models that wouldn't fit in memory otherwise, making it possible to scale models without needing exceptionally high-end hardware.

Faster training
: By reducing memory overhead and optimizing calculations, DeepSpeed accelerates training, which is crucial when dealing with large datasets or complex models.

Reduced costs
: DeepSpeed makes efficient use of resources, allowing users to train large models without excessive hardware or cloud costs.

Why DeepSpeed matters

DeepSpeed's memory and computation optimizations make it easier for researchers and developers to work with massive models that were previously limited to organizations with extensive resources. With DeepSpeed, training a model like GPT or BERT becomes more feasible for smaller teams, opening doors to experimentation with advanced AI architectures.

`DeepSpeedTorchDistributor` is a tool within the PySpark framework, specifically designed to enhance distributed training of PyTorch models by leveraging Microsoft's DeepSpeed library. This distributor provides several optimizations that make it easier to train large, complex models more efficiently on a Spark cluster, especially when using multiple GPUs.

Key features of DeepSpeedTorchDistributor

Following is a concise summary of the main capabilities of `DeepSpeedTorchDistributor` in Spark-based PyTorch training:

Enhanced memory and speed optimization
: DeepSpeed is known for its ability to optimize memory and speed, which is crucial for training large language models and other memory-intensive architectures. By using ZeRO, it reduces memory usage, allowing larger models to be trained even on limited hardware.

Efficient distributed training on Spark
: `DeepspeedTorchDistributor` enables you to distribute PyTorch models across Spark's clusters, handling parallelization, synchronization, and communication between nodes more efficiently than standard PyTorch distributed approaches.

AMP
: `DeepspeedTorchDistributor` supports automatic mixed-precision training, which helps to reduce memory usage and speed up training by using lower precision (e.g., FP16) calculations, especially beneficial for deep learning tasks on GPUs.

Using DeepspeedTorchDistributor

With this distributor, you can set up distributed training in a Spark environment as follows:

```
from pyspark.ml.torch.distributor import DeepspeedTorchDistributor

# Define your training function
def train_model():
    import torch
    import deepspeed
    # Define model and data loaders
    # Setup DeepSpeed configuration
    # Train your model using DeepSpeed optimizations

# Set up the DeepspeedTorchDistributor
distributor = DeepspeedTorchDistributor(num_processes=4, use_gpu=True)

# Run the distributed training
distributor.run(train_model)
```

Benefits in practice

`DeepspeedTorchDistributor` simplifies the setup for distributed training, allowing developers to leverage Spark's scalability with the efficiency of DeepSpeed. This tool is ideal for large-scale language models, image models, or any high performance computing tasks that require a scalable and resource-efficient approach.

Wait, Your Processor Matters!

In Chapter 9, we discussed multiple processors;RAPIDS is a critical part of them.

The *RAPIDS* AI ecosystem plays a pivotal role. RAPIDS is a collection of open source libraries built on CUDA that enable GPU acceleration for data science and machine learning pipelines. By leveraging RAPIDS with Apache Spark, practitioners can significantly speed up data preprocessing and model training workflows, thus enhancing the efficiency of deep learning pipelines.

Distributed training with RAPIDS and TorchDistributor

The *TorchDistributor API* in Spark 3.4 enables seamless distributed training for PyTorch models, but integrating RAPIDS ensures that preprocessing workloads leading up to training are also optimized. RAPIDS accelerates operations like filtering, aggregations, and feature transformations on GPUs, significantly reducing the time spent preparing datasets for DL frameworks.

Once preprocessing is complete, the TorchDistributor API takes over, launching the distributed PyTorch processes across Spark executors. With RAPIDS, these processes benefit from GPU-accelerated data transfer and processing, ensuring that large datasets are handled efficiently without significant serialization overhead.

What changes in code? Here is an example:

```
import cudf
gdf = cudf.from_pandas(df.toPandas()) # Convert to cuDF
```

Instead of using Spark's CPU-based operations, we convert the dataset into a *cuDF DataFrame*, unlocking GPU power.

Now, we can perform operations like logarithmic transformations on the data, which would normally take minutes, in mere seconds:

```
gdf["feature"] = gdf["feature"].log1p()  # Apply log1p transformation
```

Now, we can feel a sense of accomplishment—preprocessing is no longer a bottleneck.

Here's a simplified example of how RAPIDS complements the TorchDistributor workflow (see "df.toPandas()" for a warning):

```
from pyspark.ml.torch.distributor import TorchDistributor
import cudf
import dask_cudf

# Accelerated data preprocessing with RAPIDS
df = spark.read.parquet("path/to/data")
gdf = cudf.from_pandas(df.toPandas())  # Convert Spark DataFrame to cuDF for
GPU acceleration
gdf["feature"] = gdf["feature"].log1p()  # Example transformation

# Save preprocessed data back to disk for distributed training
gdf.to_parquet("path/to/preprocessed/data")

# Distributed training with TorchDistributor
def main_fn(checkpoint_dir):
    # PyTorch distributed training code
    ...

output_dist = TorchDistributor(num_processes=2, local_mode=False,
use_gpu=True).run(main_fn, checkpoint_dir)
```

df.toPandas()

While the previous example works, this part collects the *entire Spark DataFrame* to the *driver* as a single pandas DataFrame. This is not distributed anymore and can blow up driver memory if the dataset is large. This is *not the typical "Spark + RAPIDS" pattern*. With RAPIDS Accelerator for Apache Spark, Spark itself can run many DataFrame operations on GPUs *without* converting to pandas/cuDF manually. The `toPandas()` → `cuDF` pattern is usually a small-data/prototyping approach, not a scalable Spark approach.

Here is a more comprehensive, distributed approach:

```
from pyspark.sql import functions as F
from pyspark.sql.types import DoubleType
from pyspark.ml.torch.distributor import TorchDistributor

# Distributed GPU-accelerated preprocessing (no df.toPandas(), no driver
collect)
# Option A (recommended): RAPIDS Accelerator for Apache Spark
# This keeps the DataFrame distributed and lets Spark run many ops on GPU.
# ------------------------------------------------------------------

# Read distributed
df = spark.read.parquet("path/to/data")
# Example transformation: log1p(feature)
# This stays distributed. With RAPIDS Accelerator enabled, Spark can exe
cute it on GPU.
df_pre = df.withColumn("feature", F.log1p(F.col("feature").cast(Double
Type())))
# Write distributed (many files/partitions). Torch jobs can read this back
in.
df_pre.write.mode("overwrite").parquet("path/to/preprocessed/data")

# ------------------------------------------------------------
# Distributed training with TorchDistributor
# Each worker reads its shard of parquet; no driver-side materialization.
# ------------------------------------------------------------
def main_fn(checkpoint_dir):
    import os
    import torch
    import torch.distributed as dist
    from torch.utils.data import DataLoader
    import pyarrow.parquet as pq

    # Example: each rank reads a subset of files (simple sharding)
    rank = int(os.environ.get("RANK", "0"))
    world_size = int(os.environ.get("WORLD_SIZE", "1"))

    # Initialize distributed (TorchDistributor sets env vars)
    dist.init_process_group(backend="nccl")
```

```
        data_path = "path/to/preprocessed/data"
        # NOTE: you'll want a robust file listing method in production
        # (e.g., fsspec / Hadoop FS APIs). This is intentionally simple.
        import glob
        files = sorted(glob.glob(os.path.join(data_path, "*.parquet")))
        shard_files = files[rank::world_size]

        # Load shard and train...
        for f in shard_files:
            table = pq.read_table(f)
            # convert to tensors, run training steps, checkpoint to check
    point_dir
            # ...
        dist.destroy_process_group()
    checkpoint_dir = "path/to/checkpoints"
    output_dist = TorchDistributor(
        num_processes=2,
        local_mode=False,
        use_gpu=True
    ).run(main_fn, checkpoint_dir)
```

What changed. Preprocessing stays in a distributed Spark DataFrame and writes Parquet back out; each Torch worker reads its own shard from storage. No `toPandas()` and no cuDF conversion on the driver. If you *specifically* need cuDF for custom GPU logic, the scalable pattern is "per-partition" processing (Arrow/pandas UDF or `mapPartitions`), so each executor converts only its partition, not the whole dataset, to cuDF.

Distributed inference with RAPIDS: Making predictions at scale

With the model trained, we face the final hurdle: inference. Predictions must be made on the same scale as the training data, and the results must flow back into the Spark pipeline.

The new *predict_batch_udf API* introduced in Spark 3.4 simplifies distributed inference for batch, but it can further benefit from RAPIDS for batch preparation and data handling. RAPIDS accelerates the translation of Spark DataFrames into NumPy arrays and optimizes batch creation to maximize GPU utilization.

During inference, RAPIDS also facilitates efficient model loading and caching on GPU-enabled Spark executors, minimizing the performance overhead typically associated with broadcasting large model weights across the cluster.

We begin by writing a function to load the trained model and perform predictions. The model is loaded onto the GPU, ready to process batches of input data:

```
import torch

def make_predict_fn():
    model = torch.load("path/to/model.pt").to("cuda")  # Load model onto GPU

    def predict(inputs):
        gpu_inputs = torch.from_numpy(inputs).to("cuda")
        outputs = model(gpu_inputs)
        return outputs.cpu().detach().numpy()  # Return predictions as NumPy
arrays

    return predict
```

Next, we want to integrate this prediction function into the Spark pipeline using the `predict_batch_udf` API. This API automates batching, GPU utilization, and data movement. But first, let's create the UDF function `mnist` and read the batch data:

```
from pyspark.ml.functions import predict_batch_udf
from pyspark.sql.types import ArrayType, FloatType

# Distributed inference with RAPIDS-accelerated preprocessing df =
spark.read.parquet("path/to/test/data")
gdf = cudf.from_pandas(df.toPandas())
gdf["normalized"] = gdf["data"] / gdf["data"].max()

mnist = predict_batch_udf(
    make_predict_fn,
    input_tensor_shapes=[[1, 28, 28]],
    return_type=ArrayType(FloatType()),
    batch_size=1000,
)
```

Finally, we apply the `mnist` UDF to the test dataset and collect predictions:

```
df = spark.read.parquet("path/to/test/data")
predictions = df.withColumn("predictions", mnist("data")).collect()
```

Here is the complete workflow:

```
[ Raw Data ] → [ RAPIDS-Accelerated Preprocessing ]
     → [ TorchDistributor Training ]
                → [ RAPIDS-Enhanced Inference with predict_batch_udf ]
```

This workflow showcases the synergy between RAPIDS and Spark for real-world applications, enabling end-to-end acceleration from data preparation to deep learning inference.

Why RAPIDS matters for Spark deep learning

By integrating RAPIDS into Spark deep learning workflows, organizations can:

Accelerate data preprocessing
: Handle data transformations directly on GPUs, reducing the bottlenecks often faced with large datasets.

Optimize training pipelines
: Leverage GPU-accelerated distributed training with minimal overhead using RAPIDS and TorchDistributor.

Enhance inference efficiency
: Use RAPIDS to batch and preprocess data, ensuring that inference workloads fully utilize GPU resources.

With RAPIDS, Spark practitioners can build high performance deep learning pipelines that are scalable, efficient, and ready for production.

Inference in the Real World: Beyond Batch Prediction

In the previous example with RAPIDS, we focused on Spark's *predict_batch_udf API*, which provides a powerful and scalable solution for batch inference. But batch prediction is just one piece of the puzzle when it comes to deploying machine learning models for real-world applications. The world of inference is diverse, with different approaches tailored to specific use cases and operational requirements. Let's explore the broader landscape of inference workflows, from batch prediction to models running as a service or embedded within services.

Batch Prediction: Scaling Predictions with Spark

What it is. Batch prediction involves running inference on large datasets at regular intervals. This is ideal for scenarios where predictions don't need to be made in real time, such as:

- Generating recommendations for all users at the end of the day.
- Scoring large datasets of leads in a marketing campaign.

Why use Spark. Spark's predict_batch_udf API excels in batch prediction because:

- It distributes the workload across a cluster, ensuring scalability.
- It integrates seamlessly with Spark's data processing workflows, allowing preprocessing and inference to happen in the same pipeline.

When to use batch prediction:

- You have large, static datasets.
- Latency is not a critical concern.

Model as a Service: Real-Time Predictions at Scale

What it is. A *model-as-a-service* approach involves deploying your trained model to a web service or API. This setup allows real-time predictions by exposing endpoints that applications can call.

Why it's different from batch prediction:

- Predictions are made one request at a time, typically within milliseconds.
- Suited for latency-critical applications like fraud detection, personalized recommendations, or chatbot responses.

Integration with Spark. While Spark isn't typically used for real-time inference, it plays a crucial role in:

Preprocessing
: Preparing data to feed into the real-time model.

Training and retraining
: Updating models regularly using Spark's distributed training capabilities.

Here's an example workflow:

1. Train your model using Spark and save it as a serialized file.
2. Deploy the model to a scalable service like TensorFlow Serving, Flask, or FastAPI.
3. Use Spark to preprocess incoming data streams and pass them to the service for real-time predictions.

Model in a Service: Embedding Inference into Workflows

What it is. This approach embeds inference into a larger service or system, where the model is not a standalone entity but part of a broader application.

Here are examples:

- A recommendation engine integrated into an ecommerce platform.
- A dynamic pricing model embedded within a financial application.

Integration with Spark. Spark supports this setup by:

- Feeding preprocessed data directly into the embedded inference system.
- Enabling real-time stream processing with tools like *Structured Streaming* to handle dynamic data feeds.

Why Spark fits:

- Spark's ability to handle streaming data and batch processing ensures the application receives clean, consistent inputs.
- Spark UDFs, including `predict_batch_udf`, can bridge the gap between distributed data processing and embedded inference.

A World of Inference Possibilities

Inference isn't a one-size-fits-all solution. From batch prediction pipelines powered by Spark's `predict_batch_udf` to dynamic APIs serving real-time requests, each approach has its place. Spark's versatility ensures it can support these diverse workflows, acting as the glue that connects data, models, and applications in scalable, efficient pipelines. Whether your needs are batch-based, real-time, or embedded, Spark has a role to play in making your AI applications come to life.

When deciding between batch prediction, model as a service, or model in a service, consider the following factors:

Latency requirements
: Real-time applications demand low latency, favoring model-as-a-service approaches.

Data volume
: Large datasets are better suited to batch prediction with Spark.

Operational complexity
: Deploying a model as a service requires more infrastructure but offers flexibility.

Integration needs
: Embedded models simplify workflows by combining inference with existing systems.

Evaluation of Models

A model is only valuable if it fulfills its intended purpose. Evaluating the model in the context of your specific application is essential to ensure it delivers meaningful results. One major challenge in deploying AI systems is effective evaluation, which is often underestimated but plays a critical role in achieving success. Evaluation helps

to measure not just accuracy but also robustness, fairness, scalability, and usability in real-world conditions.

This process is critical for mitigating risks and ensuring reliability. For example, a model might perform well in a controlled test environment but fail in production due to unanticipated edge cases or data shifts. These issues highlight the importance of testing under realistic conditions and stress scenarios.

Think critically about potential failure points in your system. Identify vulnerabilities such as biases in the training data, inaccuracies in rare scenarios, or dependencies on unstable features. Redesign the system to improve visibility into failures through techniques like logging, monitoring, and explainability tools. This not only helps to debug issues but also builds trust among stakeholders.

Evaluation should be an iterative process, starting with small-scale testing and expanding to larger, more diverse environments. Use quantitative metrics such as precision, recall, and F1 score, but also consider qualitative feedback from end users to ensure the system meets their needs. Testing should also include adversarial scenarios to expose weaknesses and assess the model's ability to handle unexpected inputs.

Ultimately, a well-evaluated model is more than a technical achievement; it is a step toward building AI systems that are reliable, transparent, and aligned with your goals. Continuous evaluation, along with regular updates based on new data, ensures the model remains relevant and effective over time.

While Spark has some evaluation capabilities, all of them refer to classic ML work and not generative AI. Hence, we will not cover them here. If you are interested in the Classic ML eval, check out Adi's book—*Scaling Machine Learning with Spark* (O'Reilly, 2023). This book covers all the scenarios, including creative ways to continuously evaluate your model in production.

Case Studies

The gap between a demo and a system you can trust is almost never the model. It's the data: where it comes from; how you shape it; how you keep it fresh; and how you prove, repeatedly, that it still works when reality changes underneath you. The two case studies in this section live on opposite ends of the enterprise AI spectrum, but they rhyme.

- *Fight Health Insurance* starts with a painful constraint: the organizations that have the data (insurers) have zero incentive to share it. So the problem becomes: how do you build a reliable model when the training signal is scarce, biased, or inaccessible?

The answer is pragmatic: anchor synthetic data in the best public ground truth you can find (California's independent medical review data), then industrialize generation and updates with a pipeline that's incremental, reproducible, and cheap enough to rerun.

- *Uber's Enhanced Agentic-RAG (EAg-RAG)* starts with the opposite constraint: there's *so much* data, policies, docs, tables, edge cases, and naïve retrieval turns "pretty good" into "dangerously wrong." So the problem becomes: how do you make answers both accurate and grounded in a high-stakes environment?

  The answer is architectural: enrich documents so retrieval has the right structure to work with, add agents that narrow and refine the search space, and introduce automated evaluation (LLM-as-a-Judge) so you can iterate faster than human review cycles.

If there's a unifying theme here, it's this: *production AI is mostly disciplined data engineering plus ruthless evaluation*. The model is the easy part. The hard part is building the factory line that keeps the model honest.

Fight Health Insurance

Fight Health Insurance is a generative AI tool that creates appeals to health insurance denials, a growing problem in America. Since those with most of the data are health insurance companies, and it's not in their best interest to share the data, fine-tuning the model for Fight Health Insurance needed to use a lot of synthetic data. Now, rather than hallucinate training records from scratch, a practice unlikely to produce good results, we used California's independent medical review data as a basis to produce the training records.



If you want, you can try out the resulting model at *https://oreil.ly/Ts2V_*, and while we hope you never have to deal with a health insurance denial, if you do, we hope you'll try out the tool to generate an appeal.

So, how did we build it? This is our technical story.

First, we created synthetic data based on California, WA, and NY medical review open data. Rather than using the GPUs and model inference on the Spark workers themselves, we used DeepInfra to produce the synthetic data. This allowed us to access larger models than we would typically have during the great GPU crunch (see Example 10-1).

Example 10-1. Using Spark to parallelize creating synthetic training data

```
def run_iteration(data_source_name: Optional[str] = None) -> bool:
    """Run an iteration, return true if more to be done."""
    # Limit rows when running in standalone mode
    max_rows = None
    # We only need to do batching in local mode, cluster is fine.
    if is_local:
        max_rows = 160
    print(f"Running itr with max_rows {max_rows}")
    # At this stage, only missing or incomplete data will trigger AI calls.
    # If all required data is present, the pipeline just combines/processes it—no
new AI calls.
    joined = make_combined_prompts_existing_df(
        data_source_name=data_source_name, max_rows=max_rows
    ).alias("joined")
    results = joined.mapInPandas(query_model_from_prompts,
schema=results_schema).alias(
        "results"
    )
    joined = None
    results.persist(StorageLevel.DISK_ONLY)
    results_count = results.count()
    print(f"Generated {results_count} results")
    try:
        results.show(3, truncate=100)
    except Exception as e:
        raise RuntimeError(
            "Failed to display results DataFrame - this may indicate issues with
model querying or result processing"
        ) from e

    # Define default values for key fields
    defaults = {CommonFields.INDEX.value: "", "results": {}}

    # Write the data out locally since we're going to do our training locally
    # You could also just write to S3
    results_itr = results.toLocalIterator(prefetchPartitions=True)
    c = 0
    for r in results_itr:
        idx = r[CommonFields.INDEX.value]
        print(f"Writing out for {idx}")
        for response_type, responses in r["results"].items():
            i = 0
            for response in responses:
                if not response or len(response) < 2:
                    # Skip None/empty
                    continue
                target_file = join(gen_loc, f"{idx}MAGIC{midx}-{i}
{response_type}.txt")
                print(f"File is {target_file}")
                i = i + 1
```

```
                if not os.path.exists(target_file):
                    with open(target_file, "w") as f:
                        c = c + 1
                        f.write(response)
                else:
                    print(f"Skipping {target_file}")
        print(r)
    return c > 2
```

There are a few things that we've done differently here than you might in a typical Spark job. One is that in collecting the result, we use `toLocalIterator` and save the results locally since we will use them for fine-tuning locally. Our results were small enough that they didn't require more. Having said that, if you're going to use distributed fine-tuning, you'll want to save it to a distributed filesystem instead.

The other is that since our input IMR data will update, we've manually written incremental processing using a left anti join so that when we retrigger the job, we'll only generate synthetic training data for new IMR records rather than the whole dataset.

This is very much not the traditional big-data pattern, but shows how you can use Spark to speed up slow small data jobs too.

Uber's Enhanced Agentic-RAG (EAg-RAG): When Chatbots Deliver Near-Human Precision

Uber Engineering published an article describing *Genie*, an internal Slack-based on-call copilot designed to answer thousands of questions with citations from internal documentation. To validate quality in a high-stakes domain (engineering, security, and privacy), Uber's subject-matter experts curated a *golden set of 100+ test queries* and evaluated against *40+ policy documents*—and found that many answers were incomplete, inaccurate, or missed key details.

Uber reports that by moving from a traditional RAG pipeline to Enhanced Agentic-RAG (EAg-RAG), it increased the share of acceptable answers (relative +27%) and reduced incorrect advice (relative –60%).

Enhanced Agentic-RAG (EAg-RAG) is Uber's term for its RAG architecture that upgrades "retrieve top-k chunks → ask the LLM" into a *multistep, agent-driven pipeline* that improves retrieval quality and how context is assembled before generation.

Let's go over the architecture in Figure 10-1 to see what Uber changed.

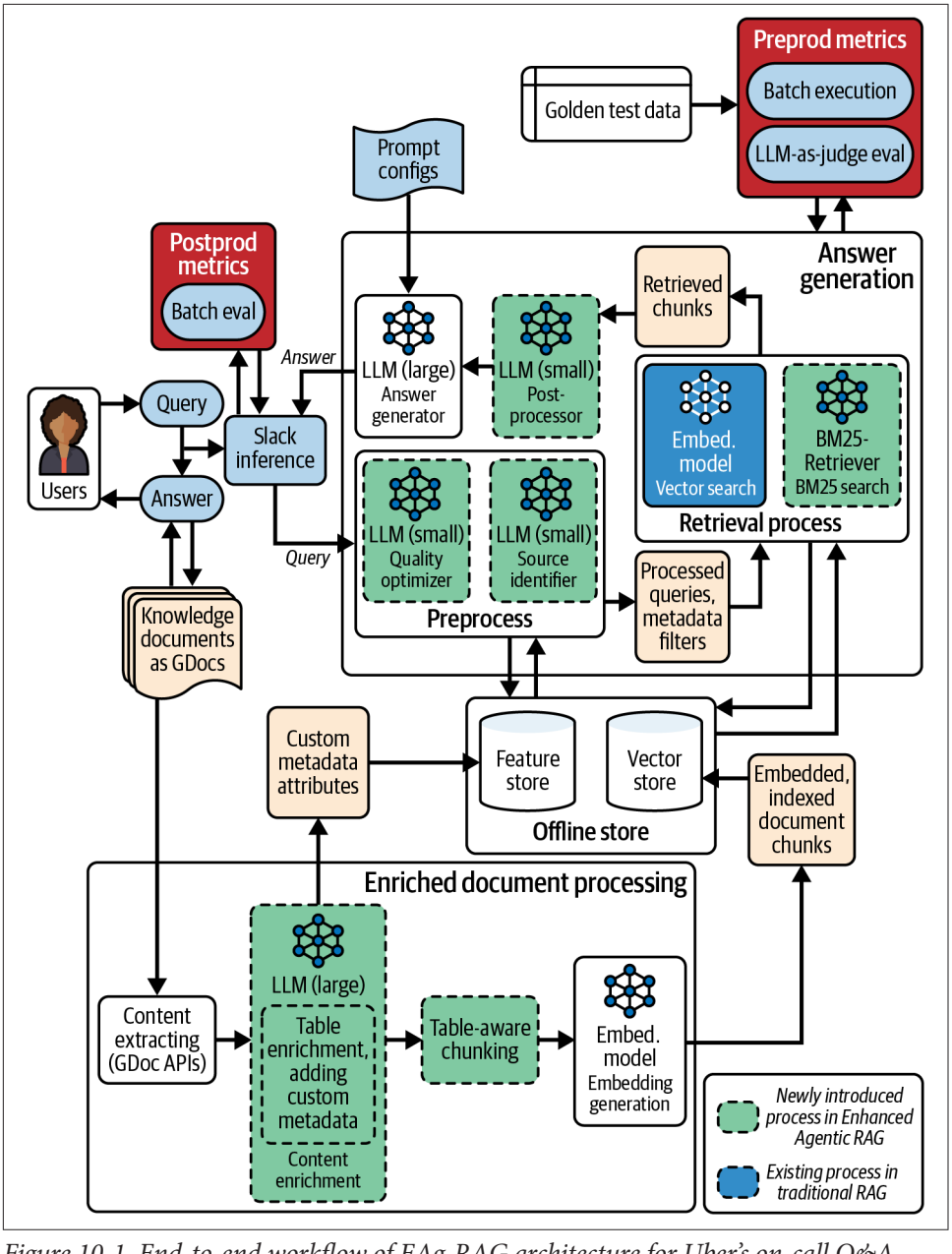


Figure 10-1. End-to-end workflow of EAg-RAG architecture for Uber's on-call Q&A copilot

At a high level, it has two components: *enriched document processing*, an offline process, and agentic answer generation. You can think of it as inference happening in real time. Let's dive deeper.

1. Enriched document processing (offline)

Uber found that typical PDF loaders often lose important structure (bullet points, tables), which then degrades chunking, embeddings, and retrieval—especially for complex multipage tables where cell context matters.

To address this, they:

- Transitioned from PDFs to *Google Docs with HTML formatting* for more accurate extraction and access control suitability in a security-sensitive setting.
- Built a *custom Google Docs loader* and used an LLM-powered enrichment step to convert tables into markdown-formatted tables while adding metadata identifiers so table chunks remain intact during chunking.
- Added lightweight table-oriented metadata (e.g., short summaries and keywords) to improve retrieval relevance.
- Enriched metadata further with document summaries, chunk-aligned FAQs, and keywords—used both to improve retrieval and to support later pre/postprocessing steps.

2. Agentic answer generation (near real time)

Uber explains that in its domain, "simple semantic similarity" can pull subtly wrong policy context (different personas, geographies, retention rules), which then cascades into wrong answers.

Uber's EAg-RAG flow introduces agents before and after retrieval:

Query optimizer
: Refines ambiguous queries, decomposing complex ones into simpler subqueries

Source identifier
: Narrows the retrieval scope to a subset of likely relevant documents, using offline artifacts like titles, summaries, and FAQs (plus few-shot examples)

Hybrid retrieval
: Combines vector search with a BM25-based retriever over enriched metadata; the retrieval set is the union of both

Postprocessor
: Deduplicates retrieved chunks and orders/structures them using their original positions in the source documents before passing context to the answer LLM

This approach overcomes the challenges of high SME involvement and plateauing accuracy of the system.

3. Faster iteration via automated evaluation (LLM-as-a-Judge)

Uber highlights two practical blockers: heavy SME involvement (evaluation took weeks) and plateauing gains. Uber introduced automated evaluation with an *LLM-as-a-Judge* approach (see Figure 10-2), scoring responses on a *0–5 scale* and generating reasoning/feedback to guide experiments.

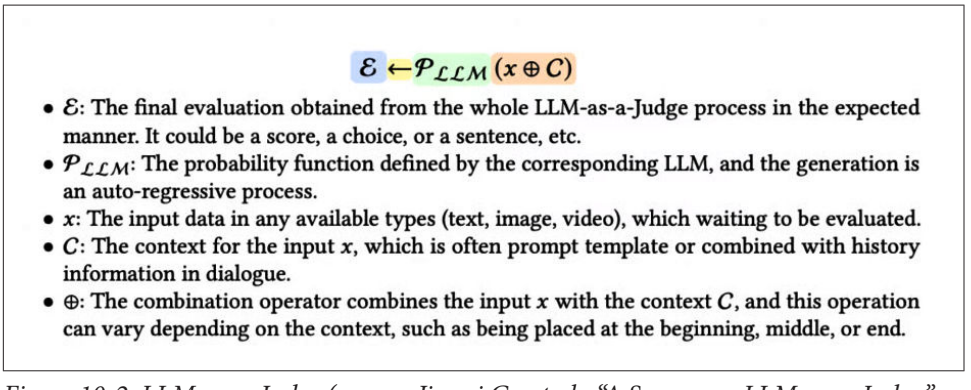


Figure 10-2. LLM-as-a-Judge (source: Jiawei Gu et al., "A Survey on LLM-as-a-Judge," preprint, arXiv, October 19, 2025)

In recent years, the *LLM-as-a-Judge* framework has become a practical standard for scaling evaluation of RAG and chatbot systems. The core idea is straightforward: instead of relying exclusively on slow, expensive human review, we use an LLM to act as a consistent evaluator that can score responses across large batches, identify systematic failure modes, and produce actionable feedback for improvement cycles.

Formally, the judge model evaluates a chatbot response x under a provided context C, which typically includes the user query, scoring rubric, expected answer or SME feedback, and any relevant ground truth evidence. The judge returns *structured outputs*, such as numerical scores (e.g., 1–5), categorical labels (e.g., correct/incorrect/partially correct), and short natural-language reasoning that explains why the response passed or failed. *This structure is important:* it makes results computable (for dashboards and regressions) while still producing qualitative insights that engineers and subject-matter experts can use to debug.

Why LLM-as-a-Judge works well for RAG systems. RAG failures are often subtle and multidimensional. A response can be fluent but wrong, correct but not grounded, or grounded but incomplete. Traditional automated metrics (exact match, BLEU, ROUGE) don't capture these nuances reliably. LLM judges, when given the right evidence and rubric, can evaluate along the dimensions that actually matter in production:

Correctness
: Does the response match the expected answer or policy truth?

Faithfulness/grounding
: Does it stick to the retrieved evidence or invent details?

Completeness
: Does it answer the full question, including constraints and edge cases?

Helpfulness and clarity
: Is it understandable and actionable for the user?

Safety and compliance (if relevant)
: Does it avoid disallowed advice or sensitive leakage?

This makes LLM-as-a-Judge especially valuable for organizations where success isn't simply "sounds good" but "matches our internal SME expectations."

Figure 10-3 is a diagram showing how Uber built it.

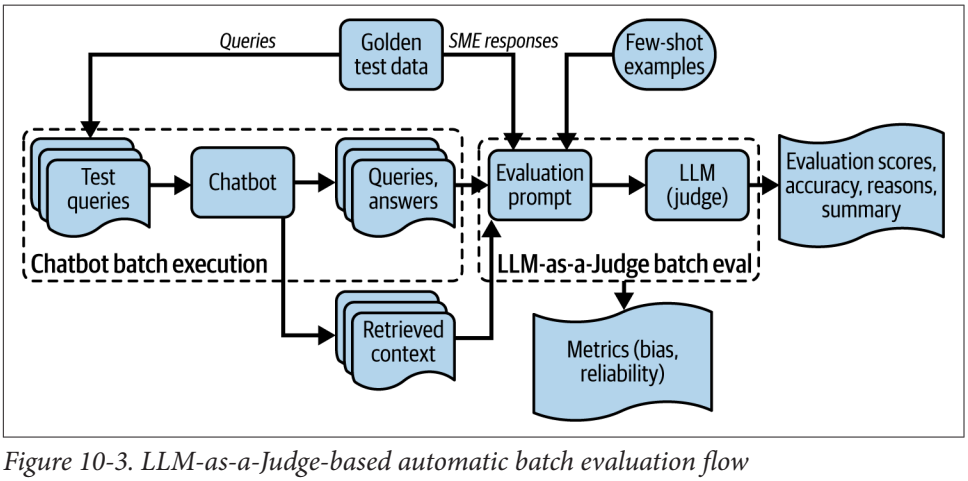


Figure 10-3. LLM-as-a-Judge-based automatic batch evaluation flow

This diagram shows an LLM-as-a-Judge automatic batch evaluation pipeline for a chatbot. First, a set of test queries is selected from golden test data and run through the chatbot in batch, producing a dataset of (query, chatbot answer) pairs plus the retrieved context the bot used (e.g., RAG snippets). In parallel, the golden dataset provides SME reference responses, and the evaluation setup may include few-shot examples to calibrate the judge. These inputs—user query, chatbot answer, SME response, retrieved context, and evaluation instructions—are assembled into an evaluation prompt, which is sent to an LLM acting as the judge. The judge returns structured evaluation outputs such as scores, correctness/accuracy labels, and concise reasons/summaries; these results are then aggregated into evaluation metrics (including signals like bias and reliability) to track performance and compare chatbot versions over time.

How does Spark fit in? As described in Uber's blog, the Genie system is based on Spark. Spark is the "factory line" that prepares the knowledge base for retrieval; it's not the chatbot itself, but the engine that continuously turns messy internal docs into clean, searchable vectors. Uber describes the RAG "data prep → embedding → artifact push" steps as a workflow that can be generalized using Apache Spark.

Concretely, Uber runs a custom Spark application (executed across Spark executors) that:

1. Fetches content from sources like Engwiki and internal Stack Overflow into a Spark DataFrame
2. Chunks that content and generates embeddings using OpenAI, implemented via PySpark UDFs—producing a DataFrame that maps each chunk to its vector
3. Pushes the embedding artifacts into Uber's storage/indexing pipeline (Terrablob and its vector database stack), including Spark jobs for index build/merge and ingestion

The payoff is simple: when a Slack question arrives, the online system can quickly retrieve relevant chunks because Spark already did the heavy lifting offline, keeping the vector index fresh and scalable without manually curating data.

Figure 10-4 shows an Uber diagram describing the flow.

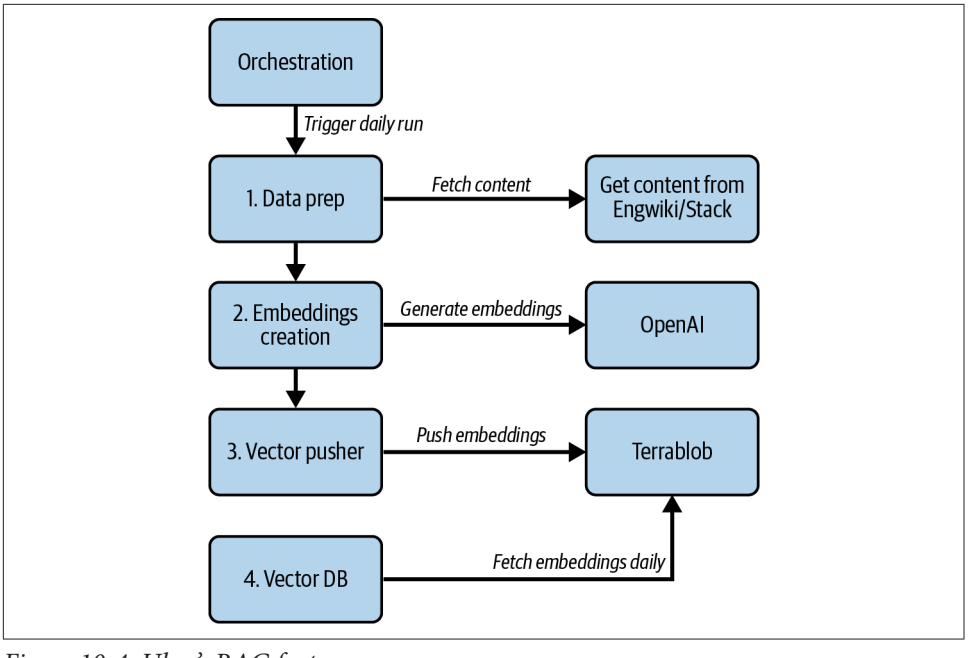


Figure 10-4. Uber's RAG factory

Optimizing recommendations

This operation can become expensive—for example, if for every UDF row entry, the system calls independently to an external system like the OpenAI model. While we don't have access to Uber's code itself, here are some optimizations we would do to drive high performance out of the Spark cluster:

Treat embedding as a separate, rate-limited distributed service
: Calling OpenAI (or any embedding endpoint) inside a row-level UDF is the *classic antipattern:* it creates *millions of tiny network calls*, amplifies retries, and makes Spark's task model fight an external rate limit.

  This would be better:

  - Use *mapPartitions* (or iterator-based pandas UDF) and do _micro-batching per partition_ (e.g., batches of 64–512 chunks).
  - Maintain a _single HTTP client per partition_ (connection pooling) instead of per row.
  - Implement *backoff + jitter + circuit breaking inside the partition loop* so one "hot" partition doesn't thrash.

  Outcome: fewer calls, higher throughput, fewer timeouts, and cleaner retry behavior.

Make the pipeline incremental and idempotent

The cheapest embedding is the one you don't recompute.

Do this:

- Compute a stable *content hash* per chunk (doc_id + chunk_id + normalized text hash).
- Keep an *embedding cache table* keyed by that hash.
- On each run: only embed "new or changed" chunks (left anti join against the cache).

In Spark terms: you want a fast "diff" job, not a full re-embed.

Partitioning strategy: partition by document, not random rows

We know partitions are opinionated space, yet due to our best experience working with Spark, one fact stays true: chunking creates a lot of small rows. Random distribution often causes:

- Skew (some docs explode into huge chunk counts)
- Excessive shuffle when you later regroup by `doc_id` for indexing

  Recommendations:

- During extraction/chunking, keep related data together: `repartition($"source", $"doc_id")`.
- For very large docs, consider salting `doc_id` or splitting by section to avoid single-partition hotspots.
- For writing: use *partitioning that matches your downstream index build* (e.g., by source/team/time).

Avoid Python overhead where it matters

If embedding is done in PySpark:

- Prefer *Arrow-enabled pandas UDFs* or *mapPartitions* over standard Python UDFs.
- Keep payloads compact: pass only needed fields to Python.
- Use *Kryo serialization* where possible.
- Ensure Arrow is configured properly.

If you can move embedding to Scala/Java (or a sidecar service), even better for CPU efficiency, but the big gain usually comes from batching + incremental.

Control data shape early: reduce, normalize, and compress

Before generating embeddings, clean and standardize the text so you don't waste tokens (and money) embedding duplicates, boilerplate, or oversized chunks.

Do this:

- Deduplicate near-identical content (templates, repeated headers/footers).
- Normalize whitespace and remove boilerplate navigation text.
- Cap chunk length consistently (don't let a few giant chunks wreck token costs).

Also do this:

- Store vectors in a compact format (array is fine, but avoid JSON blobs).
- Compress intermediate outputs; avoid "wide rows" with huge text fields after you don't need them.

Then keep the outputs lean—store vectors in compact types, compress intermediates, and drop oversized text fields early so the pipeline stays fast and cheap end-to-end.

Make indexing a first-class stage with shuffle discipline

Index build/merge often becomes the hidden cost center. For that stage:

- Use *adaptive query execution* (AQE) and tune shuffle partitions (don't leave default values).
- Prefer *bucketing* or stable partitioning if you repeatedly build/merge indexes by the same keys.
- Watch for small-file problems on object storage; coalesce output files intentionally.

Rule of thumb: embedding is network-bound; indexing is shuffle/I/O-bound. Optimize them separately.

Reliability: isolate failures and don't "fail the job" for a few bad calls

Production embedding has failures: timeouts, rate limits, and transient 5xx.[1]

Design pattern:

- Write "embedding attempts" to a side table with status and error.
- Let the Spark job succeed even if a tiny percentage fails.
- Rerun only failed chunks (small retry job).

This turns an unreliable external dependency into a manageable queue of retries.

1 A *transient 5xx* is a *temporary server-side error* (HTTP status codes *500–599*) where the *service you're calling* is failing briefly—often due to overload, a hiccup in an upstream dependency, or a short-lived outage.

Observability: measure what actually limits you
: Add metrics at the partition/batch level:

  - Requests/sec, tokens/sec
  - p50/p95 latency per batch
  - Error rates by type
  - Cost estimate per run
  - Skew indicators (top partitions by record count and runtime)

  In Spark: also track stage time, shuffle read/write, spill, and GC time. You want to know whether you're bottlenecked on the network, Python overhead, or shuffle.

Executor sizing that matches the bottleneck
: Executor sizing is how you align Spark's parallelism with the real constraint in each stage, whether that's an external embedding API's rate limits or the cluster's own shuffle and I/O throughput.

  For embedding-heavy stages:

  - Fewer cores per executor can help (too much concurrency just hits rate limits).
  - More executors with controlled concurrency per partition often work better.
  - If there are endpoint limits per API key, enforce a _global concurrency cap_ (otherwise Spark will happily DDOS your own quota).

  For shuffle/index stages: Tune for I/O (more cores and memory, fewer tiny tasks, right shuffle partitions)

  With the right executor sizing per stage, you can improve throughput while staying within API limits and keeping shuffle and I/O overhead under control.

Put simply: if you treat embedding like a real production data pipeline, batch what's expensive; cache what doesn't change; partition with intent; and measure what hurts, Spark stops being the bottleneck and becomes the boring, dependable workhorse that keeps your RAG index fresh without lighting your budget (or your on-call) on fire.

To wrap up these two use cases: none of these wins came from "finding the perfect model."

Fight Health Insurance worked because the team treated data scarcity like an engineering problem, grounded synthetic examples in real adjudication signals, generated them at scale, and made the pipeline incremental so it stayed current without becoming a GPU bonfire.

Uber's Genie improved because the company stopped pretending retrieval is a single step. It invested in document structure, retrieval scope, and automated evaluation so quality could keep climbing without weeks of SME time.

Different domains, same lesson: if you want near-human reliability, you don't ship a prompt; you ship a system. And that system is only as strong as the data pipeline that feeds it, the retrieval that frames it, and the evaluation that keeps it from drifting.

The Model Dilemma: Hosting or Leveraging a Third-Party REST API

For the sake of answering this question, we will use a fictitious character named Alex. Alex is a data engineer and aspiring machine learning expert at a fast-growing ecommerce company. Alex's team has been tasked with building a next-generation recommendation system that leverages generative AI to create hyper-personalized shopping experiences. As the project kicks off, Alex faces a pressing question:

> Should we host our own model or leverage a third-party API?

It seems straightforward at first—just pick the easiest option and run with it. But as Alex digs deeper, it becomes clear that this decision isn't so simple. The choice between *hosting* and *leveraging* a model via a third-party REST API could make or break the project's success. Enter the *seven layers framework*, Alex's compass for navigating the complexity of this decision.

Layer 1: Use Case and Domain Alignment

Alex starts by asking:

> How specific are our needs?

The team wants recommendations to feel personal, not generic. Existing APIs like OpenAI's GPT or Anthropic's Claude could work, but they're trained on general data. To generate product descriptions and recommendations with nuanced brand tone, Alex realizes the team might need a custom-trained or at least a fine-tuned model:

> "If we host our own model, we could train it specifically on our ecommerce data. But… that sounds like a lot of work."

Layer 2: Performance Requirements

The next question hits Alex like a performance bug in production:

> How fast does this need to run?

Customers expect snappy responses, especially during flash sales. Hosting a model on GPUs could provide the speed needed, but Alex worries about the cost and

complexity of managing that infrastructure. Leveraging a third-party API might add some latency, but it offloads the compute:

> Do I really want to deal with optimizing model inference times during Black Friday? Probably not…

Layer 3: Scalability Needs

Alex recalls the company's experience with traffic spikes. Last year's holiday sales crashed half of their services. The recommendation engine would need to scale dynamically.

- *Hosting* means provisioning infrastructure ahead of time, estimating peak demand (and likely over-provisioning).
- *Leveraging* means using a cloud provider that scales automatically and trusting that it will continue to operate at the rate you need it to:

> Our API provider promises global scalability. But what happens if their servers go down during a sale?

Alex makes a note to investigate service-level agreements (SLAs).

Layer 4: Cost Considerations

Next, Alex dives into budgeting with the team's finance lead:

- Hosting involves upfront investment in hardware, GPUs, and storage, not to mention ongoing maintenance costs.
- Leveraging an existing model is pay-as-you-go. Costs start low but could skyrocket with high traffic:

> I can build a model that saves money over time, but only if I don't burn our budget getting there.

Layer 5: Data Sensitivity and Security

The CTO flags a critical concern: Can we trust an external provider with our data?

Their recommendation system relies on customer purchase histories, which include sensitive data. Sending this information to an external API might breach compliance regulations in some regions. Hosting the model internally would keep all data secure:

> If we mess this up, it's not just a tech problem—it's a legal one.

Layer 6: Integration and Ecosystem Fit

Alex sketches out how the new system will fit into their Spark-based pipeline. Spark handles preprocessing terabytes of transactional data and feeds it into their model. Hosting their own model means using Spark's *TorchDistributor* for distributed fine-tuning/training, giving Alex full control. Leveraging an API simplifies integration but adds an external dependency:

> Do I trust an API to keep up with our real-time pipeline? What if latency builds up? What's the benefit of having my model hosted at my cluster?

Layer 7: Time-to-Market

Finally, Alex considers the project deadline: three months. Hosting their own model requires significant lead time to fine-tune, deploy, and test. Leveraging an existing model offers a shortcut to rapid deployment:

> We need something live soon, even if it's not perfect. Could we start with an API and move to hosting later?

The Decision

After weeks of research, debates, and sleepless nights, Alex presents the team's plan:

- Start with an existing model via an API to meet the deadline and validate the use case.
- Use this time to experiment with training their own model in parallel, leveraging Spark for distributed training and fine-tuning:

  > This way, we get the best of both worlds: quick wins now and long-term control later.

Alex's Reflection Moment

As the system goes live, Alex breathes a sigh of relief. The API integration worked smoothly, and customers loved the personalized recommendations. Months later, the team transitioned to hosting their model, cutting costs and improving performance even further.

Alex learned that the right decision isn't always about *hosting versus leveraging*—it's about flexibility, understanding the trade-offs, and adapting as needs evolve.

The Story of an Evolving Industry

As the tech industry matures and embraces AI-driven solutions, we're witnessing a pivotal transformation. The question of *hosting a model versus leveraging an existing third-party API* isn't just a technical debate—it's a reflection of how our industry evolves as products grow in complexity, scale, and impact.

This is a story that repeats itself across industries, companies, and teams. It's the story of how we balance innovation with pragmatism, agility with long-term vision, and control with convenience. It's a decision-making process that has become the cornerstone of modern AI adoption.

Conclusion: The Future of Generative AI with Spark

As we stand at the intersection of generative AI and distributed computing, Apache Spark emerges as more than a data processing engine—it's the backbone of the future's most ambitious AI innovations. Generative AI is transforming industries by enabling machines to create, not just process, and Spark is the silent hero, making this creativity scalable, efficient, and accessible.

Spark's Role in the Generative AI Revolution

The true power of Spark lies in its ability to bridge the gap between massive datasets and cutting-edge AI models. Whether it's training a deep learning model across thousands of nodes, preprocessing petabytes of data in minutes, or scaling inference pipelines to deliver real-time insights, Spark excels as the infrastructure that powers generative AI workflows:

Scalability
: Spark's distributed nature ensures that as your datasets and models grow, the system can handle the load without missing a beat.

Integration
: Tools like TensorFlowDistributor, TorchDistributor, and RAPIDS have turned Spark into an AI enabler, seamlessly connecting data pipelines with the world's most advanced deep learning frameworks.

Efficiency
: With GPU acceleration, optimized data handling, and simplified distributed training, Spark enables organizations to achieve more with less—less time, less cost, and fewer resources.

Generative AI and the Creative Industries

Generative AI, powered by Spark, is unlocking new possibilities across industries:

Healthcare
: AI-generated synthetic data is breaking privacy barriers while enabling advanced research.

Ecommerce
: Spark-driven pipelines are producing personalized recommendations and AI-generated product descriptions at scale.

Entertainment
: Creative models are generating art, music, and even scripts, reshaping the way stories are told.

Spark isn't just crunching data—it's paving the way for AI to assist, augment, and even amplify human creativity.

The Path Forward

As generative AI models grow larger and more complex, Spark will evolve alongside them. Future versions of Spark may introduce even tighter integration with AI tools, support for more advanced hardware accelerators, and optimizations for handling the colossal datasets required by next-generation models.

We can envision a future where:

- Spark clusters seamlessly orchestrate multi-trillion-parameter models.
- AI pipelines become intelligent enough to optimize themselves based on real-time data.
- Creativity becomes democratized, with tools powered by Spark enabling everyone—from researchers to small businesses—to harness the power of generative AI.

Your Next Step

This chapter gave you the core capabilities to start building generative AI pipelines on Spark. You've seen how to speed up data preparation with GPU acceleration (RAPIDS) and how to scale model training and inference across a cluster with tools like TensorFlowDistributor and TorchDistributor. With these building blocks, you can move from isolated experiments to production-grade AI workloads that are reproducible, observable, and cost-aware.

What comes next is application: where you choose to aim this stack. You can use Spark to keep retrieval indexes fresh, evaluate and iterate on agentic systems, automate high-volume content workflows, or modernize legacy data pipelines into AI-ready foundations. The opportunity isn't just "add AI"—it's designing systems where data, models, and evaluation continuously reinforce each other.

Spark will keep evolving, and your leverage will grow with it. As the ecosystem matures, Spark becomes less about running jobs and more about operating AI pipelines that automatically learn, adapt, and improve—turning generative AI from a one-off feature into a durable capability.

CHAPTER 11

Testing, Validation, and Side-By-Side Runs

Errors that are caught late often require expensive reruns of jobs, so catching them early is key to high performance data analytics. Testing, validation, and side-by-side comparisons are three key techniques for stopping errors before they hit production. These errors can lead to real-world impacts, like production outages, which more than half of people have experienced (see Figure 11-1), with almost 20% experiencing serious outages from their Spark pipelines. We don't recommend hope as a strategy when it comes to software quality; for one thing, the odds are not great (worse than half).[1] Instead, in this chapter, you will learn how to identify and stop many kinds of errors before they impact your users.[2]

We discuss unit and integration testing, validation, and side-by-side runs separately, even though they could all be (broadly) considered a form of testing. In this book, we consider testing mainly the offline process (automated or manual) of verifying that our software behaves as desired. Validation (sometimes called *audits*) is the online process of ensuring that our job is working as expected. Hidden side-by-side runs are a special kind of testing or validation that can be used when making changes to a job that you don't expect to impact its output, including performance tuning.

1 It's too late for all of the coauthors, and maybe it's too late for you to be in the happy 47%—but don't despair—you can use your knowledge of past outages to inform your testing strategy.

2 Or your job.

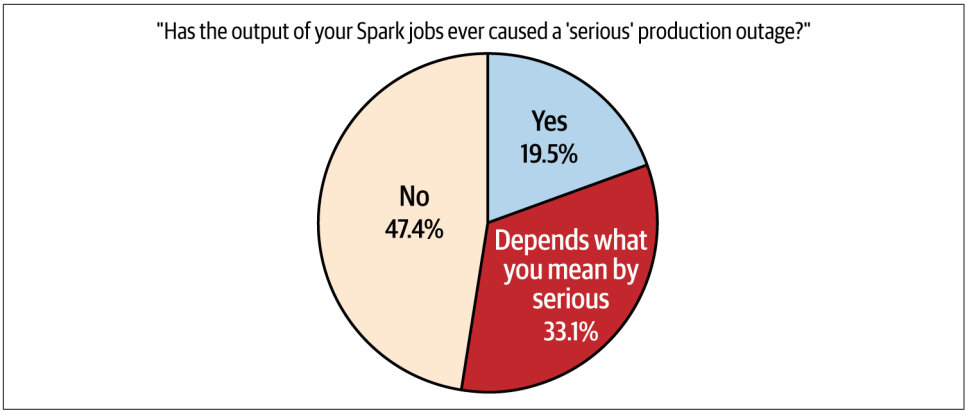


Figure 11-1. Spark production outage question (results collected from ~140 answers to a survey on Spark testing)

Automated testing in Spark is often overlooked, but with long batch jobs and complex streaming setups, manually testing and verifying functionality is time-consuming and error-prone. In fact, from that same survey, roughly 33% of folks automatically deploy some of the results from their Spark jobs to production (see Figure 11-2). Some of this comes from the prevalence of notebooks in the data analytics space, but we can work around those challenges. Effective tests allow us to develop faster and simplify when refactoring for performance.

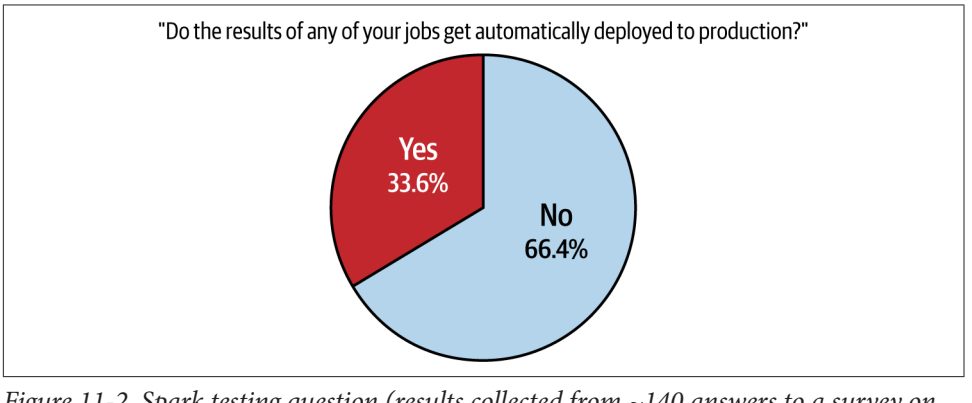


Figure 11-2. Spark testing question (results collected from ~140 answers to a survey on Spark testing)

In addition to the correctness of the system, you can also test the performance of it. Tests that verify performance have additional challenges, especially in distributed systems. However, using Spark's counters and event logs, we can get valuable metrics such as the execution time statistics from all the workers, the number of records

processed, and the number of records shuffled. These counters can serve the same purpose as system timings on a single-machine system.

Testing is an excellent way to catch the kinds of errors we can actually conceive of. Beyond that, the real world will come up with new and exciting ways to make our software fail, and sometimes, it isn't as obvious as a null pointer exception. In these cases, it is essential that we use validation (sometimes called audits) to detect potentially incorrect output to avoid making decisions with faulty models or data.

When you are improving an existing job, you can use the previous implementation as a source of truth for comparison. This can be done by creatively using the Write-Audit-Publish (WAP) pattern where the second (or untrusted jobs) audit step asserts that the data should be the same as the previous job.

Unit Testing

Unit testing allows us to focus on testing small components of functionality with dependencies (such as data sources), often mocked or simulated. Unit tests are generally faster than integration tests and are frequently used during development. If you are willing to do some refactoring, you can test much of your code without any special considerations related to Spark. Spark-specific testing libraries can significantly simplify the process for the components that must work with Spark.

This section generally follows the flow from lightweight to heavyweight in terms of the amount of overhead required to run your tests. Depending on how our Spark job is written, the smallest components of it can be tested without any Spark dependencies. We also discuss the option of using mock (or "simulated") Spark collections and why we do not recommend it for most situations.

Factoring Your Code for Testability

Since many of Spark's transformations take in functions that operate on individual elements or iterators of partitions, we can test those functions (Example 11-1) without having to create an RDD or use a `SparkContext`, unlike Example 11-2.

Example 11-1. Easy-to-test inner function

```
def tokenizeRDD(input: RDD[String]) = {
  input.flatMap(tokenize)
}

protected[tokenize] def tokenize(input: String) = {
  input.split(" ")
}
```

Example 11-2. Hard-to-test inner function

```
def difficultTokenizeRDD(input: RDD[String]) = {
  input.flatMap(_.split(" "))
}
```

Thinking back to our Goldilocks example, one of the components that retrieve the provided indexes inside of each partition could be tested in this way (see Example 8-21). When your code is factored this way, you can use the normal unit testing you are familiar with. Factoring your code this way can also improve readability, since small anonymous functions often don't stay small.



Even when you can factor your helper functions to test the internal logic of your transformations separately, ensuring that you have some end-to-end tests using Spark's data structures (DataFrames/Datasets/RDDs/DStreams) is still a good practice.

Making your code testable is not limited to Scala or Python; when writing SQL, your life will be substantially easier if you can break your large queries into smaller chunks that can be verified independently.

Mocking RDDs

Testing with a `SparkContext` involves overhead: either you or your testing library (e.g., spark-testing-base) needs to create a local Spark cluster. This overhead can slow your tests down, especially when using frameworks like ScalaCheck that generate hundreds of tests. For tests that are focused on testing the business logic rather than the specifics of your interactions with Spark (like serializability), mock RDDs can offer the ability to run more tests faster. In software testing, mocking generally refers to using a "fake" or simplified version of the actual item you want to interact with. Mocks can be used to verify that certain function calls are made (for example, checking that your code does call `filter` on the DataFrame). In distributed systems, mock objects are sometimes used to substitute a local version of a distributed object with a matching API so testing can run without as many resources.

While testing with mock RDDs can be a great way to test your business logic quickly, the testing is not as complete since it avoids testing the serialization and other Spark interactions. Daniel Westheide has made a library called kontextfrei to support a number of operations on mock RDDs. Similarly, for Python, LiveRamp made the MockRDD library (defunct). That being said, neither of these libraries appears to be actively maintained, but they do illustrate the concept.

To get a fast feedback loop during development, you can then execute those tests against Scala/Python collections locally and against RDDs on your CI server to

ensure you don't have any serializability issues. That being said, we don't personally use MockRDD/DataFrame implementations, preferring to pay the increased overhead of running local Spark contexts.

Core Spark Jobs (Testing with RDDs)

In addition to testing the functions that you provide to Spark's transformations, it's important that you test the logic expressed through these transformations as well. A simple way to test transformations is to create a `SparkContext`, `parallelize` the input, apply your transformations, and `collect` the results locally for comparison with the expected value (Example 11-3).

Example 11-3. Simple Spark unit test

```
class QuantileOnlyArtisanalTest extends AnyFunSuite with BeforeAndAfterAll {
  @transient private var _sc: SparkContext = _
  def sc: SparkContext = _sc

  val conf = new SparkConf().setMaster("local[4]").setAppName("quantileOnlyTest")

  override def beforeAll() {
    _sc = new SparkContext(conf)
    super.beforeAll()
  }

  val inputList = List(GoldiLocksRow(0.0, 4.5, 7.7, 5.0),
    GoldiLocksRow(4.0, 5.5, 0.5, 8.0),
    GoldiLocksRow(1.0, 5.5, 6.7, 6.0),
    GoldiLocksRow(3.0, 5.5, 0.5, 7.0),
    GoldiLocksRow(2.0, 5.5, 1.5, 7.0)
  )

  val expectedResult = Map[Int, Set[Double]](
    0 -> Set(1.0, 2.0),
    1 -> Set(5.5, 5.5),
    2 -> Set(0.5, 1.5),
    3 -> Set(6.0, 7.0))

  test("Goldilocks naive Solution"){
    val sqlContext = SparkSession.builder.getOrCreate().sqlContext
    val input = sqlContext.createDataFrame(inputList)
    val whileLoopSolution = GoldilocksWhileLoop.findRankStatistics(
      input, List(2L, 3L)).mapValues(_.toSet)
    val inputAsKeyValuePairs = GoldilocksGroupByKey.mapToKeyValuePairs(input)
    val groupByKeySolution = GoldilocksGroupByKey.findRankStatistics(
      inputAsKeyValuePairs, List(2L,3L)).mapValues(_.toSet)
    assert(whileLoopSolution.toMap == expectedResult)
    assert(groupByKeySolution.toMap == expectedResult)
  }
```

```
  override def afterAll() {
    // We clear the driver port so that we don't try and bind to the same port on
    // restart.
    sc.stop()
    System.clearProperty("spark.driver.port")
    _sc = null
    super.afterAll()
  }
}
```

The one unexpected bit in Example 11-3 is where we clear the port; this is done so that if we run many Spark tests in sequence, they will not bind to the same port. This will result in an exception trying to bind to a port that is already in use.

You can avoid dealing with all of the Spark setup and tear-down logic by using the `SharedSparkContext` from Holden's spark-testing-base project.[3]

If the data has become too large to directly call `collect`, you can use `toLocalIterator`, which only brings back a single partition at a time. If you are dealing with a process that creates a very large RDD, the techniques for comparing RDDs (introduced in "Computing RDD Difference" on page 308 and "Integration Testing" on page 310) can also be used, but this may also be a sign that your data test size has exceeded that of unit testing.



`toLocalIterator` can trigger multiple evaluations, so make sure to `cache` or `persist` the RDD it is called on.

Testing Streaming

With streaming tests, structured or DStream, you'll need to add extra logic to start and stop your streams as well as collect the data.

DStream streaming

Testing Spark Streaming requires special work to create test streams, collect the data locally to verify the results are as expected, and determine test completion. If it fits your application well, the spark-testing-base library provides a wrapper that allows you to write your tests by simply specifying the input and the expected output.

For simplicity, spark-testing-base provides two streaming test base classes: `StreamingSuiteBase` for transformations and `StreamingActionBase` for actions. This allows

3 Of course Holden is biased and thinks her own project is a good project.

you to write tests for transformations by specifying the expected input and output (Example 11-4). The library then takes care of interfacing with the internals and figuring out when to stop the test.

Example 11-4. `StreamingSuiteBase` example test

```
  test("really simple transformation") {
    val input = List(List("hi"), List("hi holden"), List("bye"))
    val expected = List(List("hi"), List("hi", "holden"), List("bye"))
    testOperation[String, String](input, tokenize _, expected, ordered = false)
  }

  // This is the sample function we are testing
  def tokenize(f: DStream[String]): DStream[String] = {
    f.flatMap(_.split(" "))
  }
```

Alternatively, you can look at the base classes defined in spark-testing-base to see which internals it uses (overriding the clock, a unique input stream, etc.) and implement your own version.

Structured Streaming

Structured Streaming tests are a bit easier to write than DStream tests, with built-in tools to process all of the records that have been queued up without you needing to play with a manual clock. A sample Structured Streaming test is shown in Example 11-5.

Example 11-5. Structured Streaming test

```
  test("file JSON source: sequential writes close windows via watermark (append
mode)") {
    withSpark { spark =>
      import spark.implicits._

      val inputDir = Files.createTempDirectory("json-input-it").toFile.getAbsolute
Path
      val chkDir   = Files.createTempDirectory("chk-it").toFile.getAbsolutePath
      val qName    = "json_winagg_mem_it"

      // Start the stream FIRST, using a periodic trigger and a watermark
      val q = JsonWindowedAggExample.makeQueryWith(
        spark,
        inputPath = inputDir,
        checkpointDir = chkDir,
        outputFormat = "memory",                        // assertable sink
        queryName = Some(qName),
        trigger = Trigger.ProcessingTime("250 milliseconds"),
      )
```

```
      // --- Batch 1: events in [10:00,10:10)
      Seq(
        ("2025-01-01 10:01:00", "hello"),
        ("2025-01-01 10:05:00", "hello"),
        ("2025-01-01 10:05:00", "world")
      ).map { case (ts, w) => (Timestamp.valueOf(ts), w) }
        .toDF("timestamp","word")
        .write.mode(SaveMode.Append).json(inputDir)

      // Let the stream pick up batch 1
      q.processAllAvailable() // ok in tests

      val initialCount = spark.table(qName).count()

      // We're running in complete mode, we should see some records.
      assert(initialCount > 0)

      // --- Batch 2: later event at 10:16 moves max event time to 10:16
      // Watermark = maxEventTime - 5m = 10:11 >= 10:10, so [10:00,10:10) closes
and emits.
      Seq(("2025-01-01 10:16:00", "hello"))
        .map { case (ts, w) => (Timestamp.valueOf(ts), w) }
        .toDF("timestamp","word")
        .write.mode(SaveMode.Append).json(inputDir)

      q.processAllAvailable()

      val afterBatch2 = spark.table(qName)
        .select(
          date_format(col("window.start"), "yyyy-MM-dd HH:mm:ss").as("start"),
          date_format(col("window.end"),   "yyyy-MM-dd HH:mm:ss").as("end"),
          col("word"),
          col("count")
        )
        .collect()
        .map(r => (r.getString(0), r.getString(1), r.getString(2), r.getLong(3)))
        .toSet

      val expectedAfterBatch2 = Set(
        ("2025-01-01 10:00:00", "2025-01-01 10:10:00", "hello", 2L),
        ("2025-01-01 10:00:00", "2025-01-01 10:10:00", "world", 1L),
        ("2025-01-01 10:10:00", "2025-01-01 10:20:00", "hello", 1L)
      )
      assert(afterBatch2 == expectedAfterBatch2)

      // --- Batch 3: event at 10:26 closes [10:10,10:20)
      // New watermark = 10:21 >= 10:20 ⇒ the second window can now emit.
      Seq(("2025-01-01 10:26:00", "noop"))
        .map { case (ts, w) => (Timestamp.valueOf(ts), w) }
        .toDF("timestamp","word")
        .write.mode(SaveMode.Append).json(inputDir)
```

```
    q.processAllAvailable()

    val finalOut = spark.table(qName)
      .select(
        date_format(col("window.start"), "yyyy-MM-dd HH:mm:ss").as("start"),
        date_format(col("window.end"),   "yyyy-MM-dd HH:mm:ss").as("end"),
        col("word"),
        col("count")
      )
      .collect()
      .map(r => (r.getString(0), r.getString(1), r.getString(2), r.getLong(3)))
      .toSet

    val expectedFinal = expectedAfterBatch2 ++ Set(
      ("2025-01-01 10:10:00", "2025-01-01 10:20:00", "hello", 1L),
      ("2025-01-01 10:20:00", "2025-01-01 10:30:00", "noop", 1),
    )
    assert(finalOut == expectedFinal)

    q.stop()
  }
}
```

Testing DataFrames

The primary additional complexity when testing with `DataFrames` is figuring out when our result matches our expected value. At its simplest, if we only have a few columns we care about, we can just extract the elements and compare them. This can quickly get tedious with complex `DataFrames`, so we can also compare against expected rows (Example 11-6).

Example 11-6. Check DataFrame for row equality

```
test("verify exact equality") {
  // test minHappyPandas
  val inputDF = spark.createDataFrame(pandaInfoList)
  val result = HappyPandas.minHappyPandas(inputDF, 2)
  val resultRows = result.collect()

  val expectedRows = List(Row(sandiego, "red", 2, 3))
  assert(expectedRows === resultRows)
}
```



Row equality works most of the time, but with `ByteArrays`, simple Scala equality is insufficient. In that case, the values must be compared.

spark-testing-base has similar functions and base classes available for Python and Spark 3.5 added very similar testing to Spark itself. You can write DataFrame tests in Python with spark-testing-base as shown in Example 11-7.

Example 11-7. Check DataFrame for row equality in Python with spark-testing-base

```
from sparktestingbase.sqltestcase import SQLTestCase
from pyspark.sql.functions import current_timestamp
from pyspark.sql.types import Row
from .dual_write import DualWriteExample


class DualWriteTest(SQLTestCase):
    def test_always_passes(self):
        self.assertTrue(True)

    def test_actual_dual_write(self):
        tempdir = tempfile.mkdtemp()
        p1 = os.path.join(tempdir, "data1")
        p2 = os.path.join(tempdir, "data2")
        df = self.sqlCtx.createDataFrame([Row("timbit"), Row("farted")], ["names"])
        combined = df.withColumn("times", current_timestamp())
        DualWriteExample().do_write(combined, p1, p2)
        df1 = self.sqlCtx.read.format("parquet").load(p1)
        df2 = self.sqlCtx.read.format("parquet").load(p2)
        self.assertDataFrameEqual(df2.select("times"), df1, 0.1)
```

Example 11-8 shows how to do this using Spark's new built-in functions.[4]

Example 11-8. Check DataFrame for row equality in Python with built-ins

```
import unittest
from pyspark.sql import SparkSession
from pyspark.sql.functions import current_timestamp
from pyspark.sql.types import Row
from pyspark.testing.utils import assertDataFrameEqual
from .dual_write import DualWriteExample


class DualWriteTest(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.spark = SparkSession.builder.appName(
            "Testing PySpark Example"
        ).getOrCreate()

    @classmethod
```

4 For now the base classes that set up and tear down Spark are not exposed.

```
    def tearDownClass(cls):
        cls.spark.stop()

    def test_always_passes(self):
        self.assertTrue(True)

    def test_actual_dual_write(self):
        tempdir = tempfile.mkdtemp()
        p1 = os.path.join(tempdir, "data1")
        p2 = os.path.join(tempdir, "data2")
        df = self.spark.createDataFrame([Row("timbit"), Row("farted")], ["names"])
        combined = df.withColumn("times", current_timestamp())
        DualWriteExample().do_write(combined, p1, p2)
        df1 = self.spark.read.format("parquet").load(p1)
        df2 = self.spark.read.format("parquet").load(p2)
        assertDataFrameEqual(df2.select("times"), df1, 0.1)
```

As you can see, Spark's built-ins are very similar to spark-testing-base, but require a bit more work with setting up/tearing down your `SparkSession` since it does not expose the test base classes.

Since `DataFrames` often have floating-point data in them, pure equality may not be enough. We can use `approxEqualDataFrames` from spark-testing-base, which compares two `DataFrames` for approximate equality (Example 11-9), or compare individual elements as in Example 11-10.

Example 11-9. Compare two DataFrames for approximate equality

```
test("verify simple happy pandas Percentage") {
  val expectedList = List(Row(toronto, 0.5),
    Row(sandiego, 2/3.0),
    Row(virginia, 1/10.0))
  val expectedDf = createDF(expectedList, ("place", StringType),
                                          ("percentHappy", DoubleType))

  val inputDF = spark.createDataFrame(pandaInfoList)
  val resultDF = HappyPandas.happyPandasPercentage(inputDF)

  assertDataFrameApproximateEquals(expectedDf, resultDF, 1E-5)
}
```

Example 11-10. Approximate equality using Scalatest +– Matcher on a row element

```
  assert(expectedRows.length === resultRows.length)
  expectedRows.zip(resultRows).foreach{case (r1, r2) =>
    assert(r1(0) === r2(0))
    assert(r1.getDouble(1) === (r2.getDouble(1) +- 0.001))
  }
```

Besides the different steps for equality checking, we also need to make the `SparkSession` for our test suite. We can either construct the `SparkSession` in `beforeAll`, as we did with the `SparkContext` in Example 11-3 or use the `DataFrameSuiteBase` from spark-testing-base.

Testing Codegen

While Spark does not directly expose the codegen APIs for "regular" developers, often those seeking high performance with Spark Datasets/DataFrames will write custom expressions instead of the regular user-defined function interface. To make sure that your codegen and non-codegen code paths both function, the `ScalaDataFrameSuiteBase` has `testCombined`, which can be used in place of the regular `test` entry point and will test the same code with codegen forced on and off. You must set the `SPARK_TESTING` environment variable to true when executing the tests since, otherwise, Spark does not let you control when codegen is enabled on a per-function basis.



The spark-testing-base library also has `testNonCodegen` and `testCodegenOnly` if you expect differences between the two code paths and want to ensure they are present.

Getting Test Data

So far, we've focused on using small sample test data that you create by hand. This can work well for basic unit testing, but many bugs in Spark require larger datasets to uncover. Without enough (varied) data you won't be able to discover unbalanced partitioning, incorrect handling of empty partitions, and other issues.

An excellent test data source is sampling production data, although this is often not possible due to legal and privacy concerns. Sampled production data is, in many ways, the gold standard; you don't have to worry about generating data that isn't representative.

When you need to test on datasets too large to generate or store on a single machine, Spark's MLlib provides customizable distributed random RDD generators. With these generators, it's useful to try and at least get a distribution of your real production data so that any issues related to skew will be properly discovered during testing.

In addition to explicitly generating large datasets, another technique is property-based checkers, where you specify invariants about your code and delegate test input generation to the property checker. If well constructed, these property checkers can

use Spark's built-in tools for random generation to generate truly large-scale data for testing.

Generating Large Datasets

When tracking down performance issues, you will often require large datasets for testing. Since problems often arise with key or data skew, it's important to understand the distribution of the data you are looking to generate, as discussed in Chapter 8.

Thinking back to our Goldilocks example, if we had our porridge organized by zip code (or any market data by zip code), we would end up with some zip codes having a large amount of information relative to the others.

Spark has components for generating random RDDs in the `RandomRDDs` object in `mllib`. There are built-in generator functions for exponential, gamma, logNormal, normal, Poisson, and uniform distributions as both RDDs of doubles and RDDs of vectors. If you have data for multiple distributions (as we do in Goldilocks), you can generate the different components and zip them together as in Example 11-11. For more complex data types, or for different distributions, we can implement our own element generator by extending `RandomDataGenerator`.



New versions of spark-testing-base use `RandomRDDs` as their base to support generating datasets too large for your local machine. Some other property-checking libraries do not yet support this.

Example 11-11. Generate some performance scale data for our Goldilocks example

```
/** Generate a Goldilocks data set all with the same id. We expect the zip
  * code to follow an exponential distribution and the data its self to be
  * normal. Simplified to avoid a 3-way zip.
  *
  * Note: May generate less than number of requested rows due to different
  * distribution between partitions and zip being computed per partition.
  */
def generateGoldilocks(
    sc: SparkContext,
    rows: Long,
    numCols: Int
): RDD[RawPanda] = {
  val zipRDD = RandomRDDs
    .exponentialRDD(sc, mean = 1000, size = rows)
    .map(_.toInt.toString)
  val valuesRDD =
    RandomRDDs.normalVectorRDD(sc, numRows = rows, numCols = numCols)
  zipRDD.zip(valuesRDD).map { case (z, v) =>
    RawPanda(1, z, "giant", v(0) > 0.5, v.toArray)
```

```
  }
}
```

The spark-sql-perf project, from Databricks, provides another example of generating a large, scaled-out dataset for testing. If it's easier to specify with ScalaCheck, you can also make a dummy test to save the data out and use it later.

Sampling

If it's available to you, sampling your production data can be a great source of test data. Spark's core RDD and DataFrame/Datasets support customizable random samples. When our work depends on joins between multiple tables, it is important to join from our sample table to ensure we have all the necessary records.

The simplest method for sampling, directly on the RDD and Dataset/DataFrame classes, is the function `sample`, which takes `withReplacement: Boolean, fraction: Double, seed: Long (optional)` (Example 11-12). This allows us to specify the percentage of data we want and creates a sample without any special considerations.

Example 11-12. Basic RDD sample

```
rdd.sample(withReplacement = false, fraction = 0.1)
```



The size of the result of `sample` is specified as a fraction of the input: if you need to upsample, the fraction can be set to >1 if `withReplacement` is enabled.

Under the hood, depending on if you sample with replacement or not, `sample` constructs a `PartitionwiseSampledRDD` with two different samplers (Poisson or Bernoulli). If you need more control, you can directly construct a `PartitionwiseSampledRDD` with your own sampler, provided it implements the `RandomSampler` trait from `org.apache.spark.util.random`.



Sampling has other uses, too, especially in machine learning. Sometimes it's useful to have a sample from X (e.g., `s` is some elements of X) and its inverse (`i` is the set of X that is not in `s`), and in that case, manually constructing a `PartitionwiseSampledRDD` will allow you to use the `cloneComplement()` function on your sampler to construct an inverse `PartitionwiseSampledRDD`.

When working with data of multiple classes, it's often important to ensure that you have some representation from each class. On Datasets/DataFrames the `sampleBy`

function is a part of the statistical functions and can be called with `df.stat.sampleBy` and takes the column name to treat as the key and a map of the target percentages. Similarly, on RDDs that have a key `sampleByKeyExact` and `sampleByKey` take in a map of the percentage for each key to keep allowing you to perform stratified sampling (Example 11-13). Depending on your needs, your sample could be done to try to represent the key distribution in the input or population set, or to try to create a balanced sample of an unbalanced population (HAM/SPAM), or to look at some percentage of users from each locale to validate your handling of different locales.

Example 11-13. Stratified sample

```
// 5% of the red pandas, and 50% of the giant pandas
val stratas = Map("red" -> 0.05, "giant" -> 0.50)
rdd.sampleByKey(withReplacement = false, fractions = stratas)
```

Sometimes you will want to go beyond sampling, returning a single RDD, and sample into many different RDDs at a time. The `randomSplit` function takes an array of weights and returns an array of RDDs with elements proportional to those weights.

Property Checking with ScalaCheck

ScalaCheck is a property-based testing library for Scala similar to Haskell's QuickCheck. Property-based testing allows you to specify invariants about your code (for example, all of the outputs should have the substring "panda") and lets the testing library generate different types of test input for you. Two libraries, sscheck and spark-testing-base, implement generators for Spark. A property-based check with spark-testing-base is shown in Example 11-14.

Example 11-14. ScalaCheck property-based test example

```
// A trivial property that the map doesn't change the number of elements
test("map should not change number of elements") {
  val property =
    forAll(RDDGenerator.genRDD[String](sc)(Arbitrary.arbitrary[String])) {
      rdd => rdd.map(_.length).count() == rdd.count()
    }

  check(property)
}
```

ScalaCheck will automatically generate a number of common edge conditions, but we can also specify inputs that we think are likely to cause trouble. This is the technique used by the generator itself to create RDDs with varying partition sizes, with a bias toward creating some empty partitions.

Computing RDD Difference

Bringing data back locally works really well if your test data is small enough to collect back to a local machine. If you are running your tests on a cluster and have a larger dataset, you instead need to compute the difference between the RDDs directly —without bringing the full data back to the driver.

You can do most of your unit testing with just `parallelize` and `collect`; however, RDD equality, like general collection equality, has a few different possibilities. We will examine the two cases where order matters and the order does not matter.

Let's start with comparing two RDDs where we expect the order to be the same (Example 11-15). This would be the case if we called `sortByKey`, as we do in Goldilocks after counting the unique values. If the partitioner is the same, we can `zip` the two RDDs together and compare the elements directly without doing a shuffle. Otherwise, we just repartition one of the RDDs to match the other.

Example 11-15. Comparing RDDs with order

```
/**
 * Asserts two RDDs are equal (with the same order).
 * If they are equal assertion succeeds, otherwise assertion fails.
 */
def assertRDDEqualsWithOrder[T: ClassTag](
  expected: RDD[T], result: RDD[T]): Unit = {
  assertTrue(compareRDDWithOrder(expected, result).isEmpty)
}

/**
 * Compare two RDDs with order (e.g. [1,2,3] != [3,2,1])
 * If the partitioners are not the same this requires multiple passes
 * on the input.
 * If they are equal returns None, otherwise returns Some with the first mismatch.
 * If the lengths are not equal, one of the two components may be None.
 */
def compareRDDWithOrder[T: ClassTag](
  expected: RDD[T], result: RDD[T]): Option[(Option[T], Option[T])] = {
  // If there is a known partitioner just zip
  if (result.partitioner.map(_ == expected.partitioner.get).getOrElse(false)) {
    compareRDDWithOrderSamePartitioner(expected, result)
  } else {
    // Otherwise index every element
    def indexRDD[T](rdd: RDD[T]): RDD[(Long, T)] = {
      rdd.zipWithIndex.map { case (x, y) => (y, x) }
    }
    val indexedExpected = indexRDD(expected)
    val indexedResult = indexRDD(result)
    indexedExpected.cogroup(indexedResult).filter { case (_, (i1, i2)) =>
      i1.isEmpty || i2.isEmpty || i1.head != i2.head
```

```
    }.take(1).headOption.
      map { case (_, (i1, i2)) =>
        (i1.headOption, i2.headOption) }.take(1).headOption
  }
}

/**
 * Compare two RDDs. If they are equal returns None, otherwise
 * returns Some with the first mismatch. Assumes we have the same partitioner.
 */
def compareRDDWithOrderSamePartitioner[T: ClassTag](
  expected: RDD[T], result: RDD[T]): Option[(Option[T], Option[T])] = {
  // Handle mismatched lengths by converting into options and padding with Nones
  expected.zipPartitions(result) {
    (thisIter, otherIter) =>
      new Iterator[(Option[T], Option[T])] {
        def hasNext: Boolean = (thisIter.hasNext || otherIter.hasNext)

        def next(): (Option[T], Option[T]) = {
          (thisIter.hasNext, otherIter.hasNext) match {
            case (false, true) => (Option.empty[T], Some(otherIter.next()))
            case (true, false) => (Some(thisIter.next()), Option.empty[T])
            case (true, true) => (Some(thisIter.next()), Some(otherIter.next()))
            case _ => throw new Exception("next called when elements consumed")
          }
        }
      }
  }.filter { case (v1, v2) => v1 != v2 }.take(1).headOption
}
```

If order does not matter and the equality operator is sufficient, we can `cogroup` the two RDDs (along with dummy values), as shown in Example 11-16.

Example 11-16. Comparing RDDs without order

```
/**
 * Asserts two RDDs are equal (un ordered).
 * If they are equal assertion succeeds, otherwise assertion fails.
 */
def assertRDDEquals[T: ClassTag](expected: RDD[T], result: RDD[T]): Unit = {
  assertTrue(compareRDD(expected, result).isEmpty)
}

/**
 * Compare two RDDs where we do not require the order to be equal.
 * If they are equal returns None, otherwise returns Some with the first mismatch.
 *
 * @return None if the two RDDs are equal, or Some containing
 *              the first mismatch information.
 *              The mismatch information will be Tuple3 of:
 *              (key, number of times this key occur in expected RDD,
```

```
   *             number of times this key occur in result RDD)
   */
  def compareRDD[T: ClassTag](expected: RDD[T], result: RDD[T]):
      Option[(T, Int, Int)] = {
    // Key the values and count the number of each unique element
    val expectedKeyed = expected.map(x => (x, 1)).reduceByKey(_ + _)
    val resultKeyed = result.map(x => (x, 1)).reduceByKey(_ + _)
    // Group them together and filter for difference
    expectedKeyed.cogroup(resultKeyed).filter { case (_, (i1, i2)) =>
      i1.isEmpty || i2.isEmpty || i1.head != i2.head
    }
      .take(1).headOption.
      map { case (v, (i1, i2)) =>
        (v, i1.headOption.getOrElse(0), i2.headOption.getOrElse(0)) }
  }
```

These examples may not match exactly how you need to test equality, but you should be able to generalize from them. If, for example, you are comparing RDDs of doubles, you might sort them and then use a similar function like `compareWithOrder` but check within tolerance rather than exact equality.



The same general approach works for computing `DataFrame` difference and is implemented in spark-testing-base through `equalDataFrames` and `approxEqualDataFrames`.

Integration Testing

Once you've run up against the limits of unit testing, it's time to explore integration testing. Sometimes simply faking data sources and mocking isn't enough (or becomes too complicated) to test your system. Integration testing tends to be slower than unit testing, and the failures tend to take more time to debug, but it catches an entirely different class of bugs. In this section, you will see the different trade-offs for integration environments. Integration testing can also be combined with some of our performance testing methods mentioned later to give us an idea of the entire system's performance.

Choosing Your Integration Testing Environment

At some point we will reach the limit of what we can test with our unit tests. In large-scale data systems, errors in data ingestion or misunderstandings of formats can be a common source of errors, so having some tests of the integration of your systems are important—albeit more challenging.

Production or staging clusters

Perhaps the "simplest" (but also scariest) option is reusing your production environment for integration testing. This requires careful work to ensure that your integration testing jobs do not write to "real" output tables or services and read from the desired inputs. On the one hand, it can be incredibly quick to set up, but when things go wrong, it can have significant unintended outcomes (say, the cleanup step of an integration test dropping all of the user's recommendations). This can be a great in-between if your company has a dedicated "staging" environment.

Local mode

While we've looked at using local mode mostly in the context of unit testing (see "Unit Testing" on page 295), for small enough projects, we can use many of the same techniques. Instead of mocking our data sources and using functions like `parallelize` and `queueStream`, we can set up smaller versions of our data stores and stream sources.

Kubernetes and Docker-based

Docker containers provide an easy way to package and distribute multiple virtual lightweight containers, which can vastly simplify automated deployments. This is especially useful for integration testing with distributed systems, like Spark, which require both workers and master node and whatever services your data requires (e.g., HDFS, Mongo, etc.). Several sample projects exist for setting up Docker-based Spark integration environments, including the spark-perf project from Databricks.

Various tools, like minikube, k3s, and microk8s, allow you to quickly set up and tear down a dedicated Kubernetes environment for testing. This is especially useful when working with a company that is using Kubernetes in production, although it's essential to make sure that the user does not have permission to modify the actual production cluster should something go wrong with the environment setup.

Verifying Performance

For many, the primary motivation for using Spark (and reading this book) is the ability to have highly performant processing of big data. Since this is an important motivation behind many people's use of Spark, verifying that your specific use of Spark is performing as expected is important.

Spark Counters for Verifying Performance

Spark tracks a number of counters during our job, many of which can be useful for debugging performance. You have access to many of the counters in the WebUI and can get programmatic access to them by registering a `SparkListener` to collect the

information. Spark uses callbacks to provide the metrics, and for performance info we can get most of what we need through `onTaskEnd`, for which Spark gives us a `SparkListenerTaskEnd`. We can get the bytes read, execution time, records read, and many other important metrics this way (Example 11-17).

Example 11-17. Simple `SparkListener` for execution time

```
class PerfListener extends SparkListener {
  var totalExecutorRunTime = 0L
  var jvmGCTime = 0L
  var recordsRead = 0L
  var recordsWritten = 0L
  var resultSerializationTime = 0L

  /**
   * Called when a task ends
   */
  override def onTaskEnd(taskEnd: SparkListenerTaskEnd): Unit = {
    val info = taskEnd.taskInfo
    val metrics = taskEnd.taskMetrics
    updateMetricsForTask(metrics)
  }

  private def updateMetricsForTask(metrics: TaskMetrics): Unit = {
    totalExecutorRunTime += metrics.executorRunTime
    jvmGCTime += metrics.jvmGCTime
    resultSerializationTime += metrics.resultSerializationTime
    recordsRead += metrics.inputMetrics.recordsRead
    recordsWritten += metrics.outputMetrics.recordsWritten
  }
}
```

Using this listener we can quickly validate the overall execution time (Example 11-18).

Example 11-18. Sample execution time test

```
test("wordcount perf") {
  val listener = new PerfListener()
  sc.addSparkListener(listener)
  doWork(sc)
  println(listener)
  assert(listener.totalExecutorRunTime > 0)
  assert(listener.totalExecutorRunTime < 10000)
}
```

Projects for Verifying Performance

There are two common projects used for verifying the performance of Spark jobs. The spark-perf package, from Databricks, is designed for comparing different versions of Spark but can also be extended to do performance testing of our own code. Spark-perf is written in Python but is primarily used for performance testing Scala code. The Testing Spark: Best Practices talk from Spark Summit 2014 suggests using Gatling along with the Spark Jobserver, which allows you to expose Spark jobs as a REST service for performance testing.

Validation (or Audits)

Job validation is an important part of ensuring that the results of your Spark job match your expectations before using the results. The most important artifact of your Spark job is normally the data it produces, so it makes sense to start by validating that the output matches your expectations (like size, distribution, etc.). In addition, accumulators can keep track of information important to your job, such as the number of valid and invalid records or the number of users for whom you have no recommendations.

Data Validation

Most Spark jobs are executed to consume some large collection of inputs and produce data in a table-like output. In other jobs, like machine learning, there is often a step before the model is trained where the input is effectively a table (even if it is not explicitly materialized). Streaming jobs break this mold a little bit, but you can also view their output as table-like by picturing a continuously appending output table. When the output of a Spark job is something that can be analyzed (like a table), it makes sense to make sure the output matches our expectations. It's also good practice to validate that the inputs to our job are as we expect.



If you find yourself saying, "I can't query X to see if it's correct without putting it in front of users," you can take many of the same techniques and apply them to the inputs to X.

You can use a few different tools for data validation, including Nike's Spark-Expectations, Soda, SparkConstraints, PyDeequ, AWS's Deequ, and Pandera. In this book, we focus on Spark-Expectations, Pandera, and Target's data validation packages. You don't need to limit yourself to the kinds of validation rules present in the libraries; you can also write entire jobs on your own. Pandera takes the approach of being a part of your Spark job, and Target's Data Validator is a separate program with a small domain-specific language (DSL) for specifying validation rules. Nike's Spark-Expectations combines both approaches, being intended to run as part of your Spark job—but with the rules being stored in an external database. You can, of course, write a separate validation job with Pandera or Spark-Expectations.



Pandera has multiple backends, and the schema definition between them is not compatible. For PySpark import, it is important that you use the DataFrameModel from `pandera.pyspark`. Otherwise, you may get an error like `pandera.errors.BackendNotFoundError: Backend not found for backend` and use types from `pyspark.sql.types`. Pandera support is limited with most complex things like hypothesis checking missing and some simpler things like error row counting.

Data validation can be thought of in a few different ways:

Row-based
: All or most records should satisfy some set of conditions

Schema-based
: Data types should match

Summary statistics
: The distribution of the data overall should be like this, or these two columns should be correlated

Historic
: Today's data should look like yesterday's data

Row-based validation is probably what most people think of when they think of data validation. Each of the tools we've mentioned so far has its own way of expressing row-based validation.

In Pandera, the row-based validation is combined with schema validation, as shown in Example 11-19. With Pandera, you can specify one of `eq`, `ne`, `gt`, `lt`, `le`, `in_range`, `is_in`, `notin`, `str_contains`, `str_endswith`, `str_matches`, `str_length`,[5] or `str_startswith`, or `unique_values_eq` to validate a given column. Most of these abbreviations/functions are likely familiar to you from either mathematical operators or set operators. `unique_values_eq` and `str_matches` are perhaps less familiar. `str_matches` checks a string against a regular expression, and `unique_values_eq` ensures that all of the values specified are present instead of just one with `is_in`.

Example 11-19. Validate the UK gender data using Pandera (only fields we care about)

```
import pandera.pyspark as pa
import pyspark.sql.types as T

class GenderData(pa.DataFrameModel):
    MaleBonusPercent: T.DoubleType() = pa.Field(nullable=True, le=5)
    FemaleBonusPercent: T.DoubleType() = pa.Field(nullable=True)
    CompanyNumber: T.IntegerType() = pa.Field()


    validated_df = GenderData(uk_df)
    # Print out the errors. You may wish to exit with an error condition.
    if validated_df.pandera.errors != {}:
        print(validated_df.pandera.errors)
        # sys.exit(1)
```

While we like Pandera's API, it does not yet have the option in Spark to drop the offending rows nor count them. Since at scale we often care about the percentage of errors, we think Pandera is likely better suited to postprocessing rather than input validation.

Target's Data Validator tool has a more complex set of operators, including things like distributions, ranges, etc. The DSL is complex enough, and the package is not frequently updated. We won't go into detail here, but we do show an example in Example 11-20.

Example 11-20. Target's Data Validator

```
detailedErrors: true
numKeyCols: 4
# We might have a large number of errors so just show the first 5
numErrorsToReport: 5

email:
```

5 At the time of writing, the `str_length` function is broken.

```
  smtpHost: smtp.example.com
  subject: Data Validation Summary
  from: data-validator-no-reply@example.com
  to:
    - professor-timbit@example.com

tables:
  - db: gender_paygaps
    table: uk
    # Columns that taken together uniquely specifies each row (think of groupBy)
    keyColumns:
      - CompanyNumber
      - EmployerId
      - CompanyLinkToGPGInfo
      - ResponsiblePerson
    # Used to filter
    condition: MaleBonusPercent >= FemaleBonusPercent
    checks:
      # We expect at least 500 records
      - type: rowCount
        minNumRows: 500
      # We don't expect more than 1% not companies in the dataset.
      - type: nullCheck
        column: CompanyNumber
        threshold: 0.01
```

Spark-Expectations is slightly different from the other validation libraries in a few key ways. From a developer's point of view, the rules are stored in a database, which makes rules more accessible to nondevelopers but lacks version control. It also integrates with various options for alerting, rather than depending on your workflow management system to trigger alerts, and as such, involves some extra setup as shown in Example 11-21.

Example 11-21. Spark-Expectations setup

```
se_conf = {
    "se_notifications_enable_email": False,
    "se_notifications_email_smtp_host": "mailhost.example.com",
    "se_notifications_email_smtp_port": 25,
    "se_notifications_email_from": "timbit@example.com",
    "se_notifications_email_subject": "spark expectations - data quality - notifica
tions",
    "se_notifications_on_fail": True,
    "se_notifications_on_error_drop_exceeds_threshold_breach": True,
    "se_notifications_on_error_drop_threshold": 15,
}
```

A simple workaround for the lack of version control is having a JSON/CSV with the rules loaded at the start, as shown in Example 11-22.

Example 11-22. Spark-Expectations load rules from JSON

```
from spark_expectations.config.user_config import Constants as user_config

spark.sql("DROP TABLE IF EXISTS local.magic_validation")
spark.sql("""
create table local.magic_validation (
    product_id STRING,
    table_name STRING,
    rule_type STRING,
    rule STRING,
    column_name STRING,
    expectation STRING,
    action_if_failed STRING,
    tag STRING,
    description STRING,
    enable_for_source_dq_validation BOOLEAN,
    enable_for_target_dq_validation BOOLEAN,
    is_active BOOLEAN,
    enable_error_drop_alert BOOLEAN,
    error_drop_threshold INT
)""")
# Reminder: addFile does not handle directories well.
rule_file = "spark_expectations_sample_rules.json"
sc.addFile(rule_file)
df = spark.read.json(SparkFiles.get(rule_file))
print(df)
df.write.option("byname", "true").mode("append").saveAsTable("local.magic_valida
tion")
spark.read.table("local.magic_validation").show()

# Can be used to point to your desired metastore.
se_writer = WrappedDataFrameWriter().mode("append").format("iceberg")

rule_df = spark.sql("select * from local.magic_validation")

se: SparkExpectations = SparkExpectations(
    rules_df=rule_df,  # See if we can replace this with the DF we wrote out.
    product_id="pay",  # We will only apply rules matching this product id
    stats_table="local.dq_stats",
    stats_table_writer=se_writer,
    target_and_error_table_writer=se_writer,
    stats_streaming_options={user_config.se_enable_streaming: False},
)
```

Spark-Expectations has a few different types of rules: `row_dq`, `agg_dq`, and `query_dq`. `row_dq` rules check if each row matches your specified expectation. A simple example is shown in Example 11-23. Similarly, `agg_dq` applies your specified aggregate to the DataFrame, and `query_dq` will run a raw query. The Spark-Expectations library has a large variety of example rules, including both row-based and aggregate rules.

Example 11-23. Spark-Expectations sample rule

```
{"product_id": "pay", "table_name": "local.fake_table_name", "rule_type": "row_dq",
"rule": "bonus_checker", "column_name": "MaleBonusPercent", "expectation": "MaleBo
nusPercent > FemaleBonusPercent", "action_if_failed": "drop", "tag": "", "descrip
tion": "Sample rule that the male bonuses should be higher. Thankfully
this fails (but could be lower base pay etc.)", "enable_for_source_dq_val
idation": true, "enable_for_target_dq_validation": false, "is_active": true,
"enable_error_drop_alert": true, "error_drop_threshold": 1}
```

Spark-Expectations work as a decorator in that you apply functions that generate DataFrames, as shown in Example 11-24.

Example 11-24. Integrating Spark-Expectations into your job

```
@se.with_expectations(
    user_conf=se_conf,
    write_to_table=False,  # If set to true SE will write to the target table.
    target_and_error_table_writer=se_writer,
    # target_table is used to create the error table (e.g. here
local.fake_table_name_error)
    # and filter the rules on top of the global product filter.
    target_table="local.fake_table_name",
)
def load_data():
    raw_df = spark.read.csv("data/fetched/2021", header=True, inferSchema=True)
    uk_df = raw_df.select("CompanyNumber", "MaleBonusPercent", "FemaleBonuspercent")
    return uk_df


# data = load_data()
```

You can apply Spark-Expectations to any job stage, but we recommend most rules at load time and right before write time.

Summary statistics are most often used for catching unexpected changes to datasets without having to think about each column explicitly. However, you can go beyond this, like using them to catch when data you expect to be correlated loses its relation

(for example, gender and compensation are expected to be correlated fields).[6] These unexpected changes in inputs or outputs are super useful catches. Sometimes, instead of removing a field (which would be caught with a schema check), the value of a field is no longer populated.[7]



Spark-Expectations and Target's Data Validator can automatically output summary statistics. You can also write summary stats rules with Spark-Expectations.

Schema-based validation may already be (partially) done by Spark itself depending on how you configure your job. While it's normal to specify types when loading JSON or CSV data, it's less common when loading data from a strongly typed data source. If your job does not specify the expected schema of its input tables, you should create another job that checks the schema. `columnNameOfCorruptRecord` allows you to quickly check how many columns don't meet your desired schema.

Historical comparisons can go beyond our point-in-time approach so far and can catch changes that we might not have foreseen when writing manual rules. Even with manual rules, we often expect today to be like yesterday (or the same period in the previous quarter), so writing rules that take advantage of your previous data can capture more subtle shifts. Historical comparisons are often combined with summary statistics, where we say our new snapshot should look "somewhat similar" to the previous period. Currently, the open source tools for Spark data validation do not have as much in the way of historic data validation as tools like TensorFlow Data Validation (TFDV), but additional historic rules are on the roadmap for Spark-Expectations, so keep a close eye on them. For now, it's up to you to write your own comparisons or see if one of the other tools in your system supports it (like Lakeflow's drift metrics).

6 One day this might not be true, and that's OK—you can manually pass the pipeline once we remove the gender pay gap.

7 We have many funny stories of this, which we wrote but cannot publish. We're taking WAP to heart.

A Spark-Expectations rule using the history table is shown in Example 11-25.

Example 11-25. Spark-Expectations rule with history table

```
{
  "product_id": "pay",
  "table_name": "local.3rd_fake",
  "rule_type": "row_dq",
  "rule": "bonus_checker",
  "column_name": "MaleBonusPercent",
  "expectation": "MaleBonusPercent > FemaleBonusPercent",
  "action_if_failed": "drop",
  "tag": "",
  "description": "Sample rule that the male bonuses should be higher. Thankfully
this fails (but could be lower base pay etc.)",
  "enable_for_source_dq_validation": true,
  "enable_for_target_dq_validation": false,
  "is_active": true,
  "enable_error_drop_alert": true,
  "error_drop_threshold": 1
}
{
  "product_id": "pay",
  "table_name": "local.3rd_fake",
  "rule_type": "query_dq",
  "rule": "history",
  "column_name": "MaleBonusPercent",
  "expectation": "(select count(*) from 3rd_fake_view) > (select input_count from
local.dq_stats WHERE table_name='local.3rd_fake' LIMIT 1)",
  "action_if_failed": "fail",
  "tag": "",
  "description": "We should always have more records than before",
  "enable_for_source_dq_validation": false,
  "enable_for_target_dq_validation": true,
  "is_active": true,
  "enable_error_drop_alert": true,
  "error_drop_threshold": 1
}
```

All of these validations can be applied to both your inputs and outputs. Applying these validation tools to "full" sets of your data can become expensive; you can narrow down the records to just those changed using your data catalogs time travel/CDC type feature.



It is important to remember that you can always just write your own Spark (or PySpark or SQL) job to validate your data if the validation tools don't meet your needs.

Input tables

Validating and auditing the inputs is more straightforward than the outputs, as it does not depend on intercepting the write path of your Spark job. You likely already do some implicit validation of your inputs (like filtering out nulls)—but we can (and should) go beyond this.

Output tables and WAP

Validating the output from a Spark job is more complicated, as we want to make sure we validate the data before it is consumed by someone else. If we are the only consumers of the table, that's relatively easy (just add a step in our workflow and halt if we encounter an error). Or if you can perform your validation prior to writing out to the table, like with Spark-Expectations, no special handling is required.

In the more general case, the WAP pattern allows you to stage your writes without them being visible to (therefore consumed by) downstream. The WAP pattern was popularized by Michelle Winters from Netflix in her talk "Whoops, the Numbers Are Wrong!".



In our experience, once you make data available (even if you just meant it to be for yourself), others often start depending on it.

The WAP pattern is implemented as part of Apache Iceberg and is also available through LakeFS on top of other data stores (like Hudi, Delta Lake, etc.). With Iceberg, WAP is set as a table-level property (`write.wap.enabled`) and a Spark property (either by setting `spark.wap.id` or `spark.wap.branch`), which will then stage the data on write. Once you've audited your data, you can replace the current head of the table with the snapshot from your branch or WAP ID. This pattern is shown in SQL in Example 11-26. To make sure that no one else has changed the "head" of the table during your validation, you should use Iceberg's `fast_forward` operator (which was added to its Spark API in Iceberg 1.4).

Example 11-26. Iceberg WAP example in SQL

```
DROP TABLE IF EXISTS local.wap_projects;
CREATE TABLE local.wap_projects (
       creator string,
       projectname string)
USING iceberg
PARTITIONED BY (creator);
ALTER TABLE local.projects SET TBLPROPERTIES (
    'write.wap.enabled''true'
```

```
);
-- We need a first commit, see https://github.com/apache/iceberg/issues/8849
INSERT INTO local.wap_projects VALUES("holdenk", "spark");
ALTER TABLE local.wap_projects DROP BRANCH IF EXISTS `audit-branch`;
ALTER TABLE local.wap_projects CREATE BRANCH `audit-branch`;
SET spark.wap.branch = 'audit-branch';
INSERT INTO local.projects VALUES("krisnova", "aurae");
SELECT count(*) FROM local.wap_projects VERSION AS OF 'audit-branch' WHERE creator
is NULL;
SELECT count(*) FROM local.wap_projects VERSION AS OF 'audit-branch' WHERE creator
== "krisnova";
CALL local.system.remove_orphan_files(table => 'local.wap_projects');
CALL local.system.fast_forward("local.wap_projects", "main", "audit-branch");
```

Accumulators and Built-In Counters

You can use some of the same information we examined in "Spark Counters for Verifying Performance" on page 311 to validate your jobs. The same listener can be used, and the results can be aggregated over all the stages. In general, you likely want to use the same WAP-type pipeline with counter/job metric validation, except instead of validating the data, you validate the metadata of the job before publishing. Loading the information from a previous run of Spark is shown in Example 11-27, and you can then use whichever data validation tool you prefer.

Example 11-27. Load previous runs' info

```
    def load_json_records(self, event_log_path, application_id):
        print(f"Loading {application_id}")
        full_log_path = f"{event_log_path}/{application_id}"
        df = self.session.read.json(full_log_path)
        special_events = df.filter(
            (df["Event"] == "SparkListenerExecutorAdded")
            | (df["Event"] == "SparkListenerJobEnd")
        )
        special_events.show()
```

You can then write whatever validation rules you want in normal Spark on this DataFrame representing a previous run of your job.

Spark's accumulators can also be used for validating jobs and show up in the same metadata. A common use case for accumulators is to track the number of invalid records when processing your data.

Side-by-Side Runs

When you are iterating on the performance of an existing data pipeline, you may have the unique case of having an especially large "source of truth" available in the form of the output from the existing job. To some extent, the existing tests may cover some of this, but you can go even further and run the old and new jobs side by side.



Side-by-side runs tend to be more expensive and less precise, so start with making sure your existing tests pass.

Side-by-side runs can vary greatly in the amount of time (both computer and operator/developer) required to set up and execute. In the simple case (outputting to tables managed by a metastore), the developer time in setting up side-by-side runs can be relatively easy—and in some cases, even automated. For more complicated systems, like those with streaming inputs and outputs, you will need to spend more time to set up copies of the input and output streams for comparison, and if the output goes directly into a system you don't control—this can be on-par with setting up a validation-like environment.

The Spark upgrade project has an interesting proof of concept that uses the WAP pattern to do side-by-side comparisons of pipelines where little to no change in output is expected.

Table-Level Assertions

Spark 4.1 adds the ability to specify table-level assertions, which can capture errors at write time—when the catalog supports it. This is a new and "evolving" API, so it's likely to change between when this book is published (early 2026) and when you read it, so we've skipped any concrete examples. Instead, take a look at `org.apache.spark.sql.connector.catalog.constraints`.

Conclusion

In this chapter, we have seen how to create tests, set up runtime validation, and extend validation to side-by-side comparisons for both our Spark code's functionality and performance. We've covered a technique for writing local tests and comparing RDDs and `DataFrames` when test data is too large. The tests you make using these techniques should let you feel confident as you refactor to apply the performance improvements from the previous chapters.

By adding validation to your Spark jobs, you can catch many errors before your users (or managers) notice. Validation can also improve your confidence in your jobs during upgrades and improvements while helping you sleep through the night. Side-by-side runs will give you the confidence that your new faster version will not result in you being woken up only to find out that it's faster because all of the data was null.

CHAPTER 12
Spark Components and Packages

Spark has many components designed to work together as an integrated system, and many of them are distributed as part of Spark. This differs from much of the rest of the Hadoop ecosystem, which has different projects or systems for each task. You've already seen how to effectively use Spark Core, SQL, Streaming, and ML components. This chapter will look at the projects outside of Spark itself, sometimes called external/community components (often called packages). Having a largely integrated system gives Spark two advantages: it simplifies deployment/cluster management, upgrades, and application development by having fewer dependencies and systems to keep track of.

While Spark is comparatively integrated, there are still times when bringing in outside components is well worth the increased complexity. In this chapter, we'll help you develop a framework for evaluating the trade-offs of bringing in external or third-party components into your Spark package. We'll also share some of our views about Spark's internal packages, as they are not all equally maintained.

Even early versions of Spark provided tools that traditionally would have required the coordination of multiple systems, as illustrated in Figure 12-1.

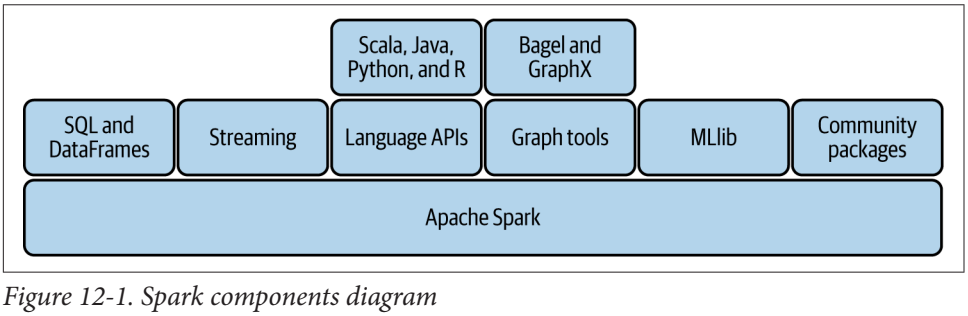


Figure 12-1. Spark components diagram

As Datasets and the Spark SQL engine have become a building block for other components inside of Spark, a minor reorganization illustrated in Figure 12-2 represents a more up-to-date version, including two of Spark's newest components, Spark ML and Structured Streaming. Much of your knowledge from working with core Spark and Spark SQL can be applied to the other components—although there are some unique considerations for each one.

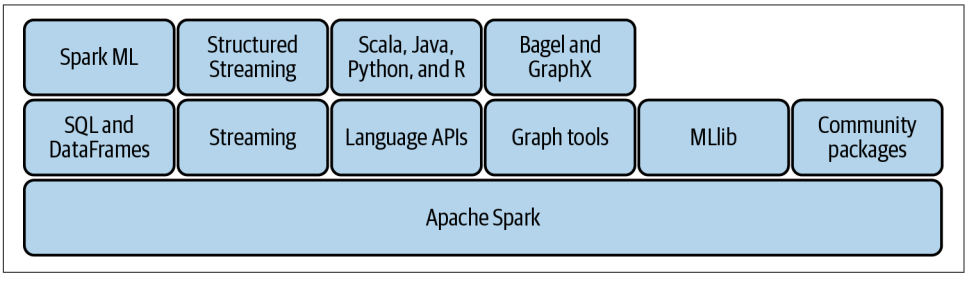


Figure 12-2. Spark 2.0+ revised components diagram



If you haven't had a chance to read Chapter 5 yet, we encourage you to jump back and at least skim it, since, as Figure 12-2 illustrates, much of the new Spark development is happening on top of Spark SQL and DataFrames/Datasets.

Beyond the integrated components, the community packages can add important functionality to Spark, sometimes even superseding built-in functionality—like with GraphFrames. Community packages provide a diverse set of functionality, ranging from additional data formats (mentioned in "Additional formats and tables" on page 98) to the testing packages discussed in Chapter 11, machine learning, and graph algorithms. These community packages can sometimes become a part of Spark if their use is wide enough, as with the spark-csv package for loading CSV data in Datasets. This chapter will wrap up with information about creating your own community package so that we all can benefit from your work—not that you should feel obliged to (but we'd certainly appreciate it).



One less directly visible benefit of this integrated approach is that each component's development can result in improvements across all of the other components. For example, streaming requires a large reduction in task creation overhead. This task overhead reduction improved Spark SQL's ability to support small queries, where the task overhead previously dominated.

GraphX

GraphX is a legacy component in Apache Spark that is no longer being updated. GraphX suffers from a number of significant performance problems, in some cases, performing iterative computation without the required checkpointing to allow Spark to clean up the DAG. As of this writing, the most promising alternative to GraphX is the community package GraphFrames, but it depends on the current Spark GraphX code for much of its functionality. We'll talk more about how to evaluate project health in the next section.

Theoretically, components without active maintenance should be marked deprecated, as with GraphX during Spark 4.0. However, some resistance exists to officially deprecating components, as it is believed to be hard to "come back from" deprecation.

Instead, if you're wondering if an integrated Spark component is actively developed, the simplest way is to look at the package Git log and see how many changes occur compared to the core Spark component. Another option is to email the user@ list (as Jacek Laskowski did for GraphX) and see if anyone responds.

Using Community Packages and Libraries

Beyond the components that ship in Spark, there is a whole host of community packages built for Spark. Some of the ones you used in this book include spark-testing-base and spark-avro. Some other notable Spark packages include ç[GraphFrames], which many people view as the successor to GraphX; ipex-llm for LLM training; and frameless for stronger compile-time type checking in Scala.

Matthew Powers started a list of Spark packages called "awesome-spark" and it is now maintained by a community and a great place to find packages. Not only is it community-maintained, but it also tracks the most recent time something has been committed to the main branch of each project.[1]

Adding a package to your Spark project can accelerate development, but it can also later slow you down when it comes time to upgrade. If a package is not keeping track of the open source Spark releases, you may be faced with the difficult task of waiting for it to upgrade or putting in the work to upgrade (and potentially fork if the maintainer is gone). For packages that are critical to your business, it makes sense to be involved in the maintenance of the package by filing bug reports and PRs when possible. If your company has the budget for it, financially supporting maintainers of projects can be a great way to ensure that the maintainers continue to have time to support the work you depend on.

1 In the first edition of this book, we recommended using spark-packages.org to find public sources, but the majority of packages no longer publish there.

Looking at whether a project is keeping up with new versions of Spark is a good start in determining if a package will be maintained, but some other important factors to consider are:

- How big is the community of developers and users?
- How frequently are there commits to the project?
- When was the last nontrivial change to a project?
- Are the project users aligned with my use case (e.g., if I'm working in aerospace and everyone else is using this software for advertising, the chance that there might be missing bits I care about are higher)?
- Do the developers follow good code practices (are there tests, a linter, etc.)?
- If I have to debug something in the code, can I read and understand it?

These are good questions to ask yourself when bringing in any new dependency, and thankfully, with open source software, it's easier to tell if software is in a "healthy" state than closed source.

Applying these evaluation metrics to graph analytics on Spark, the Neo4j Spark connector appears to be the most actively maintained library in the Spark graph analytics area. Several other community-originated options like Morpheus and SparkGraphs do not appear to be maintained (with some explicitly no longer being maintained).

Similarly, if a package is actively maintained and aligned with your use case, it's important to understand what the intentions are of the author and the scale of data that they are working with. Some companies, for example, more recently Netflix, have started to try and separate out their open source packages into experimental and non-experimental orgs to more clearly set expectations correctly. However, many companies (and even more people) do not always make the distinction between whether software is intended for production use or is more exploratory. Exploratory software can, of course, become production quality with time and collaboration, but taking on exploratory software in a production is a nontrivial amount of work.

There are a few good ways to identify exploratory packages. The first is seeing if there are multiple developers working together on a package. Commit times can also be a good sign. Looking at the issues and seeing what the rate of them being filed and responded to is. If there are not any clear users of the package, you can try reaching out to the developers to ask them if it's experimental or not and about their current deployments. While this is slow, it is a wonderful way to start the conversation, and maybe even get involved.



Spark packages don't undergo any particular scrutiny to be released, so evaluate them yourself before adding them to your project.

Beyond public packages, if your company has internal libraries you wish to include with your Spark application or use in the Spark shell, they can be added with similar mechanisms.

As you start working with a new package, you may want to explore it using the Spark shell. Thankfully, the mechanism for including a package with `spark-submit` works just as easily with the `spark-shell`: you can just add `--packages [maven_coordinates]` to your shell launch.

When you include a Spark package, at launch time the driver will fetch the JARs and its dependencies and automatically distribute them to the workers. For large JARs this can be slow when fetching from Maven Central.



In early versions of Spark, some mechanisms would result in the package only being included on the executor's classpath; in that case, you can manually add them to the driver classpath.

To add packages that you need at compile time to Java/Scala projects, you can simply copy their Maven coordinates and include them with your build. If you're already including other libraries, you may find it helpful to package all your dependencies together in an uber JAR, sometimes called an *assembly jar*. If you want the flexibility to switch your package versions at runtime with `spark-submit`, simply mark them as provided.

When adding packages and libraries, especially those not produced for the Spark ecosystem, such as a general-purpose search library, there is a chance of a "classpath conflict." A long while ago, Holden wrote some code called "user classpath first," which attempts to put JARs from the user ahead of those from the system—but it's known to be slightly buggy. Instead, you can produce shade and relocate the conflicts in your uber JAR to "hide" your dependencies from the rest of the system. Spark is increasing the amount of shading it does internally, so hopefully, the number of these conflicts should reduce over time.

The Spark packages ecosystem is not limited to the JVM. However, it is more common for non-JVM languages (like Python and R) to distribute their libraries with language-specific tools (like `PyPi` and `CRAN`). See Chapter 9 for more information on how to include custom libraries in other languages.

Popular Packages

So that's great, but it's a little abstract—let's take a look at some of the popular packages.

Holden is, of course, more than a little biased, but she thinks that spark-testing-base is a pretty awesome package. Despite how awesome it is, there are a few sort of "warning" type signs as far as packages go—one is that it is largely a single maintainer project (speaking of which, if you want to join ⊏ The other is that, compared to other packages, it can be a long time between commits. In the case of spark-testing-base, this is not such a bad sign as the project is relatively mature and only needs new commits when there are new versions of Apache Spark.

At the time of the writing (March 2026), Koalas was still listed on Awesome Spark but, in practice, has been largely subsumed by Apache Spark's built-in pandas DataFrame package. This illustrates that when looking at a package, you should check and see if you can get the same behavior inside of Spark without having to pick up an external dependency.

The Spark Cassandra connector adds the ability to read and write Cassandra data and is produced by DataStax. Another option exists to directly read from SSTables and is part of the Apache Cassandra project called Cassandra Analytics. Depending on what your goals are, offline analytics from backed-up tables or accessing more recent data, the different connectors have their own benefits. Both of these packages have a healthy number of contributors, although only one of them is in a foundation, which can help with continuity and standards.

While it has recent commits, spark-redis is currently lagging behind supporting current versions of Spark. This can be OK if your organization upgrades slowly, or if you have the time to handle upgrading the package to new versions of Spark yourself.

Some projects, like Apache Bahir, will explicitly decide to "retire" or "move to the attic" or cease maintenance. Projects in this state *may* continue to work but will likely break in the near future, and you will most likely be on your own when it comes to any issues you encounter. If the alternative is building out the same functionality from scratch, forking an existing deprecated project can still be a win—but it's important to understand the implications of picking up a project in this state.

Creating a Spark Package

Spark Packages allow people outside of the Apache Spark project to release their improvements or libraries built on top of Spark. Some Spark Packages are also created by Apache Spark contributors, like spark-testing-base or Bahir. Creating a Spark Package allows your code to be easily used by others with `spark-submit` or the Spark shell.

To create a Spark Package, you'll want to set up a repository somewhere folks have access to (GitHub being by far the most popular) and publish your package to the dependency management system of your ecosystem. For Python that's generally PyPI and for the JVM Maven Central is the way to go. The only real difference between a Spark Package and a regular library is your support for Spark.

As quite a few Spark Packages depend on access to Spark internals, it's important to understand what's involved in accessing developer APIs. If your package depends on Spark internals, it can be necessary to cross-build for different versions of Spark. There is no one-size-fits-all approach to cross-building Spark versions. To get started, spark-testing-base uses a simple shell script (Example 12-1) and build variables (Example 12-2), which can serve as a base for your own cross-building solution.[2] If you depend on classes or objects that are private or protected inside of org.apache.spark instead of just Spark's developer APIs, the chance that you need to recompile against even minor version changes is relatively high.

Example 12-1. Cross-build shell script

```
#!/bin/bash
set -ex
spark_versions=(
  2.4.8
  3.0.0 3.0.1 3.0.2
  3.1.1 3.1.2 3.1.3
  3.2.0 3.2.1 3.2.2 3.2.3 3.2.4
  3.3.0 3.3.1 3.3.2
  3.4.0 3.4.1 3.4.2
  3.5.0 3.5.1 3.5.2 3.5.3 3.5.4 3.5.6
  4.0.0 4.0.1
  4.1.0
)
# sbt doesn't read the JAVA_HOME env variable.
SBT_EXTRA=""
if [ ! -z "$JAVA_HOME" ]; then
  SBT_EXTRA="-java-home $JAVA_HOME"
fi
```

2 Another common solution for users of Maven and Gradle is the creation of build profiles.

```
# Avoid m2 race conditions which "shouldn't" happen but do by running update in
serial.
for spark_version in "${spark_versions[@]}"
do
  echo "Updating $spark_version"
  sbt $SBT_EXTRA -DsparkVersion=$spark_version +update
done
for spark_version in "${spark_versions[@]}"
    do
      echo "Building $spark_version"
      build_dir="/tmp/spark-testing-base-$spark_version-magic"
      rm -rf "${build_dir}"
      mkdir -p "${build_dir}"
      cp -af ./ "${build_dir}"
      cd ${build_dir}
      sleep 2
      nice -n 10 sbt  $SBT_EXTRA -DsparkVersion=$spark_version version clean +pub-
lishSigned sonaUpload sonaRelease &
      cd -
done
wait
echo $?
```

Example 12-2. spark-testing-base sbt Scala version setting key

```
val sparkVersion = settingKey[String]("Spark version")
…
  crossScalaVersions := {
    if (sparkVersion.value >= "4.0.0") {
      Seq() // We don't cross compile for the 4.X branch (until Scala 3 support
comes).
    } else if (sparkVersion.value >= "3.2.0") {
      Seq("2.12.15", "2.13.16")
    } else if (sparkVersion.value >= "3.0.0") {
      Seq("2.12.15")
    } else {
      Seq("2.12.15")
    }
  },
```

Sometimes, recompiling is not enough, and you'll need to make different versions of your code for different versions of Spark. There are a number of different approaches taken here; for example, NVIDIA's Spark RAPIDS uses reflection to have single code files that dispatch to the correct APIs at runtime. We don't recommend this approach as it tends to be brittle in ways that are only discovered at runtime. Other projects, like Arrow DataFusion Comet and spark-testing-base, have different subdirectories for Spark version-specific chunks of code, which are selectively included by the build tool to produce different versions from one codebase. A third option is different branches of code, but this has the downside of either leaving users on

older versions of Spark out of improvements or introducing a substantial backporting burden on you, the maintainer. As with almost all software choices, there is no one true way, although Holden is notably biased toward the approach she has taken in spark-testing-base, shown here in Example 12-3.

Example 12-3. spark-testing-base dynamic code selection

```
val coreSources = unmanagedSourceDirectories in Compile  := {
  if (sparkVersion.value >= "4.0.0") Seq(
    (sourceDirectory in Compile)(_ / "4.0/scala"),
    (sourceDirectory in Compile)(_ / "3.5/scala"),
    (sourceDirectory in Compile)(_ / "3.0/scala"),
    (sourceDirectory in Compile)(_ / "2.4/scala"),
    (sourceDirectory in Compile)(_ / "2.4/java")
  ).join.value
  else if (sparkVersion.value >= "3.5.0" && scalaVersion.value >= "2.12.0") Seq(
    (sourceDirectory in Compile)(_ / "3.5/scala"),
    (sourceDirectory in Compile)(_ / "3.0/scala"),
    (sourceDirectory in Compile)(_ / "2.4/scala"),
    (sourceDirectory in Compile)(_ / "2.4/java")
  ).join.value
```

Once you've published your code, it's time to make people aware of it so that you can get users' feedback. In the past, SparkPackages was the recommended place, but now we encourage folks to list their package in the awesome-spark repo.

Building Your Own Spark or Using Packages

Another way to add additional functionality is to fork and customize your own version of Spark. This is more popular with big companies where there is a dedicated data platform team to help customize or add functionality to Spark itself. Even in those cases, every patch that the team adds on top of Spark makes it harder to upgrade to a newer version of Spark (or potentially switch to a vendor-provided solution).

Typical changes that are implemented by customizing Spark at large companies are in the security, logging, and data lineage areas, where Spark's pluggability is insufficient.

One way to minimize the cost of your customized version of Spark is to "upstream" your changes or convince people to adopt them in future versions of Spark. This is easier said than done, as the code review process tends to be slow, and the number of reviewers is not huge. Upstreaming, of course, requires that your change be useful to more than just your own company, so probably don't put in the effort to try and upstream "FooBazinatorsLoggingPony," but consider upstreaming something like persistent volume reuse.

Conclusion

After reading this chapter, you should understand how to access and evaluate community packages when Spark's built-ins are not enough and evaluate which Spark components are being kept up-to-date. You should also be able to evaluate Spark's own internal components in similar ways to make sure that they are continuing to be improved.

APPENDIX A

Just Enough Iceberg and Friends

While there are many competing catalogs, there are some important common elements in terms of the representation of data on disk to support key operations. The ability to branch and time travel is an amazing tool for data quality work, but there are some different considerations with largely immutable (that is, nonchanging or nonchangeable) files. The different file layouts and approaches to query planning are important to understand—especially so that when you get poor performance, you can understand how to fix it. While these modern table formats offer more ACID-like behavior, it's important to understand that they're still not databases, and certain access patterns are generally slow.



Importantly, all of these table formats and catalogs are at relatively early stages of development, and different engines may support vastly different features, resulting in the same query producing different results between engines. This can go as far as one engine not supporting the format of deletes (or tombstones) from another engine and data that was intended to be deleted being returned in the query.

In this appendix, we will discuss the current setup of Iceberg and similar catalogs that we believe is most impactful for performance. However, since table formats are actively evolving at the time of writing, we've decided to make this largely exercise-focused. We encourage you to try these same exercises against your desired table format to see what the outputs are.



The *Workshop.ipynb* file in the *iceberg-workshop* directory is an easy way to run through common patterns and see what metadata and data files are produced.

To get started, you need to configure Spark to use Iceberg for one of its catalogs. Even if you've got Iceberg deployed in production, having a local catalog can be useful so that you can directly look at the files in the table (both data and metadata):

```
    .config("spark.sql.extensions",
"org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
    .config("spark.sql.catalog.local", "org.apache.iceberg.spark.SparkCatalog")
    .config("spark.sql.catalog.local.type", "hadoop")
    .config("spark.sql.catalog.local.warehouse",
"/tmp/high-performance-spark-examples/warehouse")
```

Note that despite the catalog type here being listed as Hadoop, that's really just for any catalog backed by an HDFS-compatible filesystem, which includes your local filesystem.

Now that you've got Spark set up with a catalog it's time to make a table inside of it. For demo purposes you can fetch CSVs of UK gender pay disparity data with `wget "https://gender-pay-gap.service.gov.uk/viewing/download-data/2021" -O ./data/fetched/2021`:

```
val df = spark.read.option("header", "true").option("inferSchema",
    "true").csv("./data/fetched/2021")
df.write.saveAsTable("local.uk_gender_pay_data")
```

This creates an unpartitioned Iceberg table, and you can look at the two different directories, `metadata` and `data`. The metadata JSON is directly human-readable, and you can also use `fastavro` to look at the Avro files in the metadata directory. In the metadata directory the *v1.metadata.json* file shows the table definition, and you can see the schema, snapshots, manifest files, and more. The manifest file shows you the actual data files along with some related statistics.

Updates and Table Maintenance

In previous systems, like Hive or even raw Parquet files on disk, updating a table would involve overwriting existing files—but for the majority of these new catalogs, that is not the case.[1] This change allows for support of time travel, branching, and more, but the resulting file layout is markedly different. Even in the classic mode,

1 In some ways, this is similar to Lucene's approach to immutable files.

occasionally data would be written in a suboptimal manner, such as too many small files, and require future action to clean up the table.

Iceberg can still encounter the same small file issue for the same reason as traditional formats, although it attempts to minimize this through write modes, which, by default, will group together data by any partitioning key and a target file size. A common cause of small files with Iceberg is repeated small commits that are stored as separate files.

Iceberg has a collection of actions available for table maintenance implemented in Spark. These, currently, need to be manually called, although some tools exist to perform automated table maintenance. For Iceberg to be able to collapse the files together, it's important to first remove the older snapshots:

```
import org.apache.iceberg.spark.actions.SparkActions

SparkActions.get().expireSnapshots(table).expireOlderThan(System.currentTimeMil
lis()).retainLast(1).execute()
// Note: probably pick a different time
SparkActions.get().rewriteDataFiles(table).option("target-file-size-bytes",
(512L*1024L*1024L).toString).option("rewrite-all", "true").execute()
SparkActions.get().deleteOrphanFiles(table).execute()
```

Partitioning, Indices, and Statistics

Iceberg partitioning supports partitioning on top of specific functions. While you can have similar things in Hive, you have to manually add that column in both the write side and the query side. Continuing our example with the UK-based data, we can partition the data based on a trim of the postal code (like a zip code), and Iceberg automatically translates any queries with the postal code to use the filter. Iceberg calls this *hidden* partitioning:

```
CREATE TABLE local.uk_gender_pay_data_postcode USING iceberg PARTITIONED BY
(truncate(1, PostCode)) AS select * from temp_table
```

Iceberg can track basic statistics for each data file, which can also be used to filter queries later. Currently these statistics are basic things like range (upper and lower bounds), nulls, and NaNs. You can see these statistics by running the following query:

```
SELECT * FROM local.uk_gender_pay_data_postcode.files
```



As shown here, you can get the same information directly from the table metadata and manifest files.

At present, Iceberg does not have separate indices, although this may change. There have been a few proposals to add secondary indices, but none of them have been accepted yet. Some other systems, like Hudi, do have secondary indices.

This leads nicely to query planning, as Iceberg is able to use these for choosing the correct data files to read.

Query Planning

Query Planning in Iceberg is different from classic raw file querying. Instead of depending on directory listing, the query engine delegates the reading to the metadata and manifest files. While this is normally faster than directory listing, when there is an excessive number of snapshots, this can start to have degraded performance. While there is work in progress to allow for distributed query planning, as of this writing, it is not yet ready.

Conclusion

Table formats are currently an area of active development, and the information provided here may not accurately reflect the future. We've left out a lot of important concepts, like variant types, schema migration, etc., since these are not as impactful for performance. This has also focused on Iceberg; for "friends" we strongly encourage you to check out the book by Denny Lee and Scott Haines, *Delta Lake: The Definitive Guide* (O'Reilly).

APPENDIX B
Spark Connect

Spark Connect decouples much of the driver from your Spark application. Spark Connect does not support the full Spark APIs, but if your application uses the restricted subset that is supported by Spark Connect, it gives you more flexibility to move between different backends. Since Spark Connect decouples the driver, you also have more flexibility with which language (and language version) you use to work with Spark.

With Spark Connect, a Spark application converts DataFrame/Dataset operations into protobuf messages and sends them to a remote Spark Connect server. This remote server performs many of the same tasks as the Spark driver does in the full version of Spark. Because the driver is already running separately, this can reduce the startup time for a Spark query, making Spark Connect especially well suited to smaller queries (like interactive analytics).



Some of the Spark Connect features discussed online, even in the Spark documentation, are not available in the open source "vanilla" Spark and only available from closed source vendors.

While Spark Connect gives you more flexibility to move between different execution engines, the limited API makes it poorly suited to many "high performance" applications where you want to access internals or exercise increased levels of control.

Supported API

Spark Connect supports most of the DataFrame/Dataset APIs and even has limited user-defined function support, but only in Python and Scala and only when the versions match.

Unfortunately there is currently no compile-time checking for Spark Connect compatibility; the simplest way to check if an application works in Spark Connect is by running your test suite against Spark Connect as well as full Spark.

Client Languages

In addition to the classic Spark languages (Python, Scala, Java, and R), Spark Connect has made it much easier for other languages to add (albeit varied amounts of) Spark support. Most importantly, new implementations are now present in Go and Rust, two increasingly popular languages.

One of the sometimes-claimed benefits of Spark Connect is the ability to upgrade Spark without having to upgrade Scala (or vice versa) as the driver program is decoupled. This is technically correct, but only if you don't use any user-defined functions, in which case upgrading your Scala version would often not have been an issue anyway.



There is a Spark project improvement proposal that would allow for separate JVM versions for UDF execution, which could empower this, but it will likely not be available until 2027.

Different Backends

Spark's popularity has resulted in many different engines adding partial support for Spark's API. Some of these backends are open source, including Daft's Daft Connect, and others relatively closed source, like Snowpark. The general acceptance of Spark's APIs as being worth supporting is promising, and when possible, if you can make your application work with Spark Connect, migrations between vendors are more likely to be successful.

Not all "Spark-like APIs" use Spark Connect as the intermediary, including LakeSail and DuckDB. In our experience these non–Spark Connect but still Spark-like APIs are more likely to have unexpected rough edges, so we recommend caution.



Some alternative backends claim that there are not code changes required; we have found this to not be true for the majority of complex pipelines.

These alternative backends may have very different performance implications than classic Spark. Some of these alternative backends use some of the same core accelerator code from Arrow DataFusion.

Conclusion

There have been increasing moves to make Spark Connect the default option for Spark over time. High performance Spark applications are more likely to use original, internal, or other parts of Spark that are not directly available in Spark Connect. For applications that don't need the full API, Spark Connect provides increased flexibility with switching backends, programming language versions, and more. In our view, Spark Connect is an excellent idea, a small bit ahead of its time, with great potential in future versions of Spark. As a reminder, not all Spark applications need to be high performance.

APPENDIX C

When Not to Use Spark

Apache Spark is an amazing tool for handling large-scale data with massive parallelism, but not all tasks are well suited to data parallelism. In this appendix you'll learn how to quickly look at a problem and see some possible warning signs that Spark might do more harm than good as well as some other tools you might want to consider in these cases. While some situations are more obvious, like `coalesce(1)`, `collect`, and `toLocalIterator`, not all cases where data parallelism doesn't help are quite so direct. The most conclusive way to tell if using Spark is beneficial is to write your program with and without Spark and compare the performance, but that's normally impractical, so instead let's explore some simple heuristics to sort it through.



If you have an existing Spark job and you find that it's frequently executing a single partition *or* is spending a substantial amount of time without any tasks running on the cluster, using Spark might not be worth it. If you don't want to refactor your code to rip out Spark, you can try switching it from distributed mode into local.

We tend to think of the biggest downside of using Apache Spark over other tools is the distributed overhead. While we can reduce some of that downside by using local mode, and Spark has continued to reduce its overhead, there are other downsides that come from using Spark. Most, but not all, local tools have better debugging experiences than distributed-capable tools like Apache Spark. Coroutines and multithreaded development can be similarly difficult to debug as Spark is. In our experience, if you find yourself turning to async operations, coroutines, threads, or similar for analytics tasks, it can start to make sense to explore using tools like Apache Spark.



Debugging Spark is an extra level of complexity, especially when working with non-JVM languages. It is possible, albeit uncommon, to end up with stack traces in all of Rust, Java, and Python in a single Spark failure, which can be difficult for even the most experienced developer to debug.

There are now many local tools that can read from and write to large distributed data sources. For example, both DuckDB and Polars offer high performance single-machine analytics while being able to talk to historically distributed metastores (Iceberg, Delta, Polaris, Hudi, etc.) and filesystems (Amazon S3, HDFS, etc.). Distributed tools, including Spark, Daft, and Ray, can all run in local mode as well. This allows for you to create complex pipelines, which may perform complex filtering or transformations and produce smaller summary datasets for final local analytics.

The main reason to use Spark is increased performance; the next few sections explore situations where Spark is unlikely to make your programs any faster.

Small Data and Small Tasks

Apache Spark gained its popularity for "big data," and what exactly *big* is has been shifting over time. If your entire data fits in memory, Excel, or a free USB key, Apache Spark might very well be overkill. Data parallelism is beneficial when your data is too big *or* the aggregate processing time is too long. When the processing time is low and the data fits in memory, you're often better suited to using single-machine-optimized tools, which avoid the coordination overhead added by Apache Spark.

If your data is small, but your tasks are slow, Apache Spark can still be beneficial. This is especially common with so-called "reasoning" large language models, where a substantial amount of reasoning tokens can be generated in addition to the target result; distributing your work can still be beneficial. When this is the case, it's important to make sure that you have a large number of partitions.

If you're in a situation where you have small data today and small tasks but hope to grow to big data tomorrow, using Spark can be more reasonable. Alternatively, for analytics use cases, use nonvendor-specific code, which can give you the flexibility to upgrade engines when the time comes. Avoiding vendor-specific extensions can be easier said than done, as illustrated by how many years it took for Amazon to migrate off of Oracle. In theory Apache Beam solves this problem, but in practice Apache Beam did not deliver on allowing pipelines to actually be portable, and historically the compatibility matrix has not been sufficiently tested at scale for us to depend on it. When possible, having continuous integration (CI) running your query on multiple systems is a good way to catch the potential introduction of nonportable code.

Real-Time Updates and Online Transaction Processing

Spark is built for analytics use rather than online transaction processing (OLTP). While you can update data you're querying with Spark Streaming or manually refresh a table in regular Spark, the length of time to reconcile an update is much higher than a traditional database. Even when the data does not fit in a single machine, OLTP use cases are normally better suited to sharded databases, like distributed Postgres.



While some data warehouse technologies, like Delta, Hudi, and Iceberg, do have row-level updates and deletes, these are still generally much slower than the same operations with OLTP-optimized databases.

Just because Spark itself is not designed for OLTP does not mean you cannot use it in addition to OLTP tools. It is common to have Spark query database dumps or query read-replicas with distributed databases. You can also have Spark run on top of live data, but as with any analytics queries, running them on live data has the potential to negatively impact your user experience.

User-Facing Interactive Use Cases

While you *can* use Spark to power interactive use cases, it's generally not designed with this in mind. If you've got a user/customer-facing web page that needs to do online analytic queries, it's easy for Spark to exceed the timelines available for a single page to load. Instead trigger Spark jobs off of a queue and write out the results to a database, which you serve the results from.

On the flipside, Spark is perfectly fine for handling interactive analytics for many business cases. Spark Connect makes this even more palatable, as the startup time for the driver can be avoided on each query.

Long and Highly Variable Task Processing Time

Spark assumes that each partition will be processed in roughly similar amounts of time. We've mostly looked at cases where the partition processing time comes from unbalanced data, but in some cases when talking with external APIs or tools, you can have balanced data but unbalanced processing time (especially if there is a caching layer involved). If you're in the situation where individual function calls can have vastly different processing times, you should make sure you've disabled Spark's speculative execution; otherwise Spark will treat the slow-running partitions as possible computer slowness, duplicating execution. Additionally, you'll likely want to make sure you have dynamic allocation enabled so if you end up with a long pole partition, the unused machines are released for other jobs.

Untrusted and Multitenant Queries

While Spark does expose a SQL interface and standardized connectors, allowing you to plug various analytics tools into it, Spark SQL queries can execute arbitrary code. This can be especially problematic in multiuser deployments where different users may have different permissions. Arbitrary code execution means that it may be possible for one user to extract another user's credentials in the same Spark deployment. Some vendors have added restricted execution, which allows for sharing of Spark executors, but this is not currently available in open source.

Your Job Has Side Effects per Record

If you find yourself writing user-defined functions that call non-idempotent APIs or using foreach without the ability to roll back using Apache Spark, you run the risk of applying the unsafe logic multiple times. Some examples of non-idempotent APIs include using billing APIs without specific transaction IDs to prevent double applications.

If you need to do per-record side effects, there are three ways to work around Spark's dynamic retry logic. The simplest option is to perform the side effects exclusively in the driver by collecting the result as a local iterator. While you can run into issues, for example, if the driver computer or program crashes, it's on par with normal software. The general solution is to use an external database to track the state of whether a side effect has been executed on a specific record. This can introduce contention around the machine running the database, but there are many databases that can be safely distributed to avoid contention.

Huge Individual Records/Rows

If you've got individual records that are larger than 1 GB, you may start to encounter some difficulty. Arrow automatically "chunks" large records, which breaks the Python/Java interoperability, meaning you can't use the "pandas" UDFs, `mapInPandas`, or similar Arrow-powered tools. Spark only splits up data on row/record boundaries, so your parallelism is limited to the number of rows/records present.

The general solution to huge individual records is finding ways to split up your records, but this is often "easier said than done." Pivoting the data can be a good solution if your operations don't require information from multiple columns at the same time and your row + column combination is more reasonable in size. Similarly, exploding any arrays/maps present in the data can help increase the potential parallelism provided that you don't need to process the entirety of the map/array in one go.

No Ops Support and No Budget for a Platform

Running Apache Spark is best with some level of operations support. If you don't have an operations staff, there are many commercial platforms (including Snowflake, Databricks, Amazon, Azure, Cloudera, Tangram, etc.)[1] available for running Spark that can provide some level of operational support with different platform vendors providing very different SLAs. Large companies like the FAANG ones (Alphabet, Amazon, Apple, Meta, and Netflix) will frequently have dedicated Spark teams along with their data science and data engineering teams.

Highly Unreliable Backends

While Spark's retry mechanism is generally effective, if you've got code with a sufficiently high failure rate, this retry mechanism can end up retrying too much work. If your failures are not specific to your machine, for example, calling a flaky remote API, you should wrap your flaky calls with some level of retry logic. In Python the backoff decorator from backoff, `@backoff.on_exception`, makes it easy to add complex retries without substantial code changes.

Different Functions Versus Same Function on Different Data

Spark is designed to run the same function on many different records rather than many different functions on the same data. You can work around this with many asynchronous actions or using a single function with a large switch based on an element ID along with an RDD with a series of IDs for each different target operation. These are both kind of hacky and could be a good sign that using a tool like Dask or Ray could be better suited.

Frequently Accumulated Updates

If you find yourself extensively using Spark's built-in accumulators and redistributing the result back out, this can be a sign that using the actor model is better suited to your task. Most tools that provide the actor model are language specific, with Ray providing an actor model in Python and Akka providing an actor model in Scala and JVM languages.

1 The authors own shares in some of these companies, and this is not an exhaustive list of all companies providing Spark-powered platforms.

Global In-Order Processing Required

If you need global in-order processing, data parallelism doesn't provide any benefit during that part of your workload. You can force in order processing with a global sort followed by `toLocalIterator`, but by definition, global in-order processing does not benefit from even multithreaded computation.

When You Can, Now, Use Spark

Historically Spark was not well suited to mixed hardware requirements and non-partially retriable work; this has changed since our first book. Mixed hardware support has been added largely thanks to the work of people from NVIDIA, and individual tasks can now request specific hardware, most commonly GPUs. Relatedly, Spark now supports gang scheduling, making it more suitable for use for distributed training. That being said, we still normally recommend writing out the data to persistent storage before training so as to allow reuse of the same data with different training configurations.

Despite what some say, Spark is perfectly well suited to processing textual data. We've used Spark to power large text-processing pipelines to build training data for Fight Health Insurance and so many other projects. You'll likely want to use Python for access to a more full-featured set of natural language processing tools here, though.

Previously, Spark usage was limited to a few core languages; you can now use Spark via Spark Connect in a large number of different languages. This does not offer the full API but offers more than was previously available with the JDBC connectors in noncore languages.

Conclusion

While Spark is good for many problems, it's not a universal silver bullet for all problems, let alone all analytic problems. As memory and single-machine computation have kept increasing, the scale of problems you can handle without resorting to distributed computing has been increasing. However, similarly, the amount of data generated has also been increasing, especially with generative AI.

APPENDIX D

Advanced Task Scheduling: Gangs and Resource Profiles

Spark's default scheduler works well in cases where individual tasks can be retried and resources are, if not equal, fairly similar per executor. You can use resource profiles when specific tasks require unique resources, such as more memory or hardware accelerators (e.g., GPUs). Similarly, Gang Scheduling, also called *barrier execution mode* in Spark, solves the problem of requiring all tasks in a given stage to execute at the same time and of retrying an entire block of computation, not just a single partition, on failure. Gang scheduling is most often used for specific ML workloads.

You designate certain operations as needing gang scheduling/barrier execution mode or specific resource profiles on RDDs and Datasets/DataFrames.

Gang Scheduling/Barrier Execution Mode

On RDDs, for gang scheduling you just need to call `barrier`, which will give you back a special `BarrierRDD`. The operations you do on the `BarrierRDD` will be gang/barrier scheduled, and you can use `mapPartitions` to transform each partition.

On Datasets/DataFrames the API assumes you are using barrier execution mode in Python and as such is available for `mapInPandas` and `mapInArrow` as a boolean flag.

Resource Profiles

Resource profiles have a similar RDD and Dataset/DataFrame API but require a bit of extra setup work to predefine the different resource profiles. On RDDs you call the `withResources` function, and on DataFrames you pass in the `ResourceProfile` as a named parameter to your `mapInPandas`/`mapInArrow` function.

Spark on Kubernetes has one of the more flexible APIs for vendor-specific hardware, automatically translating `spark.executor.resource.[type].amount` along with `spark.executor.resource.type.vendor` into the matching K8s request. Example D-1 requests two NVIDIA GPUs for its map task. In addition, if the automatic translation doesn't meet your needs (for example, you need to be in a different namespace with one resource), you can customize it by setting `spark.kubernetes.executor.pod.featureSteps` and writing your own translation (see `BasicExecutorFeatureStep` for an idea of what these translations can look like).

Example D-1. `ResourceProfile` example

```
    // Create a resource profile requesting 2 NVIDIA GPUs per executor and 1 per
task
    val gpuResourceProfile = new ResourceProfileBuilder()
      .require(
        new ExecutorResourceRequests().resource(
          "gpu",
          2,
          vendor = "nvidia",
          discoveryScript =
            "/opt/spark/bin/getGpusResources.sh" // See sample in Spark repo
        )
      )
      .require(new TaskResourceRequests().resource("gpu", 1))
      .build()

    // Use resource profile to run on a machine with GPUs.
    val rdd = sc
      .parallelize(1 to 4, 4)
      .withResources(gpuResourceProfile)
      .map { i =>
        // Do some special GPU stuff here my friend
        i
      }
```

While we mostly see resource profiles used for task-specific hardware, you can also use them for individual tasks that may require more general-purpose hardware (such as RAM).

Conclusion

Spark's default task scheduling is almost always what we want, with a few exceptions, most of which are focused around AI and machine learning. Both resource profiles and gang scheduling are most often used for AI/ML-specific tasks where the per-partition retry/scheduling or assumption that all are equal doesn't work.

APPENDIX E
Spark Streaming

Cowritten with Tomasz Magdanski

Spark Streaming has two modes of running, micro-batching and continuous, with very different performance characteristics. Spark Streaming exposes both DStreams and Streaming Datasets, which are similar to RDDs and Datasets, respectively, but with some additional considerations. We will focus on Streaming Datasets in this appendix, but know if you want RDD-like semantics, they can be available with the classic DStream API. Modern Spark refers to this Dataset-based API as *Structured Streaming*, and this is where virtually all new development occurs.

In both modes, since streaming applications generally run for longer than batch applications, sometimes without any normal ending time, and on a long enough timeline, the survival rate for a Spark driver drops to zero. Additionally, any small resource leakage in the main component of your driver will eventually balloon up. This applies equally to executor-side leaks for continuous mode, where long-lived tasks run indefinitely.

Despite these additional considerations and constraints, you can largely use the same Spark code between streaming and "regular" applications. This is especially appealing for those who want close-to-real-time updates with the ability to backfill. In modern Structured Streaming this is commonly achieved by reusing the same query and switching triggers; for example, using `Trigger.AvailableNow` for batch backfill and a regular processing-time trigger for ongoing streaming. A variation of this is often called the "lambda" architecture.

In this appendix you will learn how Spark Streaming differs from regular/batch, the different operation modes of streaming, and some of the key limitations you need to keep in mind if you're considering using Spark for streaming. These limitations are around reliability, exactly-once semantics, and latency.

Simplest Example

Let's start with everyone's favorite word count example, ported to streaming in Example E-1.

Example E-1. Basic streaming network socket example

```
val lines = spark.readStream
  .format("socket")
  .option("host", "localhost")
  .option("port", 9999)
  .load()

val words = lines.select(explode(split(col("value"), " ")).alias("word"))
val counts = words.groupBy("word").count()

val query = counts.writeStream
  .outputMode("complete")
  .format("console")
  .start()

query.awaitTermination()
```

The notable changes between Example E-1 and a nonstreaming version are the input source (`readStream`), output sink (`writeStream`), and `awaitTermination`. This elementary streaming example will run until the driver crashes, which will likely happen as it builds up state for each new word encountered and never evicts old ones. In a real production application, you would also configure a checkpoint location and avoid nonreplayable sources like the socket example. Without windows or watermarks, Spark has no mechanism to drop state.

Driver Considerations

The key operational difference in Spark Streaming is that long-running drivers, especially for stateful queries, tend to accumulate pressure over time and will eventually require a restart. Stateless streaming applications can run for much longer, but in practice most teams still schedule periodic driver restarts to surface issues in a controlled manner rather than during an unexpected failure. Spark Streaming relies on checkpoints to recover from driver and executor failures; however, with nonreplayable receivers, even with checkpoint recovery, you may experience data loss. Similarly, with non-idempotent sinks, restarting can cause records to be double-processed. When performing graceful restarts, you can call `query.stop()` and wait for termination, but nongraceful kill/recovery tests should also be performed.



In our experience, while not strictly necessary, having periodic (at least weekly) restarts of your driver during business hours is a good way to ensure that driver restart issues are found and addressed, not at 2 a.m. This can be graceful, but you should also test non-graceful recovery separately.

When writing your Spark program, to ensure it can be restarted, avoid using any Spark accumulators and stick to the state that Spark itself keeps track of via aggregations. Any state you set in "regular" variables, for example, building a `HashMap` inside the driver, will be lost on recovery.

You will need a separate process outside of your Spark driver to monitor and relaunch the driver on failure. Depending on your deployment, this can be the cluster manager; for example, the Kubernetes pod deployment can be set to restart on failure automatically.

Operation Modes

Spark Structured Streaming exposes three execution modes: micro-batching, real-time, and continuous processing. Recently, *async progress tracking* has been added, which is a hybrid of micro-batch and continuous operation modes, although not one that the authors have used.

Spark Structured Streaming triggers micro-batches based on different conditions:

`Trigger.ProcessingTime`
: This is the classic microbatch trigger. Spark starts a new micro-batch at the specified interval (or as fast as possible when no trigger is set). This trigger keeps the driver running and supports all streaming features like state, join, or aggregations.

`Trigger.Once`
: This trigger processes *all* available data into a *single* micro-batch and then stops the query. It is useful for small backfills. This mode could easily overload the driver or executors with data too large for the cluster size. Each micro-batch needs to be scheduled externally, and cluster cold start should be added to the overall batch processing time.

`Trigger.AvailableNow`
: This trigger, introduced in Spark 3.4, processes all available data but splits it into multiple micro-batches as needed and then stops the query. Each micro-batch needs to be scheduled externally, and cluster cold start should be added to the overall batch processing time. The size of the micro-batches created by `Trigger.AvailableNow` is controlled by the source's own rate-limiting options (for

example, `maxFilesPerTrigger` for file-based sources or `maxOffsetsPerTrigger` for Kafka).

`Trigger.RealTime`
: This trigger, introduced in Spark 4.1, signals that the query should run in the new ultra-low-latency execution mode (around 5 ms–300 ms p99 latencies). A time interval can also be specified, e.g., "300 seconds," to indicate how long of a window of time to record transactions within a specific checkpoint. Unlike ProcessingTime execution, 300 seconds in this case wouldn't cause Spark to wait for 300 seconds between jobs but instead manages where a specific transaction is committed given that the processing mode is no longer micro-batch-by-micro-batch but instead record-by-record for an unbounded streaming of records.

Continuous processing remains an *experimental* feature in Spark 3.x and 4.x, with significant functional limitations and no built-in fault tolerance. It is not generally recommended for production use and likely to be deprecated as real-time mode gains stability. While parts of continuous execution continue to evolve, for example, in Spark 4.1, it is still far less commonly deployed than micro-batching and should be approached cautiously. Continuous execution in Spark works by prescheduling the tasks and keeping each task as a long-running consumer and processor of data, allowing for records to be processed as they arrive without any batch delay. Continuous mode is implemented as a separate low-latency execution engine using long-lived tasks and epoch commits. In addition to these core modes, you can use micro-batching along with async progress tracking.

Micro-Batching

Micro-batching is the first approach to adding streaming support to Spark. The primary downside of micro-batching compared to continuous processing is slower end-to-end processing time for records. If you're going to be doing any kind of aggregation or otherwise collecting state, micro-batching is the only workable option in Spark Streaming currently. Micro-batch is also the mode that supports the full API surface: aggregations, joins, arbitrary state, and "exactly-once" sinks when paired with replayable sources and idempotent sinks.

In modern Spark, the default micro-batching behavior is to trigger a new batch as soon as the previous batch finishes processing. In its standard configuration, micro-batching cannot have two micro-batches in-flight, so even if you specify a short trigger window, this will not make the stream process any faster. You usually can reduce resource utilization by specifying a longer trigger interval.

Real-Time Mode

Real-time mode is a trigger type for Structured Streaming that enables ultra-low latency data processing with end-to-end latency as low as five milliseconds. Real-time mode is intended to replace Spark's experimental continuous mode. This is ideal for operational workloads that require immediate response to streaming data, such as fraud detection, real-time personalization, and instant decision-making systems. This new execution mode was introduced in Spark 4.1 and differs greatly from the common analytical workloads that have made up the general Spark use cases for over a decade.

This mode executes long-running batches (defaulting to 5 minutes) where the continuous flow of data is handled record-by-record rather than micro-batch-by-micro-batch. In order to process each record (on its own), Spark will now schedule all stages of the query simultaneously. This removes blocking on a per-stage basis, enabling immediate processing as long as a task slot is available within the cluster.

At the time of writing, there are a number of limitations that are on the roadmap to be resolved in future releases of Spark. At a high level, the only supported output mode is *Update*, and usage of real-time mode is limited to the Apache Kafka and Amazon Kinesis data sources. Lastly, `mapGroupsWithState`, `flatMapGroupsWithState`, and `transformWithState` are slated to be supported in future releases.

Async Progress Tracking

Normally, when a micro-batch has finished processing some range of data, it will not begin processing new data until after that has been recorded in external storage. This can become a bottleneck, and instead, you can have Spark periodically nonblockingly store its progress using the `asyncProgressTrackingEnabled` and configuring the `asyncProgressTrackingCheckpointIntervalMs`. This can make micro-batching somewhat similar to continuous processing, but you lose any state/aggregation support while retaining the ability to recover automatically from executor failures. Async progress tracking allows commit/offset logging to occur in the background while the next micro-batch begins. It does not remove support for aggregations or state—the query still executes in micro-batch mode. It reduces bottlenecks caused by synchronous log writes but adds slightly more complex failure-handling semantics. Not supported with `Trigger.Once` or `Trigger.AvailableNow`.

Output Modes

Similar to the table write modes in regular Spark, Spark Streaming has its own output modes: Append, Complete, or Update.

Update and *Append* are functionally identical, with the restriction that Append can only be used on queries without any aggregations. Both output just the records that have changed per micro-batch (or process). As such, we recommend using `Update` in place of Append since it offers more flexibility. Append mode may only emit rows once they are known to be final, typically after watermarks advance; Update mode emits intermediate results as well.

Complete outputs all of the records still in the Dataset; any record in the Dataset has been updated. Complete is only available for streaming queries with some form of aggregation. Complete mode can be expensive for large states, so many systems instead write to an upsert table and materialize full results downstream.

Time, Aggregations, Joins, and State

While much of streaming is similar to batch processing, records have a new attribute of arrival time, which is not present in batch. Often we want to create aggregations, windows, or track state over some period of time for the records. It is useful to distinguish event time, ingestion time, and processing time; Structured Streaming primarily uses event time for correctness.

If all of your records "arrive on time"—that is, the arrival time is the same (or close to the same) as the time that you're looking to aggregate over—then these aggregations can be relatively simple. Otherwise, you can use watermarking to provide a safety margin for data that may arrive late-ish.



If you're OK with it, you can get the "processing time" of a current record with the "current_timestamp()" UDF and use that as your event time column. In that case, you can set your watermark to zero. Using processing time as event time effectively assumes all data is on time and simplifies watermarking.

Aggregations

Spark Streaming aggregations are a little different than "normal" Dataset aggregations, as Spark needs to be able to not have all of the records ever received in memory to produce the aggregate. You can help Spark only have the necessary data in memory by using windows, along with watermarking for late-arriving data. Additionally, for a user-defined aggregation, you'll also have to add an extra "expiry" operation over what you're used to.

Windows

Streaming windows work similarly to "regular" Spark, except with the consideration that Spark needs to keep around the aggregated data for any window until a record can no longer be updated. For example, if, as in our streaming word count in Example E-1, without any time-based consideration, all keys could be updated indefinitely, then the entire dictionary of words will remain in the Spark cluster until the end of the application. Structured Streaming supports tumbling, sliding, and session windows over event time.

If there is a time element to your window, Spark can use this to discard records once they are no longer needed; for example, if we can modify Example E-1 to take JSON blobs with timestamps and aggregate over the last 10 minutes as in Example E-2. To allow Spark to remove records once they'll no longer be updated, you'll have to also tell it how late data can arrive using watermarking.

Example E-2. Streaming JSON with a windowed aggregation

```
val df = spark.readStream
  .format("json")
  .schema("timestamp TIMESTAMP, word STRING")
  .load(inputPath)

val windowed = df
  .groupBy(window(col("timestamp"), "10 minutes"), col("word"))
  .count()
```

Watermarking

Watermarking "solves" the late-arriving data problem by ensuring that data is kept around for the duration of the watermark to allow for any required aggregations to be completed within that watermark. The larger the watermarking, the more data that has to be stored in Spark to support updating data arriving later. Internally, Spark tracks the maximum observed event-time and subtracts the watermark delay to determine lateness.

If an event arrives after the max delay of the watermark, it will be discarded. This practice is often best done when combined with some kind of periodic batch fill job, which will produce a more complete picture of events, including those that were ignored during online processing. Many cases of potential missing data turn out to be misconfigurations of watermarking logic.

We can update Example E-2 to allow Spark to discard records after they have been around for the length of the window + watermark duration by adding a `.withWatermark("timestamp", "42 minutes")` before our `groupBy` call as shown.

User/arbitrary state

In addition to Spark's built-in aggregates and your own user-defined aggregate functions (UDAFs), Spark Streaming adds a `transformWithState` operation, which gives you more control over state management. You don't directly build state in native Scala/Python primitives; instead, you implement functions that serialize the state in a format Spark SQL can work with.

To implement your custom streaming state, you'll extend the `StatefulProcessor` class implementing a few key functions: `init`, `handleInputRows`, and `handleExpiredTimer`. The first two functions map well to the "fold" concept in functional programming, with `handleExpiredTimer` being unique to streaming where we don't want to have to keep state forever. These APIs allow per-key arbitrary logic implemented in `StatefulProcessor`, using timers to model expiry or timeouts.

For state, any state that can be encoded in a DataFrame works with `ValueState`, but if you need to do lookups or appends as part of your state, you can use the `MapState` or `ListState` wrapper. Modern versions allow multiple named state variables per key, simplifying complex logic and schema evolution.

Stream-Stream Joins

As with aggregations, joins across multiple streams require some form of windowing (expressed as a time-based join constraint) to avoid storing all records received so far. It's essential that windowing be applied to both sides of the join for Spark to clean up the stream state as it progresses. For example, this stream-stream join (Example E-3) will build state indefinitely. Both streams must have time constraints and watermarks to bound the state; otherwise, the state grows unbounded.

Example E-3. Basic stream-stream join

```
val left = stream1.alias("left")
val right = stream2.alias("right")
val joined = left.join(
  right,
  expr(
    "left.key = right.key"
  )
)

val query = joined.writeStream
  .outputMode("append")
  .format("console")
  .option(
    "checkpointLocation",
    "./tmp/checkpoints/stream_stream_no_windowing"
  )
```

```
    .start()
query.awaitTermination()
```

By having windowing on both sides as in Example E-4, Spark only needs to keep a finite amount of state. When doing a stream-stream join if the watermarks are not the same, Spark will select the minimum watermark by default (potentially resulting in data loss), but you can change this to the maximum with `spark.sql.streaming.multipleWatermarkPolicy`.

Example E-4. Stream-stream join with watermark

```
// We provide the streams aliases so we can select specific keys.
val left = stream1.alias("left").withWatermark("timestamp", "10 seconds")
val right = stream2.alias("right").withWatermark("timestamp", "5 seconds")

val joined = left.join(
  right,
  expr(
    "left.timestamp >= right.timestamp - interval 5 seconds " +
      " AND left.timestamp <= right.timestamp + interval 5 seconds " +
      " AND left.key = right.key"
  )
)
```

Stream Table (or Stream Static) inner joins don't require the same collection of state, and as such, it's not necessary to have time conditions to allow for state cleanup.

Left or right outer stream-stream joins are only supported when the opposite table has watermarking and a time-based join constraint expressed so Spark can know when there is no match without waiting forever. Full outer stream-stream joins require that at least one of the streams have a time-based join constraint, although you should have both for state cleanup.

Full outer joins also require at least one time condition, preferably both, to maintain a bounded state.

State store

While windows and watermarking can be used to minimize the amount of state Spark Streaming needs to keep track of, it still needs to be kept track of somewhere. Spark Streaming's default state store keeps all of the state in memory while running, which can cause memory pressure and failures. In addition to the default tracker, Spark now has a built-in RocksDB-powered state store, which we recommend using. This allows for less memory pressure, and since it's built-in, you don't need to do any additional deployment; just set `spark.sql.streaming.stateStore.providerClass` to `org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider` and "party on."

Ensure `spark.local.dir` points to fast disks, and checkpointing persists compact changelogs for recovery.

In both state tracking mechanisms, data is persisted externally to allow recovery. Since it relies on processes outside of Spark, enabling RocksDB may increase latency in some cases, but the memory savings trade-off of using it typically makes the most sense.

Using foreachBatch: Dropping into batch semantics inside Streaming

One of the most powerful features in Structured Streaming is the `foreachBatch` sink, which allows you to process each micro-batch, using the regular batch DataFrame/Dataset APIs. Instead of being constrained to the strict streaming operators, `foreachBatch` hands you a static DataFrame containing only the records for the current micro-batch and lets you run arbitrary batch logic on them. This provides a controlled escape hatch from streaming semantics into full batch flexibility.

At a conceptual level, while the streaming engine manages input offsets, state, watermarks, and scheduling, the work you perform inside `foreachBatch` is just normal Spark code. This enables a number of patterns that are either impossible or strongly discouraged within pure streaming operators:

Static table joins and lookups
: Since each batch is a static DataFrame, you can perform point-in-time dimensional lookups, joins with large static tables, or use broadcast joins without the constraints of stream-stream joins. You can also use arbitrary SQL or Dataset transformations that aren't supported natively in streaming mode.

Writing to multiple destinations
: Standard streaming sinks write to exactly one destination per query. With `foreachBatch`, you can write the batch to several sinks—transactional tables, analytical stores, message queues or external systems—using whichever DataFrame write APIs make sense. Each write can have its own mode, partitioning, and output format. It is important to note that `foreachBatch` does *not* provide exactly-once semantics by itself. If the streaming query fails after partially completing the work inside a given micro-batch, Spark will rerun the entire micro-batch on recovery.

Using data sources not available in streaming
: Many data sources only support batch reads or writes. Inside `foreachBatch`, these can be used normally since the data appears as a bounded batch DataFrame. This includes JDBC sinks, many NoSQL systems, cloud warehouses, and custom connectors.

Implementing fine-grained, or multistep, business logic
: Because you control the entire micro-batch, you can chain complex transformations, validations, enrichments, or multistep workflows that would be awkward or impossible to express with stateful streaming operators.

Working around the limitations of structured streaming joins or state
: Suppose your join or stateful logic does not fit neatly into Structured Streaming's event-time model, watermarking constraints, or memory profile. In that case, you can implement custom logic per batch without forcing the streaming engine to maintain a long-lived state.

Using `foreachBatch` does come with some considerations. The code inside the callback must be idempotent if you require exactly-once semantics, especially when writing to external systems. Since Spark may retry a micro-batch on failure, any side effects in `foreachBatch` must tolerate reprocessing. For this reason, you should design the operations within the callback with the same discipline applied to streaming sinks—transactional updates, idempotent upserts, or deduplicated writes where appropriate.

Finally, `foreachBatch` still inherits the micro-batch boundaries of the streaming query. The engine determines which records belong to each batch and handles checkpoints, progress tracking, and fault recovery. Your logic only deals with the batch's contents, which allows mixing streaming ingestion with batch-style transformations safely and efficiently.

In practice, `foreachBatch` becomes a bridge between streaming ingestion and batch-style output logic, and is one of the most commonly used mechanisms for integrating Structured Streaming into existing batch-oriented data ecosystems. It provides the "best of both worlds"—continuous ingestion with Spark's full batch processing capabilities applied one micro-batch at a time.

Reliability: YOLO-ish, At Least Once, At Most Once, and Exactly Once

Spark's ability to support exactly-once output (sometimes called *execution*) depends on the data sources, called receivers, and sinks (places we write output to) to have very specific semantics. Largely, Spark has outsourced the difficulties of exactly-once semantics to the data sources and sinks.



Spark does not support the idea of a record only being processed once. Spark's approach to fault tolerance, recomputing on failure, means that even with all of the constraints, an executor failure can mean that a record may need to flow through the transformations more than once.

"Exactly-once" refers to the effect on the sink, not to avoiding reprocessing inside Spark.

The classic "getting started" Spark Streaming example is that of a socket/network receiver that writes its outputs to stdout. This gives us perhaps one of the least reliable ways to construct a Spark Streaming program, as shown at the start in Example E-1.

The socket receiver cannot replay, so on failure, all records that have not been processed are lost. Similarly, the `collect` mechanism does not have any explicit state storage or tracking, so if a replayable receiver were used, Spark might output the same data twice.

Thankfully, the same core application code can be made reliable by configuring checkpointing and swapping the source and sync for replayable and idempotent options, respectively.

Checkpointing

Spark Streaming uses checkpointing to recover from driver failure. Checkpoints contain information about how far along Spark has processed the records; we can quickly add checkpointing as shown with Example E-5 on the writestream call or globally with `spark.sql.streaming.checkpointLocation` on the SparkConf.

Checkpoints contain offset logs, commit logs, and state snapshots. On recovery, Spark determines how far the query progressed by comparing the offset log, which records the maximum input offsets planned for each batch, with the commit log, which records which of those batches were successfully completed.

Example E-5. Checkpoint specified with write options

```
val writer = joined.writeStream
  .outputMode("append")
  .format("console")
  .option(
    "checkpointLocation",
    "./tmp/checkpoints/stream_stream_join_both_side_watermark"
  )
```

In micro-batch mode, checkpoints are updated after each batch is processed, and in continuous mode they are updated periodically.



For Spark to be truly fault tolerant, your checkpoint location should be "durable" to driver loss (e.g., not on the driver itself), although Spark will accept local paths, which can be faster but have no tolerance on machine loss.

In addition to recovery from unintended failures, you can use Spark checkpoints to "recover" from a shutdown and switch of execution mode (streaming to continuous or vice versa). However, if you change your logic or some key configurations (like partitioning), you cannot depend on recovering from a checkpoint and instead will need to manually have your own high watermark to restart from.

Changing schema, partitioning, or stateful operator layout typically requires starting from a new checkpoint or a manually supplied starting offset.

Replayable and Nonreplayable Receivers

Spark uses replayable receivers to provide "at least once" semantics through replaying any messages that it does not have a record of fully processing in its last checkpoint on recovery. Examples are: Kafka, Kinesis, and file-based sources that allow rereads.

With nonreplayable receivers, once Spark has received the data, it cannot go back and request the same record again. Spark Structured Streaming's write-ahead logs only record the offsets read for the purposes of requesting replays from replayable receivers and do not store the actual records read. Write-ahead logs track offsets/metadata, not the actual records.

Idempotent and Non-Idempotent Sinks

An idempotent sink is one where writing the same record to it multiple times does not change the final result. Spark depends on sinks to track which records have been written to them, allowing Spark to rewrite the same record many times. A non-idempotent sink is one where writing the same record out multiple times will result in multiple outputs of that record. Examples include transactional Kafka producers, Lakehouse/Delta upserts, or RDBMS UPSERT operations.

One of the simplest ways to turn a non-idempotent sink into an idempotent sink is by tracking which records have been written out in an external store and ensuring no records are output that have already been recorded.[1]



If this sounds like Spark has delegated responsibility to avoid more than one write to the sink, that's exactly correct. This is part of why we're a little more reserved on Spark's "exactly-once" promises.

1 Although there of course exists a race condition unless you can make the update atomic.

True "Real-Time" and Reasonable Expectations

Spark Streaming has improved greatly since the first edition of this book and is likely going to continue to improve. However, in our opinion, it is not suitable for mission-critical situations, as it does not offer hard "real-time" guarantees. By virtue of Spark's approach to recomputing on partitions on failure and recovering from checkpoints on driver failure (or manual restart on executor failure in continuous mode), if someone's life is at stake if a deadline is missed, we advise you to explore other systems. For strict real-time or subsecond SLAs, engines like Flink or Kafka Streams may be more appropriate.



At one of the authors' previous jobs, moving from a more "true" real-time system to Spark Streaming resulted in an almost 10× reduction in compute cost and ability to unify the codebase between streaming and batch processes.

Conclusion

Depending on what your goals are in terms of "high performance" Spark Streaming applications, you're likely to proceed on two different tracks. If your goal is to process individual records as quickly as possible end-to-end, Spark's continuous execution mode (and the restrictions associated with it) is your best bet, whereas if your goal is to process large amounts of records relatively quickly, you'll want to explore Spark's micro-batch options. While Spark Streaming continues to make improvements, it is still not suited to all streaming use cases.

APPENDIX F

The Spark Web UI: Debugging and Optimizing Your Jobs

The Spark Web UI is one of the most powerful yet underutilized tools in a data engineer's arsenal. While we've referenced it throughout this book for checking partition sizes, monitoring spill behavior, and examining cached RDDs, this appendix provides a comprehensive guide to understanding and effectively using every aspect of this built-in monitoring interface. Rather than presenting a dry reference manual, we'll learn to use the Web UI by debugging a real performance problem, a scenario that every data engineer encounters sooner or later. In this section, we hope to give you the tools to think about how to interpret what the Web UI is telling you and, more importantly, how to translate those observations into actionable performance improvements.

A Real-World Performance Problem

The first time many of us develop a big data application with Apache Spark, something unexpected happens: a simple job that should complete in seconds takes minutes, or never finishes at all. The culprit is often small files. We partition data incorrectly and accidentally write millions of tiny files to Amazon S3 or HDFS. The code looks correct, and the logic is sound, but the performance is terrible.

Let's re-create this exact scenario with a simple Spark script that reads sales data, filters by quantity, and writes the results partitioned by quantity:

```
val df = spark.read.load("~/data/store_sales")

val dfFiltered = df.filter(df("ss_quantity") > 1)

dfFiltered.write
  .mode("overwrite")
```

```
    .partitionBy("ss_quantity")
    .parquet("/tmp/store_sales")
```

By statically analyzing this code, you couldn't find any performance issue. The logic is straightforward: read, filter, write. Yet on a typical development machine, this simple script takes around a minute to process just 273 MB of data, far longer than it should.

This is where the Spark Web UI becomes invaluable. Let's use this problematic job to learn how to navigate and interpret every tab in the interface.

Accessing the Spark Web UI

When running Spark locally, the Web UI is accessible at `http://localhost:4040` by default. If you're running multiple Spark applications, subsequent applications will use ports 4041, 4042, and so on. For cluster deployments (YARN, Kubernetes, or standalone), the UI is typically accessible through the driver node's hostname on port 4040, though your cluster manager may provide its own proxy URLs.

The Web UI is only available while your Spark application is running. For postmortem analysis of completed jobs, you'll need to use the Spark History Server, which we'll cover later in this appendix. To enable history server support, set `spark.eventLog.enabled=true` and configure `spark.eventLog.dir` to point to an accessible storage location.

Understanding the Spark Execution Hierarchy

Before diving into the specific tabs, it's essential to understand how Spark's execution model maps to what you'll see in the interface. As we discussed in Chapter 2, the hierarchy flows from applications down to tasks. This distinction is important because it has strong implications for how you interpret the metrics displayed in each tab of the Web UI.

Application

At the highest level, an application represents a single `SparkContext` or `SparkSession`. Our "Sales Filterer" script is one application. Each application gets its own Web UI instance, hosted by the driver process. An application can contain multiple jobs, all sharing the same set of executors.

Jobs

A job is triggered each time you call an action. In our example, the `write` operation triggers a job. Actions include `count()`, `collect()`, `save()`, `show()`, etc. The number of jobs should equal the number of actions. If you see more jobs than expected, some operation likely triggered an implicit action; schema inference on CSV files is a common culprit, as is calling `show()` during debugging.

Stages

Each job is divided into stages. Stage boundaries are determined by shuffle operations; any transformation that requires data redistribution creates a new stage. In our sales filtering example, if we add a `repartition()` call, that creates an additional stage with a shuffle boundary. Common shuffle-inducing operations include `groupByKey()`, `reduceByKey()`, `join()`, `repartition()`, and `sort()`. As we discussed in Chapter 7, wide transformations (those requiring shuffles) have significant performance implications because they limit parallelization and require data movement across the cluster.

Tasks

Tasks are the smallest unit of work. Each stage is divided into tasks, with one task per partition. If your source data has 1,000 small files, you'll have 1,000 tasks in the read stage, and this is exactly where our performance problem originates. Tasks run on executor cores; if you have 100 executor cores but 1,000 tasks, Spark runs them in waves. The number of tasks corresponds to the number of output partitions, as we explained when discussing narrow versus wide dependencies (see Chapter 7).

The Jobs Tab: Our First Look at the Problem

Let's open the Web UI and navigate to the Jobs tab while our slow sales filtering job runs. The Jobs tab serves as your entry point for understanding application execution.

What we see

The Jobs tab displays all jobs submitted to Spark, organized by status: active, completed, or failed. For our simple script, we see a single job triggered by the write operation. The job shows:

Job ID and description
: A unique identifier and the action that triggered the job (in our case, something like "save at NativeMethodAccessorImpl.java").

Duration
: How long the job has been running; our problem job shows 51 seconds for what should be a subsecond operation.

Stages
: The number of stages and progress. Our job has two stages: one for reading and filtering, one for writing.

Progress bar
: Visual indication of completion percentage.

The Event Timeline

At the top of the Jobs tab, the Event Timeline provides a visual representation of when jobs are executed. This timeline helps identify unexpected gaps (indicating driver-side computation or blocking operations) or overlapping execution (in fair scheduling mode). For our single-job example, we see one continuous bar, but the length of that bar immediately signals a problem.

The Stages Tab: Finding the Bottleneck

The Stages tab is where serious performance debugging begins. Click on your slow job from the Jobs tab, then click on individual stages to investigate. This tab provides a granular view of what's happening during execution.

Stage summary view

The summary page shows stages grouped by status: active, pending, completed, skipped, and failed. For each stage, you see Input Size, Output Size, Shuffle Read, and Shuffle Write metrics. Looking at our sales filtering job:

Stage 0 (read + filter)
: This shows input of 273 MB but has an unusually high task count relative to the data size.

Stage 1 (write)
: This shows output being written with even more tasks.

The clue is in the task count. If we're reading 273 MB but have thousands of tasks, something is wrong with our file sizes. As we discussed in Chapter 8, the number of tasks directly impacts parallelization efficiency.



You can sort stages by duration to quickly identify the slowest stages. This is particularly useful in complex jobs with dozens of stages.

Stage detail view

Clicking on a stage reveals critical debugging information.

DAG visualization. The directed acyclic graph (DAG) shows RDD lineage for the stage. Each node represents an operation; edges show data flow. Nodes are grouped by operation scope, look for labels like `WholeStageCodegen` (indicating optimized, fused operations), `Exchange` (shuffle), or `FileScan`. For our job, we see a simple flow: FileScan → Filter → Write. The DAG looks reasonable, so the problem lies elsewhere.

Understanding the DAG is important because it shows you exactly how Spark is executing your logical plan. If you see unexpected nodes or a more complex graph than you anticipated, it may indicate that Spark's optimizer made decisions you didn't expect.

Event Timeline. The Event Timeline (hidden behind a dropdown) is one of the most valuable debugging tools. It shows task execution as horizontal bars, plotted by start time and grouped by executor. Bars are color-coded: green for computation, blue for shuffle read, orange for shuffle write, red for serialization.

For our problematic job, the Event Timeline reveals the issue: thousands of tiny tasks, each completing almost instantly but with significant scheduling overhead between them. The timeline shows more gaps than computation, a classic small files symptom. Ideally, we want to see fewer, longer tasks dominated by green (compute time).



The Event Timeline is particularly useful for identifying stragglers, tasks that take much longer than others due to data skew or resource contention.

Summary Metrics. The Summary Metrics table provides statistical distributions (min, 25th percentile, median, 75th percentile, max) for task metrics. Key columns for our investigation:

Duration
: If min and max are both tiny (milliseconds), tasks are too small. If max is 10× the median, you have skew. This distinction matters for choosing the right optimization strategy.

Input size/records
: For our job, we see each task processing only kilobytes—confirming small files.

GC time
: If tasks spend more than 10% in garbage collection, you have memory pressure. The Executors tab highlights rows in red when GC time exceeds this threshold.

Shuffle spill (memory/disk)
: Nonzero values indicate memory pressure during shuffles. As we discussed in Chapter 2, spilling to disk significantly degrades performance.

Aggregated metrics by executor. This section shows metrics per executor. Uneven distribution could indicate data locality issues or executor-level problems. For our local-mode job with one executor, this section is less relevant, but on a cluster it helps

identify struggling nodes; perhaps one executor has a failing disk or is colocated with a noisy neighbor process.

At this point, the native Spark UI has given us clues: too many tiny tasks, minimal compute time per task, and excessive overhead. We can infer it's a small files problem, but we had to dig through multiple screens and interpret raw numbers. This is where we can see the UI's academic heritage; it provides data but leaves diagnosis to you.

The SQL Tab: Understanding Query Plans

For DataFrame and SQL operations, the SQL tab provides query-specific information. Since our sales filtering job uses the DataFrame API, we can examine it here. This tab is essential for understanding how Catalyst, Spark's query optimizer, transformed your logical operations into a physical execution plan.

Query details

Each query shows duration, associated jobs, and links to execution plans. The detail view reveals:

Parsed Logical Plan
: Initial representation as parsed from DataFrame operations

Analyzed Logical Plan
: After resolving table and column references

Optimized Logical Plan
: After Catalyst optimizations like predicate pushdown and column pruning

Physical Plan
: The actual execution plan with specific operators

Understanding these four stages of query planning helps you identify where optimizations are (or aren't) being applied. For example, if you expected predicate pushdown but don't see it in the physical plan, your filter may not be pushdown compatible.

Key Physical Plan operators

Understanding these operators helps interpret the plan:

FileScan parquet
: Reading from Parquet files, the metrics show the files read and their size. Look for "PushedFilters" to verify predicate pushdown.

Filter
: Our `ss_quantity > 1` condition. If this appears after FileScan without pushdown, the filter is applied in Spark rather than at the storage layer.

`Exchange`
: A shuffle operation. Multiple Exchange nodes can indicate performance issues; each represents a stage boundary.

`InsertIntoHadoopFsRelationCommand`
: Writing output files.

`WholeStageCodegen`
: Multiple operators fused into optimized generated code, generally good. This is generally special inline Java code.

`BroadcastHashJoin` *versus* `SortMergeJoin`
: For joins, these indicate whether Spark is broadcasting the smaller table (fast) or sorting both sides (slower but necessary for large tables).

For our simple filter-and-write query, the physical plan looks reasonable. The SQL tab confirms we're doing what we expect, but it doesn't directly highlight the small files issue. We need to look elsewhere or use better tooling.

The Executors Tab: Resource Utilization

The Executors tab provides detailed information about each executor, including the driver. This is essential for understanding resource utilization and identifying executor-level problems. The key metrics are:

Storage memory
: Used versus available memory for caching. If near limits, cached data may be evicted. Recall from our discussion of persistence (see Chapter 5) that Spark uses LRU caching to determine which partitions to evict.

Task time (GC time)
: Total garbage collection time. Rows highlighted in red (>10% GC) indicate memory pressure. This is often the first sign that you need to increase executor memory or reduce partition sizes.

Shuffle read/write
: Data is shuffled by this executor. Uneven distribution suggests skew, with one executor doing disproportionate work.

Tasks (completed/failed)
: High failure rates on specific executors warrant investigation. Check the logs for that executor.

Input/output
: Data is read/written to external storage.

The Problem with the Native UI

After navigating through the tabs and examining DAGs, timelines, and metrics, we can *infer* that our job has a small files problem. But the native Spark UI never explicitly told us:

- "You are reading small files (average 85KB per file)."
- "You are writing even smaller files due to partitioning."
- "Here's how to fix it!"

The UI provides raw data but leaves diagnosis entirely to you. This is where DataFlint transforms the debugging experience.

Enter the DataFlint Tab

DataFlint is an open source performance monitoring plugin (Apache 2.0 licensed) that adds a modern tab to the Spark Web UI. It transforms job monitoring from archaeology into engineering by telling you what's wrong, why it matters, and how to fix it.

Installation

To install DataFlint, add two configuration lines to your `SparkSession`:

```
val spark = SparkSession
...
  .config("spark.jars.packages", "io.dataflint:spark_2.12:0.2.7")
  .config("spark.plugins", "io.dataflint.spark.SparkDataflintPlugin")
...
```

Or via spark-submit:

```
--packages io.dataflint:spark_2.12:0.2.7 \
--conf spark.plugins=io.dataflint.spark.SparkDataflintPlugin
```

No agents to deploy, no data export, no infrastructure changes. The plugin runs inside your Spark driver.

Identifying the problem immediately

Once DataFlint is installed, a new DataFlint tab appears in the Spark Web UI. Opening it for our sales filtering job, we immediately see what took us 20 minutes to diagnose with the native UI:

During the query
: DataFlint identifies a "reading small files" alert in real time. We don't have to wait for the job to complete.

After the query
: DataFlint identifies a "writing small partitioned files" alert, explaining that our `partitionBy` is creating many tiny output files.

Quantified impact
: The summary shows 51 seconds to process 273 MB, making the performance problem undeniable.

The DataFlint interface

DataFlint organizes insights into several areas accessible from the left sidebar:

Status
: Real-time query visualization with live updates. Instead of manually refreshing the native UI and parsing verbose operator trees, see your query plan as it runs with dynamic stage updates.

Summary
: Comprehensive metrics including execution time, resource utilization, and data processed. This is the high-level "How did this job perform?" view.

Resources
: CPU usage, memory allocation, disk I/O, network activity across executors.

Configuration
: Spark settings in an organized, searchable format.

Alerts
: Centralized performance warnings with specific issues, explanations, and code-level fixes.

Visual query plans

Where the native UI shows "Scan parquet," "Filter," and "Execute InsertIntoHadoopFsRelationCommand" with metadata-heavy tooltips, DataFlint shows the same operations as a clean flow: Read → Filter → Write. Each node displays:

- Row counts and data volume changes
- Partition counts and average file sizes
- Storage locations and formats
- Filter conditions with selectivity

A heat map provides instant visual feedback; problematic sections glow red, allowing immediate identification of slow operations without scanning through complex execution graphs.

The Fix: Actionable Recommendations

DataFlint doesn't just identify the "writing small partitioned files" issue; it suggests a specific fix: repartition the data in-memory by the partition key before writing.

Let's apply the fix:

```
val df = spark.read.load("~/data/store_sales")

val dfFiltered = df.filter(df("ss_quantity") > 1)
val dfRepartitioned = dfFiltered.repartition(df("ss_quantity"))  // THE FIX

dfRepartitioned.write
  .mode("overwrite")
  .partitionBy("ss_quantity")
  .parquet("/tmp/store_sales")
```

Why this works

The problem was that for each small source file, Spark reads the data, applies the filter, partitions the remaining data by quantity, and saves each partition of that small file, resulting in even smaller output files. This is a consequence of how Spark processes data partition-by-partition without global coordination.

By hash-repartitioning by quantity before writing, we ensure each Spark partition contains all records with the same quantity value. Now each Spark task writes exactly one file per table partition instead of many tiny fragments. This is an example of how understanding Spark's execution model, specifically that tasks operate on partitions independently, helps you structure operations for better performance.

The result

After applying the fix, the updated query plan in DataFlint shows two new steps: repartition by quantity hash (defaulting to 200 partitions), and then the Spark optimizer coalesces to 4 partitions based on data size.

Most importantly, *the writing alert disappears*. DataFlint confirms the fix worked.

The result: *15 seconds instead of 51 seconds*, more than a 3× speedup by adding a single line of code (on a cluster, this speedup could be even larger due to network latency and distributed storage metadata overhead that compounds with each additional file).

Other DataFlint Performance Alerts

Small files are just one of many issues DataFlint detects. Other automated alerts include:

Partition skew
: When some partitions are orders of magnitude larger than others, a few executors do all the work while others sit idly. DataFlint identifies skewed partitions and suggests balancing approaches like salting keys.

Memory issues
: Stages spilling to disk due to insufficient heap or executors with 64 GB allocated using only 8 GB. DataFlint flags misconfigurations and recommends right-sizing.

Idle cores
: When parallelism is misconfigured, CPU resources go unused—often because the partition count is lower than available cores.

Failure pinpointing
: When jobs fail, DataFlint extracts the actual error from complex JVM stack traces and shows exactly which stage failed; no more hunting through logs.

These aren't generic warnings. Each alert ties to a specific part of your query plan and includes the configuration change or code modification needed to fix it.

The Spark History Server

The live web UI disappears when your application terminates. For postmortem analysis of completed jobs, use the Spark History Server.

Enable event logging in your applications:

```
spark.eventLog.enabled=true
spark.eventLog.dir=hdfs:///spark-history
```

Start the History Server:

```
$SPARK_HOME/sbin/start-history-server.sh
```

Use `spark.history.fs.logDirectory` to configure it to read from your event log directory.

These should be considered:

Log size
: Large applications generate substantial logs. Enable compression with `spark.eventLog.compress=true`. For very long-running applications, logs can grow to gigabytes.

Scale limits
: The History Server can struggle with tens of thousands of jobs or very large event logs. Consider increasing server memory or using alternative monitoring for very large applications.

Index

A

accelerators, 243-252
  Application-Specific Integrated Circuits, 252
  Comet, 246
  Databricks Photon, 245
  Project Gluten, 247-250
  Spark RAPIDS, 250-252
  Velox, 247-249
accumulators
  RDDs and transformations, 167
  reducing setup overhead for transformations, 150-154
  validating jobs with, 322
actions
  key/value pairs, 175
  multiple on the same RDD, 156
  transformations functions versus, 20
adaptive query execution (AQE), 95
agg API, 71
aggregate functions
  aggregateByKey, 134, 182
  aggregateColumnFrequencies, 204
  user-defined, 83-89, 229
aggregates (Spark SQL), 70-74
aggregation operations
  avoiding memory allocation, 182
  preventing out-of-memory errors, 182
aggregations (Spark Streaming), 356-358
  user/arbitrary state, 358
  watermarking, 357
  windows, 357
Alluxio (Tachyon), 163
AMP (Automatic Mixed Precision), 264
Apache Arrow
  Project Gluten and, 247-250
  PySpark user-defined functions with/without, 228
Apache Arrow DataFusion Comet, 246
Apache Bahir, 330
Apache Beam, 344
Apache Software Foundation (ASF), 1
Application-Specific Integrated Circuits (ASICs), 252
applications (Spark), 23-25, 366
approxEqualDataFrames, 303
AQE (adaptive query execution), 95
Arrow (see Apache Arrow)
ASF (Apache Software Foundation), 1
ASICs (Application-Specific Integrated Circuits), 252
async (parallel) actions, 25
async progress tracking (Spark Streaming), 355
as[ElementType] function, 80
audits (see validation)
automated testing, 294
Automatic Mixed Precision (AMP), 264
Awesome Spark, 12

B

barrier execution mode (gang scheduling), 349
batch prediction, scaling predictions, 271
big data ecosystem, 8-12
broadcast hash joins, 116, 124
broadcast nested loop joins, 124
broadcast variables, transformation setup overhead and, 149

C

C#, 236
caching, checkpointing versus, 161
cartesian (cross) joins, 122
cartesian product (shuffle and replicate) joins, 124
Cassandra connector, 330
checkpointing
  caching versus, 161
  off_heap, 157
  RDDs, 161
class tags, fake, 223
ClickHouse, 249
CLR (Common Language Runtime), 236
cluster managers, 9
co-loaded RDDs, 188
co-located joins, 112
co-partitioned RDDs, 188
coalesce operation, 185
coalescence, 132
codegen, testing, 304
cogroup function, 70-74, 113, 184
collectAsMap action, 178
column metadata, 62
combineByKey function, 113, 180
Comet (see Apache Arrow DataFusion Comet)
Common Language Runtime (CLR), 236
community packages and libraries, 327-329
components and packages (Spark), 325-334
  building your own Spark versus using packages, 333
  community packages/libraries, 327-329
  creating a Spark package, 331-333
  GraphX, 327
  popular packages, 330
core Spark
  components, 11-12
  jobs testing with RDDs, 297
  joins, 111-118
    execution plans, choosing one, 115-118
    join types, 113-115
    partial manual broadcast hash join, 118
    speeding up joins by assigning a known partitioner, 115
    speeding up joins using a broadcast hash join, 116
countByKeyApprox function, 118
counters
  validating jobs with, 322
  verifying performance with, 311
cross (cartesian) joins, 122
custom code, 225-234
  accessing backing Java objects/mixing Scala code, 230-233
  how PySpark works, 225
  installing PySpark, 234
  pandas API on Spark, 228
  PySpark DataFrames/Datasets, 227
  PySpark dependency management, 233
  PySpark RDDs, 226
  user-defined functions in PySpark with/without Arrow, 228
custom encoders, 89
custom expressions, 87-89

D

DAG (directed acyclic graph), 26, 368
DAG Scheduler, 26
Dask, compared to Spark, 8
data validation, 313-321
  input tables, 321
  output tables and WAP, 321
Data Validator (Target tool), 315
Databricks Photon, 245
DataFlint, 372-375
  actionable recommendations provided by, 374
  installation, 372
  interface, 373
  various automated performance alerts, 375
  visual query plans, 373
DataFrame API, 63-77
  (see also Spark Connect)
  multi-DataFrame transformations, 76
  SQL queries/interacting with metastore tables, 77
  transformations, 63-76
DataFrame joins, in Spark SQL, 118-123
  complex join conditions, 122
  cross/cartesian joins, 122
  self joins, 121
DataFrameReader, 90
DataFrames, 55-79
  column metadata, 62
  data loading/saving functions, 90-102
    DataFrameWriter/DataFrameReader, 90
    formats, 90-99
    merge, 100

partitions, 101
save modes, 99
data representation in DataFrames/Datasets, 78
DataFrame API, 63-77
multi-DataFrame transformations, 76
SQL queries/interacting with metastore tables, 77
transformations, 63-76
interoperability with Datasets, 79-81
PySpark, 227
schema basics, 60-63
unit testing, 301-304
DataFrameWriter, 90
Dataset joins, in Spark SQL, 125
Datasets, 79-83
compile-time strong typing, 81
grouped operations on, 82
interoperability with RDDs/DataFrames/Local Collections, 79-81
multi-Dataset relational transformations, 82
PySpark, 227
RDD-like transformations, 81
relational transformations, 81
debugging
counters for, 311
lazy evaluation and, 16
local tools versus Spark, 343
Spark 4.2 changes, 48
Spark SQL queries, 108
declarative pipelines, 52
DeepSpeed, distributed training with, 263-266
benefits in practice, 266
DeepSpeedTorchDistributor, 265-266
why DeepSpeed matters, 265
dependencies() method, 19
df.toPandas(), 268-269
diffusion models, in generative AI, 258
directed acyclic graph (DAG), 26, 368
distinct operation, 113
Docker, integration testing with, 311
DStream streaming tests, 298
dynamic resource allocation, 23

E

enableHiveSupport function, 58
Enhanced Agentic-RAG (EAg-RAG)
agentic answer generation, 279
enriched document processing, 279
faster iteration via automated evaluation, 280
LLM-as-a-Judge for RAG systems, 281
Spark and, 282
optimizing recommendations, 283-287
enriched document processing, with generative Ai, 279
equals method, 187
Event Timeline (Spark Web UI), 369
exactly-once output, 361
execution hierarchy (Spark), 366
application, 366
jobs, 366
stages, 367
tasks, 367
execution operators, concrete SQL joins, 123-125
executor sizing, 286
Executors tab (Spark Web UI), 371

F

fair scheduler, 166
fault tolerance, 15, 131
Fight Health Insurance (generative AI tool), 275
flatMap function, 174
foldByKey function, 183
foreachBatch sink, 360-361
foreign function interfaces (FFIs), 220
Fortran, 242
full outer joins, 120

G

gang scheduling (barrier execution mode), 349
garbage collection (GC), 134
generative adversarial networks (GANs), 257
generative AI, 255-292
basics, 256-260
applications, 258
deep learning and, 259
how it works, 257-258
case studies, 274-287
Enhanced Agentic-RAG at Uber, 277-287
Fight Health Insurance, 275
future of generative AI with Spark, 290-292
creative industries, 291
path forward, 291

role of Spark in generative AI revolution, 290
hosting versus leveraging a third-party REST API, 287-290
evolving industry, 290
layer 1: use case/domain alignment, 287
layer 2: performance requirements, 287
layer 3: scalability needs, 288
layer 4: cost considerations, 288
layer 5: data sensitivity/security, 288
layer 6: integration/ecosystem fit, 289
layer 7: time-to-market, 289
inference in the real world, 271-274
batch prediction, 271
embedding inference into workflows, 272
factors when choosing a model, 273
model evaluation, 273
real-time predictions at scale, 272
role of Spark, 260-271
processors/RAPIDS, 266-271
PyTorch scaling with TorchDistributor, 262
supercharging distributed training with DeepSpeed, 263-266
TensorFlow scaling with TensorFlowDistributor, 261-262
getPartition method, 187
Gluten (see Project Gluten)
Goldilocks example (key/value data)
postmortem, 215-216
version 0: iterative solution, 172-174
version 1: groupByKey solution, 176-180
why groupByKey fails, 178-180
version 2: secondary sort, 198-202
combining the elements associated with one key, 201
defining the custom partitioner, 199
filtering on each partition, 199
performance, 201
version 3: sorting on cell values, 207
determining location of rank statistics on each partition, 205
filter for rank statistics, 206
map to (cell value, column index) pairs, 203
sort/count values on each partition, 203
version 4: reducing to distinct on each partition, 210-216
aggregating to ((cell value, column index), count) on each partition, 210
sorting/finding rank statistics, 211-215
GPUs, 243, 252, 261
gradient management, 264
GraphX, 12, 45, 327
groupBy function, 70-74
groupByKey function
dangers of, 176-180
Goldilocks example: version 1, 176-180
why it fails, 178-180
groupByKeyAndSortValues function, 195-198
groupSorted function, 201

H

hash partitioning, 186
hashcode method, 188
historic comparisons, data validation, 319
Hive, 58, 77

I

Iceberg, 77, 335-338
partitioning/indices/statistics, 337
query planning, 338
updates/table maintenance, 336
immutability of RDDs, 18-19
inner joins, 119
integration testing, 310
interprocess communication (IPC), 220
iterative algorithms, 105
iterative computations, reusing RDDs, 155
iterator object (Scala), 142
iterator(p, parentIters) method, 18
iterator-to-iterator transformations, 141-147

J

Janino, 88
Java API, 222
Java Native Access (JNA), 241
Java Native Interface (JNI), 238-241
Java virtual machine (see JVM)
JDBC, 91-93
JDBC/ODBC server, 108
JdbcDialect, 92
jobs (Spark)
anatomy, 26-30
DAG, 26
jobs, 27

stages, 27
tasks, 28
scheduling, 23-25
default scheduler, 25
resource allocation, 23
the Spark application, 23-25
in Spark execution hierarchy, 366
jobs tab (Spark Web UI), 367
joins, 111-126
(see also specific joins)
cogroup function, 184
complex conditions in DataFrame joins, 122
core Spark joins, 111-118
execution plans, choosing one, 115-118
join types, 113-115
leveraging co-loaded/co-partitioned RDDs, 188
RDD-type, 111-118
self joins, 121
Spark 4.2 changes, 47
Spark SQL joins, 118-125
concrete SQL join execution operators, 123-125
DataFrame joins, 118-123
Dataset joins, 125
supported join types, 119
JVM (Java virtual machine), 8, 221-236
CLR and C#, 236
custom code beyond Scala/JVM, 225-234
how SparklyR works, 234

K

key/value data, 169-217
actions on key/value pairs, 175
choosing an aggregation operation, 180-183
co-grouping, 184
Goldilocks example, 171-175
Goldilocks version 0: iterative solution, 172-174
Goldilocks version 1: groupByKey solution, 176-180
how to use PairRDDFunctions/OrderedRDDFunctions, 174
groupByKey function, dangers of, 176-180
multiple RDD operations, 184
OrderedRDDFunctions class, dictionary of operations, 192
out-of-memory error prevention, 182
partitioners, 185-191
custom partitioning, 187
dictionary of mapping/partitioning functions, 190
hash partitioning, 186
leveraging co-loaded/co-partitioned RDDs, 188
preserving partitioning information across transformations, 188
range partitioning, 186
Spark partitioner object, 186
secondary sort/repartitionAndSortWithinPartitions, 193-208
Goldilocks version 2: secondary sort, 198-202
Goldilocks version 3: sort on cell values, 202-206
how not to sort by two orderings, 198
leveraging repartitionAndSortWithinPartitions for a groupByKeyAndSortValues function, 195-198
SortByKey, 193
straggler detection/unbalanced data, 208-216
Goldilocks version 4: reduce to distinct on each partition, 210-216
known partitioner joins, 112, 115
Kubernetes
cluster managers, 9
for integration testing, 311
support in Spark 4.2, 49

L

lazy evaluation, 13-16
debugging, 16
fault tolerance, 15
performance/usability advantages, 14
transformations and, 128
left anti joins, 121
left outer joins, 115, 120
left semi joins, 121
LLM-as-a-Judge, 281
local checkpointing option, 163
Local Collections, interoperability with Datasets, 79-81
LRU caching, 163

M

map function, flatMap function versus, 174
mapPartitions function

example of complicated routine, 143-147
iterator-to-iterator transformations, 141-147
preserving partitioning, 188
reducing setup overhead, 148
MapReduce, 14
Maven Central, 12
memory management
outside of the JVM, 253
RDDs, 17
merge API, 100
metastore tables, SQL queries interacting with, 77
micro-batching, 354
mocking RDDs, for unit testing, 296
model parallelism, 264

N

narrow dependencies, in transformations, 21-22
narrow transformations, 128-132
nonaggregate UDFs, 84
numPartitions method, 187

O

off_heap, 157
one known partitioner joins, 112
optimized skew joins, 124
OrderedRDDFunctions class, 169
defining an implicit ordering for, 175
dictionary of operations, 192
how to use, 174
OrderedRDDOperations, 193
out-of-disk-space error, 165
out-of-memory errors
managing memory outside of JVM, 221, 253
preventing in aggregation operations, 182

P

PairRDDFunctions class, 169
cogroup function, 184
dictionary of mapping/partitioning functions, 190
how to use, 174
pandas
API, 228
df.toPandas(), 268-269
Parameterized SQL, 50
partial manual broadcast hash joins, 118
partition pruning, 46
partitionBy function, 185
partitioner() method, 19
partitioners, key/value data and, 185-191
custom partitioning, 187
dictionary of mapping/partitioning functions, 190
hash partitioning, 186
leveraging co-loaded/co-partitioned RDDs, 188
preserving partitioning information across transformations, 188
range partitioning, 186
Spark partitioner object, 186
partitioning functions, 191
partitions() method, 18
partitions, in Spark SQL, 101
per-record side effects, 346
persist function, 155
persisting, 157
pipe function, 236
pipe interface, 236
pipeline parallelism, 264
pipeline steps, 52
pivot, 73
Postgres, Spark versus, 15
predicate pushdown, 46
preferredLocations(p) method, 19
printSchema function, 60
Project Gluten, 247-250
ClickHouse + Gluten, 249
future, 249
getting started, 248
Velox + Gluten, 249
Project Panama, 242
Project Zen, 51
property checking, with ScalaCheck, 307
PySpark
DataFrames/Datasets, 227
dependency management, 233
how it works, 225
installing, 234
RDDs, 226
user-defined functions with/without Arrow, 228
Python
code/logic changes in Spark 4.2, 51
ease of use of API, 4

Project Zen, 51
upgrading Spark, 38
PyTorch, scaling with TorchDistributor, 262

Q

queries
large query plans/iterative algorithms, 105
optimizer, 102-106
untrusted/multitenant, 346
query optimizer, 102-106
code generation, 105
large query plans/iterative algorithms, 105
logical/physical plans, 102
Spark adaptive query execution (AQE), 104
static optimizer rules, 103-104
query planning
Iceberg, 338
Spark Web UI SQL tab and, 370-371

R

RAG systems, 281
range partitioning, 186
RAPIDS (see Spark RAPIDS)
Ray, Spark versus, 8
RDD-type joins, 111-118
RDDs (resilient distributed datasets), 4, 11-22
accumulators, 167
cases for reuse
if the cost to compute each partition is very high, 157
iterative computations, 155
multiple actions on the same RDD, 156
co-grouping, 184
computing RDD difference, 308
functions on RDDs, transformations versus actions, 20
immutability and the RDD interface, 18-19
in-memory persistence/memory management, 17
interoperability with Datasets, 79-81
joins, 111-118
lazy evaluation, 13-16
debugging, 16
fault tolerance, 15
performance/usability advantages, 14
leveraging co-loaded/co-partitioned RDDs, 188
mocking for unit testing, 296
multiple RDD operations, 184
reusing, 155-167
shuffle files, 164
types of, 19
types of reuse, 159-163
checkpointing, 161
checkpointing example, 162
persist and cache, 159-161
unit testing core Spark jobs with, 297
wide versus narrow dependencies, 21-22
read-eval-print loops (REPLs), 4
reading/writing data, formats for, 90-99
going beyond existing options, 98
JDBC, 91-93
JSON, 90
local collections, 97
metastore tables, 94
Parquet, 93
RDDs, 95-97
various formats/tables, 98
receivers, replayable/nonreplayable, 363
recompute, 157
reduceByKey function, 183
repartition function, 185
repartitionAndSortWithinPartitions, 194
Goldilocks version 2: secondary sort, 198-202
Goldilocks version 3: sort on cell values, 202-206
how not to sort by two orderings, 198
leveraging for groupByKeyAndSortValues function, 195-198
REPLs (read-eval-print loops), 4
resilient distributed datasets (see RDDs)
resource allocation, 23
resource profiles, 349
REST APIs, third-party, 287-290
right outer joins, 120
RStudio, 234
runtime exceptions, 34

S

save modes (Spark SQL), 99
Scala, 3-5
advantages for Spark development, 4
code/logic changes in Spark 4.2, 51
disadvantages of, 4
ease of use, 4
iterator object, 142
learning resources for, 5

Scalafix, simplifying Spark upgrading with, 35-38
ScalaCheck, 307
Scalafix, 35-38
schema-based data validation, 319
schemas, 60-63
secondary sort, 193-208
Goldilocks version 2: secondary sort, 198-202
Goldilocks version 3: sort on cell values, 202-206
how not to sort by two orderings, 198
leveraging for groupByKeyAndSortValues function, 195-198
selectExpr function, 77
self joins, 121
set operations, 147-148
seven layers framework, 287-289
shading, 34
shared variables, 149
shuffle and replicate joins, 124
shuffle files, 164
shuffle hash joins, 115, 124
shuffle joins, 111
shuffle sort joins, 124
ShuffleDependencies, 28
shuffles, key/value operations and, 170-171
(see also Goldilocks example)
side effects, per-record, 346
side-by-side runs, 323
singleton builder pattern, 57
sinks, idempotent/non-idempotent, 363
sortBy function, 172
sortByKey function
sorting by two keys with, 193
wide dependencies, 131
Spark (generally), 1-6
about, 1
Big Data ecosystem, 8-12
catalog system, 10
cluster managers, 9
multitool pipelines, 10
Spark components, 11-12
calling other languages, 236-243
getting to the GPU, 243
Java Native Access (JNA), 241
Java Native Interface (JNI), 238-241
pipe interface, 236
Project Panama, 242
underneath everything Is Fortran, 242
customizing, 333
execution hierarchy, 366
how it works, 7-32
job scheduling, 23-25
default scheduler, 25
resource allocation, 23
the Spark application, 23-25
parallel computing: RDDs, 13-22
functions on RDDs: transformations versus actions, 20
immutability and the RDD interface, 18-19
in-memory persistence/memory management, 17
lazy evaluation, 13-16
types of RDDs, 19
wide versus narrow dependencies, 21-22
performance, 1
recent improvements, 348
Scala and Python, 3-5
advantages of Scala for Spark development, 4
disadvantages of Scala, 4
ease of use, 4
learning resources for, 5
Spark Connect, 30-32
Spark job anatomy, 26-30
DAG, 26
jobs, 27
stages, 27
tasks, 28
upgrading (see upgrading Spark)
versions, 3
when not to use, 343-348
frequently accumulated updates, 347
global in-order processing, 348
highly unreliable backends, 347
huge individual records/rows, 346
long/highly variable task processing time, 345
no ops support/no budget for platform, 347
per-record side effects, 346
real-time updates/online transaction processing, 345
running many different functions on same data, 347
small data/small tasks, 344

untrusted/multitenant queries, 346
user-facing interactive use cases, 345
Spark 4.2, changes since 2.4, 45-53
additional upgrades required (not code), 48
code/logic changes needed, 49-51
debugging, 48
GraphX, 45
joins, 47
looking ahead, 53
partition pruning/predicate pushdown, 46
prioritizing upgrades, 51
totally new features, 52
Spark adaptive query execution (AQE), 104
Spark Cassandra connector, 330
Spark Connect, 30-32, 50, 339-341
Spark Datasets, 49
Spark declarative pipelines, 52
Spark ML, 12, 45
Spark project improvement proposals (SPIP), 53
Spark RAPIDS, 250-252
AI ecosystem, 266-271
distributed inference with RAPIDS, 269
distributed training with RAPIDS/TorchDistributor, 266-269
why RAPIDS matters for Spark deep learning, 271
going beyond RAPIDS on the GPU, 252
project health, 252
setting up/configuring Spark RAPIDS, 250
Spark SQL, 11, 50
data loading/saving functions, 90-102
dependencies, 58-59
joins, 118-125
concrete SQL join execution operators, 123-125
DataFrame joins, 118-123
Dataset joins, 125
supported join types, 119
Spark Streaming, 12, 351-364
aggregations, 356-358
arrival time attribute, 356
checkpointing, 362
driver considerations, 352
exactly-once output, 361
idempotent/non-idempotent sinks, 363
operation modes, 353-355
async progress tracking, 355
micro-batching, 354
real-time mode, 355
output modes, 356
real-time guarantees versus reasonable expectations, 364
replayable/nonreplayable receivers, 363
stream-stream joins, 358-361
foreachBatch sink, 360-361
state store, 359
word count example, 352-353
Spark Web UI, 365-376
real-world performance example
accessing Spark Web UI, 366
basic scenario, 365
DataFlint tab, 372-375
Executors tab, 371
jobs tab, 367
limitations of native Spark UI, 372
Spark execution hierarchy, 366
SQL tab, 370-371
stages tab, 368-370
Spark History Server, 375
Spark-Expectations (validation library), 316
spark-redis, 330
SparkContext, 18, 23
SparklyR, 234
SparkSession, 57, 106-107
SPIP (Spark project improvement proposals), 53
SQL
plain SQL queries interacting with metastore tables, 77
upgrading Spark, 39
SQL joins, in Spark, 118-125
concrete SQL join execution operators, 123-125
DataFrame joins, 118-123
Dataset joins, 125
SQL UDFs/UDTs, in Spark 4.2, 53
SQLContext, 57
stages
in Spark hierarchy, 367
in Spark jobs anatomy, 27
stages tab, in Spark Web UI, 368-370
Standalone Cluster Manager, 9
state store (Spark Streaming), 359
static resource allocation, 23
stream testing, 298
StreamingActionBase, 298
StreamingSuiteBase, 298

StructField, 61
Structured Streaming tests, 299
Summary Metrics table (Spark Web UI), 369
summary statistics data validation, 318

T

table-level assertions, 323
Tachyon, 163
tasks, 28
  scheduling, 349-350
    gang scheduling/barrier execution mode, 349
    resource profiles, 349
  Spark execution hierarchy, 367
  Summary Metrics table, 369
TaskScheduler, 27
TensorFlowDistributor, 261-262
testing, 293-324
  (see also validation)
  codegen testing, 304
  computing RDD difference, 308
  getting test data, 304-307
    generating large datasets, 305
    sampling, 306
  integration testing, 310
    Kubernetes/Docker-based, 311
    local mode, 311
    production or staging clusters, 311
  ScalaCheck for property checking, 307
  side-by-side runs, 323
  table-level assertions, 323
  unit testing, 295-304
    factoring code for testability, 295
    mocking RDDs, 296
    testing core Spark jobs with RDDs, 297
    testing DataFrames, 301-304
    testing streaming, 298-299
  verifying performance, 311
    counters, 311
    projects for, 313
textual data, 348
TorchDistributor, 262
transformations, 20, 127-167
  actions functions versus, 20
  DataFrame API, 63-76
    aggregates/groupBy/cogroup, 70-74
    beyond row-by-row transformations, 70
    simple transformations/SQL expressions, 64-68
    sorting, 76
    specialized transformations for missing/noisy data, 69
    windowing, 74-75
  iterator-to-iterator transformations with mapPartitions, 141-147
    example of complicated routine, 143-147
    iterator-to-iterator transformation defined, 142
    space/time advantages, 143
  minimizing object creation, 134-141
    reusing existing objects, 134-138
    using smaller data structures, 138-141
  multi-DataFrame transformations, 76
  multi-Dataset relational transformations, 82
  narrow versus wide transformations, 128-132
    coalescence, 132
    fault tolerance implications, 131
    performance implications, 130
  RDD-like transformations, 81
  reducing setup overhead, 148-154
    accumulators, 150-154
    broadcast variables, 149
    shared variables, 149
  relational transformations, 81
  reusing RDDs, 155-167
    Alluxio/Tachyon, 163
    cases for reuse, 155-157
    deciding if recompute is inexpensive enough, 157
    interaction with accumulators, 167
    LRU caching, 163
    noisy cluster considerations, 166
    shuffle files, 164
    types of reuse, 159-163
  set operations, 147-148
  type of RDD returned by, 132
  wide versus narrow dependencies, 21-22
transformers, in generative AI, 258
transformWithState operation, 358
treeAggregate function, 183
Tungsten, 78

U

UDAFs (user-defined aggregate functions), 83-89, 229
UDFs (see user-defined functions)
UDTFs (user-defined table functions), 89

unit testing, 295-304
  factoring code for testability, 295
  mocking RDDs, 296
  testing core Spark jobs with RDDs, 297
  testing DataFrames, 301-304
  testing streaming, 299
unpivot, 73
upgrading Spark, 33-43
  finding what needs changing, 33-35
    compile time changes, 34
    differing results, 35
    exceptions at runtime, 34
  performance-degrading upgrades, 42
  updating your code, 35-39
    Python, 38
    Scala, 35-38
    SQL, 39
  verifying correctness/performance, 39
user-defined aggregate functions (UDAFs), 83-89, 229
user-defined functions (UDFs), 83-89
  custom encoders, 89
  custom expressions, 87-89
  user-defined table functions, 89
user-defined table functions (UDTFs), 89

V

validation (audits), 293, 313-322
  counters/accumulators, 322
  data validation, 313-321
variational autoencoders (VAEs), 258
Velox + Gluten, 249

W

WAP (Write-Audit-Publish) pattern, 321
wide transformations, 128-132
wide versus narrow dependencies, in transformations, 21-22
writing data (see reading/writing data)

Z

ZeRO (zero redundancy optimizer), 263
zipWithIndex function, 172

About the Authors

Holden Karau is a queer transgender Canadian, Apache Spark committer, Apache Software Foundation member, and an active open source contributor. As a software engineer, she's worked on a variety of distributed computing, search, AI, and classification problems at Apple, Netflix, Google, IBM, Alpine, Databricks, Foursquare, and Amazon. She graduated from the University of Waterloo with a bachelor of mathematics in computer science. In addition to her big data work, she cofounded Fight Health Insurance to help patients appeal health insurance denials. Outside of software, she enjoys playing with fire, welding, riding motorcycles, eating poutine, and dancing.

For most of **Adi Polak**'s professional life, she dealt with data and machine learning. As a data practitioner, she developed algorithms to solve real-world problems using machine-learning techniques. As an engineer, she brought her hands-on machine learning experience to various Fortune 500 companies' products and services by building upon cutting-edge and emerging technologies. Adi has been working and contributing to the Apache Spark community since 2013 and taught Spark to thousands of students throughout the year. Adi is an official Databricks ambassador, the author of the successful book, *Scaling Machine Learning with Spark* (O'Reilly), and a respected worldwide presenter.

Rachel Warren is a data scientist and software engineer at Alpine Data Labs, where she uses Spark to address real-world data processing challenges. She has experience working as an analyst both in industry and academia. She graduated with a degree in computer science from Wesleyan University in Connecticut.

Colophon

The animal on the cover of *High Performance Spark* is a fire-tailed sunbird (*Aethopyga ignicauda*) native to Southeast Asia and the Indian subcontinent. Sunbirds are very distant relatives of the hummingbirds of the Americas and the honeyeaters of Australia.

As their name suggests, these birds (especially males) have very vivid coloring. Their tails and the back of their necks are red, their wings are green, their bellies are yellow and orange, and their heads are an iridescent blue. Male fire-tailed sunbirds are slightly larger than females, at an average of 15 centimeters long. Both members of a mating pair participate in feeding their chicks.

The fire-tailed sunbird's preferred habitat is conifer forest, where it consumes a diet of insects and nectar. The downward curve of its beak and a tubular tongue help it reach into flowers for food.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on a black-and-white engraving from Wood's *Illustrated Natural History*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.



O'REILLY®

Learn from experts. Become one yourself.

60,000+ titles | Live events with experts | Role-based courses
Interactive learning | Certification preparation | Verifiable skills

Try the O'Reilly learning platform free for 10 days.

©2026 O'Reilly Media, Inc. O'Reilly is a registered trademark of O'Reilly Media, Inc. 1035600_7x9.1875